VBD

Based on R.T.M. Nagpur University New Syllabus

Based on New Syllabus

SYSTEM PROGRAMMING

Prof. M. Shirbhate

SOLVED QUESTION BANK
+
UNIVERSITY PAPER SOLUTIONS

B.TECH. IV SEMESTER

COMP. SCI. & ENGG., COMP. ENGG.
(CSE, CE)

B.TECH. III SEMESTER

INFO. TECH. (IT)

! Xerox center सावधान !

प्रिय विद्यार्थी, VBD की Xerox करने वाले Xerox center की संपूर्ण जानकारी दीजिए और इनाम पाइए। यदि हमारी जांच में आपके द्वारा दी गई जानकारी सही पाई गई तो आपको 5000/- रुपये का इनाम दिया जाएगा।* उपरोक्त जानकारी WhatsApp No. 9373644557 या Email id : vbdnagpur@gmail.com पर भेजें।

Vedika S. Raut

VBD

SYSTEM PROGRAMMING

FOR
B.Tech. IV SEMESTER
Computer Science & Engineering
(CS, CE)

B.Tech. III SEMESTER
Information Technology
(IT)

By

Prof. M. Shirbhate
B.E., M.Tech. (CSE)
Assistant Professor

Based on R.T.M. Nagpur University New Syllabus

VBD PUBLICATIONS

Distributed by:
VBD **Publications Pvt. Ltd.**
477, Golchha Marg
Sadar Nagpur – 440001.

Sales Office:
Plot No. 236, Near Ram Coolers,
Besides Ganesh Temple,
Singada Market, Navi Shukrawari,
Mahal, Nagpur - 32
☎: 0712 - 2722235, 7620266004, Fax : 0712 - 2722236
Website : www.vbdpublications.com
E-mail : vbdbooks@gmail.com

© Publishers

Price : Rs. 380/-

Edition : 2022

The shelf-life of this book is 6 months.

फोटोकॉपी (झेराक्स) करने से मैटर बहुत छोटा हो जाता है और इसे पढ़ने से आपकी आँखें कमजोर होती हैं।
As per Indian Copyright Act 1957, photocopying of a book is punishable under law.

Books Available At ALL LEADING BOOK SELLERS
OR CONTACT:
VIJAY BOOK DEPOT
GOLCHHA MARG, SADAR, NAGPUR
☎: 0712 - 2520496

For online purchases, please visit WWW.VBDPUBLICATIONS.COM

The text of this publication, or any part thereof, should not be reproduced or transmitted in any form or stored in any computer storage system or device for distribution without the prior written permission of the publisher.

Note: While every care and precautions have been taken regarding the contents and data of this book, the publisher does not hold responsibility for any error or omissions. Please refer prescribed text books. Any dispute will be subject to Nagpur Jurisdiction only.

Published by : ABD Publishers & Printers Pvt. Ltd., Nagpur.

Statistical Analysis

Exam. / Unit No.	S - 12	W - 12	S - 13	W - 13	S - 14	W - 14	S - 15	W - 15	S - 16
Unit - I	26 M	27 M	27 M	30 M	28 M	26 M	28 M	28 M	28 M
Unit - II	26 M	23 M	13 M	20 M	26 M	28 M	26 M	26 M	26 M
Unit - III	18 M	24 M	30 M	16 M	30 M	26 M	26 M	26 M	26 M
Unit - IV	19 M	26 M	19 M	22 M	22 M	26 M	26 M	28 M	26 M
Unit - V	21 M	21 M	27 M	23 M	26 M	26 M	26 M	26 M	28 M
Unit - VI	26 M	19 M	21 M	21 M	28 M	28 M	28 M	28 M	28 M
Questions covered in VBD	136 M (100%)	140 M (100%)	137 M (100%)	132 M (100 %)	160 M (100%)	160 M (100 %)	160 M (100%)	160 M (100 %)	160 M (100%)

Exam. / Unit No.	W - 16	S - 17	W - 17	S - 18	W - 18	S - 19	W - 19
Unit - I	26 M	26 M	28 M	26 M	28 M	28 M	28 M
Unit - II	26 M	26 M	26 M	28 M	26 M	26 M	26 M
Unit - III	28 M	28 M	26 M	28 M	28 M	26 M	26 M
Unit - IV	26 M	26 M	26 M	26 M	26 M	26 M	26 M
Unit - V	26 M	26 M	28 M	26 M	26 M	26 M	26 M
Unit - VI	28 M	28 M	26 M	26 M	26 M	28 M	28 M
Questions covered in VBD	160 M (100 %)	160 M (100%)	160 M (100 %)	160 M (100%)	160 M (100 %)	160 M (100%)	160 M (100 %)

Note:- Due to Covid - 19 Summer - 2020 to Winter - 2021 exams were conducted online by the respective Colleges.

PREFACE

This *VBD* on **System Programming** has been prepared for the students of **Fourth Semester Computer Science and Computer Engineering and Third Semester Information Technology Bachelor of Technology (CBCS)-** as per the New Syllabus prescribed by the RTM-Nagpur University.

The subject matter has been thoroughly dealt with, covering 100 % syllabus.

Each unit begins with its syllabus. The previously asked University questions are fully covered in the order as per the syllabus.

In each unit, sufficient theory is given then followed by sample Assembly language programs.

Every type of Assembly language program is included alongwith previously asked.

Fully solved University Question Papers are included at the end with solutions reference inside.

I am grateful to the publishers and the staff of *VBD* Publications, Nagpur, who have taken considerable pains and shown extreme co-operation during the preparation of this book.

I will appreciate suggestions from teachers and students for the improvement of the book in the future editions.

Author

SYLLABUS

[B. Tech. IV Semester Computer Engineering]

UNIT - I

IBM 360/370 and Assembler : Introduction to System Programming and its components, M/c Architecture, Instruction Formats, Data Formats Data Formats and Register Formats, Concept of assembler, Design of single pass and two pass assembler.

UNIT - II

Macro Processor : Concept of macro, macro call within macro, macro definition within macro, recursive macro calls, design of macro processor.

UNIT - III

Linkers and Loaders: Basic Loader functions, Loader schemes, "Compile and Go" Loaders, general Loader scheme, absolute loaders, subroutine linkages, relocating loaders, direct linking loaders, other loader schemes Binders, linking loaders, Overlays, Dynamic Binders, Design of an absolute Loaders, Design of a Direct – Linking loaders.

UNIT - IV

Compiler : Phases of Compilers, Overview of Databases and Algorithms required for all phases. Role of lexical analyzer, recognition of tokens, Study of LEX and YACC.

UNIT - V

Unix Device Drivers: Definition, Anatomy and Types, Device programming, Installation, Incorporation of driver routines, Basic device operation, Implementation with Line Printer, Comparative study between device drivers for UNIX and Windows. Case study of Intel®64 and IA-32 Processors Basic architecture.

CONTENTS

[B. Tech. IV Semester Computer Engineering]

UNIT - I

IBM 360/370 AND ASSEMBLER

- Introduction to System Programming and its components — P.1-3
- M/c Architecture — P.1-6
- Instruction Formats — P.1-6
- Data Formats and Register Formats — P.1-10
- Concept of assembler — P.1-10
- Design of single pass and two pass assembler — P.1-15, 17, 19

UNIT - II

MACRO PROCESSOR

- Concept of macro — P.2-3
- Macro call within macro — P.2-6
- Macro definition within macro — P.2-6
- Recursive macro calls — P.2-7
- Design of macro processor — P.2-7, 9, A-38

UNIT - III

LINKERS AND LOADERS

- Basic Loader functions — P.3-3
- Loader schemes, "Complie and Go" Loaders — P.3-4
- General Loader scheme — P.3-5
- Absolute loaders — P.3-4
- Subroutine linkages — P.3-6
- Relocating loaders — P.3-7
- Direct linking loaders — P.3-8

Other Loader Schemes:

- Binders
- Linking loaders
- Overlays — P.3-8 to 3-13
- Dynamic Binders — P.3-11
- Design of an absolute Loaders
- Design of a Direct - Linking loaders — P.3-4 to 3-18

UNIT - IV

COMPILER

- Phases of Compilers P.6-4
- Overview of Databases and Algorithms required for all phases P.6-5, A-1 to A-11
- Role of lexical analyzer P.6-7
- Recognition of tokens P.6-7
- Study of LEX and YACC P.6-10

UNIT - V

UNIX DEVICE DRIVERS

- Definition P.5-3, 4
- Anatomy and Types P.5-3 to 5-7
- Device programming P.5-8
- Installation P.5-8
- Incorporation of driver routines P.5-8
- Basic device operation P.5-8
- Implementation with Line Printer P.5-12
- Comparative study between device drivers for UNIX and Windows P.5-16
- Case study of Intel®64 and IA-32 Processors, Basic architecture A-11

SYLLABUS

[B. Tech. IV Semester Computer Science Engineering]

UNIT - I

INTRODUCTION TO SYSTEMS PROGRAMMING

Introduction of Components of System Software:

Assemblers, Loaders, Macros, Compilers and Formal Systems. Operating System, computer language, Machine Architecture IBM 360/370, Instruction Formats, Data Formats, System Software Development, Recent Trends in Software Development, Levels of System Software, computer languages: Machine language, assembly language.

UNIT - II

ASSEMBLER

Elements of Assembly language programming, Data base for assembler design, Types of Assemblers, design of two-pass assembler and single pass assembler.

UNIT - III

MACRO AND MACRO PROCESSORS

Introduction, Macro Definition and Call, Macro Expansion, Functions of a Macro Processor, Basic Tasks of a Macro Processor, Features of macro, Design Issues of Macro Processor, design of macro processor.

UNIT - IV

LINKER AND LOADER

Introduction, Task of Loader, Relocation and Linking concepts, Compile-and-Go Loaders, General Loader Schemes, Absolute Loaders, Relocating Loaders, design of direct linking loader. Linker's v/s Loaders.

UNIT - V

COMPILER, INTERPRETERS, DEBUGGERS & DEVICE DRIVER

Compilers: Basic compilers function, Phases of compilers, Lexical Analysis- Role of Finite State Automata in Lexical Analysis, Design of Lexical analyzer, Data structures used in Syntax Analysis- Role of Context Free Grammar in Syntax analysis, Study of LEX and YACC. Benefits of Interpretation, Overview of Interpretation, The Java Language Environment, Java Virtual Machine, Types of Errors, Debugging Procedures, Comparative study between device drivers for UNIX and Windows

CONTENTS

[B. Tech. IV Semester Computer Science Engineering]

UNIT - I
INTRODUCTION TO SYSTEMS PROGRAMMING

Introduction of Component of Systems programming

- Assemblers, Loaders, Macros, Compilers and Formal Systems — P.1-1
- Operating System — P.1-5, PS-25, A-44
- Computer language — A-16
- Machine Architecture IBM 360/370 — P.1-6
- Instruction Formats — P.1-6
- Data Formats — P.1-8
- System Software Development — A-17
- Recent Trends in Software Development — A-18
- Levels of System Software — A-19

Computer Languages:

- Machine language — A-21
- Assembly language — A-21, 24

UNIT - II
ASSEMBLER

- Elements of Assembly language programming — A-21
- Data base for assembler design — P.1-15
- Types of Assemblers, Design of two-pass assembler and single pass assembler — P.1-19 to 1-22

UNIT - III
MACRO AND MACRO PROCESSORS

- Introduction — P.2-3
- Macro Definition and Call — P.2-3
- Macro Expansion — P.2-5
- Functions of a Macro Processor — P.2-4, 5, 6
- Basic Tasks of a Macro Processor — P.2-4
- Features of macro — P.2-4
- Design issues of Macro Processor — P.2-13, A-43
- Design of Macro Processor — P.2-7, 2-9, A-38

UNIT - IV
LINKER AND LOADER

- Introduction P.3-3
- Task of Loader P.3-3
- Relocation P.3-13
- Linking concepts P.3-11, 15
- Compile-and-Go Loaders P.3-4
- General Loader Schemes P.3-5
- Absolute Loaders P.3-4
- Relocating Loaders P.3-7
- Design of direct linking loader P.3-8
- Linkers v/s Loaders PS-96

UNIT - V
COMPILER, INTERPRETERS, DEBUGGERS AND DEVICE DRIVER

- Basic compilers function P.6-2, 3
- Phases of compilers P.6-4
- Lexical Analysis P.6-7
- Role of Finite State Automata in Lexical Analysis A-25
- Design of Lexical analyzer A-26
- Data structures used in Syntax Analysis PS-58
- Role of Context Free Grammar in Syntax analysis A-27
- Study of LEX and YACC P.6-10
- Benefits of Interpretation A-28
- Overview of Interpretation A-28
- The Java Language Environment A-30
- Java Virtual Machine A-32
- Types of Errors A-34
- Debugging Procedures A-35
- Comparative study between device drivers for UNIX and Windows. P.5-16

SYLLABUS

[B. Tech. III Semester Information Technology]

UNIT - I

Overview of System Software Introduction, Software, Software Hierarchy, Systems Programming, Machine Structure, Interfaces, Address Space, System Software Development, Recent Trends in Software Development, Levels of System Software. Overview of Language Processors Programming Languages and Language Processors, Language Processing Activities, Program Execution, Fundamental of Language Processing, Symbol Tables Data Structures for Language Processing: Search Data structures, Allocation Data Structures.

UNIT - II

Elements of Assembly Language Programming, Design of the Assembler, Assembler Design Criteria, Types of Assemblers, Two-Pass Assemblers, One-Pass Assemblers, Single pass Assembler for Intel x86, Algorithm of Single Pass Assembler, Multi-Pass Assemblers, Advanced Assembly Process, Variants of Assemblers Design of two pass assembler,

UNIT - III

Introduction, Macro Definition and Call, Macro Expansion, Nested Macro Calls, Advanced Macro Facilities, Design Of a Macro Pre-processor, Design of a Macro Assembler, Functions of a Macro Processor, Basic Tasks of a Macro Processor, Design Issues of Macro Processors, Features, Macro Processor Design Options, Two-Pass Macro Processors, One- Pass Macro Processors.

UNIT - IV

Introduction, Relocation of Linking Concept, Design of a Linker, Self-Relocating Programs, Linking of Overlay Structured Programs, Dynamic Linking, Loaders, Different Loading Schemes, Sequential and Direct Loaders, Compile-and-Go Loaders, General Loader Schemes, Absolute Loaders, Relocating Loaders, Practical Relocating Loaders, Linking Loaders, Relocating Linking Loaders, Linkers v/s Loaders.

UNIT - V

Compilers, Causes of Large Semantic Gap, Binding and Binding Times, Data Structure used in Compiling, Scope Rules, Memory Allocation, Compilation of Expression, Compilation of Control Structure, Code Optimization. Interpreters & Debuggers Benefits of Interpretation, Overview of Interpretation, The Java Language Environment, Java Virtual Machine, Types of Errors, Debugging Procedures, Classification of Debuggers, Dynamic/Interactive Debugger.

SOLVED UNIVERSITY QUESTION PAPERS

SUMMER - 12	PS-1 to PS-4
WINTER - 12	PS-5 to PS-8
SUMMER - 13	PS-9 to PS-12
WINTER - 13	PS-13 to PS-22
SUMMER - 14	PS-23 to PS-30
WINTER - 14	PS-31 to PS-48
SUMMER - 15	PS-49 to PS-52
WINTER - 15	PS-53 to PS-60
SUMMER - 16	PS-61 to PS-68
WINTER - 16	PS-69 to PS-72
SUMMER - 17	PS-73 to PS-78
WINTER - 17	PS-79 to PS-82
SUMMER - 18	PS-83 to PS-90
WINTER - 18	PS-91 to PS-98
SUMMER - 19	PS-99 to PS-104
WINTER - 19	PS-105 to PS-108
APPENDIX	A-1 to A-46

Note:- Due to Covid - 19 Summer - 2020 to Winter - 2021 exams were conducted online by the respective Colleges.

UNIT - I
IBM 360/370 AND ASSEMBLER

SYLLABUS

Introduction to System Programming and its components, M/c Architecture, Instruction Formats, Data Formats and Register Formats, Concept of assembler, design of single pass and two pass assembler.

UNIVERSITY PAPER SOLUTIONS
SINCE WINTER – 2009

Note to the students :
- The questions given below are from previous Nagpur University examination papers..
- Though these questions are from old university papers, they have been included in the new syllabus.
- Questions which are out of new syllabus are not included here.

WINTER - 09

Q.1. What is system programming? What are the objects of learning system programming? Explain various system programs in brief. **8M**

Ans. P.1-3, Q.1

Q.2. Explain the instruction format of IBM 360. **5M**

Ans. P.1-6, Q.7

Q.3. What are pseudo instructions? Explain USING, DROP and LTORG instruction of IBM 360. **4M**

Ans. P.1-11, Q.14

Q.4. Explain Pass 1 of Assembler in detail with the help of flowchart and database. **9M**

Ans. P.1-19, Q.27

SUMMER - 10

Q.1. "A defined constant or storage of fullwords type must be generated there, where the location counter is updated to fullword boundary". Why? **2M**

Ans. P.1-11, Q.13(b)

Q.2. Explain atleast three advantages of using base addressing in IBM 360. **4M**

Ans. P.1-10, Q.10

Q.3. For the following ALP show its symbol table, literal table, base table and generated machine code

```
TEST    START
BEGIN   BALR    15,0
        USING   BEGIN + 2, 15
        SR      4, 4
        L       3, = F '10'
LOOP    L       2, DATA (4)
        A       2, = F '49'
        ST      2, DATA (4)
        A       4, = F '4'
        BCT     3, * - 16
        BR      14
DATA    DC      F '1,2,3,4'
        END
```
8M

Ans. P.1-22, Q.29

Q.4. Differentiate between :

(1) Open subroutine and closed subroutine.

(2) Pure procedure and Impure procedure. **4M**

Ans. P.1-14, Q.21

Q.5. Discuss pass-2 assembler with algorithm and flowchart. **9M**

Ans. P.1-19, 21, Q.27, Q.28

WINTER - 10

Q.1. Explain the following with example :

(1) USING (2)BALR (3) DS

(4) DC (5) LTORG (6) DROP. **6M**

Ans. P.1-11, Q.14

Q.2. What is an operating system? State its function. Explain the different user viewpoints of an operating system. **7M**

Ans. P.1-5, Q.4

Q.3. For the following program

(1) Show symbol table at the end of pass 1.

(2) Show literal table at the end of pass 1.

(3) Show changes in base register table during pass 2.

(4) Show generated machine code from pass 2.

PROGRAM 1 :

```
        START
BEGIN   BALR    15, 0
        USING   BEGIN +2, 15
        SR      FOUR, FOUR
        L       R3, TEN
LOOP    L       R5, DATA (4)
        A       R5, thirty
        ST      R5, DATA(4)
        A       4, =F '4'
        BCT     R3, LOOP
        BR      14
        LTORG
FOUR    DC      F '4'
R3      EQU     3
R5      EQU     5
TEN     DC      F '10'
Thirty  DC      F '30'
DATA    DC      F ' 1, 2, 3, 4, 5, 6, 7,8,9,10
```

11M

Ans. P.1-24, Q.30

Q.4. Write the databases in pass 1 and pass 2 of an assembler. **3M**

Ans. P.1-17, Q.26

SUMMER - 11

No question asked in RTMNU exam.

WINTER - 11

Q.1. Define system programming. Explain the various system programs in brief. **8M**

Ans. P.1-3, Q.1

Q.2. What are the of database, required to maintain to convert assembly language program to object code? **6M**

Ans. P.1-15, Q.24

Q.3. Explain one pass assembler with algorithm and flowchart. **9M**

Ans. P.1-19, 21, Q.27, 28

Q.4. Explain when literals are given value of LC. What happens if LTORG is present in the assembly program? Explain with example. **4M**

Ans. P.1-12, Q.16

SUMMER - 12

Q.1. Discuss the formats of data structure needed for two pass assembler design in detail. **9M**

Ans. P.1-15, Q.24

Q.2. What are the advantages and disadvantages of using the base register in IBM - 360? **4M**

Ans. P.1-10, Q.10

Q.3. Differentiate between the following:

(i) Forward Reference and Backward Reference.

(ii) USING and BALR

(iii) Open subroutine and Close subroutine. **9M**

Ans. P.1-13, 14, Q.20, Q.21, Q.22.

Q.4. Explain when literals are defined as value of LC. What happens if LTORG is present in the assembly program? Explain with example. **4M**

Ans. P.1-12, Q.16

WINTER - 12

Q.1. "Operating system as user viewpoint" comment on this. **4M**

Ans. P.1-5, Q.4

Q.2. What are advantages of base addressing in IBM 360? **5M**

Ans. P.1-10, Q.10

Q.3. What are pseudo instruction ? Explain USING, DROP, LTORG, EQU. **5M**

Ans. P.1-11, Q.14

Q.4. Explain various database required in design of two pass assembler with their format. **9M**

Ans. P.1-15, Q.24

Q.5. Draw a neat sketch of IBM 360 machine architecture. **4M**

Ans. P.1-6, Q.6

SUMMER - 13

Q.1. Define system programming. Explain the various system programs in brief. **8M**

Ans. P.1-3, Q.1

Q.2. With neat sketch explain the general structure of IBM 360 machine. **6M**

Ans. P.1-6, Q.6

Q.3. Explain the design of a single pass assembler with a flowchart. **8M**

Ans. P.1-19, Q.27

Q.4. Explain the format of MOT and POT in the assembler design. **5M**

Ans. P.1-15, Q.24

Note : Also refer Paper Solutions at the end of the book.

SOLVED QUESTION BANK

[Sequence given as per syllabus]

INTRODUCTION

This chapter deals with system programming and its components, machine architecture of IBM 360 along with various data, register and instruction formats. Assembler and design of passes is also studied.

System programming is the activity of computer programming system software. Various system programs include compiler, assembler, loader, macro programs. An assembler is a program that accepts an assembly language program as input and produces its machine equivalent along with information for the loader as output.

The IBM 360 has arithmetic, logical, control or transfer and special interrupt instruction. It has various registers to store data. The data can be characters, integers, packed decimal, zoned decimal and floating point numbers.

INTRODUCTION TO SYSTEM PROGRAMMING AND ITS COMPONENTS

Q.1. Explain evolution of components of a programming system.

OR What is system programming? What are objective of learning system programming? Explain various system programs in brief.

W-09(8M)

OR Define system programming. Explain the various system programs in brief.

W-11(8M), S-13(8M)

Ans. System programming :

- System programming is the activity of computer programming system software.
- The primary distinguishing characteristics of system programming when compared to application programming aims to produce software which provides services to the user, whereas system programming aims to produce software which provides services to the computer hardware.

Objectives :

There are two main objectives of system programming :

(1) To deal with issues involved in accessing kernel services and designing of software system.

(2) To deal with issues involved in communicating between disparate user (Kernel) processes.

System Programs :

The system programs such as compiler, loader, assembler, macro programs (processor), loader operating system were developed to make computer better adapted to the need of their user.

(1) Compiler :

- Compiler are system programs that accepts human readable language and translate them into machine language.
- Compiler is a program that accepts program written in high-level language and produces the object program.
- Modern compiler must be able to provide complex facilities that are needed by the programmer to develop an software.

(2) Assembler :

- Programmers find difficulty in writing machine language programs. Hence mnemonics (symbol) are develop for each instruction.
- The mnemonics would easily get converted to machine language programs (codes)
- Such language that is understood by processor using symbols is called as assembly language.
- Programs written to translate the assembly language to machine language is called as assemblers.
- The input to an assembler program is called source program, the output is a machine code called as object program.

(3) Loader :

- Loaders are system program that prepare machine language program for execution.
- Loader is a program that places programs into memory and prepares them for execution.
- Loader places into memory the machine language versions of user's program and transfer control of it.

(4) Macro programs :

- Macro processing in operating system provides facility to programmer for defining an abbreviation of program and use the abbreviation in the program.
- The macro processor uses an abbreviation as macro definition and saves the definition.
- The macro processor replaces all the abbreviation by definition called the macro program definitions. (macro call)

Q.2. Draw a neat diagram of foundation of system programming.

Ans.

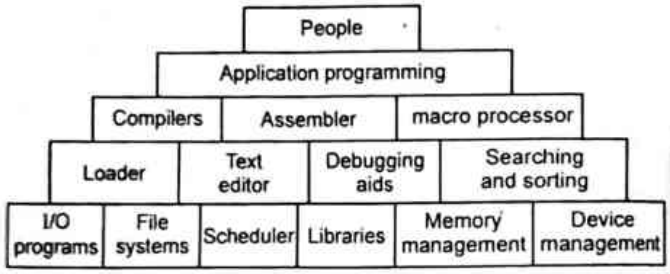


Fig. Foundation of system programming

Q.3. Explain machine structure of a computer system.

Ans.

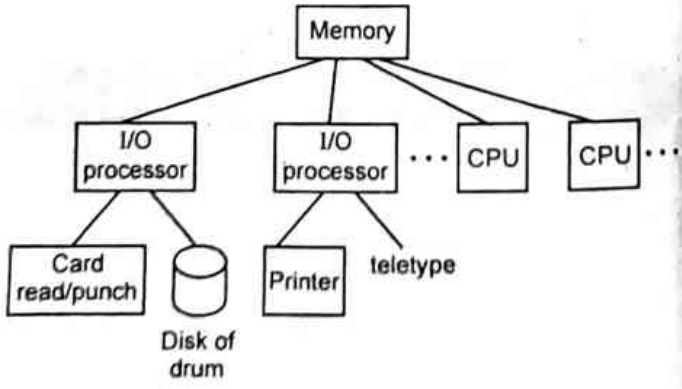


Fig. General hardware organisation of a computer system

- Memory is the device where information is stored.
- Processor are the devices that operate on this information.
- CPU is the central processing unit used to control the complete system.
- Each digit in binary is called as bits i.e. ones and zero.

- Bits are combined together to form words, character or bytes.
- Each memory location is given an unique address where each address indicates byte, word or character.
- The content of word may be an data or instruction.
- The sequence of operations are performed by processor as specified or indicated by instructions in memory.
- Following example shows same address has some specified data.

Address	Control			
10,000	0001	0000	0000	0010
10,001	0011	0000	0000	0100
10,002	0100	0000	0000	1000

- Information is the group of bits that may be interpreted as numbers, data, characters or instruction.
- Code indicates the set of rules for interpreting groups of bits. Code also indicates decimal digits (BCD) or instruction (operations).
- I/O processor are responsible for I/O operation i.e. transfer of data between memory and peripheral devices. CPU are responsible for manipulating of data stored in memory.

Q.4. What is operating system? State its functions. Explain operating system user viewpoint of an operating system. **W-10(7M)**

OR "Operating system as user view point" comment on this. **W-12(4M)**

Ans. Operating system :

- Operating system are the system programs that are responsible for performing different user specified tasks i.e. for solving problems of user to perform particular task.
- An operating system may be defined as organized collection of software routines, which acts as an interface between the user and the computer.
- It provides an environment for execution of programs.

Functions of Operating system according to user viewpoint :

The operating system perform following function.

(1) Job sequencing and traffic controller operations.

(2) Input/output processing/ programming.

(3) Protecting itself from the user, protecting the user from other users.

(4) Secondary storage management.

(5) Error handling.

(1) Job sequencing and traffic controller operations:

- The function of operating system is to perform operations in particular sequence.
- It is possible to do input and output simultaneously while executing program, hence such functions are performed by many computer system processes, all these functions are controlled and scheduled by traffic controller. This case of input and output can occur when both processes are submitted simultaneously.

(2) I/O programming :

- The job of input and output programming is efficiently done by the operating system.
- User are not interested in complex details of the program, hence processing of the data of program is very necessary, therefore these facilities are provided by operating system i.e. processing of input and output without giving any complex details to user.

(3) Protecting itself from user, protecting user from other users :

- Operating system provides a special feature of protecting user from malicious or accidental activities done to the data of user by the other users.
- Operating system ensure inviolability.
- Operating system protects user from each other or some unwanted activity that may be performed accidentally which might "crash" the system.

(4) Secondary storage management :

- One of the important task of operating system is storage management.
- The storage management is a task performed by an operating system in combination with use of disk, tapes and other secondary storage for a user programs and data.

(5) Error Handling :

- One of the important task of operating system is error handling.
- If error has occurred in the execution of program it is necessary to handle those errors because it may cause system crash or damage to the system.

Q.5. What are operating system user viewpoint, facilities?

Ans. Following facilities are typically provided by modern operating system :

(1) Assemblers
(2) Compilers such as FORTRAN, COBOL and PL/I.
(3) Subroutine libraries such as SINE, COSINE, SQUARE ROOT.
(4) Linkage editor and program loader that bind subroutines together and prepare program for execution.
(5) Utility routine such as SORT/ MERGE and TAPE COPY.
(6) Application package such as circuit analysis or simulation.
(7) Debugging facilities such as program tracing and "core dump".
(8) Data management and file processing.
(9) Management of system hardware.

MACHINE ARCHITECTURE OF IBM 360

Q.6. Draw general machine structure and explain each component in brief.

OR Draw a neat sketch of IBM 360 machine architecture. *W-12(4M)*

OR With neat sketch explain the general structure of IBM 360 machine. *S-13(6M)*

Ans.

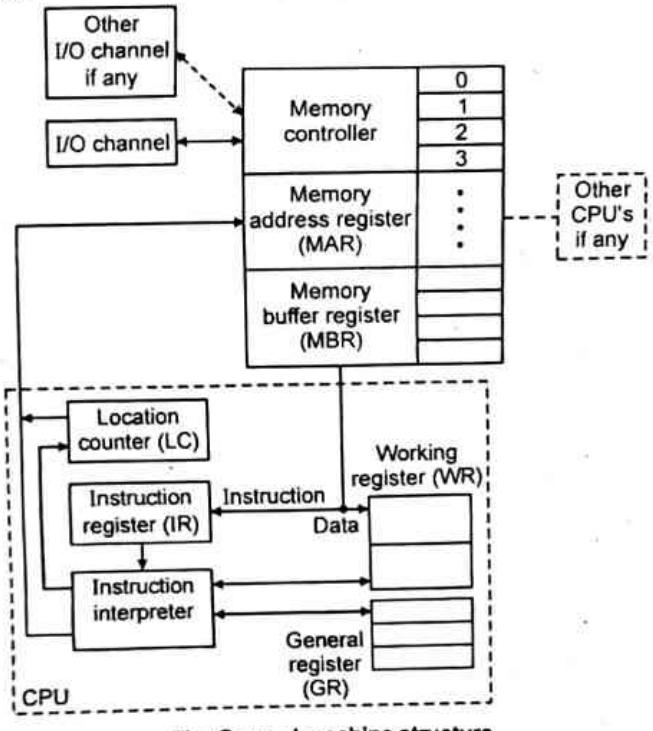


Fig. General machine structure

- In general machine structure the CPU consists of instruction interpreter, instruction register, general register, working register, location counter.

(1) **Instruction interpreter :** It consists of group of electrical circuits that translate the instruction fetch from and memory into human readable language.

(2) **Location counter or Instruction counter :** It is a hardware memory device which denotes or represent the location of current instruction being executed.

(3) **Instruction Register :** It stores copy of current instruction.

(4) **Working Register :** It generally serve as "scratch pads" for instruction interpreter.

(5) **General register :** It is used by programmer for temporary storage while executing an instruction. Primary interface between memory and CPU is memory address register (MAR), and memory buffer register (MBR).

(6) **Memory address register (MAR) :** The address of the memory location that is read from memory or stored in memory are stored in MAR.

(7) **Memory Buffer Register (MBR) :** It stores copy of the particular memory location given by MAR after a 'read' or the new content of memory location prior to a write.

(8) **Memory controller :** It transfers data between MBR and core memory location address of which is in MAR.

(9) **I/O channels :** It interprets instruction to perform the input and output operation of information from memory.

INSTRUCTION FORMAT OF IBM 360

Q.7. Explain instruction format of IBM 360. *W-09(5M)*

Ans. Instruction format :

- The IBM 360 has arithmetic, logical, control or transfer and special interrupt instruction.
- The following figure shows instruction format of IBM 360.
- The instruction format differ from each other by the type of operand.

- Register operand refers to data stored in one of register i.e. 16 registers, each register is of 32 bits.

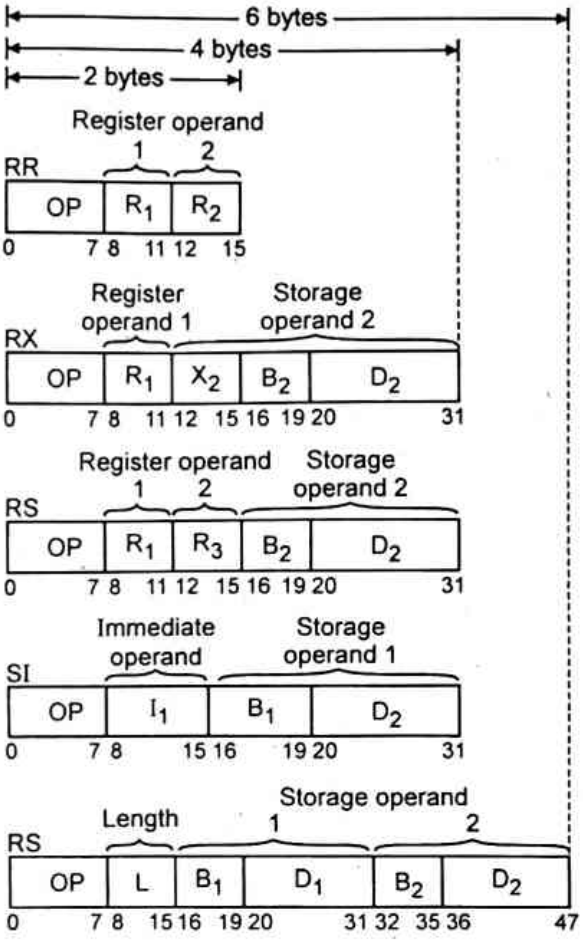


Fig. Basic 360 instruction format

Mnemonics used :

OP → Operation code

Ri → Contents of general register used as operand

Xi → Contents of general register used as index

Bi → Contents of general register used as base

Di → Displacement

Li → Immediate data

L → Operand length

- The 360/370 has five instruction formats.
- Each format is associated with a single addressing mode and has a set of operations defined for that format.
- While some operations are defined in multiple formats, most are not.
- Instructions in the IBM 360 are two, four or six bytes in length, with the opcode in byte 0. Instructions have one of the following formats :

(1) RR (register-register) (two bytes) :

Generally byte 1 specifies two 4-bit register numbers, but in some cases e.g., SVC, byte 1 is a single 8-bit immediate field.

(2) RS (register storage) (four bytes) :

Byte 1 specifies two register numbers; byte 2-3 specify a base and displacement.

(3) RX (register indexed) (four bytes) :

Byte 1 specifies either a register number or a modifier; byte 4-7 specifies the number of the general register to be used as an index; bytes 2-3 specify a base and displacement.

(4) SI (storage immediate) (four bytes) :

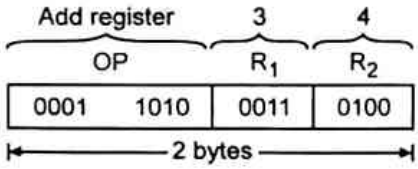
Byte 1 specifies an immediate field; bytes 2-3 specify a base and displacement.

(5) SS (storage storage) (six bytes) :

Byte 1 specifies two 4-bit length fields or one 8-bit length field; bytes 2-3 and 4-5 each specify a base and displacement. The encoding of the length fields is length-1.

- Instructions must be on a two byte boundary in memory; hence the low-order bit of the instruction address is always 0.

Example : Instruction ADD register 3, 4 causes the contents of general register 4 (32 bits) to be added to the content of register 3 (32 bits) and result is stored in register 3.



- Storage operand refers to data stored in memory.
- Length of format depend upon data types.
- Operand and data field that are longer than one byte are specified by address of the lowest address byte.
- The address of i^{th} storage operand is computed from the instruction in following ways.
  Address = C(Bi) + C (Xi) + DI (RX format)
  or C (Bi) + DI (RS , SI, SS format)
  where C (Bi) and C (Xi) denote the content of general register Bi and Xi respectively.
  Exception if Xi = 0, then C(Xi) is treated as 0; likewise for Bi = 0
- Immediate operands are a single byte of data and are stored as part of instruction.

Q.8. What types of data can be handled by the IBM 360? Can it handle character, numbers, logical data? How is this data stored?

OR Draw and explain data format of IBM 360.

Ans.

- The IBM 360 may store several different types of data as shown in figure.

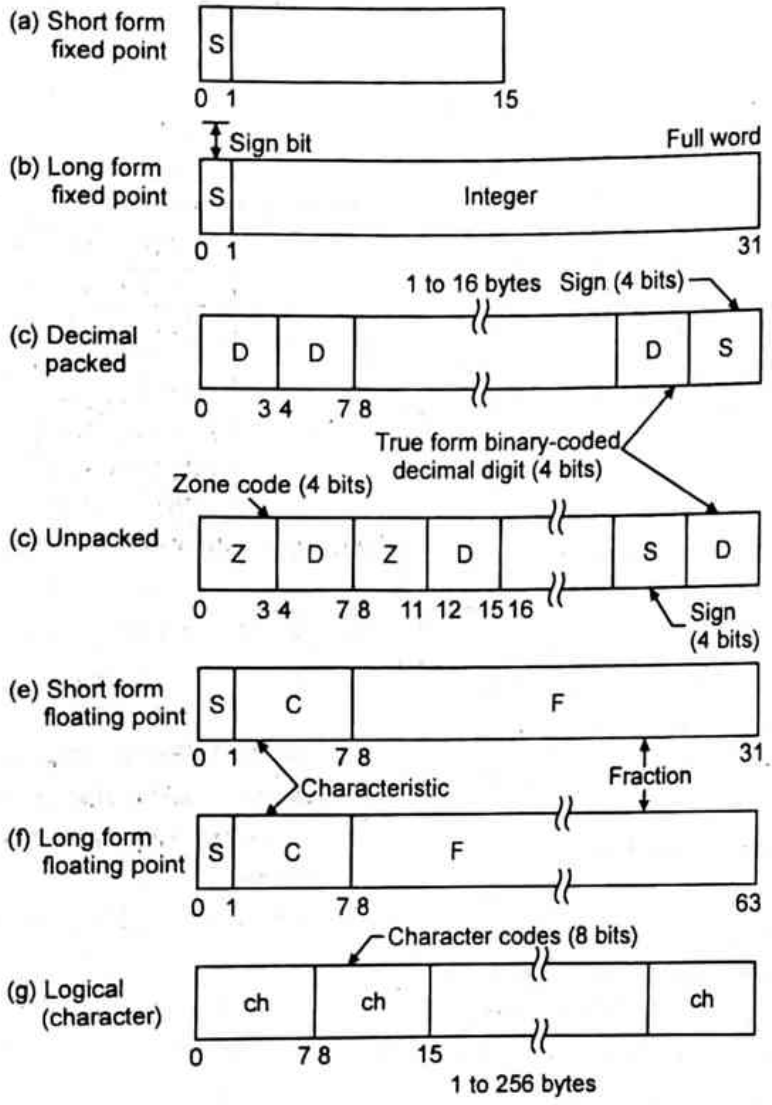


Fig. Data formats of IBM 360

- The groups of bits stored in memory are interpreted by a 360 processor in several ways. If a 360 interprets the contents of two bytes as an integer, it interprets the first bit as a sign and the remaining 15 as a binary number.
- If a 360 interprets contents of two bytes as a packed decimal it would interpret the first byte as two BCD coded digits, the first four bits of the second byte as a BCD digit, and the last four as a sign.
- All data and instructions are physically stored as sequence of binary ones and zeros. Thus, a 16-bit fixed point halfword with decimal value +300 would be stored in binary as '0000 0001 0010 1100'.
- The 360 allows the storage of numbers in decimal form. That is, numbers may be stored not as binary numbers but in a formal closely approximating the decimal representation.

The 360 allows floating-point numbers, logical

data, and character strings to be represented in memory.

- The IBM 360 architecture defines formats for characters, integers, decimal integers and hexadecimal floating point numbers.
- Character and integer instructions are mandatory, but decimal and floating point instructions are part of the decimal arithmetic and floating point arithmetic features.
- Various data formats are as follows :

(1) **Characters :** They are stored as 8-bit bytes.

(2) **Integers :** They are stored as two's complement binary halfword or fullword values.

(3) **Packed decimal :** These numbers are stored as 1-16, 8-bit bytes containing an odd number of decimal digits followed by a 4-bit sign. Sign values of hexadecimal A, C, E and F are positive and sign values of hexadecimal B and D are negative. Digit values of hexadecimal A-F and sign values of 0-9 are invalid, but the PACK and UNPACK instructions do not test for validity.

(4) **Zoned decimal :** These numbers are stored as 1-16 8-bit bytes, each containing a zone in bits 0-3 and a digit in bits 4-7. The zone of the rightmost byte is interpreted as a sign.

(5) **Floating point numbers :** These are only stored as fullword or double word values on older models. On the 360/85 and 360/195 there are also extended precision floating point numbers stored as quadwords. For all three formats, bit 0 is a sign and bits 0-7 are a characteristics precision, the low order double word has its own sign and characteristic, which are ignored on input and generated on output.

Q.9. Explain register format of IBM 360.

Ans. General purpose registers

0	R0
1	R1
2	R2
3	R3
4	R4
5	R5
6	R6
7	R7
8	R8
9	R9
10	R10
11	R11
12	R12
13	R13
14	R14
15	R15

Floating-point register

FP0 (bits 32-63)	FP0
FP0 (bits 0-31)	
FP2 (bits 32-63)	FP2
FP2 (bits 0-31)	
FP4 (bits 32-63)	FP4
FP4 (bits 0-31)	
FP6 (bits 32-63)	FP6
FP6 (bits 0-31)	

Program status word

	PSW (bits 0-31)	Program status Word
(bits 32-39)	IA (bits 40-63)	Instruction address

Fig. IBM 360 register

- Note that IBM documentation numbers the bits in reverse order to that shown above, i.e. the most significant (leftmost) bit is designated as bit number 0.

System/360 architecture provides the following registers :

- It has 16 32-bit general purpose registers.
- In addition there are 4 64-bit floating point registers.
- It has 64-bit processor status register (PSW) that contains the value of location counter, protection information and interrupt status and includes a 24-bit Instruction Address.
- It has 24bit (16 MB) byte-addressable memory space.
- It has Big-endian byte/word order.
- The general purpose registers may be used for various arithmetic and logical operations and as base registers.
- When the programmer uses them in arithmetic or logical operations, these registers act as scratch

pads to which numbers are added, subtracted, compared, and so forth.

- When used as base registers, they aid in the formation of the address.

Example :

Index register

A 1, 901 (2, 15)
offset Base register

- It is interpreted as an add instruction. A number is to be added to the contents of register 1.
- The location of the number is 901 (offset) plus the contents of register 2 (index) plus the contents of register 15(base)
- That is, if those three numbers were added together, the result would be the address of the memory location whose contents we wish to add to the contents of register 1.

DATA FORMAT AND REGISTER FORMAT

Q.10.What are advantages and disadvantages of using base addressing in IBM 360?

S-12(4M), W-12(5M)

OR Explain atleast three advantages of using base addressing in IBM 360. *S-10(4M)*

Ans. Advantages :

(1) Base register aids in the process of relocation of a program.

(2) Relocation is a process of moving an entire program from one series of locations to another by changing the contents of base register.

(3) A major motivation of employing base register is to promote efficient addressing of core.

Disadvantages :

(1) The disadvantage of base register is overhead associated with the formation of address since offset is 12 bits long can specify a number from 0 to 4095.

(2) Thus without using an index register and without changing contents of base register, the core location cannot be address further than 4095 locations away from the core location to which the base register is pointing.

Q.11. Explain the use of base register in IBM 360 machine. Also explain the strategy of choosing the base register of more than one base register are available in the assembly program?

Ans. Use of base register :

(1) The general purpose registers perform all the addressing involved in referencing main storage. When used for addressing, the register is called a base register.

(2) In the case of system/360 the base address is contained in one of 15 general registers called as base register.

Defining and choosing multiple base registers that are defined in an assembler program:

```
BALR 12, R0
USING*, R12, R10, R11
BASE LM R10, R11, BASES
B MAIN
BASES DC A (BASE + 4096)
CS A(BASE + 8192)
MAIN MVC IN OUT
```

- In above example three base registers are defined i.e. R12, R10 and R11.
- R12 is loaded with the address of the instruction following USING. R10 is loaded with the address of R12 + 4096 and R11 is loaded with address of R12 + 8192.

CONCEPT OF ASSEMBLER

Q.12.Explain assembler in detail.

Ans. Assembler:

- With machine language we must communicate directly with a computer in terms of bits, registers, and very primitive machine operations.
- Since a machine language program is nothing more than a sequence of 0's and 1's, programming a complex algorithm in such a language is terribly tedious and filled with opportunities for mistakes.
- Programmer finds it difficult to write or read program in machine language.
- In their quest for a more convenient language they began to use a mnemonic (symbols) for each machine instruction, which they would use for the translation of assembly language into machine language.

The input to an assembler program is called that subsequently translates into machine language. Such a mnemonic machine language is called an assembly language.

Programs known as assemblers were written to automate source programs and output is a machine language translation (object program).

An assembler is a program that accepts an assembly language program as input and produces its machine equivalent along with information the loader as output.

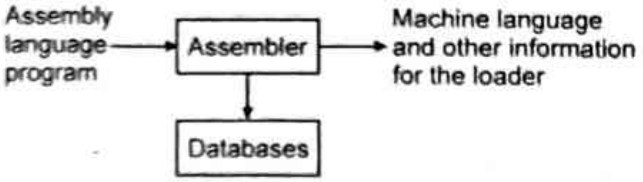


Fig. Function of an assembler

.13.(a) State the various translation phases of an assembler.

ns. The six translation phases while designing an assembler are :

) Specify the problem.
) Specify data structures.
) Define format of data structures.
) Specify algorithm.
) Look for modularity.
) Repeat 1 through 5 on modules.

.13.(b) "A defined constant or storage of fullwords type must be generated there, where the location counter is updated to fullword boundary". Why? ***S-10(2M)***

ns. The define constant (DC) or define storage (DS) instructions are pseudo-ops that instruct the assembler to place a 10, a 4 and 49 in 3 consecutive full words ("F") in memory and leave 10 more for data. A number before the F would cause multiple allocation, e.g. DS 100F causes the assembler to set aside a 100 full word area.

Q.14. What are pseudo instruction? Explain USING, DROP, EQU, LTORG instruction of IBM 360. ***W-09(4M),W-12(5M)***

OR Explain meaning of following instruction of IBM-360 : (1) USING (2) BALR (3) DS (4) DC (5) LTORG (6) DROP. ***W-10(6M)***

Ans. Pseudo instruction :

An assembly language instruction that specifies an operation of the assembler is known as pseudo instruction.

(i) USING :

It is a pseudo-op that indicates to the assemble which general register is used as a base registers and what its content is. The assembler is thus able to produce the machine code with the current base register and offset.

(ii) BALR :

It is the instruction to the computer to load the register with the next addresses and branch to the address in the second field. If the second field is zero then it will goto the next instruction. BALR is different than USING. USING only specify the base register and its contents whereas BALR actually load the base register.

(iii) START :

It is a pseudo-op that tells the assembler the start or beginning of the program and allow user to give the name to their program.

(iv) END :

It is a pseudo-op that tells the assembler that the last part of the program has been received. It indicates the end of the program.

(v) BR :

BR 14 is a branch to the location where address is in general register 14. By conversion, calling programs leave their return address in register 14.

(v) LTORG :

It gives the value of location to the literal first. LTORG instruction specifies that all the encountered literals should be given the value of next double word location of the location counter i.e. first * as double word of 8*.

(vii) DROP :

It is the pseudo-op that tells the assembler if the operand is not available. It marks the base table entry according to the drop opcode. If USING then available, If DROP then unavailable.

(viii) DC : Define constants i.e.

Data constants are inserted into an object program.

(ix) DS (Define Storage)

A block of memory can be reserved for data using define storage.

Note : Offset = Value of symbol – Content of base register]

(x) **EQU :**

It allows the programmer to define variables. Any sort of valid arithmetic expression may appears as operand of EQU statement.

Example : If one wish to use a different base register, he would need only to change the EQU statement.

Q.15. Explain with examples how LTORG statement is processed by an assembler.

Ans. Processing of LTORG statement :

- The LTORG directive instruct the assembler to assemble the current literal pool immediately.
- The assembler assemblers the current literal pool at the end of every code section. The end of a code section is determined by the AREA directive at the beginning of the following section, or the end of the assembly.
- These default literal pools can sometimes be out of range of some LDR, VLDR, and WLDR pseudo-instructions. Use LTORG to ensure that a literal pool is assembled within range.
- Large programs can require several literal pools. Place LTORG directives after unconditional branches or subroutine return instructions so that the processor does not attempt to execute constants as instructions.

Syntax :

LTORG

Example :

Start **funcl**	AREA BL	EXAMPLE, CODE, READONLY	
	BL	Funcl	; function body
	; code LDR	R1m – 0 × 55555555	; ⇒ LDR R1, [pc, #offset to ' Literal Pool 1']
	'code		;end function
	MOV	Pc, lr	;Literal Pool 1 contains
	LTORG		;Literal &5555555
data	%	4200	;Clears 4200 bytes of memory
			;starting at current location
	end		;Default literal pool is empty

Q.16. Explain when literals are given value of location counter. What happens if LTORG is present in assembly program? Explain with example. *W-11(4M), S-12(4M)*

Ans.

- When LTORG instruction is encountered by the assembler in assembly language program then literals in program above the LTORG are given value of LC.
- The assembler instruction that directs the location of the literal pool is LTORG. It does not have any operands. If we do not use any LTORG instructions in our program, the literal pool is placed at the end of the first control section (CSECT) of our program.
- When the assembler encounters a LTORG instruction within a program, it creates a literal pool at that point, containing all the literals specified since the last LTORG instruction, or since the beginning of the program, if this is the first LTORG.
- The classic form of the statement is as follows, where the "L" of "LTORG" is to be found in column 10 of the listing.

  LTORG*

- Generally, this statement should be placed near the end of the listing, as in the next example taken from an actual program.

	240 *	LITERAL POOL
	241	**********
000308	242	LTORG*
000308 0000001	243	=F'1'
000000	244	END LAB1

Here, line 243 shows a literal that is inserted by the assembler.

Example :

30		JEQ	ENDGIL	EXIT IF EOF FOUND
35		*JSUB	WRREC	WRITE OUTPUT RECORD
40		J	CLOOT	LOOP
45	ENDFIL	LDA	=C'EOF'	INSERT END OF FILE MARKER
50		STA	BUFFER	USE= to represent a literal
55		LDA	#3	SET LENGTH = 3
60		STA	LENGTH	
65		*JSUB	WRREC	WRITE EOF
70		J	@RETADR	RETURN TO CALELR
93		LTORG		
95	RETADR	RESH	1	
100	LENGTH	RESH	1	LENGTH OF RECORD

- If we do not use a LTORG on line 93, the literal = C'EOF' will be placed at the end of the program.
- This operand will then begin at address 1073 and be too far away from the instruction that uses it. PC relative addressing mode cannot be used.

- LTORG thus is used when we want to keep the literal operands close to the instruction that uses it.

Q.17. Define the execution time, compile time, and load time.

Ans.

(i) **Execution time :** The period of execution of a user's program is called execution time.

(ii) **Compile Time :** The period of translating a user's source program is called as assembly or compile time.

(iii) **Load time :** Load time refers to the period of loading and preparing an object program for execution.

Q.18. What is Absolute relocation and Relative relocation?

Ans. Absolute relocation :

Location Counter does not change if the program is moved in core. Absolute relocation is denoted by 'A'.

Relative relocation :

Location counter change if program is moved in core. Relative relocation is denoted by 'R'.

Q.19. Differentiate between pseudo-op and machine-op.

Ans.

Sr. No.	Pseudo-op	Machine-op
(1)	Pseudo-op specify the operation of the assembler.	Machine-op specify the machine instruction to the assembler.
(2)	While converting the assembly language program into m/c language the value of location counter will not be updated for pseudo-op.	While converting the assembly language program into m/c language the value of location counter will be updated for machine-op.
(3)	**Example :** USING START END LTORG DC DS EQU	**Example :** A AC AR L ST BR BALR

Q.20. State the difference between BALR and USING. *S-12(3M)*

Ans.

Sr. No.	USING	BALR
(1)	USING is a pseudo-op that indicates to the assembler which general registers to be used as a base register and what its contents will be	BALR is an instruction to load a register with the next address and branch to the address in the second field.
(2)	Format of USING is USING BEGIN + 2, 10; this pseudo-opcode indicates to assembler register 10 is base register and its content is address of the next instruction.	Format of BALR instruction is BALR 10, 0; this sets register 15 to the address of the next instruction.
(3)	The programmer must inform the assembler which registers to use and how to use them with USING.	In the BALR instruction the second operand is zero. Execution proceed with the next instruction.
(4)	The assembler is able to produce the machine code with the correct base register and offset.	In a BALR instruction if the register does not contain the address that the USING says it should contain, a program error may result.

Q.21. Differentiate between :

(a) Open subroutine and close surbroutine **S-10(2M), S-12(3M)**

(b) pure and impure procedure. **S-10(2M)**

(c) Assembler and compiler.

Ans. (a)

Sr. No.	Open Subroutine	Close Subroutine
(1)	An open subroutine is the one whose code is inserted into main program.	Close subroutine is stored outside the main routine and controls transfer to the subroutine.
(2)	An open subroutine call is used to create a channel between the caller and a graphic input device driver.	The close subroutine is used to remove a channel created by the open subroutine call.
(3)	An open subroutine is a subroutine that adds the instruction to the already existing block of code. That is, it replaces the one line call with the entire set of instructions.	When a closed subroutine is called, the program branches to the file/ section of code, executes, and then returns to the line after the calling line.
(4)	Open subroutine is used in assembly language like IBM360.	Close subroutines are used in high level language like C, C++, etc.

(b)

Sr. No.	Pure Procedure	Impure Procedure
(1)	A pure procedure does not modify itself.	Procedures that modify themselves are called impure procedures.
(2)	To ensure that the instructions are the same each time a program is used, pure procedure (re-entrant code) are employed.	Each processor executing an impure procedure modifies its contents.
(3)	Using pure procedure an intelligent system learns itself and updates knowledge.	Another processor attempting to execute the same procedure may encounter different instructions or data. Thus, impure procedure are not readily reusable.

4] Pure Procedures do not have any side effects

4] Impure Procedures have side effects

(c)

Sr. No.	Asssembler	Compiler
(1)	An assembler can be considered a special type of compiler which translates only assembly language to machine code.	Compiler is a computer program that reads a program written in one language and translates it into another language.
(2)	Assembler is a program, which converts a assembly language program into machine language program.	A compiler is a program and output of assembler is machine language program.
(3)	Input to assembler is assembly language program and output of assembler is machine language program.	Input to compiler is high level language program and output of compiler is object program.
(4)	Assemblers produce an object code which might have to be linked using linker programs in order to run on a machine.	Compilers usually produce the machine executable code directly from a high level language.

Q.22. Differentiate between forward reference and backward reference. **S-12(3M)**

Ans.

Sr. No.	Forward reference	Backward reference
(1)	Referencing of the entity or variable precedes its definition is called as forward reference.	Referencing of the entity or variable succeeds its definition is called as backward reference.
(2)	Forward referencing is used in assembly level language like IBM 360.	Backward referencing is used in high level language like C, C++.
(3)	**Example :** A1, DATA --------- --------- DATA DC + 'A'	**Example :** int p, n, r Int r = (p*n*r)

Q.23. In the EQU statement, the operand expression can contain a reference to a program symbol. eg.

X EQU Y + 25

(i) Assume Y to be a backward reference, explain how the EQU statement can be processed in 2 pass assembly scheme?

(ii) If Y is a forward reference, how will the EQU statement be processed?

Ans.

- Consider the statement
  X EQU Y + 25
- The EQU pseudo-op allows the programmer to define variables.
- The variables associated with EQU statement are entered into the symbol table along with their values.
- If Y is a backward reference, the expression with EQU will be evaluated. This expression is the operand of EQU.
- Its value will be assigned to the symbol X. X is defined in pass 1, the value of X will not be available in pass 1.
- So the symbol is defined in pass 1 and the second pass will generate the instructions and the addresses.

Q.24. Explain in detail, with suitable example, the formats and contents of databases used in assembler design.

OR Explain various database required in design of two pass assembler with their format. ***W-12(9M)***

OR What are the contents of database required to maintain to convert assembly language into object code? ***W-11(6M)***

OR Explain the format for MOT and POT of the assembler. ***S-13(5M)***

OR Discuss the format of data structure needed for two pass assembler design in detail. ***S-12(9M)***

Ans. Assembler design contains the four main databases :

(1) Machine op table
(2) Pseudo-op table.
(3) The symbol table
(4) Base table.

MOT :

- A machine operation table contains name, length, binary code and format whereas pass 1 requires only name and length.
- We could use two separate tables with different formats and contents or use the same table for both the passes.
- Once we decide what information belongs in each database, it is necessary to specify the format of each entry. The MOT and POT are examples of fixed tables.
- The contents of these tables are not filled in or altered during the assembly process.
- Figure shows a possible contents and format of the machine op table.

← 6 - bytes per entry →

Mnemonic op-code (4-bytes) (characters)	Binary op-code (1- byte) (hexadecimal)	Instruction length (2-bits) (binary)	Instruction format (3-bits) (binary)	Not used in this design (3-bits)
"Abbb"	5A	10	001	
" AHbb"	4A	10	001	
'ALbb"	5E	10	001	
"ALRb"	1E	01	000	
"Arbb"	1A	01	000	
...	...	...	....	
"MVCb"	D2	11	100	
...	...	...	...	

Fig. Machine –op table for pass 1 and pass 2

b → represents the character "blank"

Codes	instruction length	Instruction format
01 =	1 half words = 2 bytes	000 = RR
10 =	2 half-words = 4 bytes	001 = RX
11 =	3 half words = 6 bytes	010 = RS
		011 = SI
		100 = SS

- The opcode is the key and its value is the binary opcode equivalent, which is stored for use in the generating machine code.
- The instruction length is stored for use in updating the location counter.
- The instruction format is useful forming the machine language equivalent.

POT :

- Figure depicts a possible pseudo-op table.
- Each pseudo-op is listed with an associated pointer to the assembler routine for processing the pseudo-op.

ST and LT :

- The symbol table and literal table include for each entry not only the name and assembly time value fields but also a length field and a relative location indicator.
- The length field indicates the length (in bytes) of the instruction or datum to which the symbol is attached.
- The relative location indicator tells the assembler whether the value of the symbol is absolute (does not change if the program is moved in core), or relative to the base of the program.
- The assembler can use a simple heuristic to decide which class a symbol falls.
- If the symbol is defined by equivalence with a constant or an absolute symbol, then the symbol is absolute otherwise it is a relative symbol. The relative-location field in the symbol table will have a "R" in it if the symbol is relative, or an "A" the symbol is absolute.

Symbol (8- bytes) (characters)	Value (4- bytes) (hexadecimal)	Length (1- byte) (hexadecimal)	Relocation (1-byte) (binary)
"JOHNBbbb"	0000	01	"R"
'FOURbbbb"	000C	04	"R"
"FIVEbbbb"	0010	04	"R"
"TEMPbbbb"	0014	04	"R"

Fig. Symbol table for pass 1 and pass 2

- In the actual assembler a substantially more complex algorithm is generally used.

Base table :

- Base table shows a possible base table that is used by the assembler to generate the proper base register reference in machine instructions and to compute the correct offsets.
- Basically, the assembler must generate an address for most symbolic references.
- The symbol table contains the address of the symbol relative to the beginning of the program.
- When generating an address, the assembler may use the base register table to choose a base register that will contain a value closest to the symbol references. The address is then formulated.
- Base register number = the base register containing a value closest to the symbolic references offset +(value of symbol in symbol table) – (contents of base register)

4 – bytes per entry

Availability Indicator (1-byte) (character)	Designated relative - address contents of base register (3- bytes = 24 bit address) (hexadecimal)
"N"	–
"N"	–
:	
"N"	–
"Y"	00 00 00

Fig. Base table

Code → Availability
Y → register specified in USING pseudo-op
N → register never specified in USING pseudo-op
or subsequently made unavailable by the DROP pseudo-op

Pseudo-op table :

Pseudo-op table contain pseudo-op and address of routine to process the pseudo-op. Figure shows a possible pseudo-op table. Each pseudo-op is listed, in following figure, with an associated pointer to the assembler routine for processing the pseudo-op.

8- bytes per entry

Pseudo - op (5 - bytes) (character)	Address o routine to process pseudo-op (3 - bytes = 24 bit address)
"DROPb"	P1 DROP
"ENDbb"	P1 END
"EQUbb"	P1 EQU
"START"	P1 START
" USING"	P2 USING

These are presumably labels of routines in pass 1 ; the table will actually contain the physical addresses.

Q.25. What is the function of an assembler? What are the purposes of pass-1 and pass-2 of an assembler?

Ans. The function of assembler are as follows :

(1) **Generate instructions:**

(a) Evaluate the mnemonic in the operation field to produce its machine code.

(b) Evaluate the sub-fields and find the value of each symbol, process literals and assign addresses.

(2) **Process pseudo ops :**

Pass 1:

Purpose: Define symbols and literals.

(1) Determine length of machine instructions (MOTGET).

(2) Keep track a location counter (LC).

(3) Remember values of symbols until pass 2 (STSTO).

(4) Process some pseudo ops, e.g. EQU, DS (POTGET 1).

(5) Remember literals (LITSTC).

Pass 2: Purpose – Generate object program.

(1) Look up value of symbols (STGET).

(2) Generate instructions (MOTGET2).

(3) Generate data (for DS, DC, and literals).

(4) Process pseudo ops (POTGET2).

Q.26. State the various steps for assembler design to establish the databases in Pass-1 and Pass-2. Draw the flow chart.

OR Write the databases in pass 1 and pass 2 of an assembler. *W-10 (3M)*

Ans. The steps for assembler design to establish the databases for.

Pass 1 databases:

(1) Input source program.

(2) A Location Counter (LC), used to keep track of each instruction's location.

(3) A table, the Machine-Operation Table (MOT), that indicates the symbolic mnemonic for each instruction and it's length (two, four, or six bytes).

(4) A table, the Pseudo-Operation Table (POT), that indicates the symbolic mnemonic and action to be taken or each pseudo-op in pass 1.

(5) A table, the Symbol Table (ST), that is used to store each label and it's corresponding value.

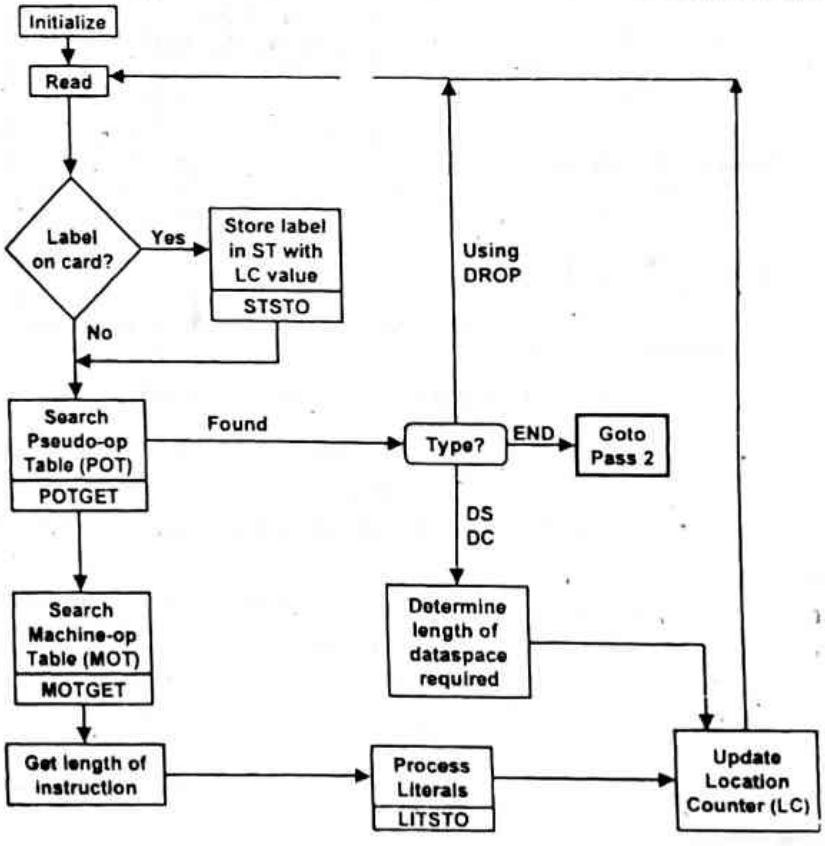


Fig. Pass 1 overview - Define symbols

(6) The Literal Table (LT), is used to store each literal encountered and it's corresponding assigned location.

(7) A copy of the input to be used later by Pass 2.

Pass 2 databases :

(1) Copy of source program input to pass 1.

(2) Location Counter (LC).

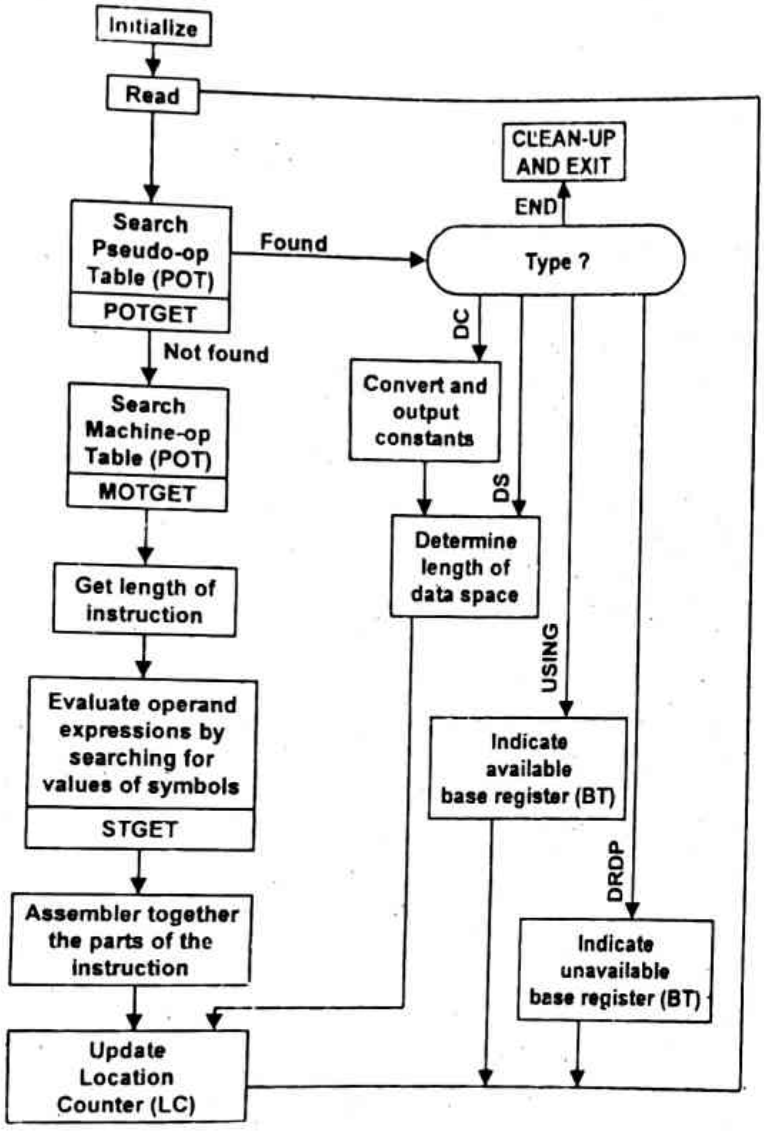


Fig. Pass 2 overview - Generate object program

(3) A table, the Machine Operation Table (MOT), that indicates for each instruction : (a) symbolic mnemonic; (b) length; (c) binary machine operation and (d) format (e.g. RS, RX, SI).

(4) A table, the Pseudo-Operation Table (POT), that indicates for each pseudo-op the symbolic mnemonic and the action to be taken in pass2.

(5) The Symbol Table (ST), prepared by pass 1, containing each label and it's corresponding value.

(6) A table, the Base Table (BT), that indicates which register is currently specified as base register by USING pseudo-ops and what are the specified contents of these registers.

(7) A work-space, INST, that is used to hold each instruction as its various parts are being assembled together.

(8) A workspace, PRINT LINE, used to produce a printed listing.

(9) A workspace, PUNCH CARD, used prior to actual outputting for converting the assembled instructions into the format needed by the loader.

(10) An output deck of assembled instruction in the format needed by the loader.

Q.27. Explain pass 1 and pass 2 of assembler in detail with the help of flowchart and database.

OR Explain pass 1 of assembler in detail with the help of flow chart and database.

S-13(8M), W-09(9M), W-11(5M)

OR Explain pass-2 of assembler with flowchart.

S-10(5M)

Ans. Pass 1: Define symbol.

- The purpose of the first pass is to assign a location to each instruction and data defining pseudo instruction, and thus to define values for symbols appearing in label fields of the source program.
- Initially, the location counter (LC) is set in the first field of the source program (relative address 0) Then a source statement is read.
- The operation code field is examined to determine if it is a pseudo-op, if it is not the table of machine opcode field. (MOT) is searched to find a match for the source statement's op-code field.
- The matched MOT entry specifies the length (2, 4 or 6 bytes) of the instruction.
- The operand field is scanned for the presence of a literal. If a new literal is found, it is entered into literal table (LT) for later processing.
- The label field of the source statement is then examined for the presence of a symbol. If there is a label the symbol is saved in the symbol table (ST) along with the current value of the location counter.
- Finally, the current value of location counter is incremented by the length of the instruction and a copy of source card is saved for use by pass 2.
- The above sequence is then repeated for the next instruction.
- The simplest procedure occurs for USING and DROP. Pass 1 is only concerned with pseudo-ops that define symbols (labels) or affects the location counter, USING and DROP do neither. The assembler need only to save the USING and DROP cards for pass 2.
- In case of EQU pseudo-op during pass 1 we are concerned only with defining the symbol in the label field. This requires evaluating the expression in the operand field.
- The DS and DC pseudo-ops can affect both the location counter and the definition of symbols in pass 1.
- The operand field must be examined to determine the number of bytes of storage required. Due to requirements for certain alignment condition, it may be necessary to adjust the location counter before defining the symbol.
- When the END pseudo-op is encountered pass 1 is terminated.

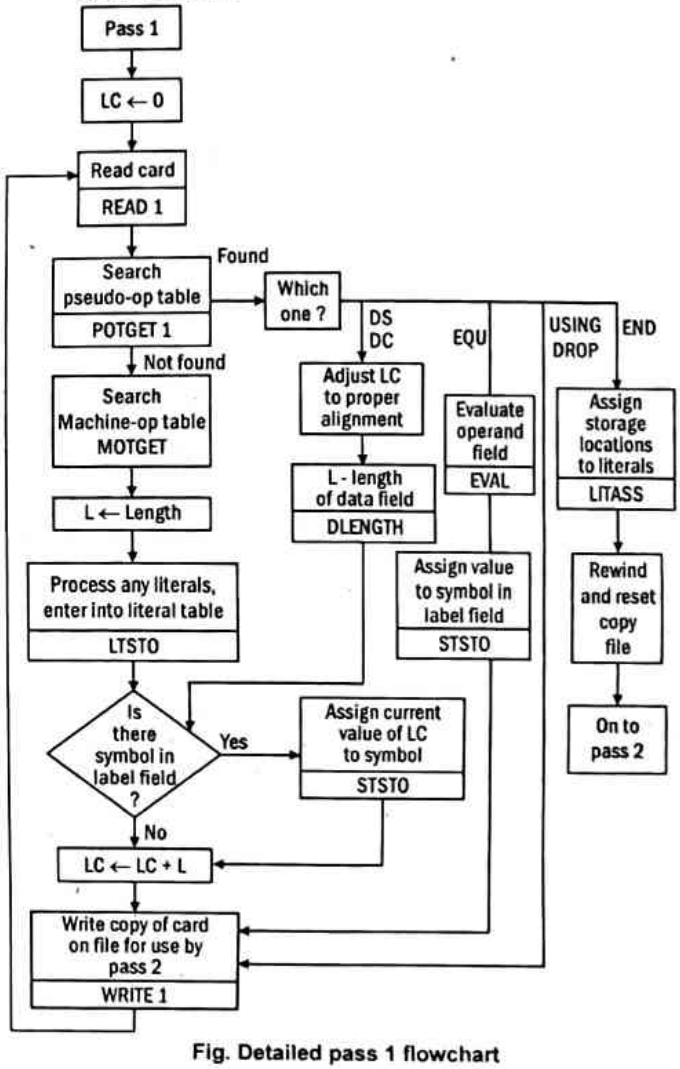


Fig. Detailed pass 1 flowchart

Pass 2 : Generate code

- After all the symbols have been defined by pass 1, it is possible to finish the assembly by processing each card and determining values for its operation code and its operand field.
- In addition, pass 2 must structure the generated code into the appropriate format for later processing by the loader, and print an assembly listing containing the original source and the hexadecimal equivalent of the bytes generated.
- The location counter is initialized as in pass 1 and the processing continues as follows.

- A card is read from the source file left by pass 1. As in pass 1, the operation code field is examined to determine if it is a pseudo-op, if it is not, the table of machine op-codes (MOT) is searched to find a match for the card's opcode field.
- The matching MOT entry specifies the length, binary op-code, and the format type of instruction The operand field of the different instruction format type require somewhat different processing.
- For RR format instruction, each of the two register specification field is evaluated. This evaluation may be very simple, as in

```
      AR      2, 3
Or    more    complex,  as in :
      MR      EVEN      EVEN+1
```

- The 4 bit base register specification and 12-bit displacement field are then assembled into third and fourth bytes of the instruction. Only RR and RX instruction types are explicitly shown in the flow-chart. The other instruction format are handled similarly.

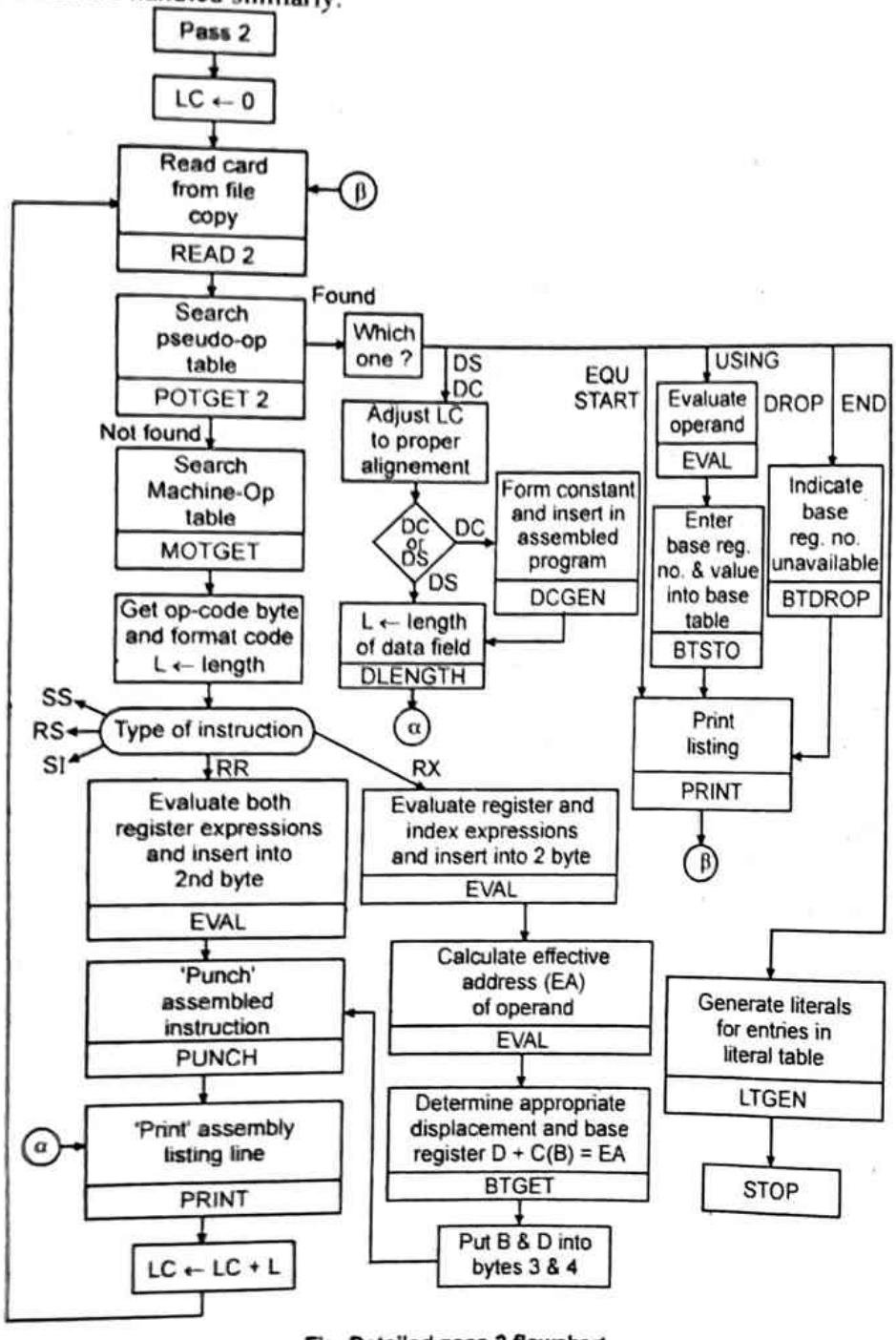


Fig. Detailed pass 2 flowchart

- After the instruction has been assembled, it is put into he necessary format for later processing by the loader.
- As in pass 1, each of the pseudo-ops calls for special processing.
- The USING and DROP pseudo-ops which were largely ignored in pass 1, require additional processing in pass 2. The operand field of the pseudo-ops are evaluated the corresponding base table entry is either marked as available, if USING or unavailable, if DROP.
- The DS and DC pseudo-ops are processed essentially as in pass 1.
- In pass 2, however, actual code must be generated for the DC pseudo-op. Depending upon the datatypes specified this involves various conversion. The END pseudo op indicates the end of the source program and terminates the assembly.

Q.28. Explain one pass assembler with algorithm.

W-11(9M)

OR Discuss pass 2 assembler with algorithm.

S-10(4M)

Ans. Pass 1 algorithm :

The algorithm for the first pass of the assembler is as given below –

Algorithm Assembler First Pass :

Step (i) loc_cntr : = 0; (default value)
pooltab_pr:=1 ; POOLTAB [1] : = 1;
littab_ptr: = 1:

Step (ii) : While next statement is not an END statement

(a) If label is present then
this_label := symbol in the label field:
Enter (this_table, loc_cntr) in SYMTAB

(b) If an LTORG statement then

(1) Process literals LITTAB [POOLTAB[pooltab_ptr]]... LITTAB [littab_ptr-1] to allocate memory and put the address in the address field. Update loc_entr accordingly.

(2) pooltab_ptr = pooltab_ptr] := littab_ptr;

(3) POOLTAB[pooltab_ptr]: = littab_ptr;

(c) If a START or ORIGIN statement then
loc_cntr : = value specifid in the operand field;

(d) If an EQU statement then

(1) this_addr : = value of <address spec>;

(2) correct the symtab entry for this_label to (this_label, this_addr)

(e) If a declaration statement then

(1) code : = code of the declaration statement;

(2) size := size of memory area required by DC/DS.

(3) loc_cntr : = loc_cntr + size;

(4) Generate IC '(DL, code)....'.

(f) If an imperative statement then

(1) code : = machine opcode from OPTAB;

(2) loc_cntr : loc_cntr + instruction length form OPTAB;

(3) If operand is a literal then
this_literal; = literal in operand field;
LITTAB [littab_ptr]; = this_literal;
littab_ptr : = littab_ptr + 1;
else (i.e., operand is a symbol)
this_entry : = SYMTAB entry number of operand;
Generate IC '(IS, cod) (s, this_entry)';

(iii) (Processing of END statement)
(a) Perform step (ii) b.
(b) Generate IC '(AD, 02)';
(c) Go to pass II.

Pass 2 algorithm :

The algorithm for second pass of the assembler is as given below. It has been assumed that the target code is to be assembled in the area. named code_area.

Algorithm Assembler Second Pass :

Step (i) code_are_address :=address of code_area;
pooltab_ptr := 1;
loc_cntr := 0;

Step (ii) While next statement is not an END statement
(a) Clear machine_code_buffer;
(b) If an LTORG statement

(1) Process literals in LITTAB [POOLTAB [pooltab_ptr]] .... LITTAB [POOLTAB [pooltab_ptr + 1]] – 1 similar to processing of constant in a DC statenebt, i.e. assembler the literals in machine_code_buffer.

(2) size : = size of memory are required for literals;

(3) pooltab_ptr : = pooltab+ptr + 1;

(c) If a START or ORIGIN statement then
(1) loc_cntr : = value specified in operand field;

(2) size : = 0;

(d) If a declaration statement

(1) If a DC statement then

Assemble the constant in machine code_buffer.

(e) If an imperative statement

(1) Get operand address from SYMTAB or LITTAB

(2) Assemble instruction in machine_code_buffer.

(3) size : = size of instruction;

(f) If size # 0 then

(1) Move contens of machine_code_buffer to the address code_area_address + loc_cntr;

(2) loc_cntr : = loc_cntr + size ;

Step (iii) Processing of END statement)

(a) perform steps 29b) and 2(f)

(b) write code_area into output file.

Q.29. For the following ALP show its symbol table, literal table, base table and generated machine code

```
TEST    START
        BEGIN   BALR    15,0
        USING   BEGIN + 2, 15
        SR      4, 4
        L       3, = F '10'
LOOP    L       2, DATA (4)
        A       2, = F '49'
        ST      2, DATA (4)
        A       4, = F '4'
        BCT     3, * – 16
        BR      14
DATA    DC      F '1,2,3,4'
        END
```

S-10(8M)

Ans.

```
                                    Location counter
TEST    START                       : 0
        BEGIN   BALR    15,0        : 0
        USING   BEGIN + 2, 15       : 0
        SR      4, 4                : 2
        L       3, = F '10'         : 4
LOOP    L       2, DATA (4)         : 8
        A       2, = F '49'         : 12
        ST      2, DATA (4)         : 16
        A       4, = F '4'          : 20
        BCT     3, * – 16           : 24
        BR      14                  : 28
DATA    DC      F '1,2,3,4'         : 30
                                    (utility)
        END                         34 (required)
```

Initially set LC as in column 3 of above table.

Step 1:

Set the location counter for each statement (instruction) except pseudo set 2 byte for register instruction and 4 bytes for normal instruction.

Step 2 :

Construct the database table as following.

Step 3 :

Machine code : Normal Instruction R_1, offset (X, Base) offset = value – content of base ; RR instruction $R_1 R_2$.

Symbol table :

Symbol	Value	Relocation
TEST	0	R
BEGIN	0	R
LOOP	8	R
DATA	30	R

Literal table :

Literal	Value	Relocation
F'10'	4	R
F'49'	12	R
F'4'	20	R

Base table :

Reg. No.	Value
15	2

Machine op table :

Instruc-tion	Value	comment
BALR	15, 0	R_1 R_2
SR	4, 4	R_1 R_2
L	3, 2 (0, 15)	R_1, (4 -2) [0, 15]
L	2, 28 (4, 15)	R_1 (30 - 2) [4, 15]
A	2, 10 (0, 15)	.
ST	2, 28 (4, 15)	.
A	4, 18 (0, 15)	.
BCT	3, 10 (0, 15)	.
BCR	14, 15	.
DC	F, '1, 2, 3, 4,5'	

Q.30. For the following program

(1) Show symbol table at the end of pass 1.

(2) Show literal table at the end of pass 1.

(3) Show changes in base register table during pass 2.

(4) Show generated machine code from pass 2.

```
PROG. 1  START
BEGIN    BALR     15, 0
         USING    BEGIN +2, 15
         SR       FOUR, FOUR
         L        R3, TEN
LOOP     L        R5, DATA (4)
         A        R5, thirty
         ST       R5, DATA(4)
         A        4, =F '4'
         BCT      R3, LOOP
         BR       14
         LTORG
FOUR     DC       F '4'
R3       EQU      3
R5       EQU      5
TEN      DC       F '10'
Thirty   DC       F '30'
DATA     DC       F' 1, 2, 3, 4, 5, 6, 7,8,9,10
```

W-10(11M)

```
                                          Location
                                          counter
PROG. 1  START                            : 0
BEGIN    BALR     15, 0                   : 0
         USING    BEGIN +2, 15            : 0
         SR       FOUR, FOUR              : 2
         L        R3, TEN                 : 4
LOOP     L        R5, DATA (4)            : 8
         A        R5, thirty              : 12
         ST       R5, DATA(4)             : 16
         A        4, =F '4'               : 20(32)
         BCT      R3, LOOP                : 24
         BR       14                      : 26
         LTORG                            : 30
FOUR     DC       F '4'                   : 36
R3       EQU      3                       : -
R5       EQU      5                       : -
TEN      DC       F '10'                  : 40
Thirty   DC       F '30'                  : 44
DATA     DC       F' 1, 2, 3, 4, 5, 6, 7,8,9,10  : 48
                                          ------
                                          52
                                          (required)
```

LTORG : Instruction tells the assemblers to encounter the literal and assign location counter to the literal after the call of LTORG instruction. The location counter are shown in column 3 of program.

Symbol table :

Symbol	Value	Relocation
PROG 1	0	R
BEGIN	0	R
LOOP	8	R
FOUR	36	R
R3	3	A
R5	5	A
TEN	40	R
thirty	44	R
DATA	48	R

Literal table :

Literal	Value	Relocation
F' 4'	32	R

Base table :

Reg. No.	Value
15	2

Machine OP table :

Instruction	Value
BALR	15, 0
SR	36, 36
L	3, 38 (0, 15)
L	5, 46 (4, 15)
A	5, 42(0, 15)
ST	5,46(4, 15)
A	4, 30(0, 15)
BCT	3, 6 (0, 15)
BCR	14, 15
DC	F' 4
DC	F' 10
DC	F' 30

Q.31. For the following program find symbol table, literal table, Base table, Machine output table.

```
BEGIN   START
        BALR    15, 0
        USING   *, 15
        L       R1, TWO
        A       R1, TWO
        ST      R1, FOUR
        A       1, = F'5'
T       EQU     4
IN      EQU     5
        EQU     *
        USING   SET, 15
        A       IN, = F' 4'
        LTORG
SA      DS      100F
        LR      1, T
        C       IN, = F'8000
        ST      AC, = SA(IN)
        L       AC, = Data (IN)
AC      EQU     *
DATA    DC      F' 30, 40.
                80[100 nos]
R1      EQU     1
TWO     DC      F'2
FOUR    DC      F
        END
```

Ans.

```
                                        Location
                                        counter
BEGIN   START                           : 0
        BALR    15, 0                   : 0
        USING   *, 15                   : 0
        L       R1, TWO                 : 2
        A       R1, TWO                 : 6
        ST      R1, FOUR                : 10
        A       1, = F'5'               : 14
T       EQU     4                       : –
IN      EQU     5                       : –
        EQU     *                       : –
        USING   SET, 15                 : –
        A       IN, = F' 4'             : 18(28)
        LTORG                           : 22
SA      DS      100F                    : 32
        LR      1, T                    : -
        C       IN, = F'8000            : 432
        ST      AC, =SA(IN)             : 438
        L       AC, = Data (IN)         : 4342
AC      EQU     *                       : –
DATA    DC      F' 30, 40.
                80[100 nos]             : 446
R1      EQU     1                       –
TWO     DC      F'2                     : 450
FOUR    DC      F                       : 454
        END
                                        ---
                                        : 458
                                        (required)
```

Symbol table :

Symbol	Value	Relocation
BEGIN	0	R
T	4	A
IN	5	A
SET	18	R
SA	32	R
AC	446	R
DATA	446	R
R1	1	A
TWO	450	R
FOUR	454	R

Literal table :

Literal	Value
F'5'	24
F'4'	28
F'8000	434

Base table :

Reg. No.	Value
15	2
15	8

Machine-op-table :

Machine Instruction	Value
BALR	15, 0
SR	1, 448 (0, 15)
A	1, 448 (0, 15)
ST	5, 452(0, 15)
A	1,22(0, 15)
A	5, 26(0, 15)
LR	1, 4
C	5, 432 (0, 15)
ST	446, 14 (5, 15)
L	446, 428 (5, 15)
DC	F'30, 40 ...80 [10 ones]
DC	F'2'

POINTS TO REMEMBER :

(1) System programming is the activity of computer programming system software.

(2) An operating system may be defined as organized collection of software routines, which acts as an interface between the user and the computer.

(3) Various instruction formats supported by IBM 360 are :
- *(i) RR (register - register)*
- *(ii) RX (register- indexed)*
- *(iii) RS (register -storage)*
- *(iv) SI (storage-immediate)*
- *(v) SS (storage - storage)*

(4) Various registers provided by IBM 360 architecture are :
- *(i) 16 general purpose register of 32-bit.*
- *(ii) 4 floating point registers of 64-bit.*
- *(iii) 64-bit processor status register word (PSW)*
- *(iv) 24-bit byte- addressable memory space.*

(5) Various data format supported by IBM 360 are :
- *(i) Short form fixed point.*
- *(ii) Long form fixed point.*
- *(iii) Decimal packed.*
- *(iv) Decimal unpacked.*
- *(v) Short form floating point.*
- *(vi) Long form floating point.*
- *(vii) Logical (character).*

(6) An assembly language instruction that specifies an operation of the assembler is known as pseudo-instruction.

(7) An assembler is a program that accepts an assembly language program as input and produces its machine equivalent along with information for the loader output.

(8) The period of execution of a user's program is called execution time.

(9) The period of translating a user's source program is called as assembly or compile time.

(10) Load time refers to the period of loading and preparing an object program for execution.

(11) The purpose of Pass 1 of assembler is to define symbols and literals.

(12) The purpose of pass 2 of assembler is to generate object program.

Notes :-

UNIT - II
MACROPROCESSOR

SYLLABUS

Concept of macro, Macro call within macro, Macro definition within macro, Recursive macro calls, Design of macro processor.

UNIVERSITY PAPER SOLUTIONS
SINCE WINTER - 2009

Note to the students :
- The questions given below are from previous Nagpur University examination papers.
- Though these questions are from old university papers, they have been included in the new syllabus.
- Questions which are out of the new syllabus are not included here.

WINTER - 09

Q.1. Explain advantages and disadvantages of combining macro processor with assembler. **4M**

Ans. P.2-13, Q.9

Q.2. For the following Assembly program prepare MDT, MNT and ALA. Also write the expanded source code: **9M**

```
        MACRO
        OPERATE     &MACRON, &OP
        MACRO
        &MACRON     &A1, &B1
        L           1, &A1
        &OP         1, &B1
        ST          1, &B1
        MEND
        MEND
        ......
        ......
        ......
DATA1   DC          F'20'
DATA2   DC          F'30'
DATA3   DC          F'40'
        ......
        ......
        ......
        OPERATE     ADD, A
        OPERATE     SUB, S
        ......
        ......
        ADD         DATA 2, DATA 3
        ......
        ......
        END
```

Ans. P.2-15, Q.12

Q.3. Write short note on Conditional Macro. **4M**

Ans. P.2-5, Q.4

SUMMER - 10

Q.1. For the following program show MNT, MDT, ALA and expanded assembly language program. **8M**

```
        MACRO
        STORE       & ANS
        ST          1, & ANS
        MEND
        MACRO
        TRY         & ADD, & DEST
        SR          1,1
        MACRO
        & ADD       & A, & B
        L           1, & A
        A           1, & B
        MEND
& DEST  DS          F
```

```
        MEND
        ARITH   START   0
        USING   *, 15
        TRY     PLUS, RESULT
        PLUS    N1, N2
        STORE   RESULT
        N1      DC      F'4'
        N2      DC      F'5'
        END
```

Ans. P.2-14, Q.11

Q.2. Explain the necessity of stack for implementing macro call within macro definition. **5M**

Ans. P.2-7, Q.7

WINTER – 10

Q.1. Explain various macro instruction facilities in details with help of example. **8M**

Ans. P.2-4, 5, 6, Q.2, Q.3, Q.4, Q.5(a), Q.5(b)

Q.2. What are basic task that any macro instruction processor must perform? How do you define a macro in a program? Give a general format. **5M**

Ans. P.2-3, 4, Q.1(a), Q.1(b)

Q.3. Write short note on : **13M**

(i) Macro call within macro and its stack implementation.

Ans. P.2-7, Q.5(a), Q.7

(ii) One pass macro-processor for macro definition within macro definition.

Ans. P.2-7, Q.6, Q.7.

SUMMER - 11

No Question asked in RTMNU

WINTER - 11

Q.1. Differentiate positional argument and keyword argument. When to use the keyword argument? **4M**

Ans. P.2-7, Q.6(b)

Q.2. Explain the various macro instruction facilities. **9M**

Ans. P.2-4, 5, 6, Q.2, Q.3, Q.4, Q.5(a), Q.5(b)

SUMMER - 12

Q.1. Explain in detail the two pass Macro processor with the help of neat and clean flowchart. What macro facilities are restricted in this design? **13M**

Ans. P.2-10, Q.8(a)

Q.2. Explain how MDI and MDLC help in designing a macro capable of handling macro definition within macro. **4M**

Ans. P.2-12, Q.8(b)

Q.3. Discuss the advantages and disadvantages of combining a macro processor with pass-1 of assembler. **4M**

Ans. P.2-13, Q.9

Q.4. What are the two special pseudo-op used in conditional macro? Explain with example. **5M**

Ans. P.2-5, Q.4

WINTER - 12

Q.1. Given the entries in MNT, MDT, ALA for the following definition and call. **13M**

```
                MACRO
                OPRTN   & Macnm, & OP
                MACRO
                & Macnm & A, & B
                L       1, & A
                & OP    1, & B
                ST      1, & B
                MEND
                MEND
                START
                :
DATA 1          DC      F'10'
DATA 2          DC      F'11'
DATA 3          DC      F'20'
                :
                OPRTN   ADD A
                OPRTN   SUB S
                :
                ADD     DATA1, DATA2
                :
                SUB     DATA 1, DATA 3
                END
```

Ans. P.2-15, Q.12

Q.2. What is a use of stack frame in macro processor? Give the format of stack frame along with example. **6M**

Ans. P. 2-7, Q.7

Q.3. Write short note on Positional argument and keyword argument. **4M**

Ans. P. 2-7, Q.6(b)

SUMMER - 13

Q.1. What databases are used by the two passes of macro processor? **5M**

Ans. P.2-13, Q.10

Q.2. What are the basic tasks that any macro instruction processor must perform? **5M**

Ans. P.2-4, Q.1(b)

Q.3. What is the use of conditional macro calls? **3M**

Ans. P.2-5, Q.4

Note : Also refer Paper Solutions at the end of the book.

SOLVED QUESTION BANK

[Sequence given as per syllabus]

INTRODUCTION

In assembly language programmes usually need to repeat some block of code many times in complete range of a particular program.

The block may have information or code to save or to exchange set of register.

In such scenario programmers often write a block to be repeated only once and by using the defined keyword that block of code is repeated.

Macro instructions are single line abbreviations for groups of instructions. Macro operations simply debugging and facilitates standardization.

MACRO CONCEPT AND DEFINITION

Q.1.(a) How do you define a macro in a program? Give a general format. ***W-10(2M)***

Ans. Macro instruction provide following features or task :

(1) By defining a macro instruction, we can increase the level of programming in a convenient manner.

(2) Macro provides task of easy programming code.

(3) Macro also provides conciseness in coding high level language without losing the basic advantage of assembly language program.

(4) Macro simplify debugging and also provide program modification and provide standardization in programs.

Macro :

Macro is an abbreviation for a sequence of operations.

Example :

```
:
A   1, DATA     Add content of
                DATA to register 1
A   2, DATA     Add content of
                DATA to register 2
A   3, DATA     Add content of
                DATA to register 3
```

```
        :
A   1, DATA     Add content of
                DATA to register 1
A   2, DATA     Add content of
                DATA to register 2
A   3, DATA     Add content of
                DATA to register 3
        :
DATA DC F'5'
        :
```

In the above program the sequence below occurs twice.

```
A   1, DATA
A   2, DATA
A   3, DATA
```

A macro facility permits us to attach a name to the sequence. A macro processor effectively constitutes a separate language processor with it's own language.

Syntax for defining macro :

MACRO → Start of macro definition.

[ ] → Macro name.

-------- } → Sequence to be abbreviated.
-------- }
-------- }

MEND → End of macro definition.

Description :

(1) **MACRO :** This Pseudo-op is the first line of definition and identifies the following line as macro instruction name.

(2) **MEND :** This pseudo op denote end of macro.

Once macro has been defined, the use of macro name as an operation mnemonic in an assembly program is equivalent to the use of corresponding instruction sequence.

Q.1(b). What are the basic tasks that any macro instruction processor must perform?

W-10(3M), S-13(5M)

Ans. There are four basic tasks that any macro instruction processor must perform.

(1) **Recognize macro definitions:**

- A macro instruction processor must recognize macro definitions identified by the MACRO and MEND pseudo-ops. This task can be complicated when macro definitions appear within macros.
- When MACROs and MENDs are nested, the macro processor must recognize the nesting and correctly match the last or outer MEND with the first MACRO.
- All of the intervening text, including nested MACROs and MENDs, defines a single macro instruction.

(2) **Save the definitions:** The processor must store the macro instruction definitions, which it ill need for expanding macro calls.

(3) **Recognize calls:** The processor must recognize macro calls that appear as operation mnemonics. This suggests that macro names be handled as a type of op-code.

(4) **Expand calls and substitute arguments:** The processor must substitute for dummy or macro definition arguments the corresponding arguments from a macro call; the resulting symbolic text is then substituted for the macro call. This text may contain additional macro definitions or calls.

FEATURES OF MACRO FACILITY

Q.2. List various macro instruction facilities in details with help of example. ***W-10(1M)***

OR What are various macro instruction facilities? ***W-11(1M)***

Ans. The various macro instruction facilities are as follows :

(1) Macro instruction arguments
(2) Conditional macro expansion
(3) Macro calls with macros.
(4) Macro instruction defining macro.

Q.3. Explain macro instruction argument. ***W-10,11(2M)***

Ans. Macro Instruction Arguments :

- An important feature provided by macro is that the block of instruction can be inserted in place of macro calls.

- All macro call can be replaced by identical blocks of code.
- **Advantage :** Provide a facility of argument, or parameter in macro calls, such parameter is called as macro instruction argument or dummy argument.

Example :

```
        A    1,   Data 1
        A    2,   Data 1
        A    3,   Data 1
             ⋮
        A    1,   Data 2
        A    2,   Data 2
        A    3,   Data 2
Data 1  DC   F'5'
Data 2  DC   F'10'
```

- In above program 1st sequence perform operation using Data 1 as operand and 2nd sequence uses Data 2 as operand, both operand perform same operation with variable or parameter or argument, such parameter are macro instruction argument or dummy argument.
- Dummy argument is specified on macro name line and distinguished by "&" (ampersand) symbol.

Program implementing macro can be given as:

```
                Source                                  Expanded source
MACRO           macro xyz has one argument
  xyz           & ARG
  A             1, & ARG
  A             2, & ARG
  A             3, & ARG
 MEND
   ⋮
  xyz           DATA 1    use DATA 1 as argument (operand)   A  1, DATA 1
                                                             A  2, DATA 1
   ⋮                                                         A  3, DATA 1
  xyz           DATA 2    Use DATA 2 as operand              A  1, DATA 2
                                                             A  2, DATA 2
                                                             A  3, DATA 2
   ⋮
DATA 1 DC       F'5'
DATA 2 DC       F'10'
```

Q.4. What is the use of conditional macro call? *S-13(3M)*

OR Write Short note on Conditional Macro. *W-09(4M), W-10,11(2M)*

OR What are the two pseudo-op used in conditional macro? Explain with example. *S-12(5M)*

Ans. Conditional macro expansion :

- Two important macro processor pseudo-op AIF and AGO permit conditional reordering of the sequence of macro expansion.
- This facilitates conditional selection of machine instruction that appears in expansion of macro call.

Example :

```
Loop 1    A    1, Data 1
          A    2, Data 2
          A    3, Data 3
               ⋮
Loop 2    A    1, Data 3
          A    2, Data 2
               ⋮
Loop 3    A    1, Data 1
               ⋮
Data 1    DC   F'5'
Data 2    DC   F'10'
Data 3    DC   F'15'
               ⋮
```

The macro program for above program sequence can be written as follows :

```
          Source                                  Expanded source
               ⋮
          MACRO
& ARG0    VARY A    & count, & ARG 1, ARG 2, ARG3,
          AIF       (& count EQ1).FINI 2, & ARG 2
          A         1, & ARG 1
          AIF       (& count EQ2). .FINI
          A         3, & ARG3
FINI      MEND
               ⋮
```

```
Loop 1    VARY    3, Data 1, Data 2,      Loop 1 { A 1, Data 1
                  Data 3                           A 2, Data 2
                                                   A 3, Data 3
Loop 2    VARY    2, Data 3, Data 2       Loop 2 { A 1, Data 3
                                                   A 2, Data 2
                                                   :
Loop 3    VARY    1, Data 1               Loop 3   A 1, Data 1
          :
DATA 1    DC      F' 5'
DATA 2    DC      F' 10'
DATA 3    DC      F' 15'
```

- The label starting with period (•) Such as • FINI are macro label and do not appear in output.
- Statement AIF (& count EQI) •FINI direct the macro processor to skip the statement labeled •FINI if value of count = 1 otherwise continue with statement following AIF pseudo- op.
- AIF is a conditional branch pseudo-op perform arithmetic test and branches if condition is true.
- AGO is an unconditional branch pseudo-op or go to statement.

Q.5.(a) Define Nesting of Macro.

OR Write short note on macro call within macro.

W-10,11(2M)

Ans. Macro call within macro :

- Macro can be called within macro easily.
  MACRO calls within macros can involve several levels.

  Example :

```
MACRO
ADD 1    & ARG
L        1, & ARG
A        1, = F'1'
ST       1, & ARG
MEND
MACRO
ADDS     & ARG1, &ARG 2, & ARG 3
ADD 1    & ARG 1
ADD 1    & ARG 2
ADD 1    & ARG 3
MEND
```

- The macro ADDs might be called within the definition of another macro.
- In fact conditional macro facilities (such as AIF and AGO) make it possible for a macro to call itself.
- In above program the definition of macro 'ADDs' are three separate calls to a previously defined macro 'ADD1'
- The use of the macro 'ADD1' has shortened the length of definition of 'ADDs' and thus has made it more easily understood.

Q.5(b). Explain macro instructions defining macros.

W-10,11(2M)

Ans. Macro Instructions Defining Macros :

- The following example defines a macro instruction DEFINE, which when called with a subroutine name defines a macro with the same name as the subroutine.
- The individual macros generated, bear the names of their associated subroutines.

Defination of macro DEFINE :

```
MACRO
DEFINE &SUB
DEFINE
MACRO
&SUB     & Y
CNOP     0,4
BAL      2,* +8
DC       A(&Y)
L        15,V(&SUB)
BALR     14,15
MEND
MEND
```

- The user might then call this macro with the statement

  DEFINE COS

  defining a new macro named COS : the statement expands into a new macro definition.
- The user might subsequently call the COS macro as follows :

  COS AR

  and the macro processor will generate the calling sequence :

```
BAL     1,*+8
DC      A(AR)
L       15,=V(COS)
BALR    14,15
```

Macro definitions within macros are sometimes called "macro definitions within macro definitions."

RECURSIVE MACRO CALLS

Q.6.(a) What is the use of recursive macro calls?

Ans. Recursive macro calls :

- It is used for calling a macro within another macro.
- Recursive procedures operate by means of a stack, which allocates a separate storage area for the variables associated with each call to the procedure.
- Because of the separate space or stack frame is associated with each recursive call, the status of unfinished computation is preserved, which is otherwise lost if recursion is not used which can then result in unexpanded lines in the macro.

Q.6.(b) Differentiate positional and keyword argument. When to use the keyword argument? *W-11(4M)*

Ans.

Sr. No.	Positional Argument	Keyword Argument
(1)	The arguments which depends upon their physical position in macro prototype are called positional arguments.	The arguments which refers dummy arguments by name as well as by position are called as keyword arguments.
(2)	It can be written as & <Argument name>	It can be written as <Formal parameter> = <ordinary string>
(3)	**Example :** & SAMPLE & DATA etc.	**Example :** INCR_M = B REG = ARG etc.

DESIGN OF MICROPROCESSOR

Q.7. Explain the necessity of stack for implementing macro call within macro definition. *S-10(5M)*

OR Write short note on stack implementation. *W-10(4M)*

OR Write short note on one pass macro-processor for macro definition within macro definition. *W-10(7M)*

OR Explain the implementation of macro call within macro definition draw flowchart for one pass macro processor capable of handling macro call within macro definition.

OR What is a use of stack frame in microprocessor? Give the format of stack along with example. *W-12(6M)*

Ans.

- To handle the macro calls within macro definitions, the macro processor must be able to work recursively that is to process one macro before it is finished with another, then to continue with the previous, or outer one.
- Recursive procedure usually operates by means, of a stack, a storage scheme that allocates a separate storage area for the variables associated with each call to the procedure.
- Because a separate storage space or stack frame is associated with each recursive call, the status of unfinished computations is preserved conceptually, a stack is an unbounded array that is treated in a last-in, first-out manner i.e. the last element stored is the first one removed.
- A stack Pointer (SP) indicates the position or frame at the top of the stack. (Pointer is a term that refers to a data item which is an address or index. A pointer points to information stored in core).
- **Example :**

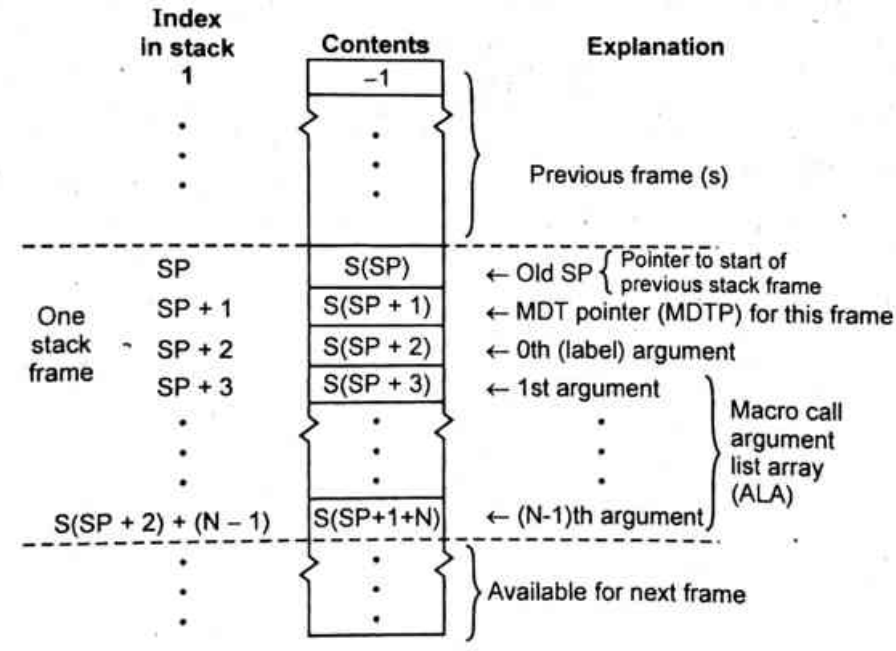
```
MACRO
ADD1    & ARG
L       1, & ARG
A       1 = F, '1'
ST      1, &ARG
```

```
MEND
MACRO
ADDS    &ARG1, &ARG2, &ARG3
ADD1    &ARG1
ADD1    &ARG2
ADD1    &ARG3
MEND
```

The Macro definition table (MDT) for the above example is :

Index	Contents	
1	ADD1	&ARG
2	L	1, # 1
3	A	1, = F '1'
4	ST	1, # 1
5	MEND	
6	ADDS	& ARG1,
		& ARG2
		& ARG3
7	ADD1	#1
8	ADD1	#2
9	ADD1	#3
10	MEND	

- In example, the pointer MDTP and the argument list ALA associated with each call belong in the stack. The switch MDI must also be saved.
- The stack pointer SP which indicates the beginning of the current stack frame can serve in place of MDI.
- In each succeeding frame. However we must save the previous value at SP to prevent it from being lost.
- Recursive handling of macro definitions was easy because the inner definitions don't have any associated data.
- Ignoring the actual method of data representation we consider the stack to be an array of pointers and character string; S(SP) refers to the beginning of the current (top) stack frame. The position indexed by SP.
- The organization of each stack frame is shown in figure given below.



N : number of arguments on macro name line
Sp : current value of stack pointer
S(m) : contents of m^{th} position on the stack

Fig. Stack organisation

- S(SP) contains the previous value of SP; for the first frame where SP =1, S(SP) = – 1. The condition SP = – 1 like MDI = 0 in the non-recursive algorithm takes the macro processor that it is not within a macro call expansion.
- S (SP+1) contains the current value of MDTP, the MDT pointer, S (SP+2).....S(SP+N+1) contains the N character strings of the current argument list.

One pass macro processor capable at handling macro calls within macro definitions & a detail of READ function for recursive macro expansion is given in the figure.

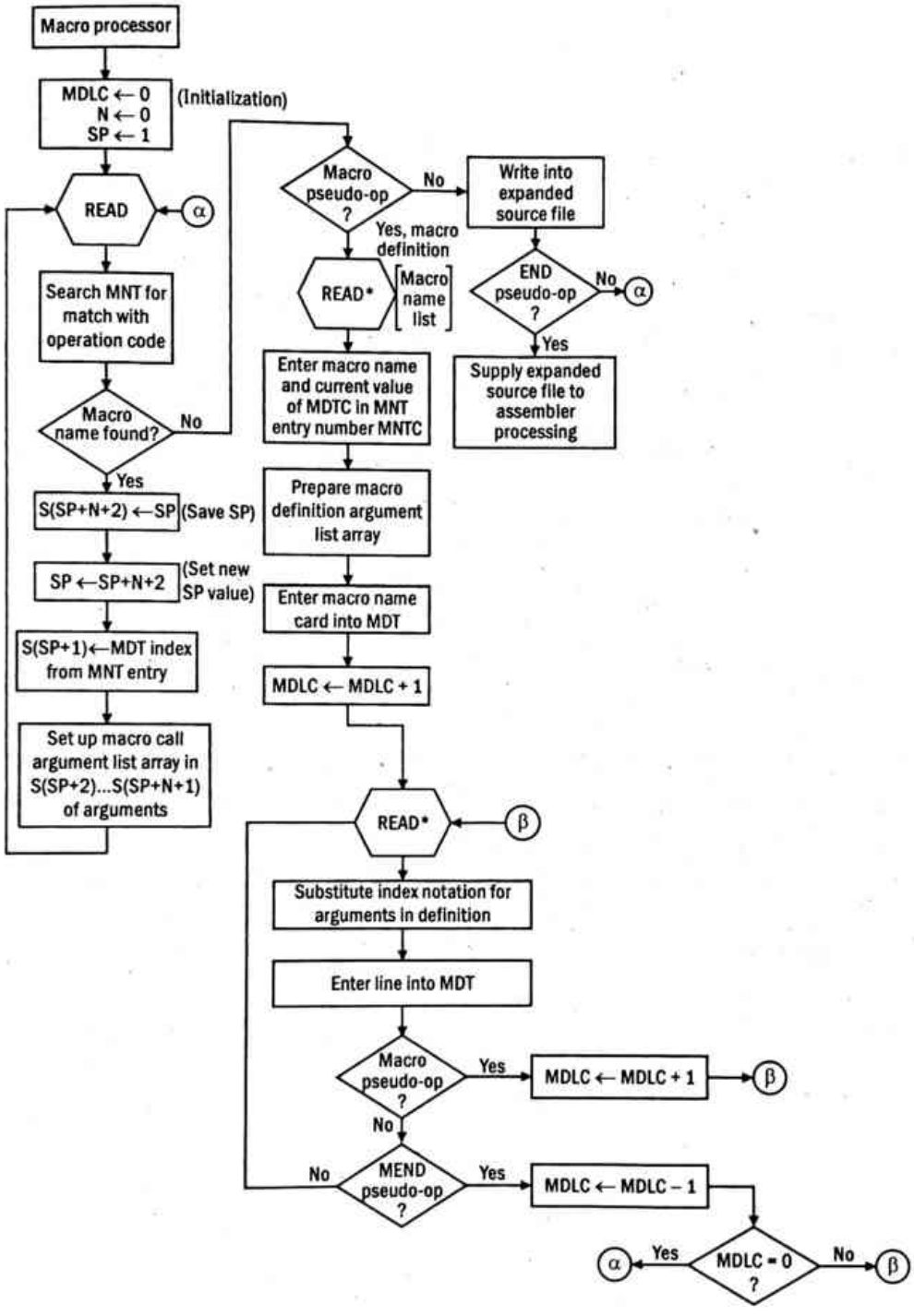
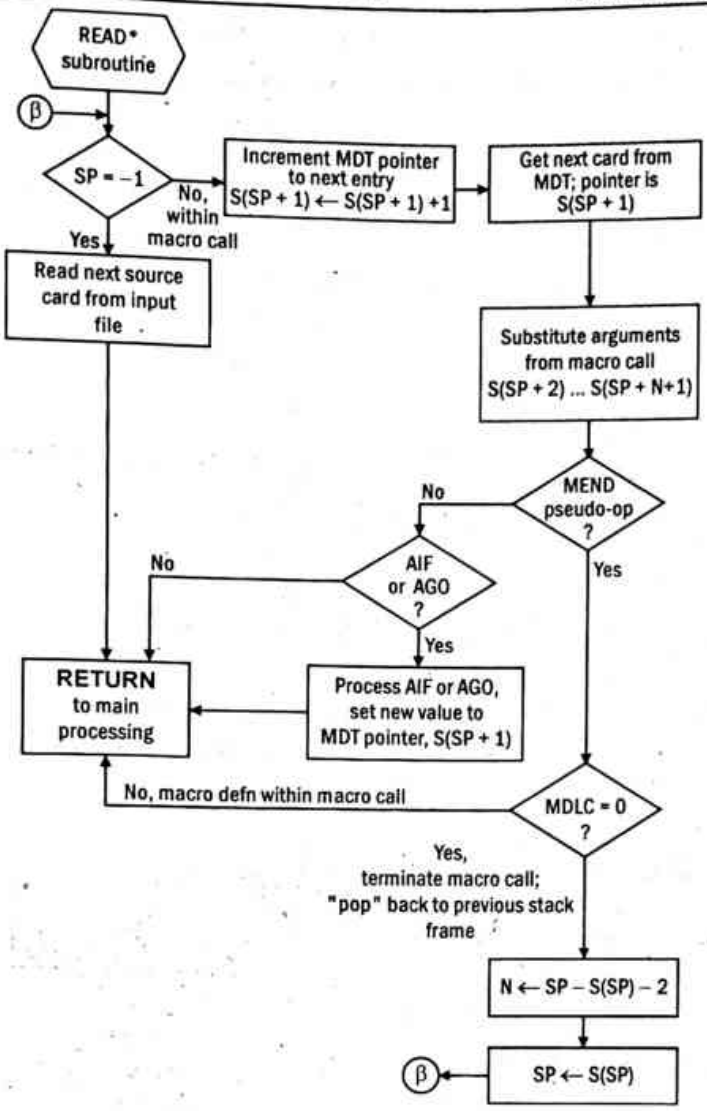


Fig. one-pass macro processor



Q.8(a). Explain in detail the two pass macro processor with the help of neat and clean flowchart. What macro facilities are restricted in this design? ***S-12(13M)***

Ans. pass 1 : Macro Definition :

- The algorithm for pass 1 tests each input line. If it is a MACRO pseudo-op, the entire macro definition that follows is saved in the next available locations in the Macro Definition Table (MDT).
- The first line of the definition is the macro name line. The name is entered into the Macro Name Table (MNT), along with a pointer to the first location of the MDT entry of the definition.
- When the END pseudo-up is encountered, all of the macro definitions have been proposed so control transfers to pass 2 in order to process macro calls.

Pass 2 : Macro Calls And Expansion :

- The algorithm for pass 2 tests the operation mnemonic of each input line to see if it is a name in the MNT.
- When a call is found, the call processor sets a pointer, the Macro Definition Table Pointer (MDTP), to the corresponding macro definition stored in the MDT.
- The initial value of the MDTP is obtained from the "MDT index" field of the MNT entry. The macro expander prepares the Argument List Array (ALA) consisting of a table of dummy argument indices and corresponding arguments to the call.

- This list is simply a succession of symbols ordered to match the dummy arguments on the name card.
- Arguments not represented in a call are considered blank and superfluous arguments are ignored. In the case of argument reference by position, this scheme is completely straight-forward.
- For references by name, the macro processor locates the dummy argument on the macro name line in order to determine the proper index.
- Reading proceeds from the MDT; as each successive line is read, the values from the argument list are substituted for dummy argument indices in the macro definition.
- Reading of the MEND line the MDT terminates expansion of the macro and scanning continues from the input file.
- When the END pseudo-op to encountered, the expanded source deck is transferred to the assembler for further processing.

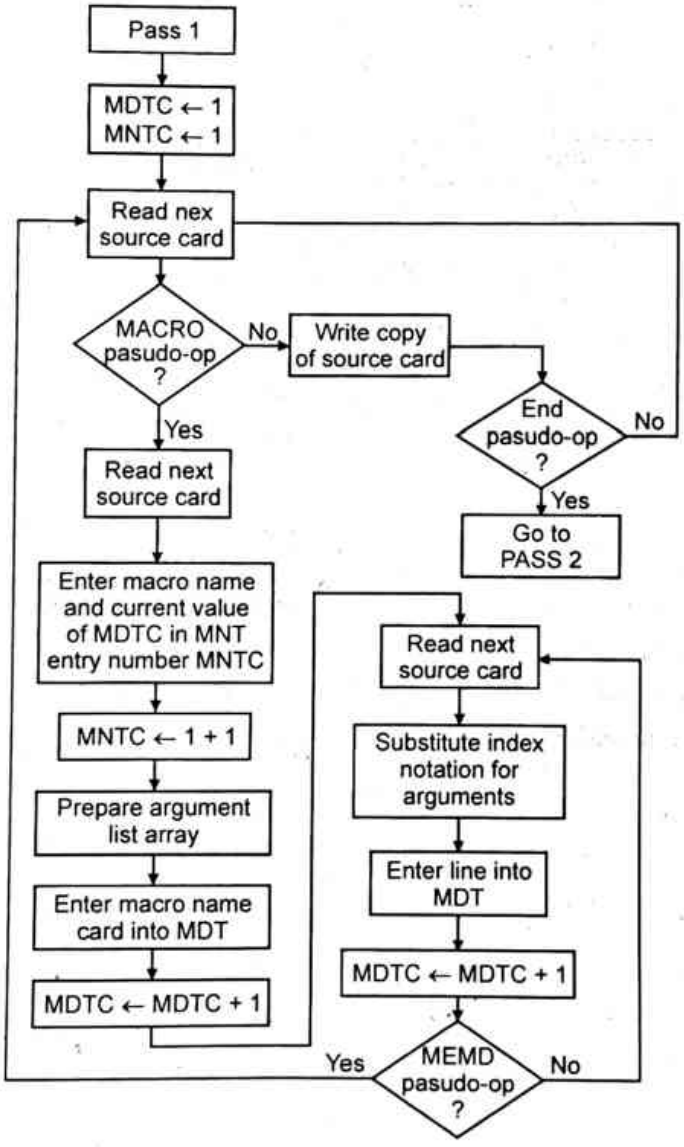


Fig. Pass 1 : Processing macro definitions

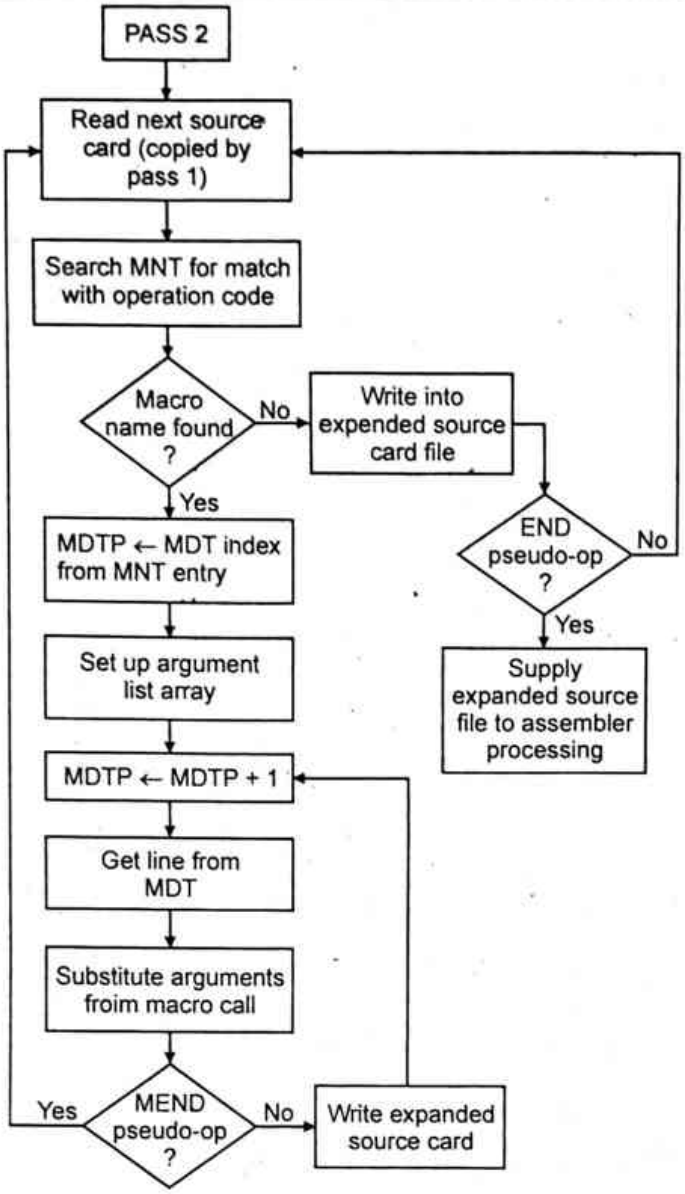


Fig. Pass 2 : Processing macro calls and expansion

Q.8.(b) Explain how MDI and MDLC help in designing a macro capable of handling macro definition within macro. ***S-12(4M)***

Ans.

- The MDI and MDLC are switches (counter) used to keep track of macro calls and macro definition.

MDI :

- The MDI indicator has the value "on" during expansion of a macro call and the value "OFF" at all other time.
- The actual expansion of macro calls is performed in the READ box as given in figure.
- READ test the switch MDI
- If it is "ON" lines are read from the macro definition table (MDI).
- The reading of a MEND line indicates the end of a macro and terminates expansion of a call.
- MDI is reset to "OFF" and the next line is obtained from the regular input stream.

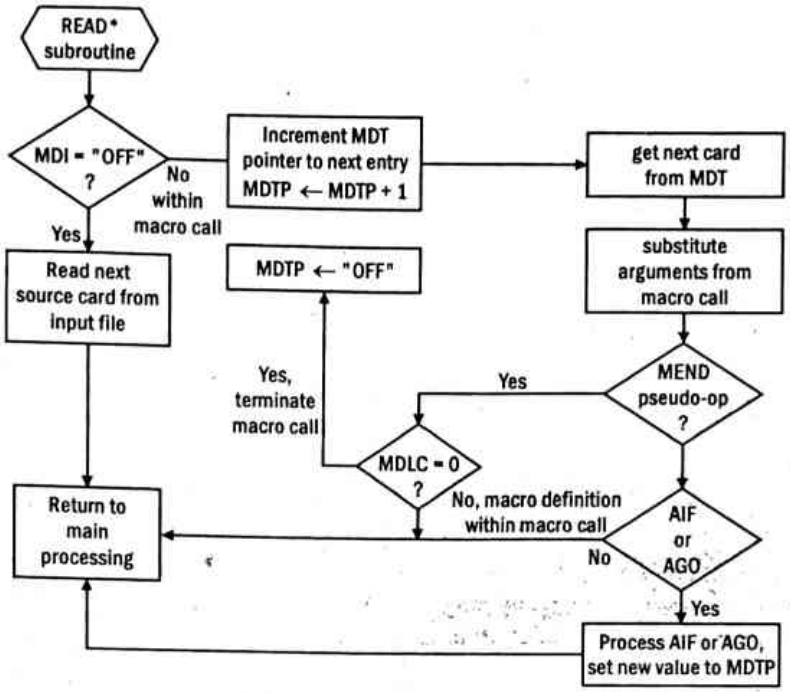


Fig. Details of read function used for macro easy expansion

MDLC :

- MDLC is used to ensure the entire macro definition, including macro and MEND get stored in MDT.
- If program is to read entire expansion, it must not stop at first right parenthesis but rather it must keep on a count of left and right parenthesis and recognize sound parenthesis is the true end of expansion.
- The MDLC acts as a counter for the difference between the number of MACRO and MEND, it counts how many more MACRO's and MEND's have been read.

Q.9. What are the major advantages of incorporating the macro processor into pass of the assembler? What are the major disadvantages?

OR Explain advantages and disadvantages of combining macro processor with assembler.

W-09(4M), S-12(4M)

Ans. Advantages :

(1) Macro function do not implemented twice. (**Example :** read a, cord, test for statement type)

(2) There is less overhead during processing functions are combined and its not necessary to create intermediate file as output from macro processor and input to the assembler.

(3) Here flexibility is available to the programmer so that it may use all the features of the assembler (**Example :** EGU statement) in conjunction with macros.

Disadvantages :

(1) The combined pass 1 of the assembler and macro processor may be too large a program to fit into, core of some machine.

(2) The complexity of program increases.

(3) The data structure becomes complex.

Q.10. What databases are used by the two passes of macro processor? ***S-13(5M)***

Ans. The following databases are used of two passes of macro processor.

Pass 1 :

(1) Input macro source desk.

(2) Output macro source desk copy for use pass 2.

(3) The macro definition table (MDT) used to store the body of macro definitions.

(4) The macro name table (MNT) used to store the names of defined macro.

(5) The macro definition table counter (MDTC) used to indicate the next available entry in MDT.

(6) The macro name table counter (MNTC) used to indicate the next available entry in the MNT.

(7) The argument list array (ALA) used to substitute index markers for dummy arguments before storing a macro definition.

Pass 2 :

(1) The copy of the input macro source desk.

(2) The output expanded source deck to be used as input to the assembler.

(3) The macro definition table (MDT) created by pass 1.

(4) The macro name table (MNT) created by pass 1.

(5) The macro definition table pointer (MDTP) used to indicate the next line of text to used during macro expansion.

(6) The argument list array (ALA) used to substitute macro call arguments for Re-index markers in stored macro definition.

MACRO PROBLEMS

Q.11. For the following assembly program find

(1) MDT (2) MNT

(3) Expanded assembly language program

S-10(8M)

```
          MACRO
          STORE     & ANS
          ST        1, & ANS
          MEND
          MACRO
          PERFORM   & ADD, & DEST
          SR        1, 1
          MACRO
          &ADD      & A, & B
          L         1, & A
          A         1, & B
          MEND
& DEST    DC        F
          MEND
ARITH     START     O
          USING     *, 15
          PERFORM   PLUS, RESULT
          PLUS      NUM1, NUM 2,
          STORE     RESULT
NUM 1     DC        F'4'
NUM 2     DC        F'5'
          END
```

Ans. (1) Macro definition table :

Index	Card	
⋮		
⋮		
⋮		
15	STORE	& ANS
16	ST	1, # 1
17	MEND	
18	PERFORM	&ADD, &DEST
19	SR	1, 1
20	#1,	&A, &B
	L	1, # 3
	A	1, # 4
	MEND	
	# 1 DC .	F

(2) Macro name table :

Index	Name	MDT index
⋮		
3	STORE	15
⋮		⋮
10	PERFORM	18
⋮		⋮
12	# 1,	12

(3) Expanded source code :

```
          START
          USING     * 15
          L         1, NUM 1
          A         1, NUM 2
          ST        1, RESULT
          DC        F'4'
          DC        F'5'
          END
```

Q.12. For following assembly program prepare MDT, MNT and ALA. Also write the expanded source code: ***W-09(9M),12(13M)***

```
        MACRO
        OPERATE   &MACRON, &OP
        MACRO
        &MACRON   &A1, &B1
        L         1, &A1
        &OP       1, &B1
        ST        1, &B1
        MEND
        MEND
        ......
        ......
        ......
DATA 1  DC        F' 20'
DATA 2  DC        F' 30'
DATA 3  DC        F' 40'
        ......
        ......
        ......
        OPERATE   ADD, A
        OPERATE   SUB, S
        ......
        ......
        ADD       DATA 2, DATA 3
        ......
        ......
        END
```

Ans. Macro definition table :

Index	Card
0	MACRO
1	OPERATE & macron, & OP
2	MACRO
3	# 1 $\&A_1, \&B_1$
4	L 1, # 3
5	2 1, # 4
6	ST 1, # 4
7	MEND
8	MEND

ALA (Argument list array)

Index	Argument
0	"macronbb"
1	"opbbbbbb"
2	"A1 bbbbbb"
3	"B1 bbb bb"

MNT (Macro name table)

Index	Name	MDT index
0	"OPERATE bb"	1
1	"macron bb"	3

Macro expansion code :

```
MACRO
OPERATE     ADD A
MACRO
ADD         DATA 2,   DATA3
L           1,        & DATA 2
A           11,       & DATA 3
ST          1,        & DATA 3
MEND
MEND
⋮
DATA 1      DC        F'20'
DATA 2      DC        F'30'
DATA 3      DC        F'22'
OPERATE     ADD       A
OPERATE     SUB       S
⋮
L           1,        & DATA 2
A           1,        & DATA 3
ST          1,        & DATA 3
⋮
END.
```

POINTS TO REMEMBER :

(1) A macro instruction is a single line abbreviations for groups of instructions.

(2) The macro facility is classified into four categories, they are :

(i) Macro instruction arguments.

(ii) Conditional macro expansion.

(iii) Macro calls within macros.

(iv) Macro instruction defining macros.

(3) The macro calls which are used for calling a macro within another macro is called as Recursive macro calls.

(4) The four basic tasks that any macro processor must perform.

(i) Recognize macro definition.

(ii) Save the definitions.

(iii) Recognize calls.

(iv) Expand calls and substitute arguments.

UNIT - III
LINKER AND LOADER

SYLLABUS

Concept of static and dynamic relocation, External symbols, Design of linker, Design of object file for different loading schemes.

UNIVERSITY PAPER SOLUTIONS
SINCE WINTER - 2009

Note to the students :

- The questions given below are from previous Nagpur University examination papers.
- Though these questions are from old university papers, they have been included in the new syllabus.
- Questions which are out of new syllabus are not included here.

WINTER - 09

Q.1. What are the basic four functions of loader? How does Relocating loader perform these functions? Explain in detail. **7M**

Ans. P.3-3,7, Q.1 & 7.

Q.2. Explain the following type of address constant with example: **6M**

i) Absolute ii) Simple Relocatable

iii) Complex Relocatable.

Ans. P.3-18, Q.24.

Q.3. Write short note on : **8M**

i) Absolute Loader

ii) Binders and Module Loader.

Ans. P.3-4,11, Q.4 & 13.

SUMMER - 10

Q.1. What is the order of RLD, ESD, TXT, END and LDT cards in a bunch of object decks of more than one subroutine and why are they in this order? **4M**

Ans. P.3-13, Q.19.

Q.2. Why two passes are required by a direct linking Loader? Draw and explain flowchart for pass - 2 of loader. **9M**

Ans. P.3-9, Q.12.

Q.3. Write short note on Link Editor. **4M**

Ans. P.3-12, Q.16.

WINTER - 10

Q.1. Explain following terms : **3M**

(a) Transfer vector .

(b) Binder.

(c) Overlay.

Ans. P.3-11, Q.14.

Q.2. Write short note on : **8M**

(a) Dynamic linking and dynamic loading.

(b) BSS loader.

Ans. P.3-7, 11, Q.8 & 15.

Q.3. For the program given below, show the contents of ESD and RLD card. Also show the contents in GEST.

```
PGA     START
        ENTRY    PGA1
        EXTRN    PGB, PGB2
        DC       A(PGA), A(PGB + 4)
PGA1    DC       A(PGA1 – PGA),
                 A(PGB2 + PGA)
        END
PGB     START
        ENTRY    PGB2
        ENTRY    PGB1
PGB1    DC       A(PGB1-4), A(PGA1)
PGB2    DC       A(PGB+4),A(PGB1 – PGB)
        END
```

10M

Ans. P.3-18, Q.25.

SUMMER - 11

No question asked in University exam.

WINTER - 11

Q.1. Explain 'LOCAL' loader. **5M**

Ans. P.3-11, Q.15.

Q.2. How the ID numbers are provided on ESD card? What is its advantage? Why these ID numbers are not provided to locally defined symbols? **5M**

Ans. P.3-15, Q.21.

Q.3. Explain Assembler and Go loader in brief. **3M**

Ans. P.3-4, Q.3.

Q.4. Write short note on link editor. **3M**

Ans. P.3-12, Q.16.

Q.5. Show the entries in ESD, TXT and RLD cards for the following program :

```
JOHN      START
          EXTRN   SUM, DATA
          ENTRY   LOOP, POINTER
          BALR    15, 0
          USING   *, 15
          SR      4, 4
          L       1, FOUR
          A       2, FOUR
          ST      2, FOUR
          BR      14
FOUR      DC      F' 4'
LOOP      DC      A (SUM + 4)
POINTER   DC      A (LOOP-DATA)
          DC      A (POINTER-LOOP)
          DC      A (POINTER)
          END
```

13M

Ans. P.3-25, Q.30.

SUMMER - 12

Q.1. Write short note on Link Editor. **4M**

Ans. P.3-12, Q.16.

Q.2. Consider the following three programs :-

```
PGA       START
          ENTRY   PGA1
          EXTRN   PGB, PGC, PGC2
          DC      A(PGA), A(PGB+3)
PGA1      DC      A(PGA1 – PGA), A(PGC2
                                  – PGC)
          END
PGB       START
          ENTRY   PGB1
          EXTRN   PGA, PGC1
PGB1      DC      A(PGC – 2), A(PGB1)
PGB2      DC      A(PGB+4), A(PGB1 – PGB)
PGB3      DC      A(PGC1+PGB – PGA – 16)
          END
PGC       START
          ENTRY   PGC1, PGC2
          DC      A(PGC2 – PGC)
PGC1      DC      A(PGC1 – 4), (* + 4)
PGC2      DC      A(PGC+PGC2 – PGC1)
          END
```

Assume that these programs are loaded at location 400 in the order PGA, PGB, then PGC, Obtain ESD, TXT, RLD, GEST and LESA. **14M**

Ans. P.3-22, Q.28.

WINTER - 12

Q.1. Explain following address constant. **7M**

(a) Absolute

(b) Simple relocatable

(c) Complex relocatable.

What role they play in design of loader?

Ans. P.3-18, Q.24.

Q.2. Write short note on : **4M**

Binder and Module loader

Ans. P.3-11, Q.13.

Q.3. Prepare ESD, RLD and GEST for following program :

```
PGA       START
          ENTRY   SYM11
          EXTRN   PG2, SYM23
          DC      A(PG1), A(PG2 + 4)
Sym11     DC      A(SYM11-PGI),
                  A(SYM23-PG-2)
          END
PG-2      START
          ENTRY   SYM23
          ENTRN   PG1, SYM11
```

```
Sym21   DC      A(Sym11-2), A(Sym21)
Sym22   DC      A(PG2+4),A(SYM21-PG2)
Sym23   DC      A(Sym21-PG1)
        END
```

13M

Ans. P.3-24, Q.29.

SUMMER - 13

Q.1. What are the functions of loader? **2M**

Ans. P.3-3, Q.1.

Q.2. Explain relocating loader in detail with advantages and disadvantages. **8M**

Ans. P.3-7, Q.7.

Q.3. What is meant by Binder? Also explain the module loader. **3M**

Ans. P.3-11, Q.13.

Q.4. Show the entries in ESD, TXT and RLD cards for the following program :

```
JOHN      START
          ENTRY   SUM, DATA
          EXTRN   LOOP, POINTER
          BALR    15, 0
          USING   *, 15
          SR      4, 14
          L       1, FOUR
          A       2, FOUR
          ST      2, FOUR
          BR      14
FOUR      DC      F '4'
LOOP      DC      A (SUM + 4)
POINTER   DC      A (LOOP – DATA)
          DC      A (POINTER – LOOP)
          DC      A (POINTER)
          END
```

13M

Ans. P.3-25, Q.30.

Q.5. Write short note on link editor. **4M**

Ans. P.3-12, Q.16.

Note : Also refer Paper Solutions at the end of the book.

SOLVED QUESTION BANK

[Sequence given as per syllabus]

INTRODUCTION

Loader is a utility program which takes object code as input prepares it for execution and loads the executable code into the memory. This loader is actually responsible for initiating the execution of any process. Loader is one of the essential stage in the process of starting a program. Loading of program involves reading the content of the executable file containing instructions into memory and then carrying out other required task to prepare the program for execution. Once loading is complete the operating system starts program by passing control to the loaded program code.

A linker is program that is used to properly combine all the object program generated by compiler and to convert them into the final executable program, which is a load module. It means that a linker takes object program file (modules), and fits them together to assemble them into the program's final executable form.

Q.1. What is loader? What are the basic function of a loader? ***W-09(3M), S-13(2M)***

Ans. Loader : The loader is a program which accepts the object program decks, prepares these program for execution by the computer; and initiate the execution.

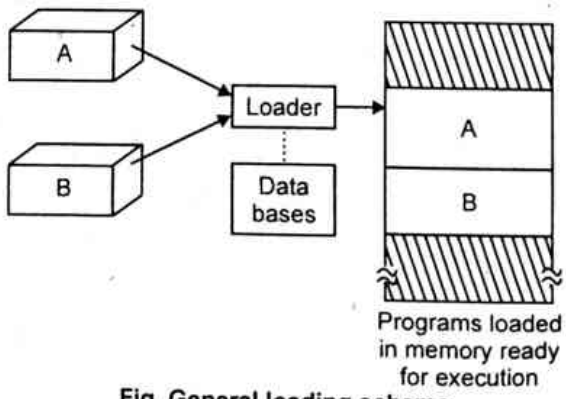


Fig. General loading scheme

Functions of Loader :

The loader is responsible for the activities such as allocation, linking, relocation and loading.

(1) Loader allocates the space for program in the memory, by calculating the size of the program. This activity is called **allocation.**

(2) Loader resolves the symbolic references (code/data) between the object modules by assigning all the user subroutine and library subroutine addresses. This activity is called **linking.**

(3) There are some address dependent locations in the program, such address constants must be adjusted according to allocated space; such activity done by loader is called **relocation.**

(4) Finally loader places all the machine instructions and data of corresponding programs and subroutines into the memory. Thus program now becomes ready for execution, this activity is called **loading.**

LOADING SCHEMES

Q.2. State different loader schemes.

Ans. The different loader schemes are :

1) Compile and Go loaders.
2) Absolute loader.
3) General loader scheme.
4) Subroutine linkages.
5) Relocating loaders.
6) Direct - Linking loaders.

Q.3. Explain assembler and Go loader in brief.

W-11(3M)

Ans. Compile-and-Go Loaders or Assembler and Go loader :

- A compile and go loader is one in which the assembler itself does the processes of compiling then place the assembled instruction in the designated memory locations.
- The assembly process is first executed and then the assembler causes a transfer to the first instruction of the program.

  Example : WATFOR FORTRAN compiler.
- This loading scheme is also called assemble-and-go.

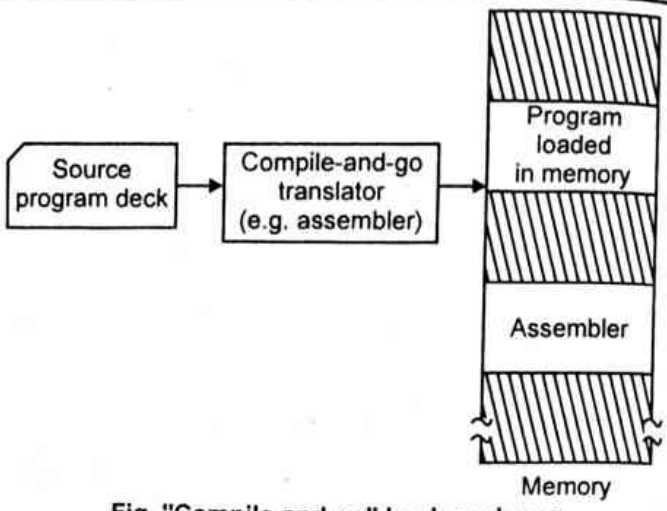


Fig. "Compile-and-go" loader scheme

Advantages :

(1) Simple and easier to implement.

(2) No additional routines are required to load the compiled code into the memory.

Disadvantages :

(1) Wastage in memory space due to the presence of the assembler.

(2) There is a need to re-assemble the code every time it is to be run.

(3) It becomes increasingly difficult to handle large number of segments when the input code is written in a variety of HLL say one routine in Pascal and one in FORTRAN and so on.

Q.4. Write a short note on Absolute loader.

W-09(4M)

OR What are absolute loaders? What are their advantages and disadvantages? How the different functions of loaders are accommodated in absolute loader scheme?

Ans. Absolute loader :

- The simplest of the loaders that fits the general loader scheme is absolute loader.
- In this scheme the assembler outputs the machine language translation of source program in almost the same form as "compile and go" scheme, except that data is punched on cards instead of being placed directly in memory.
- The loader in turn simply accepts the machine language text and places it in the memory prescribed by the assembler.

- This scheme makes more memory available to the user because assembler is not in the memory at load time.
- Figure below illustrates the operation of an absolute assembler and an absolute loader.
- The programmer must be careful not to assign two subroutines to the same or overlapping locations.
- The MAIN program is assigned to locations 100-247 and the SQRT subroutine is assigned to locations 400-477.
- If changes were made to MAIN that increased its length to more than 300 bytes, the end of MAIN (at 100 + 300 = 400) would overlap the start of SQRT (at 400). It would then be necessary to assign SQRT overlap the start of SQRT (at 400). It would then be necessary to assign SQRT to a new location by changing its START pseudo-op card and reassembling it.
- Furthermore, it would also be necessary to modify all other subroutines that referred to the address of SQRT.
- In situations where dozens of subroutines are being used, this manual "shuffling" can get very complex, tedious, and wasteful of core.

MAIN program

```
MAIN    START   100
        BALR    12, 0
        USING   MAIN+2,12
        :
        L       15, ASQRT   } Call
        BALR    14,15       } SQRT
        :
ASQRT   DC      F'400'      } Adress
        END                 } of SQRT
```

SQRT subroutine

```
SQRT    START   400
        USING   *, 15
        :                   } Compute
        :                   } square root
        BR      14) Return
        END
```

SOURCE DECK INPUT TO ABSOLUTE ASSEMBLER

MAIN program

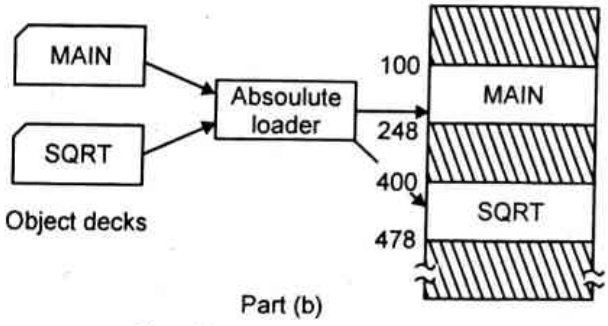
Loc-ation	Instruction
100	BALR 12,0
102	:
:	:
120	L 15,142(0,12)
124	BALR 14, 15
126	:
:	:
244	F'400'
248	

SQRT subroutine

```
400     :       :
        :       :
476     BCR     15, 14
478     :
```

OBJECT DECK OUTPUT FROM ABSOLUTE ASSEMBLER

Part (a)



Part (b)

Fig. Absolute loader example

- If the changes were made to a certain subroutine that increased its size to such an extent that its end may overlap the start of some other subroutine, then it would be necessary to shift this subroutine to some new location.
- Furthermore all subroutines that referred to changed subroutine have to be modified.
- In situations where there are dozens of subroutines, this manual shuffling can get very complex, tedious and wasteful of core.
- The four loader functions accomplished in a absolute loader scheme are as follows :
  1) Allocation - By programmer.
  2) Linking - By programmer.
  3) Relocation - By assembler.
  4) Linking - By loader.

  Advantage : Absolute loader is simple to implement.

  Disadvantages :

(1) The programmer must specify to the assembler the address of the memory where the program is to be loaded.

(2) If there are multiple subroutines, the programmer must remember the address of each and use that absolute address explicitly in his other subroutines to perform subroutine linkage.

Q.5. Draw and explain general loader scheme.

Ans. General Loader Scheme :

- Outputting the instructions and data as they are assembled circumvents the problem of wasting core for the assembler. Such an output could be saved and loaded whenever the code was to be executed.
- The assembled programs could be loaded into the same area in core that the assembler occupied, (since the translation will have been completed).

- This output form, which may be on cards containing a coded form of the instructions, is called an object deck.
- The use of an object deck as intermediate data to avoid one disadvantage of the preceding "compile-and-go" scheme requires the addition of a new program to the system, a loader (Fig.).
- The loader accepts the assembled machine instructions, data, and other information present in the object format, and places machine instructions and data in core in an executable computer form.
- The loader is assumed to be smaller than the assembler, so that more memory is available to the user.
- A further advantage is that reassembly is no longer necessary to run the program at a later date.
- Finally, if all the source program translators (assemblers and compilers) produce compatible object program deck formats and use compatible linkage conventions, it is possible to write subroutines in several different languages since the object decks to be processed by the loader will all be in the same "language" (machine language).

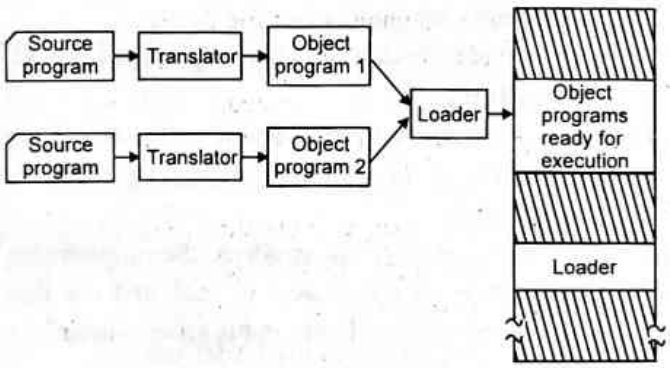


Fig. General loader scheme

Q.6. Write short note on subroutine linkages.

Ans. Subroutine linkages : The problem of subroutine linkage can be stated as :

- A main program 'A' wishes to transfer control to subroutine 'B'. The programmer in program 'A' can write a transfer instruction to sub program 'B' by

  BALR 14, B

  (i.e. load register 14 with current address and branch to address mentioned 'B')
- Alternatively we could have used statement

  L 15, = A(B)

  BALR 14, 15
- Here the assembler doesn't know the address of B and therefore assembler will generate error unless a special mechanism is provided.
- The assembler pseudo op EXTERN is normally used followed by a list of symbols.
- This indicates list of symbols defined in other program but referenced in the present program.
- Similarly variables in a program which are treated as externals in other program are normally indicated to assembler with the help of ENTRY statement.
- **For example :** The following sequence of instructions may be a simple calling sequence to another program :

```
MAIN    START
        EXTRN     SUBROUT
        ----------
        ----------
        L         15, = A (SUBROUT)
        BALR      14, 15
        •
        •
        •
        END
```

- The above sequence of instructions first declares SUBROUT as an external variable, that is, a variable referenced but not defined in this program.
- The load instruction loads the address of that variable into register 15.
- The BALR instruction branches to the content of register 15, which is the address of SUBROUT and leaves the value of next instruction in register 14.
- Linkage is resolving address of each symbol defined as.
- This can be done at different point of time like at the time of executing the program by the programmer, as in absolute loader or at the time of assembling as in "compile and go" or at the time of loading, as in direct linking or, at the execution as in dynamic linkage.
- The subroutine can have multiple entry points.

The advantages are :

1) Common loading (one or more function can be written as one single subroutine)
2) Collecting together related subroutine for easy use.
3) Better or convenient access to common database.

Q.7. How relocating loader performs the function of loader? Explain in details. *W-09(4M)*

OR Explain the relocating loader in detail with advantages and disadvantages. *S-13(8M)*

Ans. Relocating loader :

- Relocating loader is a loader that avoids possible reassembling of all subroutines when a single subroutine changes, and performs the task of allocation, relocation and linking for the programmer.
- Relocation is the process of updating the addresses used in the address sensitive instructions of a program. It is necessary that such a modification should help to execute the program from designated area of the memory.
- Operating systems need relocating loaders, which adjust addresses (pointers) in the executable to compensate for variations in the address at which loading starts.
- The relocating loader will load the program anywhere in memory, altering the various addresses as required to ensure correct referencing.
- Loaders that allow for program relocation are called relocating loaders/relative loaders.
- In this the assembler assembles each procedure segment independently and passes onto the loader the text and information as to relocation and intersegment references.
- For each source program the assembler outputs a text prefixed by the transfer vector that consists of addresses containing the names of subroutines referenced by the source program.
- The assembler would also provide to the loader the length of the program and length of the transfer vector. The loader will load each subroutine identified in the transfer vector.
- It will then place a transfer instruction to the corresponding subroutine in each entry.

Advantage :

Relocation bits are used to solve the problem of relocation, transfer vector is used to solve the problem of linking and program length info to solve allocation.

Disadvantages :

(1) Relocating loader is not suited for loading external data.
(2) Transfer vector increases the size.
(3) It does not facilitate access to data segments that can be shared.

Q.8. Write a short note on BSS (Binary symbolic subroutine) loader. *W-10(4M)*

Ans. Binary Symbolic Subroutine Loader :

- Binary Symbolic Subroutine (BSS) Loader was one of the first in about 1956. Before relocating loaders programmers could not easily write modules in different languages.
- The BSS loaders permit multiple program segments in different languages to be compiled at different times. It required all the translators to produce a common object file format.
- BSS loaders also allowed the development of independent (and separate) data segments [FORTRAN BLOCK DATA].
- To avoid possible re-assembling of all subroutines when a single subroutine is changed, and to perform the tasks of allocation and linking for the programmer.
- An example of relocating loader scheme is that of the Binary Symbolic Subroutine (BSS) loader that was used in the IBM 7094, IBM 1130, GE 635 and UNIVAC 1108.
- The BSS loader allows many procedure segments, yet only one data segment (common segment). The assembler assembles each procedure segment independently and passes on to the loader the text and information as to relocation and inter-segment references.
- The output of a relocating assembler using a BSS scheme is the object program and information about all other program it references.

- The BSS loader is often used on computers with a fixed length direct address instruction format.
- **For example :** If the format of the IBM360 RX instruction are as follows :

OP	R1	X1	A2

Where A2 was the 16 bit absolute address of the operand, this would be a direct address instruction format. Such a format works if there is less than 2**16, 65,536 bytes of storage, as was the case with most of the early computers.

Disadvantages of the BSS loader scheme :

(1) The transfer vector linkage is only useful for transfers, and is not well suited for loading or storing external data (load located in another procedure segment).

(2) The transfer vector increases the size of the object program in memory.

(3) The BSS loader, as described, processes procedure segment but does not facilitate access to data segments that can be shared. This feature can COMMON. This facility is usually implemented by extending the relocation bits scheme to use 2 bits per half word address field if the bits are 01, the half word is relocated relative to the procedure segment, and if they are 10, it is relocated relative to the address of the single common data segment. If the bits are 00 or 11, half word is not relocated.

Q.9. Explain relocation bits.

Ans. Relocation bits :

- To relocate the address portion of every instruction, computers with the direct-address instruction format have much more severe relocation problem than IBM 360 machines.
- In the absence of 360-type base registers, this problem is often solved by the use of relocation bits.
- The assembler associates a bit with each instruction or address field. If this bit equals 1, then the corresponding address field must be relocated. Otherwise the field is not relocated.
- Relocation bits are included on object decks to provide information to the loader about the instructions that need to be relocated.

Q.10. What are direct-linking loaders? What is their advantage? What information about the procedure or data segment must be provided by the assembler in this case?

Ans. Direct-linking loader :

- The direct-linking loader is general relocating loader and is perhaps the most popular loading scheme presentably used.
- The direct linking loader has the advantage of allowing the programmer multiple procedure segments as well as multiple data segments and of giving him complete freedom in referencing instructions or data contained in other segments. This provides flexible intersegment referencing and accessing ability, while at the same time allowing independent translation of programmer.
- The assembler in direct linking loader scheme provides following information regarding procedure or data segments :

(1) The length or a segment.

(2) A list of all symbols in the current program segment that may be referred by other segments and their relative location within the current segment. These symbols are defined by ENTRY statement and are called public definitions.

(3) A list of all symbols defined in other program segment that may be referred in current program segment (External references). These symbols are defined by EXTERN statement.

(4) Information as to where address constants are located in the segment and a description how to revise their values.

(5) The machine code translation of source program and the relative addresses assigned.

Q.11. Draw and explain Flowchart of Pass 1 of loader.

Ans. Flowchart for Pass 1 of the DLL is shown in figure below :

- The purpose of the Pass 1 is to assign a location to each segment, and to define the values of all external symbols.
- The main aim is to minimize the amount of memory required for the total program hence we will assign each segment the next available location after the preceding segment .

- It is necessary for the loader to know where it loads the first segment.
- This address is called the Initial Program Load Address (IPLA) which is normally determined by the operating system but in some of the systems the IPLA may be specified by programmer.
- In Pass 1 algorithm initially Program Load Address (PLA) is set to IPLA.
- An object card is then Read and a copy of this object card is written for the use by Pass 2.
- This object card is then checked for the various types such as ESD, TXT, RLD, END or LDT/ EOF.

1) If it is a TXT or RLD card, no processing is required for Pass 1. So the next card is read.
2) If it is an ESD card, initially it is checked for the type i.e. External symbol ER.

(a) SD-Segment definition or LD-Local Definition is a segment definition. ESD card is read, the length field LENGTH, from the card is temporarily saved in the variable SLENGTH the VALUE, to be assigned to this symbol is set to the current value of PLA.

The symbol and its assigned value is then stored in the GEST, if the symbol is already existed in GEST, there must have been a previous SD or LD with the same name hence this is an Error.

The symbol and its value are printed as part of the Load-map.

(b) A similar process is used for LD ESD. The value to be assigned is set to the current PLA plus the relative address (ADDR) indicated on ESD.

(c) The ER symbols do not require any processing during Pass 1.

3) If an END card is encountered, the PLA is incremented by the length of the segment and saved in SLENGTH; becoming the PLA for the next segment.
4) If it is LDT or EOF, card is finally read, Pass 1 is completed and control transfers to Pass 2.

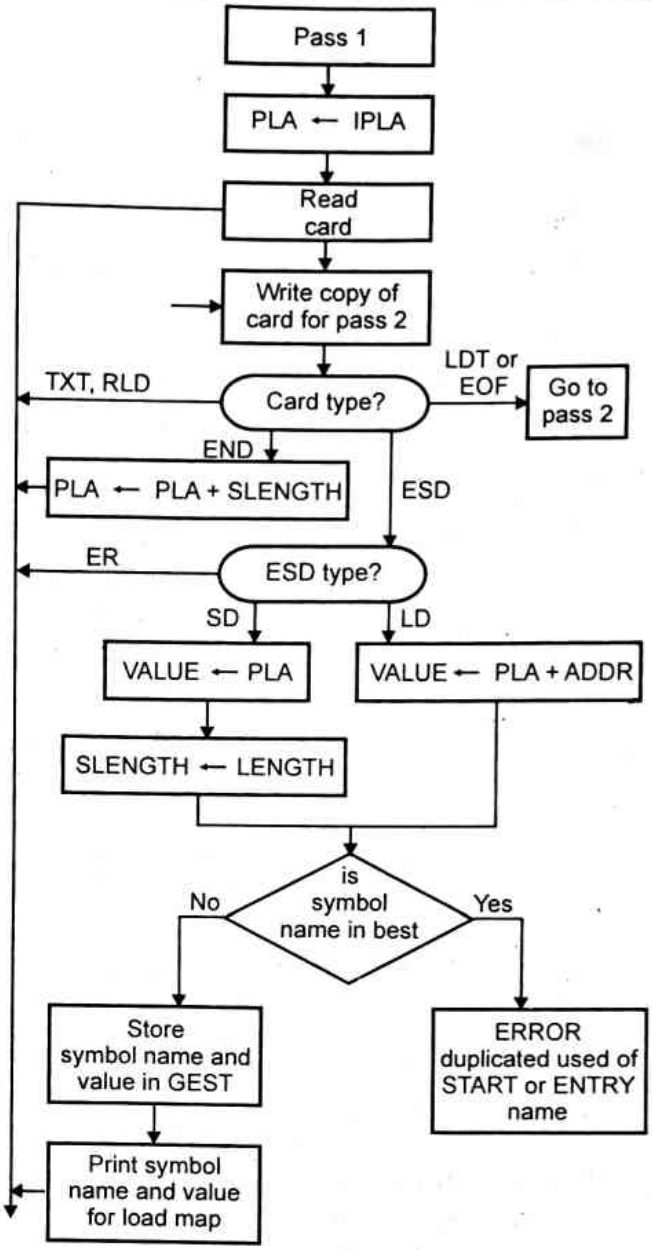


Fig. Detailed pass 1 flowchart

Q.12. Why 2 passes are required by direct linking loader? Draw and explain flowchart of pass 2 of loader. *S-10(9M)*

Ans.

- The linking process is performed in two passes.
- Two passes are necessary because the linker may encounter a forward reference before knowing its address.
- So it is necessary to scan all the DEFINITION and USE table at least once.

- Linker then builds the Global symbol table with the help of USE and DEFINITION table.
- In Global symbol table name of each externally referenced symbol is included along with its address relative to beginning of the load module.
- And during pass 2, the addresses of external references are replaced by obtaining the addresses from global symbol table.

Pass 2 :

(1) Load text and relocate/link address constants. After all the segments have been assigned locations and the external symbols have been defined by pass 1, it is possible to complete the loading by loading the text and adjusting (relocation or linking) address constants.

(2) At the end of pass 2, the loader will transfer control to the loaded program.

(3) The following simple rule is often used to determine where to commence execution :

(a) If an address is specified on the END card, that address is used as the execution start address.

(b) Otherwise, execution will commence at the beginning of the first segment.

- At the beginning of pass 2 the program load address is initialized as in pass 1, and the execution start address (EXADDR) is set to IPLA.
- The cards are read one by one from the object deck file left by pass 1. Each of the five types of cards is processed differently, as follows:

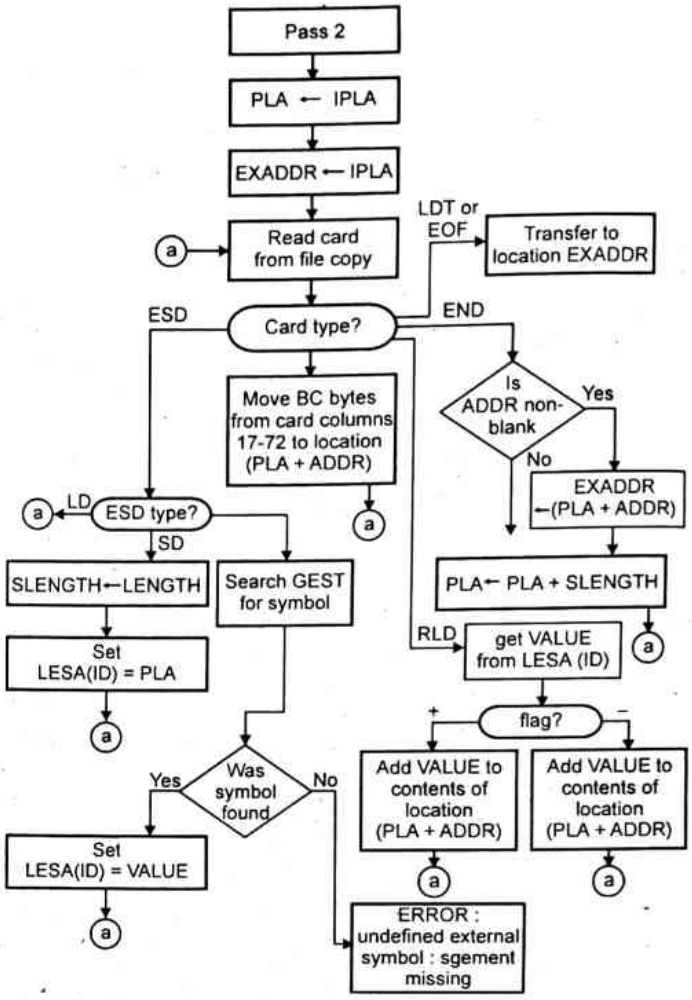
1) **ESD CARD :** Each of the ESD card types is processed differently.

i) **SD-type ESD :** The Length of the segment is temporarily saved in the variable SLENGTH. The appropriate entry in the local external symbol array, LESA(ID), is set to the current value of the Program Load Address.

ii) **LD-type ESD :** The LD-type ESD does not require any processing during pass 2.

iii) **ER-type ESD :**

- The Global External Symbol Table (GEST) is searched for a match with the ER symbol. If it is not found, the corresponding segment or entry must be missing - this is an error.
- If the symbol is found in the GEST, its value is extracted and the corresponding Local External Symbol Array entry, LES(ID), is set equal to it.



2) **TXT CARD :** When a TXT card is read, the text is copied from the card to the appropriate relocated core location (PLA + ADDR).

3) **RLD CARD :** The value to be used for relocation and linking is extracted from the local external symbol array as specified by the ID field, i.e., LESA(ID). Depending upon the flag setting (plus or minus) the value is either added to or subtracted from the address constant. The actual relocated address of the address constant is computed as the sum of the PLA and the ADDR field specified on the RLD card.

4) **END CARD :** If an execution start address is specified to the END card, it is saved in the variable EXADDR after being relocated by the PLA. The Program Load Address is incremented by the length of the segment and saved in SLENGTH, becoming the PLA for the next segment.

5) **LDT/EOF CARD :** The loader transfers control to the loaded program at the address specified by current contents of the execution address variable (EXADDR).

OTHER LOADING SCHEMES : BINDER, LINKING, OVERLAYS AND DYNAMIC BINDER

Q.13. Write short note on binder and module loader. *W-09,12(4M), S-13(3M)*

Ans. Binder and module loader :

- The program which performs allocation, relocation and linking is called binder.
- The binder performs relocation, creates linked executable text and stores this text in a file in some systematic manner.
- Such kind of module prepared by the binder execution is called load module.
- This load module can then be actually loaded in the main memory by the loader. This loader is also called as module loader.
- If the binder can produce the exact replica of executable code in the load module then the module loader simply loads this file into the main memory which ultimately reduces the overall time complexity.

Advantage :

The advantage of core image binder or linkage editor is that a program is to be bound only once and it can be used any number of times and loaded without the need to bind once again.

Q.14.Explain the following terms :

(i) Transfer vector (ii) Binder

(iii) Overlay. *W-10(3M)*

Ans.

(i) Transfer Vector :

- The translator and the loader do LINKING as a joint activity by use of the Transfer Vector (TV).
- The translator set asides an area at the start of each object file to serve as a TV area.
- The purpose of the TV is to isolate linking commands to a specific area.

(ii) Binder :

- A binder is a program that performs the same function as the direct linking loader in "binding" subroutines together but rather than replacing the relocated and linked text directly into memory, it outputs the text as a file or card deck.
- This output file is in a format ready to be loaded and such format is called as load module.

(iii) Overlay :

- The subroutine of a program are needed at different times.
- For example pass1 and pass2 of an assembler are mutually exclusive.
- By explicitly recognizing which subroutines call other subroutines, it is possible to produce an overlay structure that identifies mutually exclusive subroutines.

Q.15. Write short note on dynamic linking and dynamic loading. *W-10(4M)*

OR Explain LOCAL loader. *W-11(5M)*

Ans.

(1) Dynamic Linking :

- The process of postponing linking and loading of external reference until execution is called dynamic linking.
- In dynamic linking, the binder first prepares a load module in which along with program code the allocation and relocation information is stored.
- The loader simply loads the main module in the main memory.
- If any external reference to a subroutine comes, then the execution is suspended for a while, the loader brings the required subroutine in the main memory and then the execution process is resumed.
- Thus in dynamic linking both the loading and linking is done dynamically.

Advantages :

(1) The overhead on the loader is reduced. The required subroutine will be load in the main memory only at the time of execution.

(2) The system can be dynamically reconfigured.

Disadvantages :

(1) The linking and loading need to be postponed until the execution.

(2) During the execution if at all any subroutine is needed then the process of execution needs to be suspended until the required subroutine gets loaded in the main memory.

(2) **Dynamic Loading :**

- To obtain better memory-space utilization, we can use dynamic loading.
- With dynamic loading, a routine is not loaded until it is called.
- All routines are kept on disk in a reloadable load format.
- The main program is loaded into memory and is executed. When a routine needs to call another routine, the calling routine first checks to see whether the other routine has been loaded.
- If not, the reloadable linking loader is called to load the desired routine into memory and to update the program's address tables to reflect this change. Then control is passed to the newly loaded routine.
- The overlay structure to work, it is necessary for the module loader to load the various procedure as they are needed.
- There are many binder capable of processing and allocating an overlay structure.
- The portion of the loader, the actually intercepts, the "calls" and loads the necessary procedure is called the overlay supervisor or simply the flipper.
- This overall scheme is called dynamic loading or LOAD-ON-CALL(LOCAL)

Advantages :

1) The advantage of dynamic loading is that an unused routine is never loaded.
2) Dynamic loading does not require special support from the OS. Operating systems may help the programmer, however, by providing library routines to implement dynamic loading.

Disadvantages :

(1) With dynamic loading a routing is not loaded until it is called.
(2) All routines are kept on disk in a relocatable load format.
(3) The main program is loaded into memory and is executed. When a routine needs to call another routine, the calling routine first checks to see whether the other routing has been loaded.
(4) If not, the relocatable linking loader is called to load the desired routing into memory and to update the program's address tables to reflect this change. Then control is passed to the newly loaded routing.

Q.16. Write short note on LINK EDITOR.

W-11(3M), S-10,12,13(4M)

Ans. Link editor :

- Link editor keeps track of the relocation information so that the resulting load module as an ensemble.
- In linkage editor the module loader must perform allocation and relocation as well as loading, but it does not have complex problems of linking.
- The linkage editor binder is more complex but allows a more flexible allocation and loading scheme.

Functions :

(1) Replacement of subroutine in linkage program.
(2) Linkage editor combine common libraries at the execution time.

Q.17. Differentiate between linking loader and linkage editor.

Ans.

Sr. No.	Linking loader	Linkage editor
(1)	Performs linking and relocation at load time.	Linking is done prior to load time.
(2)	Loads the linked program directly into the memory. Therefore steps for writing and reading the linked program are avoided.	Writes a linked version of program, into a file which is later executed by the relocation loader.
(3)	Linking loader searches libraries and resolves external references every time the program is executed.	Resolution of external references and Library searching are performed only once in linkage editor.

(4)	Used in situations where program has to be reassembled for every execution. **Example :** Program development and testing	Linkage editor can be efficiently used when a program is to be executed many times without being reassembled, each time.
(5)	Linking loader has less flexibility and control.	Linkage editors offer more flexibility and control.
(6)	Less complexity and overhead.	Increase in complexity and overhead.

CONCEPT OF STATIC AND DYNAMIC RELOCATION

Q.18. Explain concept of static and dynamic relocation.

Ans.

- Memory allocation is an aspect of a more general action in software operation known as binding in terms of system programming.

  Static Binding : A binding performed before the execution of a program (or operation of a software system) is set in motion is known as static binding.

  Dynamic Binding : A binding performed during the execution of a program (or operation of a software system) is known as dynamic binding.
- Relocation may be performed in two ways :

  (1) Static (before program is executed)

  (2) Dynamic (during program execution)
- Alternative 1 : Suspend execution and relocate.
- Alternative 2 : Use a relocation register.
- Static allocation is performed by compiler, linker, or loader.
- Sizes of data structures must be known a priori.
- Dynamic allocation provides flexibility.
- Memory allocation actions constitute on overhead during operation.

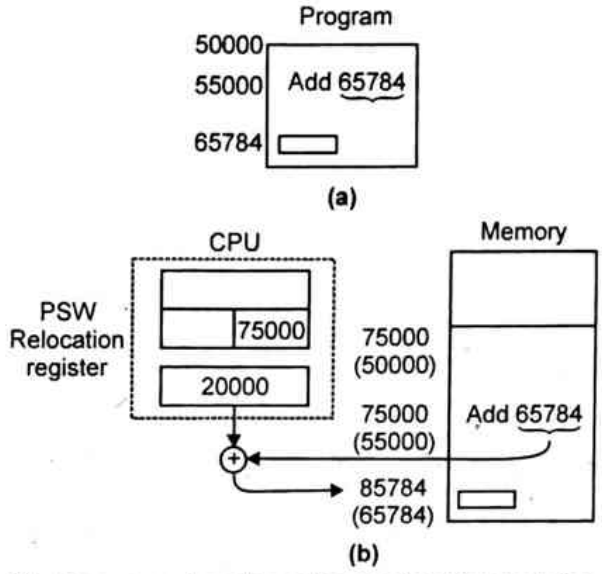


Fig. Program relocation using a relocation register (a) Program, (b) its execution.

EXTERNAL SYMBOLS

Q.19. What is the order of the RLD, ESD, TXT and END cards in an object deck? Why are they in this order? Also describe function of each of RLD, ESD, TXT and END cards.

S-10(4M)

Ans.

- The order of RLD, ESD, TXT and END cards in object deck is as follows :

  (a) ESD **(b)** TXT **(c)** RLD **(d)** END
- The loader is responsible for the activities such as allocation, linking, relocation and loading. Hence to follow these function of loader the cards in object deck are in this sequence.

(a) ESD : External Symbol Directory (ESD) cards contain information about all symbols that are defined in the present program, but that may be referenced elsewhere and all symbols referenced in this program but defined elsewhere.

(b) TXT : The Text (TXT) card contains the actual object code translated version of source program. The TXT card contains the actual assembled programs. The format and use of these cards are similar to those for the absolute loader.

(c) **RLD :** The Relocation and Linkage Directory (RLD) card contains information about those locations in the program whose contents depends on the addresses at which the programs are placed. For such locations, the assembler must supply information enabling the loader to correct their contents.

- The RLD cards contain the following information :

(1) The location of each constant that needs to be changed due to relocation.

(2) By what it has to be changed.

(3) The operation to be performed.

(d) **END :** The END card indicates the end of object and specifies starting address for execution of assembled program.

Q.20. Give the format of GEST and LESA used in direct linking loader.

OR Write short note on following :
(1) GEST (2) LESA

OR Explain necessity of GEST and LESA for implementing a two pass linker.

Ans.

(1) GEST :

- GEST (Global external symbol table) is used to store each external symbol and its corresponding assigned core address.
- The Global External Symbol Table (GEST), prepared by pass 1, contains each external symbol and its corresponding absolute address value.
- The Global External Symbol Table (GEST) is used to store the external symbols defined by means of a Segment Definition (SD) or Local Definition (LD) entry on an External Symbol Dictionary (ESD) record. When these symbols are encountered during pass 1, they are assigned an absolute core address; this address is stored, along with the symbol, in the GEST.
- The external symbol table has the same format as the local symbol table, except an offset (ifd) field has been added to the file descriptor table. This is used to locate information associated with the symbol in an auxiliary symbol table.
- Table below shows the format of external symbol table :

Declaration	Name	Description
SYMR	asym	Same as local symbol table
unsigned short	weakext : 1	Symbol is weak external
unsigned short	reserved : 15	Reserved for future use
int	ifd	Pointer to entry in file descriptor table

GEST : Value = PLA + ADD

Example with format :

EXTERNAL SYMBOL	ASSINGNED CORE ADDR.
X	104
X1(104+20)	124
X2	134
Y	168
Y1	184

(2) LESA :

- Local External Symbol Array(LESA) is used to establish correspondence between ESD ID used in ESD and RLD cards.
- An array, the Local External Symbol Array (LESA), which is used to establish a correspondence between the NAMES, used on ESD and RLD records, and the corresponding external symbol's absolute address.
- If the symbol is found in the GEST, its value is extracted and the corresponding Local External Symbol Array entry, LESA(ID), is set equal to it.
- **Format of a Local Symbol Table Entry :**

Declaration	Name	Description
long	Value [Table, Note 1]	Value of symbol
int	iss[Table Note 2]	Index into local strings of symbol name
unsigned	st : 6[Table Note 3]	Symbol type

unsigned	sc : 5 [Table Note 4]	Storage class
unsigned	reserved : 1	N/A
unsigned	index : 20 [Table Note 5]	Index into local or auxiliary symbols

Example with Format :

ID number	ASSINGNED CORE ADDR.
1	104
1	124
1	134
2	-------

- An integer representing an address, size, offset form a frame pointer. The value is determined by the symbol type.
- The index into string space (iss) is an offset from the issBase field of an entry in the file descriptor to the name of the symbol.
- The symbol type (st) defines the symbol.
- The storage class (sc), where applicable, explains how to access the symbol type in memory.

Q.21. How ID numbers are provided on ESD card? What is its advantage? Why the ID numbers are not provided to locally defined symbol? ***W-11(5M)***

OR What is purpose of using ID number on the ESD card? Why it is not needed for locally defined symbol?

Ans.

- The external symbols are used for relocation or linking is identified on the RLD cards by means of an ID number rather than the symbol's name.
- The ID number must match on SD or ER entry on the ESD cards.
- This techniques save space on the RLD cards and speeds processing by eliminating many searches of the Global External Symbol Table.
- To establish a correspondence between an ID number on an RLD card and the absolute core address value, the ESD card contain the ID numbers and symbol they correspond to, while the information relating these symbols to absolute core address values may be found in GEST.
- In pass2 of the loader the GEST and ESD information for each individual object deck is merged to produce the local external symbol array that directly related ID number and value.
- This principle is necessary to create separate LESA for each segment, but LESA's are not only produced one at a time, the same array can be reused for each segment.
- It is not necessary to search the LESA; given an ID number the corresponding value is written as LESA (ID) and can be immediately obtained.

DESIGN OF LINKER

Q.22. What is a linker? Why is it required? Give its function.

Ans. Linker :

- Generally, a modular approach is used to develop reasonably sized software.
- In this approach, the software is divided into a number of smaller subprograms called modules and separate source programs are written for each module of the software.
- It is easier to develop, test and debug smaller programs.
- Each source program file can be modified and compiled independent of other source program files to create a corresponding object program file.
- A linker is a program that is used to properly combine all the object program files (modules) of the software, and to convert them into the final executable program, which is called a load module.
- It means that a linker takes object program file (modules), and fits them together to assemble them into the program's final executable form.

Functions of Linker :

(i) Linking of the program with other programs needed for its execution.

(ii) Relocation of the program to execute from the specific memory area allocated to it.

Q.23. Explain the design of a linker in detail.

Ans. Design of a linker :

- The design of a program named LINKER which performs both linking relocation is given below.
- The output of LINKER is a binary program which resembles a program with .COM extension in MS DOS.

  Specification : The LINKER invocation command format is as follows :

  LINKER <object module names>, <executable file>. <load origin>, <list of library files>
- LINKER performs relocation and linking of all named object modules to produce a binary program with the specified load origin.
- The program is stored in <executable file>.
- When LINKER comes across an external name ext_symb_i, which is not defined in any of the object modules named in the LINKER command, then it locates an object module om_i in <list of library files> which contains ext_symb_i, as a public definition. om_i is now included in the set of object modules to be linked and relocated.
- This process of resolving an external reference by including the object module containing its definition from a library file is called auto-linking.
- LINKER execution terminates when all external references have been resolved, or when the unresolved references cannot be found in any of the library files.

Data Structures and Algorithm :

- LINKER uses a two pass strategy.
- In the first pass, the object modules are processed to collect information concerning segments and public definitions.
- The second pass performs relocation and linking.

First Pass : In the first pass, LINKER only processes the object records relevant for building NTAB.

Algorithm : First Pass or LINKER

(i) program_linked_origin : = <load origin>;
(use a default value if <load origin> is not specified)

(ii) Repeat step (iii) for each object module to be linked.

(iii) Select an object module and process its object records.

(a) If an LNAMES record, enter the names in NAMELIST.

(b) If a SEGDEF record,

(1) i : = name index; segment_name : = NAMELIST [I];

(2) If an absolute segment, enter (segment_name, segment_addr) in NTAB.

(3) If the segment is relocatable and cannot be combined with other segments – Align the address contained in program_liked_origin on the next word or paragraph as indicated in the attributes field.
- Enter (segment_name, program_linked_origin) in NTAB.
- program_linked_origin : = program load origin + segment length;

(c) For each PUBDEF record

(1) i : = base; segment_name : = NAMELIST [i];symbol : = name;

(2) segment_addr : = segment_addr + offset;

(3) sym_addr : = segment _addr + offset;

(4) Enter (symbol, symb_addr) in NTAB.

Second Pass :

- Second pass of LINKER constructs the executable program in work_area.
- Data from the LEDATA records is moved to appropriate parts of work_area.
- FIXUPP records are then processed to effect relocation and linking.
- At the end of pass, the executable program is written into the current directory under the name <executable file> specified in the LINKER command.
- It is simple to move the data from an LEDATA record into the appropriate part of work_area using segment index and data offset.
- The segment index is used to obtain the load origin of the segment from NTAB.

- Adding the data offset to it gives load address of the first byte of the data in the LEDATA record.
- Now, the address of this byte in work_area is obtained by computation.
- The segment index contains a numeric value rather than the name of a segment.
- The segment name is obtained by using this value as an index into LNAMES record(s).
- This name is reached in NTAB to obtain its load origin.
- Similarly, while processing a FIXUPP record, the target datum can be used as an index into EXTDEF or SEGDEF records to obtain the target name.
- This name is searched in NTAB to obtain its load origin.
- The second pass constructs a number of tables to eliminate indirect references to NTAB.
- To eliminate search in the LNAMES records, the NAMELIST table can be constructed once again in the second pass, and segment index can be used to index NAMELIST.
- However, the search in NTAB is still required.
- To overcome this problem, LINKER builds another table called the segment table (SEGTAB) to contain all segment names defined in the object module.
- A SEGTAB entry has the following format :

segment name	load address

where the load address information is copied from NTAB.

- Now, while processing on LEDATA record, the segment index can be used to index SEGTAB to obtain the segment's load address.
- A similar use can be made while processing FIXUPP records.
- Because a linking specifications in a FIXUPP record may contain a reference to an external symbol, LINKER builds an external symbols table (EXTAB) with a similar entry-format, viz.

external symbol	load address

- EXTAB is built by processing the EXTDEF records and copying the load addresses from NTAB.

Algorithm : Second Pass of LINKER

(i) list_of_object_modules : = Object modules named in LINKER Command ;

(ii) Repeat step (iii) until list_of_object_modules is empty.

(iii) Select an object module and process its object records.

(a) If an LNAMES record
Enter the names in NAMELIST

(b) If a SEGDEF record
i : = name index; segment_name : = NAMELIST [i];
Enter (segment_name, load address from NTAB) in SEGTAB.

(c) If an EXTDEF record

(1) external_name : = name from EXTDEF record;

(2) If external_name is not found in NTAB, then
– Locate an object module in the library which contains external_name as a segment or public definition.
– Add name of object module to list_of_object_modules.
– Perform first pass of LINKER

(3) Enter (external_name, load address from NTAB) in EXTTAB.

(d) If an LEDATA record

(1) i : = segment index; d : data offset;

(2) program_load_origin : = SEGTAB [i]. loadaddress;

(3) address_in_work_area : = address of work_area+ program_load_origin–<load origin> + d;

(4) Move data from LEDATA into the memory area starting at the address address_in_work_area.

(e) If a FIXUPP record, for each FIXUPP specification

(1) f : = offset form locat field;

(2) fix_up_address : = address_in_work_area + f;

(3) Perform required fix up using a load address from SEGTAB or EXTTAB and the value of code in locat and fix dat.

(f) If a MODEND record

If start address is specified, compute the corresponding load address (analogous to the computation while processing an LEDATA record) and record it in the executable file being generated.

DESIGN OF OBJECT FILE FOR DIFFERENT LOADING SCHEMES

Q.24. Explain the following types of address constant with example.
(i) Absolute
(ii) Simple relocatable
(iii) Complex relocatable
and what role they play in design of loader?

W-09(6M), W-12(7M)

Ans.

- In IBM System/360 through present day z/Architecture, an address constant or "adcon" is an assembly language data type which contains the address of a location in computer memory.
- An address constant can be one, two, three or four bytes long, although an adcon of less than four bytes is conventionally used to hold expression for a small integer such as a length, a relative address, or an index value, and does not represent an address at all.
- Address constants are defined using an assembler language "DC" statement.
- An address constant can be :
  1) Absolute
  2) Simple relocatable
  3) Complex relocatable.

(1) Absolute :

- If LOC1 and LOC2 are two relocatable symbols defined internal to program, the assembler can calculate the actual value, their differences.
- An expression is absolute if it refers to labels in a program, and the value of the expression is constant regardless of the location of the program.
- An absolute expression can be completely evaluated at assembly time and does not require further processing by the linkage editor.

For example : 'DC A (4900796)' has an absolute nominal value.

(2) Simple relocatable :

- If LOC1 is relocatable symbol within this procedure and LOC2 is an absolute number (eg : LOC2 EQU5), the assembler can calculate the difference between relative location of LOC1 and value of LOC2 but the loader must perform relocation by adding the program address.
- A relocatable expression is one that contains one or more terms that require relocation by the linkage editor when the program is linked, for example, in the following code 'ADCON' has a relocatable nominal value.

(3) Complex relocatable :

- If LOC1 and LOC2 are entries to some other program, the assembler can do nothing and the value must be calculated by the loader.
- A complex relocatable expression contains terms that relate to addresses in different source modules.
- **For example :** 'DC A(X–Y)' where 'X' and 'Y' are in different modules.

Example :

```
DOG    DS   CL5
CAT    DS   CL7
PIG    DS   CL9
ADR1   DC   A(PIG)          RELOCATABLE
ADR2   DC   A(PIG – DOG)    ABSOLUTE
ADR3   DC   A(PIG + DOG)    COMPLEX
                            RELOCATABLE
```

LOADER PROBLEMS

Q.25. For the program given below, show the contents of ESD card and RLD card. Also show the contents in GEST. *W-10(10M)*

```
PGA     START
        ENTRY    PGA1
        ENTRN    PBG, PGB2
        DC       A(PGA), A(PGB+4)
PGA1    DC       A(PGA1 – PGA)
        END      A(PGB2 – PGA)
```

```
PGB     START
        ENTRY   PGB2
        ENTRN   PGA1
PGB1    DC      A (PGB1-4), A(PGA1)
PGB2    DC      A (PGB+4), A(PGB1- PGB)
PBG3    DC      A(PGA1+PGB-16)
        END
```

Ans.

ESD card

Ref. No.	Symbol	Type	Relative Location	Length
01	PGA	SD	0	16
02	PGA1	LD	8	-
03	PGB	ER	-	-
03	PGB2	ER	-	-

TXT card

Ref. No.	Relative	Object code
4	0-3	Known
4	4-7	4
5	8-11	8-unknown
5	12-15	Unknown

RLD card

Ref. No.	ESD ID	Length	Flag	Ref. address
4	01	4	+	0
4	01	4	+	4
5	02	4	+	8
5	02	4	+	12

GEST : (global symbol table)

External Symbol (8 Bytes Character)	Assigned Core Address (4 Bytes Decimal)
"PGAbbbbb"	104
"PGA1bbbb"	148
"PGBbbbbb"	160
"PGB2bbbb"	176
"PGB1bbbb"	180

Q.26.For the following program write the content of ESD and RLD cards.

```
Card
No
 1                JOHN    START
 2                ENTRY   RESULT
 3                EXTERN  SUM
 4                BALR    12, 0
 5                USING   *, 12
 6                ST      14, SAVE
 7                L       1, POINTER
 8                L       15, SUM
 9                BALR    14, 15
10                ST      1, RESULT
11                L       14, SAVE
12                BR      14
13   TABLE        DC      14
14   POINTER      DC      F' 1, 7, 9, 10,
                          31
15   RESULT       DS      A' (TABLE)
16   SAVE         DS      F
17   ASUM         DC      F
18                END     A(SUM)
```

Ans.

ESD card

Ref. No.	Symbol	Table	Relative Location	Length
1	JOHN	SD	0	64
2	RESULT	LD	52	-
3	SUM	ER	-	-

RLD card

Ref. No.	Symbol	Flag	Length	Relative Location
14	JOHN	+	4	48
17	SUM	+	4	60

TXT card

Ref. No.	Relative Location	Object code
4	0	BALR 12, 0
12	24	BCR 15, 14
13	28	1
13	32	7
13	36	9
13	40	10

13	44	3
14	48	28
17	60	70 (sum)
6	2	L 1, 46 (0, 12)
7	6	L 15, 58 (0, 12)
8	10	BALR 14, 15
9	14	ST 1, 50 (0, 12)
10	16	L 14, 54 (0, 12)
11	20	BCR 15, 14

Q.27. For the assembly language program shown below.

```
PROG    START
        ENTRY     RESULT
        EXTRN     SUM
        BALR      12, 0
        USING     *, 12
        ST        14, SAVE
        L         1, PTR
        L         15, ASUM
        BALR      14, 15
        ST        I, RESULT
        L         14, SAVE
        BR        14
TABLE   DC 'F, 7, 9, 10, 3'
PTR     DC A(TABLE)
RESULT  DSF
SAVE    DSF
ASUM    DC A (SUM)
        END
```

Show :

1) Equivalent mnemonic machine language translated program.

2) Contents of ESD card, TXT and RLD card for linkage loader.

3) Object deck for direct linking loader.

Ans.

1) Equivalent mnemonic machine language translated program :

Relative Location:

0	BALR	12, 0
2	ST	14, 54 (0, 12)
6	L	1, 46 (0, 12)
10	L	15, 58(0, 12)
14	BALR	14, 15
16	ST	1, 50(0, 12)
20	L	14, 54(0, 12)
24	BCR	15, 14
26	-	
28	1	
32	7	
36	9	
40	10	
44	3	
48	28	
52	-	
56	-	
60	-	
64	-	

2) Contents of ESD, TXT and RLD card :

ESD card

Ref. No	Symbol	Type	Relative Location	Length
1	PROG	SD	0	64
2	RESULT	LD	52	-
3	SUM	ER	-	-

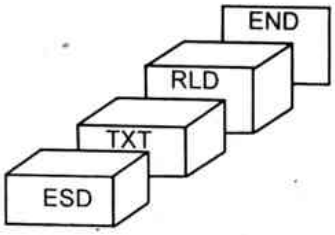
TXT card

Reference No.	Relative Location	Object code	
4	0	BALR	12, 0
6	2	ST	14, 54(0, 12)
7	6	L	1, 46 (0,12)
8	10	L	15, 58(0, 12
9	14	BALR	14, 15
10	16	ST	1, 50 (0, 12)
12	24	BCR	15, 14
13	28		1
13	32		7
13	36		9
13	40		10
13	44		3
14	48		28
17	60		0

RLD card

Ref. No.	Symbol	Flag	Length	Relative Location
14	JOHN	+	4	48
17	SUM	+	4	60

3) Object deck for direct linking loads :



Design of DLL i.e. PASS 1 and PASS 2

```
1   0          PGA   START
2                    ENTRY A1, A2
3                    EXTERNAL
                     B1, PGB
4   A1 = 20
5   A2 = 30
6   40               ADDR A(A + 1)
7   44               ADDR A(A2 + 15)
8.  48               ADDR
                     A(A2 – A1 – 3)
9   52               ADDR A (PGB)
10  56               ADDR A (PGB – A)
11  60               END
12  0          PGB   START
13                   ENTRY B1
14                   EXTERNAL A1, A2
15  16 ---- 61
16  24               ADDR A (A1)
```

Object Deck (PGA) :

ESD Record :

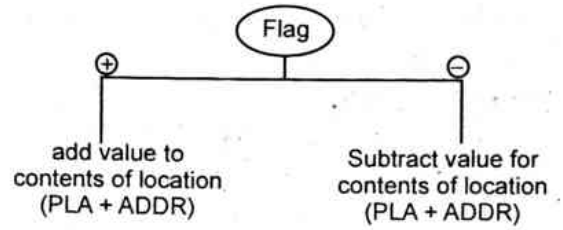
Variable Name	Type	Address	Length	Ref. to source line
PGA	Program Name	0	60	1
A_1	Local var	20		2
A_2	Local var	30		2
PGB	EXT Var	NA		3
B_1	EXT Var	NA		3

TXT Record :

Relative Address	Contents Flag.	What the assembler did	Ref. to source line
40	20	A_1 A(A_1)	6
44	45	30 + 15 (A_2 + 15)	7
48	7	30 – 20 – 3 ($A_2 - A_1 + 3$)	8
52	0	Unknown	9
56	– 16	– 20 + 4	10

RLD Record :

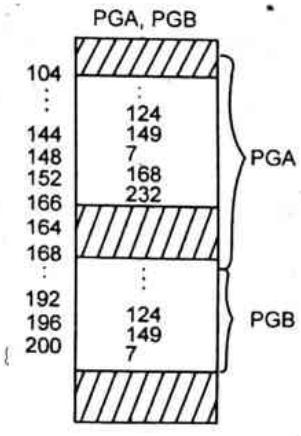
Relative address	Arithmetic operator	Variable Name	Length	Ref. to source line
40	+	PGA (A_1)	4	6
44	+	PGA (A_2)	4	7
52	+	PGB	4	9
56	+	B_1	4	10
56	+	PGB	4	10
56	–	PGB (A_1)	4	10



Assume (main storage after loading prog.)

PGA loaded at loc.104

PGB loaded at loc.168



RLD card

Relative address	Arithmetic Operator	Variable Name	Length	Ref. to source line
24	+	A_1	4	16
28	+	A_2	4	17
32	+	A_2	4	18
32	–	A_1	4	18

PGB :

ESD card

Variable Name	Type	Addr	Length	Ref. to source line
PGB	Program (SD)	0	36	12
B_1	Loc. variable (IT)	16	N/A	13
A_1	EXTR (ER) Variable	Unknown	N/A	13
A_2	EXTR variable (ER)	Unknown	N/A	14

TXT Card

Relative addr.	Content	What the assember did	Ref. to source line
24	0	0 (A_1)	16
28	15	0 + 15	17
32	– 3	0 + 15 – 3	18

Q.28. Consider the following three programs. *S-12(14M)*

```
PGA     START
        ENTRY    PGA1
        EXTERN   PGB,PGC,PGC2
        DC       A(PGA), A(PGB+3)
PGA1    DC       A(PGA1-PGA),
                 A(PGC2-PGC)
        END
```

```
PGB     START
        ENTRY    PGB1
        EXTRN    PGA, PGC1
PGB1    DC       A(PGC-2),A(PGB1)
PGB2    DC       A(PGB+4),A(PGB1-PGB)
PGB3    DC       A(PGC1+PGB-PGA-16)
        END
PGC     START
        ENTRY    PGC1,PGC2
        DC       A(PGC2-PGC)
PGC1    DC       A(PGC1-4),(*+4)
PGC2    DC       A(PGC+PGC2-PGC1)
        END
```

Assume that these programs are loaded at location 400 in the order PGA, PGB, then PGC. OBTAIN ESD, TXT, RLD, GEST and LESA.

Ans. FOR PGA :

1) ESD :

S.C.R	Symbol	Type	ID	Relocation address
1	PGA	SD	01	36
2	PGA1	LD	--	
3	PGB	ER	02	
3	PGC	ER	03	
3	PGC2	ER	04	

2) TXT :

S.C.R	Relocation address	Content
4	20-23	20
4	24-27	4
5	28-31	8
5	32-35	0

3) RLD :

S.C.R	ESD ID	Length	Flag	Relocation address
4	01	4	+	20
4	02	4	+	24
5	01	4	+	28
5	01	4	-	28
5	04	4	+	32
5	03	4	-	32

FOR PGB :

1) **ESD :**

S.C.R	Symbol	Type	ID	Relocation address
7	PGB1	SD	01	36
8	PGB1	LD	-	
9	PGA	ER	02	
9	PGC1	ER	03	

2) **TXT :**

S.C.R	Relocation address	Content
10	16-19	4
10	20-23	20
11	24-27	28
11	28-31	28
12	32-35	16

3) **RLD :**

S.C.R	ESD ID	Length	Flag	Relocation address
10	03	4	+	16
10	01	4	+	20
11	01	4	+	24
11	01	4	+	28
11	01	4	-	28
12	03	4	+	32
12	01	4	+	32
12	02	4	-	32

FOR PGC :

1) **ESD :**

S.C.R	Symbol	Type	ID	Relocation address
15	PGC	SD	01	28
16	PGC1	LD	-	
16	PGC2	LD	-	

2) **TXT :**

S.C.R	Relocation address	Content
16	12-15	12
17	16-19	12
17	20-23	24
18	24-27	8

3) **RLD :**

S.C.R	ESD ID	Length	Flag	Relocation address
16	01	4	+	12
16	01	4	-	12
17	01	4	+	16
17	01	4	+	20
18	01	4	+	24
18	01	4	+	24
18	01	4	-	24

Assuming that program are loaded with 400

LESA :

FOR PGA:

Relocation address	Symbol
400	PGA
404	PGA1
408	PGB
412	PGC
416	PGC2

FOR PGB :

Relocation address	Symbol
436	PGB
440	PGB1
444	PGA
448	PGC1

FOR PGC :

Relocation address	Symbol
422	PGC1
476	PGC1
480	PGC2

GEST :

Symbol	Absolute address
PGA	400
PGA1	404
PGB	436
PGB1	440
PGC	472
PGC1	476
PGC2	480

Q.29. Prepare ESD, RLD AND GEST for the following program. *W-12(13M)*

```
PGI     START
        ENTRY   SYM11
        EXTRN   PG2,SYM23
        DC      A(PG1), A(PG2+4)
SYM11   DC      A(SYM11-PGI)
                A(SYM23-PG-2)
        END
PG-2    START
        ENTRY   SYM23
        EXTRN   PG1,SYM11
SYM21   DC      A(SYM11-2),A(SYM21)
SYM22   DC      A(PG2+4),A(SYM21-PG2)
SYM23   DC      A(SYM21-PG1)

        END
```

Ans. FOR PG1 :

1) ESD :

S.C.R	Symbol	Type	ID	Relocation address
1	PG1	SD	01	32
2	SYM 11	LD	-	32
3	PG2	ER	02	32
3	SYM 23	ER	03	32

2) TXT :

S.C.R	Relocation address	Content
4	16-19	16 A (PGA)
4	20-23	24 A(PGB + 4)
5	24-27	24
5	28-31	0

3) RLD :

S.C.R	ESD ID	Length	Flag	Relocation address
4				16
4				20
01	4	+	24	
01	4	-	24	
03	4	+	28	
02	4	-	28	

FOR PG2 :

1) ESD :

S.C.R	Symbol	Type	ID	Relocation address
7	PG2	SD	01	36
8	SYM23	LD	-	36
9	PG1	ER	02	36
9	SYM11	ER	03	36

2) TXT :

S.C.R	Relocation address	Content
10	16-19	-2
10	20-23	20
11	24-27	28
11	28-31	28
12	32-35	32

3) RLD :

S.C.R	ESD ID	Length	Flag	Relocation address
10	03	4	+	16
10	01	4	+	20
10	01	4	-	20
11	01	4	+	24
11	01	4	-	28
12	02	4	-	32

LESA :

FOR PG1:

Relocation Value	Symbol
200	PG1
204	SYM 11
208	PG2
212	SYM 23

FOR PG2:

Relocation Value	Symbol
232	PG 2
236	SYM 23
240	PG 1
244	SYM11

Program = 268

GEST = Only take SD or LD

Symbol	Absolute address
PG 1	200
SYM 11	204
PG 2	232
SYM 23	236

Q.30. Show the entries in ESD, TXT and RLD card for following program. ***W-11, S-13(13M)***

```
JOHN     START
         ENTRY    SUM, DATA
         EXTRN    LOOP, POINTER
         BALR     15, 0
         USING    *, 15
         SR       4, 14
         L        1, FOUR
         A        2, FOUR
         ST       2, FOUR
         BR       14
FOUR     DC       F '4'
LOOP     DC       A (SUM + 4)
POINTER  DC       A (LOOP – DATA)
         DC       A (POINTER – LOOP)
         DC       A (POINTER)
         END
```

Ans. ESD : (external symbolic dictionary)

Name	Type	ID	Relative Location	Total Length
JONH	S.D.	01	0	
SUM	E.R	02	-	
DATA	E.R	03	-	38
LOOP	L.D	-	22	
POINTER	L.D	-	26	

SD : Segment definition

E.R. : External Reference

L.D. Local Definition

TXT Card :

Source Card	Reference	Relative Address	Contents	Com-ments
4	BALR	15,0		
6	SR	4,4		
7	L	1,FOUR	L	1,16 (0,15)
8	A	2,FOUR	A	2,16 (0,15)
9.	ST	2,FOUR	ST	2,16 (0,15)

RLD(Relocation and Linkage Dictionary) :

Source Card	ESD ID	Length (Bytes)	Flag	Relative Address
12	02	4	+	22
13	03	4	+	26
14	-	4	+	30
15	-	4	+	34

POINTS TO REMEMBER :

(1) Loader is program which accepts the object program deck, prepares these programs for execution.

(2) The four basic function of loader are allocation, linking, relocation and loading.

(3) The various types of loaders : 'Compile-and-go', absolute, relocating, direct-linking, dynamic loading, dynamic linking.

(4) A typical direct-linking loader requires two passes.

(5) The first pass allocates space for segments and defines the values of the external symbols.

(6) The second pass actually loads the text and uses the global external symbol table.

(7) Linker is used to link all object program files of the software and convert them into final executable program.

NOTES

UNIT - IV
COMMON OBJECT FILES FORMAT AND SYSTEM UTILITIES

SYLLABUS

Structure of object file and executable file, Section or segment headers, Symbol table, Concept of storage class, String, Various data types, Line insert, Character, Arrays structures, Source code control system, Make, link editor, Symbolic debugger, GNU debugger.

UNIVERSITY PAPER SOLUTIONS
SINCE WINTER - 2009

Note to the students :
- The questions given below are from previous Nagpur University examination papers.
- Though these questions are from old university papers, they have been included in the new syllabus.
- Questions which are out of new syllabus are not included here.

WINTER - 09

Q.1. Explain in detail the format of common object file. **13M**

Ans. P.4-2, Q.1.

Q.2. Write a short note on **8M**

(i) Symbolic debugger

Ans. P.4-11, Q.13.

(ii) Source code control system(SCCS)

Ans. P.4-10, Q.10.

SUMMER - 10

Q.1. Explain COFF symbol table and string table. **7M**

Ans. P.4-3, Q.3.

Q.2. What do you mean by source code control system? **4M**

Ans. P.4-10, Q.10.

Q.3. Write a short note on : **8M**

(i) Make Editor

Ans. P.4-11, Q.11.

(ii) Link Editor

Ans. P.4-11, Q.12.

WINTER - 10

Q.1. What is symbolic debugger ? Explain in brief. **5M**

Ans. P.4-11, Q.13.

Q.2. Write a short note on : **8M**

(i) Source code control system(SCCS)

Ans. P.4-10, Q.10.

(ii) Structure of common object file format.

Ans. P.4-2, Q.1.

SUMMER - 11

No question asked in MSBTE Exam.

WINTER - 11

Q.1. What is common object file format? Give the typical structure of an object file. **7M**

Ans. P.4-2, Q.1.

Q.2. Explain with example the type field entry of the symbol table. **6M**

Ans. P.4-6, Q.4.

Q.3. Write a short note on: **12M**

(i) SCCS

Ans. P.4-10, Q.10.

(ii) Symbolic debugger

Ans. P.4-11, Q.13.

(iii) Link editor

Ans. P.4-11, Q.12.

SUMMER - 12

Q.1. Explain symbolic debugger. **5M**

Ans. P.4-11, Q.13.

Q.2. Write a short note on: **14M**

(i) Segment header

Ans. P.4-4, Q.2.

(ii) Storage classes

Ans. P.4-7, Q.5.

(iii) COFF symbol table

Ans. P.4-5, Q.3.

(iv) Link editor

Ans. P.4-11, Q.12.

WINTER - 12

Q.1. Explain in detail common object file format. **13M**

Ans. P.4-2, Q.1.

Q.2. Write a short note on: **13M**

(i) Source code control system(SCCS)

Ans. P.4-10, Q.10.

(ii) Symbolic Debugger

Ans. P.4-11, Q.13.

SUMMER - 13

Q.1. What is common object file format? Give the typical structure of an object file. **7M**

Ans. P.4-2, Q.1.

Q.2. Explain with example the type field entry of the symbol table. **6M**

Ans. P.4-6, Q.4.

Q.3. Write short note on : **6M**

(i) Link editor

Ans. P.4-11, Q.12.

(ii) Symbolic Debugger

Ans. P.4-11, Q.13.

Note : Also refer Paper Solutions at the end of the book.

SOLVED QUESTION BANK

[Sequence given as per syllabus]

INTRODUCTION

The Common Object File Format (COFF) is an output file format produced by the compiler, assembler (as), and link editor (ld).

The Common Object File Format (COFF) is a specification of a format for executable, object code, and shared library computer files used on Unix systems.

It was introduced in Unix System V, replaced the previously used a.out format, and formed the basis for extended specifications such as XCOFF and ECOFF, before being largely replaced by ELF, introduced with SVR4. COFF and its variants continue to be used on some Unix-like systems, on Microsoft Windows, in EFI environments and in some embedded development systems.

STRUCTURE OF OBJECT FILE AND EXECUTABLE FILE

Q.1. Explain in detail the format of common object file. ***W-09, 12(13M)***

OR What is common object file format? Give the typical structure of an object file. ***W-11, S-13(7M)***

OR Write a short note on structure of common object file format. ***W-10(4M)***

Ans. Common Object File Format :

- The assembler creates object files in common object file format (COFF).
- COFF is an implementation of an object file format of the same name that was developed by AT&T for use on UNIX-based systems.
- This format encourages modular programming and provides powerful and flexible methods for managing code segments and target system memory.

COFF File Structure :

The elements of a COFF object file describe the file's sections and symbolic debugging information.

These elements include:

(a) A file header
(b) Optional header information
(c) A table of section headers
(d) Raw data for each initialized section
(e) Relocation information for each initialized section
(f) A symbol table
(g) A string tab

FILE HEADER
OPTIONAL HEADER INFORMATION
SECTION 1 HEADER
• • •
SECTION n HEADER
RAW DATA FOR SECTION 1
• • •
RAW DATA FOR SECTION n
RELOCATION INFO SECTION 1
• • •
RELOCATION INFO FOR SECTION n
LINE NUMBERS FOR SECTION 1
• • •
LINE NUMBERS FOR SECTION n
SYMBOL TABLE
STRING TABLE

Fig. COFF File Structure

(a) **File Header Structure :**

The file header contains 22 bytes of information that describe the general format of an object file.

(i) **Byte Number :** 0-1
Description: Version ID; indicates version of COFF file structure.

(ii) **Byte Number :** 2-3
Description: Number of section headers.

(iii) **Byte Number :** 4-7
Description : Time and date stamp; indicates when the file was created.

(iv) **Byte Number :** 8-11
Description : File pointer; contains the symbol table's starting address.

(v) **Byte Number :** 12-15
Description: Number of entries in the symbol table.

(vi) **Byte Number :** 16-17
Description: Number of bytes in the optional header. This field is either 0 or 28; if it is 0, there is no Optional file header.

(vii) **Byte Number :** 18-19
Description: Flags.

(viii) **Byte Number :** 20-21
Description: Target ID; magic number.

(b) **Optional File Header Information :**

The link step creates the optional file header and uses it to perform relocation at download time.

(i) **Byte Number :** 0-1
Description: Optional file header magic number.

(ii) **Byte Number :** 2-3
Description: Version stamp.

(iii) **Byte Number :** 4-7
Description: Size (in bytes) of executable code.

(iv) **Byte Number :** 8-11
Description: Size (in bytes) of initialized data.

(v) **Byte Number :** 12-15
Description : Size (in bytes) of uninitialized data.

(vi) **Byte Number :** 16-19
Description : Entry point.

(vii) **Byte Number :** 20-23
Description: Beginning address of executable code.

(viii) **Byte Number :** 24-27
Description: Beginning address of initialized data.

(c) **Section Header Structure :**

COFF object files contain a table of section headers that define where each section begins in the object.

(i) **Byte Number :** 0-7
Description : This field contains one of the following:

(1) An 8-character section name padded with nulls.
(2) A pointer into the string table if the symbol name is longer than eight characters.

(ii) **Byte Number :** 8-11
Description : Section's physical address

(iii) **Byte Number :** 12-15
Description : Section's virtual address

(iv) **Byte Number :** 16-19
Description : Section size in bytes.

(v) **Byte Number :** 20-23
Description : File pointer to raw data.

(vi) **Byte Number :** 24-27
Description : File pointer to relocation entries

(vii) **Byte Number :** 28-31
Description : Reserved

(viii) **Byte Number :** 32-35
Description : Number of relocation entries

(ix) **Byte Number :** 36-39
Description : Number of line number entries

(x) **Byte Number :** 40-43
Description : Flags

(xi) **Byte Number :** 44-45
Description : Reserved

(xii) **Byte Number :** 46-47
Description : Memory page number.

(d) **Structuring Relocation Information :**
A COFF object file has one relocation entry for each relocatable reference. The assembler automatically generates relocation entries.

(i) **Byte Number :** 0-3
Description : Virtual address of the reference

(ii) **Byte Number :** 4-5
Description : Symbol table index

(iii) **Byte Number :** 6-7
Description : Reserved

(iv) **Byte Number :** 8-9
Description : Relocation type

SECTION OR SEGMENT HEADER

Q.2. Write short note on segment header. ***S-12(4M)***

Ans. Segment Header :

- An executable or shared object file's program header table is an array of structures, each describing a segment or other information the system needs to prepare the program for execution.
- An object file segment contains one or more sections, as "Segment Contents" described below.
- Program headers are meaningful only for executable and shared object files. A file specifies its own program header size with the ELF header's e_phentsize and e_phnum members.

Segment Types : p_type

Name	Value
PT_NULL	0
PT_LOAD	1
PT_DYNAMIC	2
PT_INTERP	3
PT_NOTE	4
PT_SHLIB	5
PT_PHDR	6
PT_TLS	7
PT_LOOS	0 x 60000000
PT_HIOS	0 x 6fffffff
PT_LOPROC	0 x 70000000
PT_HIPROC	0 x 7fffffff

(i) **PT_NULL :**

- The array element is unused; other member's values are undefined.
- This type lets the program header table have ignored entries.

(ii) **PT_LOAD :**

- The array element specifies a loadable segment, described by p_filesz and p_memsz.
- The bytes from the file are mapped to the beginning of the memory segment.
- If the segment's memory size (p_memsz) is larger than the file size (p_filesz), the "extra" bytes are defined to hold the value 0 and to follow the segments initialized area.
- The file size may not be larger than the memory size.
- Loadable segment entries in the program header table appear in ascending order, sorted on the p_vaddr member.

(iii) **PT_DYNAMIC :**

- The array element specifies dynamic linking information.

(iv) **PT_INTERP :**

- The array element specifies the location and size of a null-terminated path name to invoke as an interpreter.

- This segment type is meaningful only for executable files (though it may occur for shared objects); it may not occur more than once in a file.
- If it is present, it must precede any loadable segment entry.

(v) PT_NOTE :

The array element specifies the location and size of auxiliary information.

(vi) PT_SHLIB :

- This segment type is reserved but has unspecified semantics.
- Programs that contain an array element of this type do not conform to the ABI.

(vii) PT_PHDR :

- The array element, if present, specifies the location and size of the program header table itself, both in the file and in the memory image of the program.
- This segment type may not occur more than once in a file.
- Moreover, it may occur only if the program header table is part of the memory image of the program.
- If it is present, it must precede any loadable segment entry.

(viii) PT_TLS :

- The array element specifies the Thread-Local Storage template.
- Implementations need not support this program table entry.

(ix) PT_LOOS through PT_HIOS :

Values in this inclusive range are reserved for operating - specific semantics.

(x) PT_LOPROC through PT_HIPROC :

- Values in this inclusive range are reserved for processor-specific semantics.
- If meanings are specified, the processor supplement explains them.

SYMBOL TABLE

Q.3. Explain the COFF symbol table and string table. *S-10(7M)*

OR Write short note on COFF symbol table. *S-12(5M)*

Ans. COFF Symbol Table :

FILENAME 1
FUNCTION 1
LOCAL SYMBOLS FOR FUNCTION 1
FUNCTION 2
LOCAL SYMBOLS FOR FUNCTION 2
• • •
STATICS
• • •
FILENAME 2
FUNCTION 1
LOCAL SYMBOLS FOR FUNCTION 1
• • •
STATICS
• • •
DEFINED GLOBAL SYMBOLS
UNDEFINED GLOBAL SYMBOLS

Fig. Symbol sequences within the symbol table

- The symbol table consists of at least one fixed-length entry per symbol with some symbols followed by auxiliary entries of the same size.
- The entry for each symbol is a structure that holds the value, the type, and other information.

Symbol And Meanings :

(i) .file : Filename
(ii) .text: Address of .text section
(iii) .data: Address of .data section
(iv) .bss: Address of .bss section
(v) .bb: Address of start of inner block
(vi) .eb: Address of end of inner block
(vii) .bf: Address of start of function
(viii) .ef: Address of end of function
(ix) .target: Pointer to the structure or union returned by a function

Functions Of Symbol Table :

(1) A .bf is put between the function name and the first local symbol to indicate the start of that function in the symbol table.

(2) Also, .ef is put immediately after the last local symbol to indicate the end of the function in the symbol table. Figure shows the sequence.

FUNCTION NAME
.bf
LOCAL SYMBOL
.cf

Fig. Symbols for function boundaries

String Table :

- Symbol table names longer than 16 characters are stored contiguously in the string table with each symbol name delimited by a null byte.

00	00	00	2a
's'	't'	'r'	'i'
'n'	'g'	'_'	'l'
'o'	'n'	'g'	'_'
'n'	'a'	'm'	'e'
'_'	'l'	'\0'	's'
't'	'r'	'i'	'n'
'g'	'_'	'a'	'n'
'o'	't'	'h'	'e'
'r'	'_'	'o'	'n'
'e'	'\0'		

Fig. String Table

- The first 4 bytes of the string table are the size of the string table in bytes; offsets into the string table, therefore, are greater than or equal to 4.
- Figure shows the string table format for a file containing two symbols (with names longer than 16 characters, string_long_name_1, and string_another_one).
- The index of string_long_name_1 in the string table is 4, the index of string_another_one is 23, and the size of the string table is 42(0x2a).

Q.4. Explain with example the type field entry of the symbol table. ***W-11, S-13(6M)***

Ans. Type Representation :

- The type field of a symbol table entry contains two bytes, each byte representing type information. The least-significant byte represents simple (base) data type, and the most-significant byte represents complex type, if any :

MSB	LSB
Complex type: none, pointer, function, array.	Base type: integer, floating-point, etc.

- The following values are defined for base type, although Microsoft tools generally do not use this field, setting the least-significant byte to 0.
- Instead, CodeView information is used to indicate types. However, the possible COFF values are listed here for completeness.

Constant	Value	Description
IMAGE_SYM_TYPE_NULL	0	No type information or unknown base type. Microsoft tools use this setting.
IMAGE_SYM_TYPE_VOID	1	No valid type; used with void pointers and functions.
IMAGE_SYM_TYPE_CHAR	2	Character (signed byte).
IMAGE_SYM_TYPE_SHORT	3	Two-byte signed integer.
IMAGE_SYM_TYPE_INT	4	Natural integer type (normally four bytes in Windows NT).
IMAGE_SYM_TYPE_LONG	5	Four-byte signed integer.
IMAGE_SYM_TYPE_FLOAT	6	Four-byte floating-point number.
IMAGE_SYM_TYPE_DOUBLE	7	Eight-byte floating-point number.

IMAGE_SYM_TYPE_STRUCT	8	Structure.
IMAGE_SYM_TYPE_UNION	9	Union.
IMAGE_SYM_TYPE_ENUM	10	Enumerated type.
IMAGE_SYM_TYPE_MOE	11	Member of enumeration (a specific value).
IMAGE_SYM_TYPE_BYTE	12	Byte; unsigned one-byte integer.
IMAGE_SYM_TYPE_WORD	13	Word; unsigned two-byte integer.
IMAGE_SYM_TYPE_UINT	14	Unsigned integer of natural size (normally, four bytes).
IMAGE_SYM_TYPE_DWORD	15	Unsigned four-byte integer.

- The most significant byte specifies whether the symbol is a pointer to, function returning, or array of the base type specified in the least significant byte.
- Microsoft tools use this field only to indicate whether or not the symbol is a function, so that the only two resulting values are 0x0 and 0x20 for the Type field. However, other tools can use this field to communicate more information.
- It is very important to specify the function attribute correctly.
- This information is required for incremental linking to work correctly.
- For some architectures the information may be required for other purposes.

IMAGE_SYM_DTYPE_NULL	0	No derived type; the symbol is a simple scalar variable.
IMAGE_SYM_DTYPE_POINTER	1	Pointer to base type.
IMAGE_SYM_DTYPE_FUNCTION	2	Function returning base type.
IMAGE_SYM_DTYPE_ARRAY	3	Array of base type.

CONCEPT OF STORAGE CLASSES

Q.5. Write a short note on storage classes.

S-12(4M)

Ans. Storage Class :

- A storage class defines the scope (visibility) and life time of variables and/or functions within a C program.
- There are following storage classes which can be used in a C Program
  (1) Auto
  (2) Register
  (3) Static
  (4) Extern

(1) Auto - Storage Class :

- Auto is the default storage class for all local variables.

```
{
  int Count;
  auto int Month;
}
```

- The example above defines two variables with the same storage class. auto can only be used within functions, i.e. local variables.

(2) Register - Storage Class :

- Register is used to define local variables that should be stored in a register instead of RAM.
- This means that the variable has a maximum size equal to the register size (usually one word) and can't have the unary '&' operator applied to it (as it does not have a memory location).

```
{
  register int  Miles;
}
```

- Register should only be used for variables that require quick access - such as counters.
- It should also be noted that defining 'register' not mean that the variable will be stored in a register.
- It means that it Might be stored in a register - depending on hardware and implementation restrictions.

(3) Static - Storage Class :

- Static is the default storage class for global variables. The two variables below (count and road) both have a static storage class.

```
static int Count;
    int Road;
    {
        printf("%d\n", Road);
    }
```

- static variables can be 'seen' within all functions in this source file.
- At link time, the static variables defined here will not be seen by the object modules that are brought in.
- static can also be defined within a function.
- If this is done the variable is initialized at run time but is not reinitialized when the function is called.
- This inside a function static variable retains its value during various calls.

(4) Extern - Storage Class :

- extern is used to give a reference of a global variable that is visible to all the program files.
- When we use 'extern' the variable cannot be initialized as all it does is point the variable name at a storage location that has been previously defined.
- When we have multiple files and we define a global variable or function which will be used in other files also, then extern will be used in another file to give reference of defined variable or function.
- Just for understanding extern is used to declare a global variable or function in another files.

Examples :

File 1: main.c

```
int count=5;
main()
{
  write_extern();
}
```

File 2: write.c

```
void write_extern(void);
extern int count;
void write_extern(void)
{
    printf("count is %i\n", count);
}
```

- Here extern keyword is being used to declare count in another file.
- Now compile these two files as follows
  gcc main.c write.c -o write
- This will produce write program which can be executed to produce result.
- Count in 'main.c' will have a value of 5. If main.c changes the value of count-write.c will see the new value.

CONCEPT OF STRING, DATA TYPES, LINE INSERT, CHARACTER, ARRAY STRUCTURE

Q.6. Write short note on strings used in a system.

Ans. String :

- A string is traditionally a sequence of characters, either as a literal constant or as some kind of variable. The latter may allow its elements to be mutated and/or the length changed, or it may be fixed (after creation).
- A string is generally understood as a data type and is often implemented as an array of bytes (or words) that stores a sequence of elements, typically characters, using some character encoding.
- A string may also denote more general arrays or other sequence (or list) data types and structures.
- Depending on programming language and precise data type used, a variable declared to be a string may either cause storage in memory to be statically allocated for a predetermined maximum length or employ dynamic allocation to allow it to hold variable number of elements.
- When a string appears literally in source code, it is known as a string literal.
- A string is any finite sequence of characters (i.e., letters, numerals, symbols and punctuation marks).
- An important characteristic of each string is its length, which is the number of characters in it.
- The length can be any natural number (i.e., zero or any positive integer).
- A particularly useful string for some programming applications is the empty string, which is a string

containing no characters and thus having a length of zero.

- A substring is any contiguous sequence of characters in a string.

  Example : "Nagpur", the word nagpur is a string since it is sequence or group of characters.

Q.7. Explain Data types in brief.

Ans. Data Types :

- A data type or simply type is a classification identifying one of various types of data, such as real-valued, integer or boolean, that determines the possible values for that type; the operations that can be done on values of that type; the meaning of the data; and the way values of that type can be stored.
- Data types are used within type systems, which offer various ways of defining, implementing and using them.
- Different type systems ensure varying degrees of type safety. Formally, a type can be defined as "any property of a program we can determine without executing the program."
- Almost all programming languages explicitly include the notion of data type, though different languages may use different terminology.
- Common data types may include:

  (i) Integers,

  (ii) Booleans,

  (iii) Characters,

  (iv) Floating-point numbers,

  (v) Alphanumeric strings.
- Each data type has range; between that range the numbers can be stored .

  Example : In 'C' Language int a, denote that the variable "a" can store data of type integer.

Q.8. Explain how to use line insert command.

Ans. Line Insert :

- The Insert Line activity inserts lines into a text file on a line number that the user specify.
- Before we configure the Insert Line File activity, we need to determine the following:

(i) The name of the file we want to insert text into.

(ii) The file encoding type of the file we want to insert text into.

(iii) The line number location where we want to insert the text.

- Use the following information to configure the Insert Line activity.

Settings	Configuration Instructions
File	Type the path and name of the file that user want to insert the text into, or click the ellipsis button (...) and browse for it.
File encoding	Click the ellipsis button (...) and select the format that the file is encoded in from the File encoding drop-down list. Verify that user select the correct encoding format. If the file uses a different encoding format, the activity fails.
Text	Type the text that user want to insert into the file.
Line number	Type the line number where the text will be inserted.

Q.9. Write a short note on :

(i) Character

(ii) Array Structure

Ans.

(i) Character :

- A character is a unit of information that roughly corresponds to a grapheme, grapheme-like unit, or symbol, such as in an alphabet or syllable in the written form of a natural language.
- Examples of characters include letters, numerical digits, common punctuation marks (such as "." or "-"), and whitespace.
- Control characters, which do not correspond to symbols in a particular natural language, but rather to other bits of information used to process text in one or more languages. Examples of control characters include carriage return or tab, as well as instructions to printers or other devices that display or otherwise process text.
- Characters are typically combined into strings.

  Example : In c language, char ch;

'a' + 2 *// evaluates to the character value 'c'*
// (becuase 'a' is encoded as a number,
// we can add 2 to it to get 'c')
'C' > 'A' *// evaluates to true, which is a*
// non-zero value in C (0 is false)

(ii) Array Structure :

- An array is group of same data type values.
- An array data structure or simply an array is a data structure consisting of a collection of elements (values or variables), each identified by at least one array index or key.
- An array is stored so that the position of each element can be computed from its index tuple by a mathematical formula.

  Example : Consider an array of 10 integer variables, with indices 0 through 9, may be stored as 10 words at memory addresses 2000, 2004, 2008, ... 2036, so that the element with index i has the address 2000 + 4 × i.
- Array contains all values of same data type i.e all values having homogenous nature

  Syntax : Data type array_name [Length of array];

  0 1 2 3 ..... *//Index No Of Array Elements*

  Example : char name [20]; *// the name of //character data type array can store 20 character //type values.*
- Array can be one dimensional, two dimensional or multi dimensional also. The example above is one dimensional array .

SOURCE CODE CONTROL SYSTEM

Q.10. Write short note on source code control system(SCCS). ***W-09,10,11(4M), W-12(6M)***

OR What do you mean by source code control system? ***S-10(4M)***

Ans. Source code control system(SCCS) :

- The source code control system (SCCS) is a complete system of commands that allows specified users to control and track changes made to an SCCS file.
- SCCS files allow several versions of the same file to exist simultaneously, which can be helpful when developing a project requiring many versions of large files.
- The SCCS commands support Multi byte Character Set (MBCS) characters.
- The SCCS commands form a complete system for creating, editing, converting, or changing the controls on SCCS files.
- An SCCS file is any text file controlled with SCCS commands. All SCCS files have the prefix s. , which sets them apart from regular text files.
- An SCCS file is made up of three parts:
  - **(i)** Delta table
  - **(ii)** Access and tracking flags
  - **(iii)** Body of the text

(i) Delta table in SCCS files :

- Instead of creating a separate file for each version of a file, the SCCS file system only stores the changes for each version of a file. These changes are referred to as deltas.
- The changes are tracked by the delta table in every SCCS file.
- Each entry in the delta table contains information about who created the delta, when they created it, and why they created it.
- Each delta has a specific SID (SCCS Identification number) of up to four digits.
- The first digit is the release, the second digit is the level, the third digit is the branch, and the fourth digit is the sequence.

  Example : SID = 1.2.1.4 that is, release 1, level 2, branch 1, sequence 4.

(ii) Access and tracking flags :

- After the delta table in an SCCS file, a list of flags starting with the @ (at sign) define the various access and tracking options of the SCCS file. Some of the SCCS flag functions include:

(a) Designating users who may edit the files.
(b) Locking certain releases of a file from editing.
(c) Allowing joint editing of the file.
(d) Cross-referencing changes to a file.

(iii) Body of an SCCS file :

- The SCCS file body contains the text for all the different versions of the file.

- Consequently, the body of the file does not look like a standard text file.
- Control characters bracket each portion of the text and specify which delta created or deleted it.
- When the SCCS system builds a specific version of a file, the control characters indicate the portions of text that correspond to each delta. The selected pieces of text are then used to build that specific version.

Q.11. Write a short note on make editor.

S-10(4M)

Ans. Make editor :

- A make editor is a type of program used for editing plaintext files also called as text editor.
- Text editors are often provided with operating systems or software development packages, and can be used to change configuration files and programming language source code.
- A text editor is a tool that allows a user to create and revise documents in a computer. Though this task can be carried out in other modes, the word text editor commonly refers to the tool that does this interactively.

Types of Text Editors

Depending on how editing is performed, and the type of output that can be generated, editors can be broadly classified as.

(1) **Line Editors :** During original creation lines of text are recognised and delimited by end-of-line markers and during subsequent revision, the line must be explicitly specified by line number or by some pattern context.

Example : edlin editor in early MS-DOS systems.

(2) **Stream Editors :**

- The idea here is similar to line editor, but the entire text is treated as a single stream of characters.
- Hence the location for revision cannot be specified using line numbers.
- Locations for revision are either specified by explicit positioning or by using pattern context.

Example : sed in Unix / Linux.

- Line editors and stream editors are suitable for text-only documents.

(3) **Screen Editors :**

- These allow the document to be viewed and operated upon as a two dimensional plane, of which a portion may be displayed at a time.
- Any portion may be specified for display and location for revision can be specified anywhere within the displayed portion.

Example : vi, emacs, etc.

(4) **Word Processors :** It provides additional features to basic screen editors. Usually support non-textual contents and choice of fonts, style, etc.

(5) **Structure Editors :** These are editors for specific types of documents, so that the editor recognises the structure/syntax of the document being prepared and helps in maintaining that structure / syntax.

LINK EDITOR

Q.12. Write a short note on link editor.

S-10,12,13, W-11(4M)

Ans. Link editor :

- Linkage editor keeps track of the relocation information so that the resulting load module has an ensemble .
- In linkage editor the module loader must perform allocation and relocation as well as loading, but it does not have complex problems of linking.
- The linkage editor binder is more complex but allows a more flexible allocation and loading scheme.

Functions :

(i) Replacement of subroutine in linkage program.

(ii) Linkage editor combine common libraries at the execution time.

SYMBOLIC DEBUGGER

Q.13. Explain the symbolic debugger.

S-12(5M)

OR What is symbolic debugger? Explain in brief.

W-10(5M)

OR Write short note on symbolic debugger.

W-09,11, S-13(4M), W-12(7M)

Ans. Symbolic Debugger :

- Software helps a programmer debugs a program by stopping at certain breakpoints and displaying various programming elements. The programmer can step through source code statements one at a time while the corresponding machine instructions are being executed.
- Debug commands include single stepping through/over statements/instructions, breakpoints and watch points, print and modify variables, examine types, write log file etc.
- When executing, breakpoints may be placed at selected statements, or the program may be single-stepped on a line-by-line basis.
- Debuggers also offer more sophisticated functions such as running a program step by step (single-stepping or program animation), stopping (breaking) (pausing the program to examine the current state) at some event or specified instruction by means of a breakpoint, and tracking the values of some variables.
- Some debuggers have the ability to modify the state of the program while it is running, rather than merely to observe it.
- It may also be possible to continue execution at a different location in the program to bypass a crash or logical error.
- A debugger or debugging tool is a computer program that is used to test and debug other programs
- Symbolic debugging is a method by which the programmer can receive information about bugs as well as look for bugs at the same level as the program was originally written.
- By keeping the programmer at the source level during debugging, the symbolic debugger makes the same impact on the debugging process as the compiler made on the coding process.
- The symbolic debugger is actually only one part of a series of cooperating programs.
- The symbolic debugger requires information from both the compiler and the linker.
- The basic idea behind a symbolic debugger is to be able to execute the application normally by issuing the appropriate start up commands to the host operating system.
- If, however, the program displays some aberrant behavior, the symbolic debugger may be invoked which then runs the program in a different mode making it available for debugging.

GNU DEBUGGER

Q.14. Explain the GNU debugger. Enlist the findings covered by the GNU debugger.

Ans. GNU Debugger :

- The GNU Debugger, usually called just GDB and named gdb as an executable file, is the standard debugger for the GNU operating system.
- However, its use is not strictly limited to the GNU operating system; it is a portable debugger that runs on many Unix-like systems and works for many programming languages, including Ada, C, C++, Objective-C, Free Pascal, Fortran, Java.
- A Debugger is a program that runs other programs, allowing the user to exercise control over these programs, and to examine variables when problems arise.
- GNU Debugger which is called gdb is the most popular debugger for UNIX systems to debug C and C++ Programs.
- GNU Debugger helps use in finding out following :

(i) If a core dump happened then what statement or expression did the program crash on?

(ii) If an error occurs while executing a function, what line of the program contains the call to that function, and what are the parameters?

(iii) What are the values of program variables at a particular point during execution of the program?

(iv) What is the result of a particular expression in a program?

- GDB allows us to do things like run the program up to a certain point then stop and print out the values of certain variables at that point, or step through the program one line at a time and print out the values of each variable after executing each line.
- GDB uses a simple command line interface.

POINTS TO REMMBER :

(1) The Common Object File Format (COFF) is an output file format produced by the compiler, assembler (as), and link editor (ld).

(2) The symbol table consists of at least one fixed-length entry per symbol with some symbols followed by auxiliary entries of the same size.

(3) The entry for each symbol is a structure that holds the value, the type, and other information.

(4) A string is traditionally a sequence of characters either as a literal constant or as some kind of variable.

(5) A data type is a classification identifying one of various types of data such as real-valued, integer or boolean.

(6) The insert line activity inserts lines into a text file on a line number that the user specifies.

(7) A character is a unit of information that roughly corresponds to symbols such as alphabets or syllable in the written form of a natural languages.

(8) An array is group of same data type values.

(9) The GNU debugger (GDB) is the standard debugger for the GNU operating system.

(10) Linkage editor keeps track of the relocation information so that the resulting load module has an ensemble.

(11) The source code control system (SCCS) is a complete system of commands that allows specified users to control and track changes made to an SCCS file.

(12) Symbolic debugging is a method by which the programmer can receive information about bugs as well as look for bugs at the same level as the program was originally written.

(13) By keeping the programmer at the source level during debugging, the symbolic debugger makes the same impact on the debugging process as the compiler made on the coding process.

NOTES

UNIT - V
UNIX DEVICE DRIVERS

SYLLABUS

Definition, Anatomy and Types, Device programming, Installation, Incorporation of driver routines, Basic device operation, Implementation with Line Printer, Comparative study between device drivers for UNIX & Windows.

UNIVERSITY PAPER SOLUTIONS
SINCE WINTER - 2009

Note to the students :
- The questions given below are from previous Nagpur University examination papers.
- Though these questions are from old university papers, they have been included in the new syllabus.
- Questions which are out of new syllabus are not included here.

WINTER - 09

Q.1. What are the steps involved in installing a device driver in UNIX system? Explain the steps in details. **13M**

Ans. P.5-8, Q.9.

Q.2. Design a device driver for handling a RAM Disk. **8M**

Ans. P.5-11, Q.11.

Q.3. Write a short note on STREAM driver. **5M**

Ans. P.5-5, Q.5.

SUMMER - 10

Q.1. How device drivers are installed? Explain steps in installation for both UNIX and WINDOWS operating system separately. **13M**

Ans. P.5-9, Q.10.

Q.2. Differentiate between character driver and block driver. **6M**

Ans. P.5-5, Q.4.

Q.3. Mention different registers available with line printer along with significance of each bit. **3M**

Ans. P.5-12, Q.13.

WINTER - 10

Q.1. Design a device driver for handling a RAM disk. **8M**

Ans. P.5-11, Q.11.

Q.2. Explain the following entry points and routines for device driver of a line printer
(i) init
(ii) open
(iii) close **6M**

Ans. P.5-14, Q.15.

Q.3. Write a short note on types of windows device driver. **5M**

Ans. P.5-6, Q.6.

Q.4. Explain various steps involved in installation of UNIX device driver. **7M**

Ans. P.5-8, Q.9.

SUMMER - 11

No question asked in University exam.

WINTER - 11

Q.1. Design a device driver for handling a RAM Disk. **8M**

Ans. P.5-11, Q.11.

Q.2. Compare the device driver for UNIX and WINDOWS operating system. **5M**

Ans. P.5-16, Q.16.

Q.3. Explain in details the anatomy and types of device driver in UNIX system. **13M**

Ans. P.5-4, Q.3.

SUMMER - 12

Q.1. Write down structure of device driver in any operating system. Describe each part in brief. **8M**

Ans. P.5-3, Q.1.

Q.2. Write down steps for installation of any device driver and specify the setting in system files. **5M**

Ans. P.5-8, Q.9.

Q.3. Design device driver for handling a line printer. **8M**

Ans. P.5-13, Q.14.

WINTER - 12

Q.1. Explain following entry points and routines for device driver of a line printer **8M**

(i) init
(ii) open
(iii) close
(iv) LP work routine
(v)LP restart routine

Ans. P.5-14, Q.15.

Q.2. Design a device driver for handling a RAM Disk. **8M**

Ans. P.5-11, Q.11.

Q.3. Write a short note on STREAM driver. **5M**

Ans. P.5-5, Q.5.

SUMMER - 13

Q.1. What are the different kinds of drivers in windows? **3M**

Ans. P.5-6, Q.6.

Q.2. State different types of UNIX device driver. **2M**

Ans. P.5-4, Q.3.

Q.3. State steps involved in installing a device driver in a UNIX system. **4M**

Ans. P.5-8, Q.9.

Q.4. What are major design issue in the study of device drivers? **4M**

Ans. P.5-3, Q.2.

Q.5. Compare device driver for UNIX and WINDOWS operating system. **6M**

Ans. P.5-16, Q.16.

Q.6. Explain structure of kernel mode driver. **4M**

Ans. P.5-7, Q.7.

Q.7. Write a short note on drive memory allocation. **4M**

Ans. P.5-11, Q.12.

Note : Also refer Paper Solutions at the end of the book.

SOLVED QUESTION BANK

[Sequence given as per syllabus]

INTRODUCTION

A device driver is a program that links a peripheral device to an operating system of a computer. It is essentially a software program that allows a user to employ a device, such as a printer, monitor, or mouse It is written by programmers who comprehend the detailed knowledge of the device's command language and characteristics and contains the specific machine language necessary to perform the functions requested by the application.

When a new hardware device is added to the computer, such as a CD-ROM drive, a printer, or a sound card, its driver must be installed in order to run it. The operating system "calls" the driver, and the driver "drives" the device. In Windows, for example, everything that is seen on the screen is the result of the display driver (video driver). The display driver effectuates the visual appearance of the screen according to the precise commands that Windows issues to it.

DEFINATION, ANATOMY AND TYPES

Q.1. What is Device Drivers?

OR Write down structure of device driver in any operating system? Describe each part in brief. ***S-12(8M)***

Ans. Device Drivers :

- A device driver is the glue between an operating system and its IO devices.
- Device drivers act as translators, converting the generic requests received from the operating system into commands that specific peripheral controller can understand.
- The figure below illustrates this relationship:

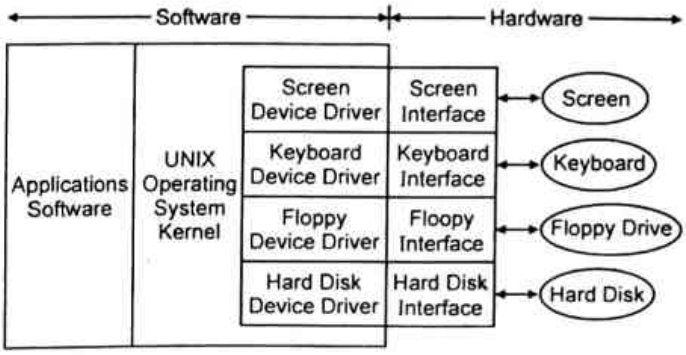


Fig. Software and Hardware relationship in device driver

- The application software makes system calls to the operating system requesting services.
- The operating system analyzes these requests and, in device driver when necessary, issues requests to the appropriate device driver.
- The device driver in turn analyzes the request from the operating system and, when necessary, issues commands to the hardware interface to perform the operations needed to service the request.
- Without device drivers, the operating system would be responsible for communicating directly with the hardware.
- This would require the operating systems designer to include support for all of the devices that users might want to connect to their computer.
- It would also mean that adding a new device would mean modifying the operating system itself.
- By separating the device driver's function from the operating system, the designer of the operating system can concern only with the issues related to the operation of the system.
- The device driver is not concerned with the management of the operating system.

Q.2. What are the major design issue in the study of device driver? ***S-13(4M)***

Ans. The design of device driver can be made easier by dividing the issues to be considered into three broad categories :

(1) Operating system/Driver communications.
(2) Driver /Hardware communications.
(3) Driver operations.

(1) Operating system/Driver communications :

- It covers all the issues related to the exchange of information between the device driver and the operating system.
- It also includes support functions that the kernel provides for the benefit of the device driver.

(2) Driver /Hardware communications :

- It covers those issues related to the exchange of information between the device driver and the device it controls.
- This also includes issues such as how software talks to hardware and how the hardware talks back.

(3) Driver operations :

- It cover issues related to the actual internal operation of the driver itself.

This includes :

(i) Interpreting commands received from the operating system ;

(ii) Scheduling multiple outstanding request for service ;

(iii) Managing the transfer of data across both interfaces ;

(iv) Accepting and processing hardware interrupts ;

(v) Maintaining the integrity of the driver's and the kernel 's data structure.

Q.3. Explain different kinds of UNIX device driver.
OR Explain in details the anatomy and types of device driver in UNIX system. ***W-11(13M)***
OR State different types of UNIX device driver. ***S-13(2M)***

Ans. There are four kinds of driver in UNIX :

(1) Block Drivers
(2) Character Drivers
(3) Terminal Drivers
(4) Stream Drivers

(1) Block driver :

- It communicate with OS through a collection of fixed-sized buffers.
- OS manages these buffer caches in order to satisfy the user request.
- The driver is invoked only if the requested data is not present in the cache, or buffer cache have been changed and must be written out.
- While accessing the data for first time with R/W system call, the actual data access is done from the device on which it is present with the help of necessary device driver.
- The accessed data is kept in buffer cache.
- When same data is required second time, it is actually accessed from the buffer cache, instead of accessing it from the origianl device storage.

For example : disks are commonly implemented as block devices.

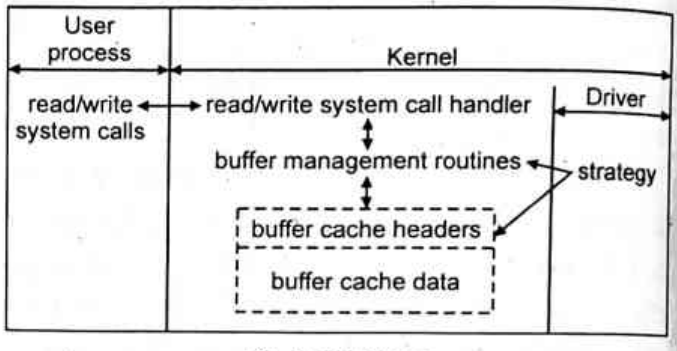


Fig. Block Drivers

(2) Character driver :

- It can handle I/O requests of arbitary size.
- It is generally used with devices that handles data in byte wise manner.

Example : Line printer.

- Difference with block device driver :

In block device driver the user process interaction with device is indirect, whereas in character device driver there is direct interaction between user process and device.

- There is no buffer cache in this driver.
- Similarly it is used with data is accessed in byte wise manner.
- Mostly used devices are Printers.

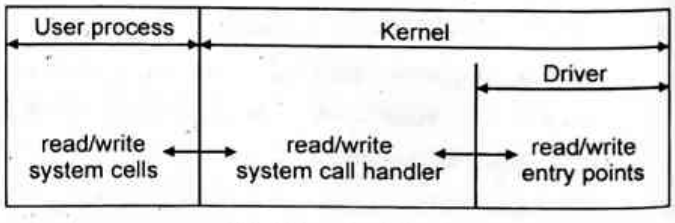


Fig. Character Driver

(3) Terminal Driver :

- They are similar to character deivce driver and deal with communication terminals that connect user to central UNIX computer system.
- These device drivers are responsible for
  - **(i)** Transportation of user data to and from user terminal.
  - **(ii)** Line editing
  - **(iii)** TAB expansion and various commands to mange text.
- Similarly in this device driver additonal features can be implemented with the help of kernel routines and specialized data structures.
- In these device drivers the device is accessed with the help of R/W entry points. These entry points are used to allow only valid access to devices.

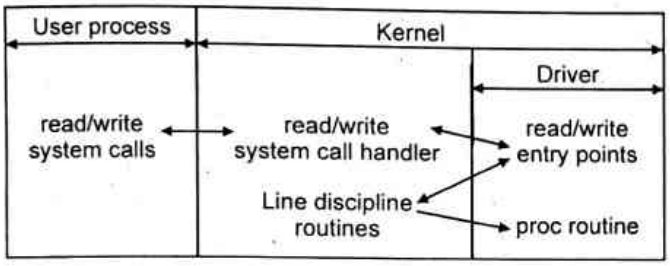


Fig. Terminal Driver

(4) Stream Driver :

- It can handle high speed communication devices such as networking adapters that deal with unusual sized chunks of data and that need to handle protocol.
- It is also known as Network Device Drivers. e.g., Network Devices.
- The character model assumes that a single driver sit between the user process and the device.
- With character drivers the user process's request is handed directly to the driver with little intervention or processing by the kernel.
- Networking devices, however, usually support a number of layered protocols.
- The character model essentially required that each layer of the protocol be implemented within the single driver.
- This lack of modularity and reusability reduced the efficiency of the system.

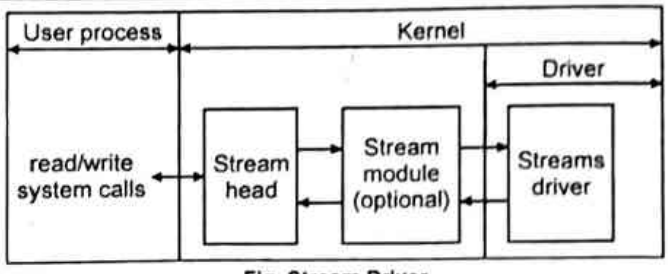


Fig. Stream Driver

Q.4. Differentiate between character driver and block driver. *S-10(6M)*

Ans.

Sr. No.	Block Driver	Character Driver
(1)	It communicates with OS using fixed size buffers cache memory.	It handles I/O request of any arbitrary size.
(2)	The user process interaction with device is indirect.	There is direct interaction between user process and device.
(3)	The buffer cache memory is present in this driver.	There is no buffer cache in this driver.
(4)	For block driver, all I/O occurs through the buffer cache.	Character driver does not handle I/O through the buffer cache.
(5)	Block drivers are particularly well-suited for disk drives	A character driver for a device such as line printer handles one character at a time.
(6)	**Example :** disk drive.	**Example :** Line printers, interactive terminals, and graphics display.

Q.5. Write short note on Stream driver. *W-09,12(5M)*

Ans. Stream Driver :

- It can handle high speed communication devices such as networking adapters that deal with unusual sized chunks of data and that need to handle protocol.
- It is also known as Network Device Drivers.

eg. Network Devices

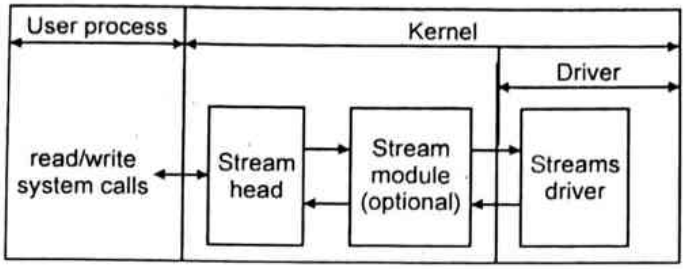


Fig. Stream Driver

- STREAM drivers can be considered a subset of device drivers in general and character device drivers in particular.
- While there are some differences between STREAM drivers and STREAMS drivers, much of the information contained in Writing Device Drivers also applies to STREAMS drivers.
- STREAMS drivers share a basic programming model with STREAMS modules.
- STREAMS drivers have five different points of contact with the kernel:

Kernel Contact Point

(1) Configuration entry points :

- These kernel (dynamically loading) routines enable the kernel to find the driver binary in the file system and load it into, or unload it from, the running kernel.
- The entry points include _init(9E), _info(9E), and _fini(9E).

(2) Initialization entry points :

- It enable the driver to determine a device's presence and initialize its state.
- These routines are accessed through the dev_ops(9S) data structure during system initialization.
- They include getinfo(9E), identify(9E), probe(9E), attach(9E), and detach(9E).

(3) Table-driven entry points :

- It is accessed through cb_ops(9S), the character and block access table, when an application calls the appropriate interface.
- The members of the cb_ops(9S) structure include pointers to entry points that perform the device's functions, such as read(9E), write(9E), ioctl(9E).
- The cb_ops(9S) table contains a pointer to the streamtab(9S)structure

(4) STREAMS queue processing entry points :

- It contained in the streamtab, read and process the STREAMS messages that travel through the queue structures.

Examples : put(9E) and srv(9E)

(5) Interrupt routines :

- It handle the interrupts from the device (or software interrupts).
- It is added to the kernel by ddi_add_intr(9F) when the kernel configuration software calls attach(9E).

Q.6. Explain different kinds of driver in windows.

OR Write short note on types of windows device driver. ***W-10(5M)***

OR What are different kinds of driver in windows? ***S-13(3M)***

Ans. There are two basic types of Microsoft Windows drivers :

(1) User-mode drivers :

- It execute in user mode, and they typically provide an interface between a Win32 application and kernel-mode drivers or other operating system components.

For example, in Windows Vista, all printer drivers execute in user mode.

(2) Kernel-mode drivers :

- It execute in kernel mode as part of the executive, which consists of kernel-mode operating system components that manage I/O, Plug and Play memory, processes and threads, security.
- Kernel-mode drivers are typically layered.
- Generally, higher-level drivers typically receive data from applications, filter the data, and pass it to a lower-level driver that supports device functionality.
- Some kernel-mode drivers are also WDM drivers, which conform to the Windows Driver Model (WDM). All WDM drivers support Plug and Play, and power management.
- WDM drivers are source-compatible (but not binary-compatible) across Windows 98/Me and Windows 2000 and later operating systems.

Q.7. Explain structure of a kernel mode driver.

S-13(4M)

Ans. Kernel mode driver :

- Kernel-mode drivers execute in kernel mode as part of the executive, which consists of kernel-mode operating system components that manage I/O, Plug and Play memory, processes and threads, security.
- Kernel-mode drivers are typically layered. Generally, higher-level drivers typically receive data from applications, filter the data, and pass it to a lower-level driver that supports device functionality.
- The following figure divides kernel-mode drivers into several types :

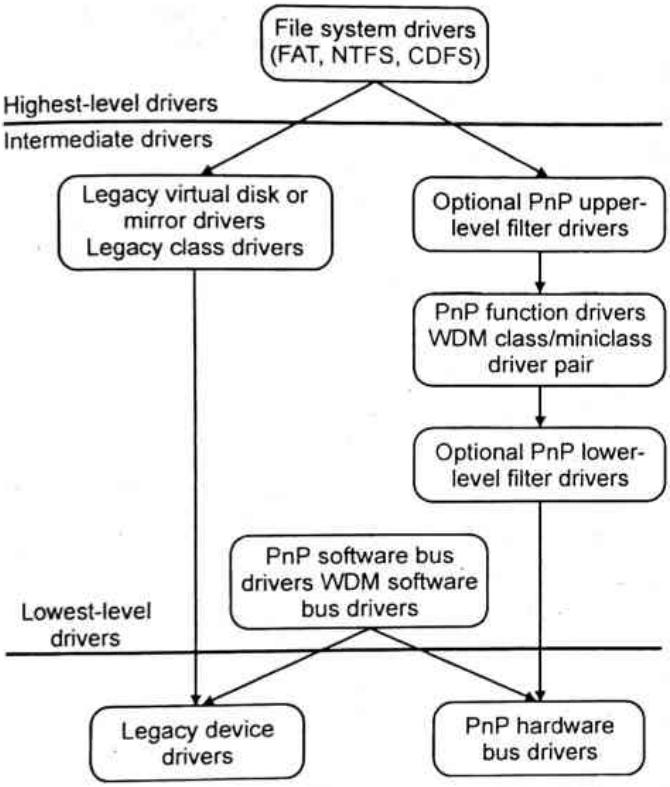


Fig. Types of kernel mode drivers

- As shown in the figure, there are three basic types of kernel-mode drivers in a driver stack: highest-level, intermediate, and lowest-level.
- Each type differs only slightly in structure but greatly in functionality.

(1) Highest-level drivers :

Highest-level drivers include file system drivers (FSDs) that support file systems, such as :

(i) NTFS

(ii) File allocation table (FAT)

(iii) CD-ROM file system (CDFS)

- Highest-level drivers always depend on support from underlying lower-level drivers, such as intermediate-level function drivers and lowest-level hardware bus drivers.

(2) Intermediate drivers :

- Intermediate drivers such as a virtual disk, mirror, or device-type-specific class driver.
- Intermediate drivers depend on support from underlying lower-level drivers.
- Intermediate drivers are subdivided further as follows:

(i) Function drivers : It control specific peripheral devices on an I/O bus.

(ii) Filter drivers : It insert themselves above or below function drivers.

(iii) Software bus drivers : It present a set of child devices to which still higher-level class, function, or filter drivers can attach themselves.

- **For example**, a driver that controls a multifunction adapter with an on-board set of heterogeneous devices is a software bus driver.
- Any system-supplied class driver that exports a system-defined class/miniclass interface is, in effect, an intermediate driver with one or more linked miniclass drivers (sometimes called minidrivers).
- Each linked class/minidriver pair provides functionality that is equivalent to that of a function driver or a software bus driver.

(3) Lowest-level drivers :

- It control an I/O bus to which peripheral devices are connected.
- Lowest-level drivers do not depend on lower-level drivers.

(i) Hardware bus drivers :

- These are system-supplied and usually control dynamically configurable I/O buses.
- Hardware bus drivers work with the Plug and Play manager to configure and reconfigure

system hardware resources, for all child devices that are connected to the I/O buses that the driver controls.

- These hardware resources include mappings for device memory and interrupt requests (IRQs).

(ii) Legacy drivers :

It directly control a physical device.

DEVICE PROGRAMMING AND BASIC DEVICE OPERATION

Q.8. Write short note on "Device programming" and its basic operations.

Ans. Device programming :

- A device programming services is used to turn programmable components into customized devices for a specific application.
- The field of computer hardware, the term programmer, chip programmer or device programmer refers to a device that configures programmable non-volatile digital circuits such as EPROMs, Flashs, PALs, FPGAs or programmmable logic circuits.
- For programming a circuit, it is either inserted into a socket (often ZIF) on top of the programmer, or the programmer is directly connected by an adapter to the circuit board (In-System Programming).
- Afterwards the data is transferred into the circuit by applying signals to the connecting pins. Some circuits have a serial interface for receiving the programming data (JTAG interface).
- Other circuits require the data on parallel pins, followed by a programming pulse with a higher voltage for programming the data into the circuit.
- Usually device programmers are connected to a personal computer through a printer connector, USB port or LAN interface.
- A software program on the computer then transfers the data to the programmer, selects the circuit and interface type, and starts the programming process.
- There are four general types of device programmers :

(i) Gang programmers for mass production.

(ii) Development programmers for development and 'small-series production'.

(iii) Pocket programmers for development and field service, and

(iv) Specialized programmers for certain circuit types only.

Example : EPROM programmers.

INSTALLATION, INCORPORATION OF DRIVER ROUTINES

Q.9. What are the steps involved in installing a device driver in UNIX system? Explain the steps in details. *W-09(13M), S-13(4M)*

OR Explain various steps involved in installation of UNIX device driver. *W-10(7M)*

OR Write down steps for installation of any device driver and specify setting system files. *S-12(5M)*

Ans. Installing the device driver in a Unix system involves the following steps :

(1) Compiling the device driver

(2) Modifying the kernel configuration tables and files

(3) Linking the device driver with the kernel object files to produce a new kernel

(4) Creating the necessary entries in the / dev directory

(5) Rebooting the system with new kernel

(1) Compiling the device driver :

- Compiling the driver is similar to compiling any other c program .
- Since the driver is not a stand-alone program it is always compiled to produce an object rather than a linked executable files
- System special flags are needed to ensure the correct part of included files are used and that the compiler generates a module appropriate for linking into the kernel.
- The information must be obtained from system's documentation.

(2) Modifying or configuring the kernel :

- Kernel Configuration tables are used to reflect the addition of new driver.
- Method varies between different Version of Unix.
- There are two Config files used by the system to generate a new kernel are the mdevice and sdevice file in /etc/conf/cd.d directory contain entry for each device driver that exists on the system.
- Sdevice contains information for each device driver that is to be incorporated into the kernel
- Mdevice contains entry for each device driver that exist on the system.

(3) Linking the device driver with the kernel object files to produce a new kernel :

- System utility to run is idbuild.
- Once the kernel configuration tables have been modified then we can generate the new kernel.
- The command to build a new kernel varies between different UNIX system.
- On SVR 3.2 systems the utility to run is idbuild. This program will generate a new UNIX kernel.
- On some SVR 3.2 systems the idbuild utility will ask us if we want the new kernel to boot by default.
- On others , the new kernel will automatically be used during the next boot.

(4) Creating the necessary entries in the / dev directory :

- dev directory reference our device driver.
- It can be done with mknod command.
- On most system it will be automatic.
- Node files has four fields :
  (i) Device Name
  (ii) Node Name
  (iii) Node Type
  (iv) Minor Device Number

(5) Rebooting the system with new kernel :

- Reboot the system.
- In computing, rebooting is the process by which a running computer system is restarted, either intentionally or unintentionally.
- Reboots can be either cold (alternatively known as hard) whereby the power to the system is physically turned off and back on again, causing an initial boot of the machine, or warm (alternatively known as soft) where the system restarts without the need to interrupt the power.
- The term restart is used to refer to a reboot when the operating system closes all programs and finalizes all pending input and output operations before initiating a soft reboot.

Q.10. How device driver are installed? Explain steps in installation for both UNIX and WINDOWS OS separately. ***S-10(13M)***

Ans.

- Device driver is software that our computer components use to function correctly. These include motherboard, printer and video cards.
- Our IDE hard drives, CD/DVD ROMS or processor does not require a driver to perform properly. If we reinstall our operating system or upgrade, some devices will need a driver.
- It is important to have all driver disks or download the drivers before changing operating systems.
- In addition, a virus or errors in the operating system may corrupt our components.
- Installing a device driver is a necessary task.

(1) Steps in installation of device driver in WINDOWS :

Following are the instructions to be followed while installing device driver in WINDOWS.

(i) Downloaded Driver

(a) Download our specific driver from Lost My Drivers, Drivers and Software.

(b) Downloads or Open Drivers. Save the file in a "New" folder.

(c) Double-click on the download file. The install process will begin. Follow any online instructions including accepting user agreements and choosing the install folder.

(d) Click "Yes" if a prompt appears to reboot the PC. Save all work before clicking "Yes". If we are not, restart the computer manually.

(ii) **Driver Disk**

(a) Place the driver disk in our CD/DVD ROM. An automatic window should appear. If not, click on "My Computer." Then double-click on our CD/DVD ROM drive. Select the "Setup" or "Install" file.

(b) Click on the "Install Driver" or "Setup (Device) Driver" option. Some deriver disks display a new window while others begin the install process. automatically.

(c) Answer any questions about our drive during the install process. Save all applications open as we may have a prompt asking to "Reboot". Click the "Yes" tab when we have finished. Regardless of the prompt, restart the PC.

(2) **Steps in installation of device driver in UNIX :** It should contain the following installation check list.

(i) The name of the system holding the drive for the installation, and its device name.

(ii) Diskless, dataless, stand-alone, or server system.

(iii) The name of the server for the new client, if it's a dataless or diskless system, should be on your sheet along with its IP address.

(iv) The name of the host and domain.

(v) The IP address

(vi) The packages to install.

(vii) How to partition the disk (This is the map of the disk drive or drives you made earlier)

(viii) Whether to use a network database (This is the name of the NIS domain, if you intend to run NIS).

Step 1 : Booting the Installation Media :

The first step in installing a UNIX system is to load the mini-root into RAM (the mini-root is basically a scaled down kernel that will give us the ability to run the UNIX installation programs).

Step 2 : Installing the Master System :

- Once the mini-root is loaded, we are generally presented with the install options.
- Some systems leave us at a shell prompt.
- If this happens, enter install to start the installation procedure.
- Our distribution may be different, or it may be automatic.

Step 3 : Installing Optional or Additional Packages.

- Once the system is installed and rebooted, we are running UNIX.
- Of course, we shall still need to perform installations from time to time to add packages and applications.
- All UNIX packages and most standard applications for System V Release 4 use the pkgadd format.
- Installation of these packages and applications is automatic using the pkgadd utility.

  Example : RedHat Linux uses rpm, and BSD uses pkgadd via installsw (to make life easier).
- To run pkgadd on the install media, place the media in the drive and enter the command.

  pkgadd-d path-name-to-device pkg_name
- Using swmtool sun's Solaries system provides an X application to guide us through running pkgadd.
- Using rpm RedHat Linux uses a packaging format called rpm. To manipulate these packages we use the rpm program.
- Using installsw, most BSW implementations use the pkgadd format, but come with a simple to use program called installsw.

Step 4 : Adding a diskless Client to a server :

- We take two steps to add a diskless client to a server.

  (i) Add the common files to support any client.

  (ii) Add the specific files for this client.
- The first needs to be done only if this is the first client of this type and second is revision of the operating system to be installed.

Q.11. Design a device driver for handling a RAM DISK. ***W-09,10,11,12(8M)***

Ans. RAM Disk Driver :

- The RAM disk driver does not require any hardware. It uses function malloc to allocate space to store files.
- The device configuration specifies the maximum size the RAM disk should have, but the driver will only allocate sectors as they are used.
- Thus, even if we request a RAM disk of 256M but write files with only 3M size to the RAM disk, only those 3M (plus some overhead for FAT and directories) will be allocated.
- When files are deleted, the space previously allocated to the files is deallocated with function free.
- RAM disks format themselves automatically as an FAT-12 or FAT-16 volume when the disk is mounted.
- The application can reformat RAM disks as FAT-32, if desired. Any number of RAM disks can be installed.
- The RAMDisk driver is reentrant as long as malloc and free can be called simultaneously from several tasks.
- Device flag RTF_DEVICE_NEW_LOCK is supported for each RAM disk.

(1) RTFDevice.DeviceType

RTFDevice.DeviceType may be set to either RTF_DEVICE_FLOPPY or RTF_DEVICE_FDISK.

(2) RTFDevice.DeviceNumber

This field is ignored and should be set to 0, even if several RAM disks are used.

(3) RTFDevice.DeviceFlags

- The RAM disk driver does not define any device dependent device flags.
- The device independent flag RTF_SINGLE_FAT is not required, because the RAM disk driver will format the RAM disk with a file system with a single FAT by default.

(4) RTFDevice.Driver

This field must point to RTFDrvRAM.

(5) RTFDevice.DriverData

The RAM disk driver requires a pointer to a structure of type RTFDrvRAMData for RTFDevice.DriverData with the following layout:

```
typedef struct
{
DWORD Sectors;
...
}
```

(6) RTFDrvRAMData :

- RTFDrvRAMData Sectors specifies the maximum size of the RAM disk, in sectors of 512 bytes.
- If set to 0, a RAM disk with a maximum size of approximately 4M is set up.
- The other fields of RTFDrvRAMData are for the driver's internal use and should be left uninitialized or initialized to 0.

Q.12. Write a short note on driver memory allocation. ***S-13(4M)***

Ans. Driver memory allocation :

- Drivers can use system-allocated space within their device extensions as global storage areas for device-specific information.
- Drivers can use only the kernel stack to pass small amounts of data to their internal routines.
- To allocate I/O buffer space, the best memory allocation routines to use are MmAllocateNon CachedMemory,MmAllocateContiguousMemory SpecifyCache, AllocateCommonBuffer (if the driver's device uses bus-master DMA or a system DMA controller's auto-initialize mode), or ExAllocatePoolWithTag.
- Nonpaged pool typically becomes fragmented as the system runs, so a driver's DriverEntry routine should call these routines to set up any long-term I/O buffers the driver needs.
- Each of these routines, except ExAllocate PoolWithTag, allocates memory that is aligned on a processor-specific boundary (determined by the

processor's data-cache-line size) to provide best performance.

- Drivers should allocate I/O buffers as economically as possible, because nonpaged pool memory is a limited system resource.
- Typically, a driver should avoid calling these support routines repeatedly to request allocations of less than PAGE_SIZE because each allocation that is less than PAGE_SIZE also comes with a pool header that is used to internally manage the allocation.
- To allocate I/O buffer memory economically, following points are considered:

(1) Each call to MmAllocateNonCachedMemory or MmAllocateContiguousMemorySpecifyCache always returns a full multiple of the system's page size, of nonpaged system-space memory, whatever the size of the requested allocation. Therefore, requests for less than a page are rounded up to a full page and any remainder bytes on the page are wasted; they are inaccessible by the driver that called the function and are unusable by other kernel-mode code.

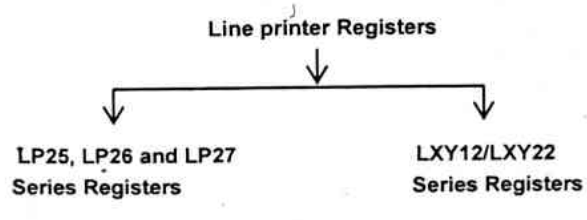
(2) Each call to AllocateCommonBuffer uses at least one adapter object map register, which maps at least one byte and at most one page. For more information about map registers and using common buffers.

IMPLEMENTATION WITH LINE PRINTER

Q.13. Mention different register available with line printer along with significance of each bit.

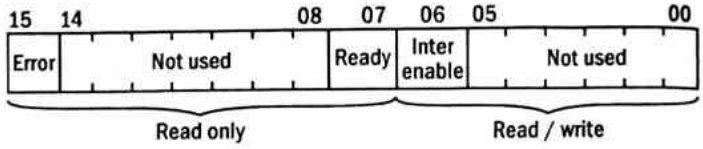
S-10(3M)

Ans. Various line printer register are categorized as follows :



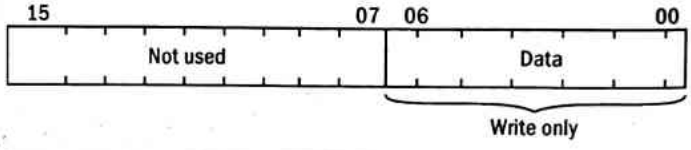
(1) LP25, LP26 and LP27 Series Registers :

(a) Control and Status Register (LXCS) 777514 :



Bit	Name
15	**Error**: Set whenever an error condition exists in the graphics line printer. Error conditions are power off, graphics line printer off-line, no paper, torn paper, band gate open, over-temperature alarm, or line-printer off-line. Generates an interrupt if INTERRUPT ENABLE . 06 is also set. Cleared by correcting the error condition.
14 – 08	**Not Used**
07	**Ready:** Set when the graphics line printer is ready for the next character to be loaded into the data buffer register. Generates an interrupt if INTERRUPT ENABLE . 06 is also set.
06	**Interrupt Enable**: When set allows an interrupt to occur when either the ERROR 15 or READY . 07 is also set. Cleared by loading with a 0. Also cleared by INIT.
05 – 00	**Not Used**

(b) Data Buffer Register (LXDB) 777516 :



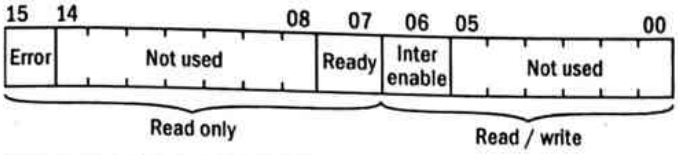
Bit	Name
15 – 07	Not Used
06 – 00	**Data:**7-bit ASCII character buffer. Characters are transferred to the lineprinter by loading this buffer.

(2) LXY12/LXY22 Series Registers :

- While the LXY12/LXY22 graphics line printers systems described here use different controllers depending on the model selected.

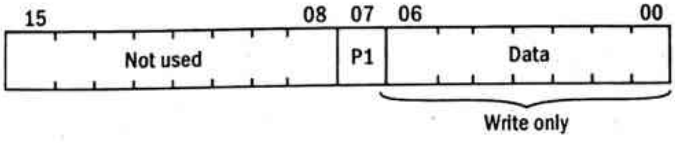
- Each controller has two registers: the Control and Status Register and the Data Buffer Register.
- The following are the controller's register drawings and bit definitions.

(a) Control and Status Register (LXCS) 777514 :



Bit	Name
15	**Error** : Set whenever an error condition exists in the graphics line printer. Error conditions are power off, graphics line printer off-line, no paper, form paper or form thickness adjustment level left up. Generates an interrupt if INTERRUPT ENABLE <06> is also set. Cleared by correcting the error condition.
14 – 08	Not Used
07	**Ready:** Set when the graphics line printer is ready for the next character to be loaded into the data buffer register. Generates an interrupt if INTERRUPT ENABLE <06> is also set.
06	**Interrupt Enable:** When set allows an interrupt to occur when either the ERROR <15> or READY <07> is also set. Cleared by loading with a 0. Also cleared by INIT.
05 – 00	Not Used

(b) Data Buffer Register (LXDB) 777516



Bit	Name
15 – 08	**Not Used**
07	**Paper Instruction (PI)**:Used to advance the paper up to 15 lines when DATA BIT <04> is also set. Data bits <03-00> contain the number of lines to be advanced. When the PLXY-11 plot software is used with some operating systems, the PI is disabled by installing jumper W5 on the controller logic board "A".
06 – 00	**Data** :Seven-bit ASCII character buffer. Characters are transferred to the line printer by loading this buffer.

Q.14. Design a device driver for handling a line printer. *S-12(8M)*

Ans. Handling a line printer :

- The line printer is an impact printer in which one line of text is printed at a time.
- They are mostly associated with unit record equipment and the early days of digital computing, but the technology is still in use.
- Print speeds of 600 lines-per-minute (approximately 10 pages per minute) were achieved in the 1950s, later increasing to as much as 1200 lpm.
- Line printers print a complete line at a time and have speeds in the range of 150 to 2500 lines per minute. The different types of line printers are drum printers and chain printers.
- The lp driver provides an interface to the port used by a printer.
- The lp driver interprets carriage returns, backspaces, line feeds, tabs, and form feeds in accordance with the modes that are set in the driver (through the splp command or configuration).
- The number of lines per page, columns per line, and the indentation at the beginning of each line can also be selected.
- The default for these modes can be found using the lsattr command.
- The following modes can be set with the LPRMODS ioctl operation:

Item	Description
PLOT	Determines if the data stream is interpreted by the device driver when formatting the text. If the PLOT mode is off, the text is formatted using the current values set with the LPRSET ioctl operation.
NOFF	If this mode is on, each form-feed

	character is replaced with a line-feed character, based on the current line value set with the LPRSET ioctl operation. This mode is ignored if the PLOT mode is active.
NONL	If this mode is on, each line-feed character is replaced with a carriage return. This mode is ignored if the PLOT mode is active.
NOCL	If this mode is off, a carriage return is inserted after each line-feed character. If the mode is on, no carriage return is inserted after the line-feed character. This mode is ignored if the PLOT mode is active.
NOTAB	If this mode off, 8 position tabs are simulated using spaces, If the NOTAB mode is on, the tab character is replaced with a space. This mode is ignored if the PLOT mode is active
NOBS	If this mode off, backspaces are sent to the printers. If the NOBS mode is on, the backspace is simulated by sending a carriage return followed by spaces to the proper print position. This mode is ignored if the PLOT mode is active.
NOCR	If this mode on, each carriage return is replaced with a line-feed character. This mode is ignored if the PLOT mode is active.
CAPS	If this mode on, lowercase characters are converted to uppercase. This mode is ignored if the PLOT mode is active.

Q.15. Explain the following entry point and routine for device driver of a line printer.

(i) init

(ii) open

(iii) close

(iv) LP work routine

(v) LP restart routine *W-10(6M), W-12(8M)*

Ans.

(i) Init Entry Point :

- Init performs all of the standard functions of a driver init entry point.
- It checks that the hardware is present, initializes the hardware, prints a message announcing the presence of the driver (and hardware), and initializes data structures internal to the driver.
- The pseudo-code below outlines the structure of the routine while the detailed code follows.
- In this driver we take a slightly more involved approach to verify that the hardware is present. Firstly (line 34)), we zero the control register and then check for the expected result.
- Then we set a specific value and make sure that the result is as expected.
- Only then we are willing to declare the hardware as present.

```
34      init entry point :
36          clear control register
37-38       if (control register ! = expected value)
            then return
39          write special value to control register
40 - 41     if (control register ! = expected value)
            then return
42          disable device by clearing control
            register
43          print : line printer exists message
44          set device exists flag
```

(ii) The Open Entry Point :

- We check to see if the device has been detected and if so, initialize it. Otherwise open subroutine return an error.
- An additional feature supported by this driver is the exclusive - open flag.
- When the user opens the device, the open call can include the O_EXCL flag indicating that no other open of the same device is to be permitted while this process has the device open

For example :

open ("/dev/lp", O_WRONLY|O_EXCL) ; .

- We start with a series of checks to see whether the device exists (line 50), is already open for

exclusive use (line 52), or is already open and this open is for exclusive use (lines 54 and 56).

- In all cases, the appropriate error code is returned by setting u.u_error.
- If no errors are detected (and the device is not already open), we initialize the printer (lines 61 and 62), mark the device as open (line 64), and check to see if this open is for exclusive use (line 65) and if so, set a flag to remember that fact.

```
46    open entry point :
50          if (device exists flag is  not set)
51                set error code
52          else if (device has already be opened
            exclusively)
53                set error code
54          else if (device has already been opened)
56                if (this open is for exclusive use)
57                  set error code
59    else
61          clear lp data register
62          set line feed, init, and select bits in
            control register
64          set lp open flag
65          if (this open is for exclusive use)
66                set exclusive open flag
```

(iii) The Close Entry Point :

- The close entry point for this driver has a problem none of previous drivers have had to cope with.
- It is possible for the driver still to be busy processing previous output requests when the application program issues the close.
- Therefore, we must wait until the printer has printed everything before complete the close operation and shut down the printer.
- In line 75 check to see if the device is still busy.
- If so, set a flag telling the rest of the driver that are waiting for the printer to complete printing and then go to sleep.
- When the printer is finished, the driver will check for the WAIT flag and if set, will issue a wake up which will cause a call to sleep to return.
- By line 80, the printer is quiet so clear all of the flags except LPEXIST (line 80) and disable the printer (line 81).

```
69    close entry point :
74          raise processor priority in order to
            disable interrupts
75            while (lp busy flag is set)
77            set flag indicating we are waiting to close
78            put the user process to sleep until
              awakened
80          clear all flags except the lp exists flag
81          clear lp control register thus disabling
            the device
82          restore processor priority
```

(iv) The Lpwork Routine :

- The lpwork routine is not a true entry point (i.e., it is not called by the operating system).
- It is a routine local to the driver that is called by other parts of the driver to start activity on the device.
- The lpwork routine is where the real work is done.
- The lpwrite routine is responsible for copying the data to be printed from the user's process to the clist.
- Lpwork is responsible for copying the data from the clist to the printer itself.
- The first half of this routine (lines 122 -135) is concerned with transferring data from the clist to the printer.
- The remainder of the routine handles cases such as the clist being empty.
- The lpwork may be called from the interrupt handler as well as from lpwrite.
- This routines are divided into two sections to make the commentary appear closer to the code which it describes.

```
116   lpwork routine :
121         set flag indicating device is busy
122         while (true i.e. forever)
124-127           spin until device is ready for
                  data or we lose patience
```

```
128          if we lost patience,
129               break out of loop
130          try to obtain byte from clist buffer
             if (no more data in clist buffer)
131               break out of loop
132          write byte to lp data register
133-134      tell printer to accept data
```

(v) The Lprestart Routine :

- When the timeout finally occurs then one-tenth of a second after the call to timeout regardless of whether the printer is busy or not, regardless of whether an interrupt has been dropped or not.
- When the time delay specified in the call to timeout expires, the kernel will call the specified routine (in this case lprestart)
- Lprestart first clears the timeout-pending flag (line 165) so that lpwork knows (in line 147) that no timeout is pending.
- It then raises the process priority and calls lpwork.
- This simulates a real interrupt.
- If an interrupt has been dropped, then lpwork will start things going again.
- If an interrupt has not been dropped, then lpwork will find the printer still busy and will do nothing (other than posting another timeout request)

```
162  lprestart routine :
165       clear the timeout pending flag
166       raise processor priority (disable
          interrupts)
167       call lpwork
168       restore processor priority
```

COMPARATIVE STUDY BETWEEN DEVICE DRIVERS FOR UNIX & WINDOWS

 # Q.16. Compare device driver for UNIX and WINDOWS operating system. ***W-11(5M), S-13(6M)***

Ans.

Sr. no	PARAMETER	UNIX	WINDOWS
(1)	Kernels and APIs	In UNIX, the API functions are called system calls. System calls are a programming interface common to all implementations of UNIX.	Windows has an API for programming calls to the executive. In addition to this, each subsystem provides a higher-level API.
(2)	Kernel Objects and Handles	In UNIX, the kernel object can be created using the system calls and it returns an unsigned integer.	In Windows, Windows APIs are used to create the kernel object and it returns a Windows-specific data type called HANDLE.
(3)	Hardware Drivers	UNIX allow dynamic loading and unloading of drivers in some implementation while in other implementation not	In Windows, the Windows driver model provides a platform for developing drivers for industry-standard hardware devices attached to a Windows-based system.
(4)	Process Management	UNIX operating systems support processes and threads.	Windows operating systems support processes and threads.

(5)	Signals, Exceptions, and Events	In UNIX, signals are used for typical events, simple inter process communication (IPC), and abnormal conditions such as floating point exceptions.	Windows supports some UNIX signals and others can be implemented using Windows API and messages. An event mechanism that handles expected events, such as communication between two processes.
(6)	User Interfaces	The current versions of UNIX support the graphical user interface using the X Windows system.	Windows contains the Graphics Device Interface (GDI) engine to support the graphical user interface.

POINTS TO REMMEBER :

(1) A Device Driver is glue between an OS and its I/O devices.
(2) Act as Translators, Requests to Command etc.
(3) Device drivers communicate directly with devices.
(4) A device is a physical or logical entity that requires control, resource management, or both from the operating system (OS).
(5) A device driver is a software module that manages the operation of a virtual or physical device, a protocol, or a service.
(6) Drivers can use system-allocated space within their device extensions as global storage areas for device-specific information. Drivers can use only the kernel stack to pass small amounts of data to their internal routines.

Notes :-

UNIT - VI
COMPILER

SYLLABUS

Phases of Compilers, Overview of Databases and Algorithms required for all phases. Role of lexical analyzer, recognition of tokens, Study of LEX & YACC.

UNIVERSITY PAPER SOLUTIONS
SINCE WINTER - 2009

Note to the students :
- The questions given below are from previous Nagpur University examination papers.
- Though these questions are from old university papers, they have been included in the new syllabus.
- Questions which are out of new syllabus are not included here.

WINTER - 09

Q.1. Explain database and algorithm for lexical phase of compiler. **8M**

Ans. P.6-5, Q.5.

Q.2. How tokens are recognized by lexical phase? Explain in details. **5M**

Ans. P.6-7, Q.9.

Q.3. Write a short note on YACC. **5M**

Ans. P.6-10, Q.12.

Q.4. Write a short note on LEX. **5M**

Ans. P.6-10, Q.13.

SUMMER - 10

Q.1. How LEX is used to design lexical analyzer? Write a l_x program to recognize identifier names, integer constants, floating point, constants and arithmetic operator. **9M**

Ans. P.6-12, Q.15.

Q.2. What is compiler? Explain in brief various databases used in lexical phase of compiler. **7M**

Ans. P.6-2 & 5, Q.1 & 5.

Q.3. What is cross compiler? How does it use boot-strapping concept? **6M**

Ans. P.6-9, Q.11.

Q.4. Write a short note on YACC. **5M**

Ans. P.6-10, Q.12.

WINTER - 10

Q.1. Explain the various phases of compiler. **8M**

Ans. P.6-4, Q.4.

Q.2. Define the following terms : **5M**
i) Token
ii) Lexeme
iii) Pattern

Ans. P.6-3, Q.3.

Q.3. Describe a tool for study of Lex. **7M**

Ans. P.6-11, Q.14.

Q.4. What is regular expression ?State basic rules for constructing regular expression for a given language. **6M**

Ans. P.6-8, Q.10.

Q.5. Write a short note on YACC. **5M**

Ans. P.6-10, Q.12.

SUMMER - 11

No question asked in University exam.

WINTER - 11

Q.1. Explain LEX in details with the help of example. **8M**

Ans. P.6-10, Q.13.

Q.2. Define the following terms: **5M**
i) Phase ii) Pass
iii) Pattern iv) Token
v) Lexeme

Ans. P.6-3, Q.3.

Q.3. Write a short note on database for lexical phase. **5M**

Ans. P.6-5, Q.5.

SUMMER - 12

Q.1. Explain in brief various databases used in lexical phases. **9M**

Ans. P.6-5, Q.5.

Q.2. Explain lexical error in details. **4M**

Ans. P.6-6, Q.7.

Q.3. Explain YACC in details. Also explain Lex. **9M**

Ans. P.6-10, Q.12 & 13.

Q.4. Explain how the tokens are specified. **4M**

Ans. P.6-7, Q.9.

WINTER - 12

Q.1. Draw a block diagram of the phases of compiler and indicate the main function of each phase. **9M**

Ans. P.6-4, Q.4.

Q.2. What is purpose of uniform symbol? Can uniform symbol be avoided? If so, what are gains and losses? **5M**

Ans. P.6-6, Q.6.

Q.3. What is cross compiler? Explain how boot strapping can be achieved. **5M**

Ans. P.6-9, Q.11.

SUMMER - 13

Q.1. State and explain the role of lexical analyzer. **6M**

Ans. P.6-7, Q.8.

Q.2. What are token recognizer? Explain the ways of designing the token recognizer. **7M**

Ans. P.6-7, Q.9.

Q.3. Write a short note on : **8M**

i) Regular expression

Ans. P.6-9, Q.10.

ii) Bootstrapping

Ans. P.6-9, Q.11.

Note : Also refer Paper Solutions at the end of the book.

SOLVED QUESTION BANK

[Sequence given as per syllabus]

INTRODUCTION

A compiler is a computer program (or set of programs) that transforms source code written in a programming language (the source language) into another computer language (the target language, often having a binary form known as object code) . The most common reason for wanting to transform source code is to create an executable program.

The name "compiler" is primarily used for programs that translate source code from a high-level programming language to a lower level language (e.g., assembly language or machine code).

A program for a computer must be built by combining these very simple commands into a program in what is called machine language. Since this is a tedious and error-prone process most programming is, instead, done using a high-level programming language.

This language can be very different from the machine language that the computer can execute, so some means of bridging the gap is required. Hence compiler is very essential

A compiler translates (or compiles) a program written in a high-level programming language that is suitable for human programmers into the low-level machine language that is required by computers. During this process, the compiler will also attempt to spot and report programmer mistakes.

COMPILER

Q.1. What is compiler? ***S-10(2M)***

Ans. Compiler :

- A compiler is a program that translates a high-level language program into functionally equivalent low-level language program.
- So, a compiler is basically a translator whose source language is the high-level language and the target language is a low-level language.
- As an important part of this translation process, the compiler reports to its user the presence of errors in the source program.

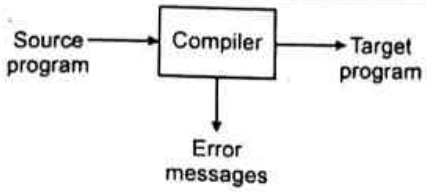


Fig. A compiler

Q.2. What is compilation process? Explain in details.

Ans. Compilation Process :

- Compilation is a process that translates a program in one language (the source language) into an equivalent program in another language (the object or target language).
- Programming language is a problem-oriented language and the target language is a machine language or assembly language (i.e. a machine-oriented language). Thus compilation is a fundamental concept in the production of software.
- It is the link between the (abstract) world of application development and the low-level world of application execution on machines.

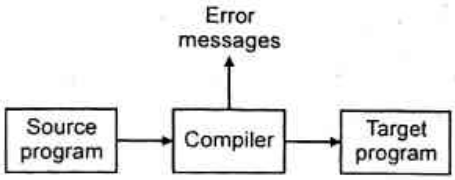


Fig. Compilation process

- Compilation program flowchart can be given as follows :

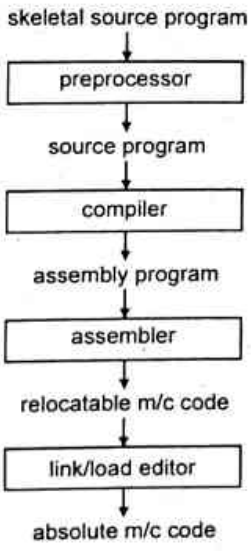


Fig. Compilation program flowchart

Q.3. Define the following :

i) Tokens

ii) Lexeme

iii) Patterns *W-10(5M)*

OR Define the following terms :

i) Phase

ii) Pass

iii) Pattern

iv) Token

v) Lexeme *W-11(5M)*

Ans. i) Tokens :

- Tokens are symbolic names for the entities that make up the text of the program.

  For example, if for the keyword 'if', and id for any identifier. These make up the output of the lexical analyser.
- Token is a string of one or more characters that is significant as a group. The process of forming tokens from an input stream of characters is called tokenization.

ii) Lexeme :

A lexeme is a sequence of characters from the input that match a pattern (and hence constitute an instance of a token).

For example, if matches the pattern for if ,and foo123bar matches the pattern for id.

iii) Patterns :

A pattern is a rule that specifies when a sequence of characters from the input constitutes a token.

For example, the sequence if for the token if , and any sequence of alphanumeric starting with a letter for the token id.

iv) Phase :

- The compiler has to undergo various different individual dependent as well as independent stages of compilation these stages of processing are called as phases of compilation.
- When the source language is large and complex, and high quality output is required, the design may be split into a number of relatively independent phases.
- Having separate phases means development can be parceled up into small parts and given to different people.

- It also becomes much easier to replace a single phase by an improved one, or to insert new phases later (e.g., additional optimizations).

v) Pass :

- Compiling involves performing lots of work and early computers did not have enough memory to contain one program that did all of this work. So compilers were split up into smaller programs which made a pass over the source (or some representation of it) performing some of the required analysis and translations.
- The ability to compile in a single pass has classically been seen as a benefit because it simplifies the job of writing a compiler and one-pass compilers generally perform compilations faster than multi-pass compilers.
- Many early languages were specifically designed so that they could be compiled in a single pass (e.g., Pascal).

PHASES OF COMPILER

Q.4. Draw a block diagram of the phases of compiler and indicate the main function of each phase. ***W-12(9M)***

OR Explain the various phases of compiler. ***W-10(8M)***

Ans. Phases of compiler :

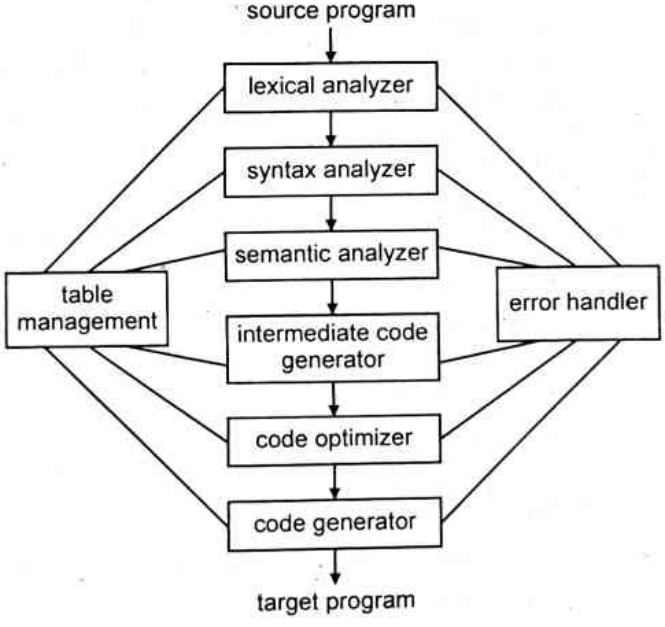


Fig. Phases of compiler

1) Lexical Analysis :

- Reads the characters of the source program and recognizes the tokens or basic syntactic components that they represent. It is able to distinguish and pass on, as single units, objects such as Numbers, Punctuation symbols, Operators, Reserved keywords, Identifiers and so on.
- It simplifies the syntax analyzer.
- By effectively reducing the size of the grammar the syntax analyzer has to handle. In a free-form language, the lexical analyzer ignores spaces, newlines, tabs and other layout characteristics, as well as comments.

Example : for i := 1 to 10 do
sum := sum + term[i];
('sum array')will be transformed by the lexical analyzer into the sequence of tokens:
for i := 1to 10 do
sum:=sum +term [i];

- Pascal, maintains a list of reserved words so that they can be distinguished from identifiers and passed to the next phase of the compiler in the form of, for example, a short integer code (or, use a symbol table).

2) Syntax Analysis :

- The syntax analyzer or parser has to determine how the tokens returned by the lexical analyzer should be grouped and structured according to the syntax rules of the language.
- Output representation of the syntactic structure often expressed in the form of a tree (the 'parse tree').
- Usually, lexical analyzer should be responsible for all the simple syntactic constructs, such as identifiers, reserved words and numbers, while the syntax analyzer should deal with all the other structures.

3) Semantic Analysis :

- To determine the semantics or meaning of the source program (the translation phase) may cope with tasks involving declarations and scopes of identifiers, storage allocation, type checking, selection of appropriate polymorphic operators, addition of automatic type transfers etc.

- This phase is often followed by a process that takes the parse tree from the syntax analyzer and produces a linear sequence of instructions equivalent to the original source program (instructions for a virtual machine).

4) Intermediate code generator :

- It uses the structure produced by the syntax analyzers to create a stream of simple instructions.
- Many styles of intermediate code are possible. One common style uses instructions with one operator and a small no of operands.
- The primary difference between intermediate code and assembly code is that the intermediate code need not specify the register to be used for each operation.

5) Code optimization :

It is an optional phase designed to improve the intermediate code so that the ultimate object program runs faster and takes less space.

6) Code Generation :

It is the final phase which takes the output from semantic analyzer or translator and output machine code or assembly language for the target hardware (machine's architecture is required to write good code generator).

7) Table management :

- It is used to keep track of the names used by the program and records essential information about each, such as its type (integer, real, etc).
- The data structure used to record this information is called symbol table.

8) Error handler :

- This phase is used to invoke when a flaw in the source program is detected.
- It must warn the programmer by issuing a diagnostic and adjust the information being passed from phase to phase so that each phase can proceed.

OVERVIEW OF DATABASE AND ALGORITHM

Q.5. Explain database and algorithm for lexical phase of compiler. *W-09(8M)*

OR Explain in detail database used in lexical phase of compiler.

OR Write a short note on database for lexical phase. *W-11(5M)*

OR Explain in brief various databases used in lexical phases. *S-10(5M), S-12(9M)*

Ans. Lexical phase :

Three task of lexical phase are :

i) To parse the source program into the basic elements or tokens of the language.

ii) To build a literal table and identifier table.

iii) To build a uniform symbol table.

Database in lexical phase :

The task given above require 5 databases

(1) Source program : Original form of program appears to compiler as a string of characters

(2) Terminal table :

- A permanent database that has an entry for each terminal symbol (e.g.: arithmetic operator, keywords, non alphanumeric symbols)
- Each entry include terminal symbol, an indication of its classification(operator, break character) and its precedence

Symbol	Indicator	precedence

Fig. Terminal table entry

(3) Literal table :

- This table is created by lexical analyzer to describe all literals used in the source program.
- There is one entry for each literal including value, a number of attributes, address denoting the location of literal at execution time and other information.
- The attribute such as data type or precision can be deduced from literal and filled in by lexical analysis

Literal	Base	Scale	Precision	Other information	Address

Fig. Literal table entry

(4) Identifier table :

- This table is created by lexical analysis to describe all identifier used in source program.

- There is one entry for each identifier.
- Lexical analysis creates the entry and places the name of the identifier into that entry.
- Identifier may be 1 to 31 symbols long; lexical phase may enter a pointer in identifier table for efficiency of storage. The pointer points to name in a table of names.
- The other parts is filled by data attributes and address of each identifier

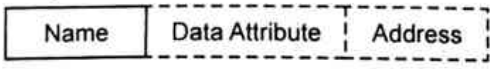


Fig. Identifier table entry

(5) Uniform symbol table :

- This table is created by lexical analysis to represent the program as a string of tokens rather than of individual character.
- Each uniform table consist of identification of table of which token is member and its index within that table.

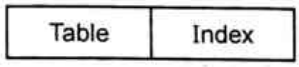


Fig. Uniform table entry

Algorithm :

Step 1 : The first task of lexical analysis algorithm is to parse the input character string into tokens.

Step 2 : The second is to make the appropriate entries in the tables.

Step 3 : Input string is separated into tokens by break character.

Step 4 : Lexical analysis identifies 3 types of tokens: terminal symbol, possible identifier, and literals. If token does not fit into any of these categories, then it's an error and particular flag is set according to the error.

Step 5 : If particular token is not in table then entry is made in the table

Q.6. What is purpose of uniform symbol? Can uniform symbol be avoided? If so, what are gains and losses? *W-12(5M)*

Ans.

- Uniform symbol table created during lexical analysis to represent the program as a string of tokens, rather than of individual characters. Each uniform symbol contains the identification of the table to which it belongs (IDN → Identifier table, LIT → Literal Table, TRM → Terminal Symbol Table)and index within that table.
- Lexical analysis creates a uniform symbol of type 'IDN' and insert it into the uniform symbol table.
- The lexical analysis may make one complete pass over the source code and produce the entire uniform symbol table.
- Another scheme is to have the lexical analyzer called only when the syntax phase needs the next token.
- Each new entry made in the literal table consist of the literal and all its attributes.
- Regardless of whether or not a new entry is made, a new uniform symbol of type 'LIT' is created and put into the uniform symbol table
- Each entry in the symbol table is for the declaration of a name.
- The format of entries does have to be uniform, because the information saved about a same depends on the usage of time.
- Each entry can be implemented as a record consisting of a sequence of consecutive words of memory.
- To keep symbol table entries uniform, it may be convenient for some of the information about a name to be kept outside the table entry, with only a pointer to this information stored in the record.

Q.7. Explain lexical error in details. *S-12(4M)*

Ans. Lexical error :

- A lexical error is a mistake in a lexeme, for example, typing technique instead of then, or missing off one of the quotes in a literal.
- There are relatively few errors which can be detected during lexical analysis.

a) Strange characters

- Some programming languages do not use all possible characters, so any strange ones which appear can be reported.
- However almost any character is allowed within a quoted string.

b) Long quoted strings (1)

- Many programming languages do not allow quoted strings to extend over more than one line; in such cases a missing quote can be detected.
- Languages of this type often have some way of automatically joining consecutive quoted strings together to allow for really long strings.

c) Long quoted strings (2)

- If quoted strings can extend over multiple lines then a missing quote can cause quite a lot of text to be 'swallowed up' before an error is detected.
- The error will probably then be reported as somewhere in the text of the next quoted string, which is unlikely to make sense as part of a program.

d) Invalid numbers

- A number such as 123.45.67 could be detected as invalid during lexical analysis (provided the language does not allow a full stop to appear immediately after a number).
- Some compiler writers prefer to treat this as two consecutive numbers 123.45 and .67 as far as lexical analysis is concerned, leave it to the syntax analyser to report an error.
- Some languages do not allow a number to start with a full stop/decimal point, in which case the lexical analyser can easily detect this situation.

ROLE OF LEXICAL ANALYZER

Q.8. State and explain the role of lexical Analyzer. *S-13(6M)*

Ans. Lexical Analyzer :

- A lexical analyser or scanner is a program that group sequences of characters into lexemes and outputs (to the syntax analyser) a sequence of tokens.
- Lexical analysis is the first phase of a compiler.
- Main task of lexical analyzer is to read the input characters and produce a sequence of tokens that will be processed by the Parser.
- Upon receiving a "get-next-token" command from the Parser, the input is read to identify the next token.
- Since the lexical analyzer is the part of the compiler that reads the source text, it may also perform certain secondary tasks at the user interface.
- One such task is tripping out from the source program comments and white space in the form of blank, tab, and newline characters.
- Another is correlating error messages from the compiler with the source program.

  For example, the lexical analyer may keep track of the number of newline characters seen, so that a line number can be associated with an error message.
- In some compilers, the lexical analyzer is in charge of making a copy of the source program with the error messages marked in it.
- If the source language supports some macro pre-processor functions, then these pre-processor functions may also be implemented as lexical analysis takes place.
- While looking for next token it eliminates comments and white-spaces.
- Tokens are treated as Terminal Symbols in the Grammar for the source program.

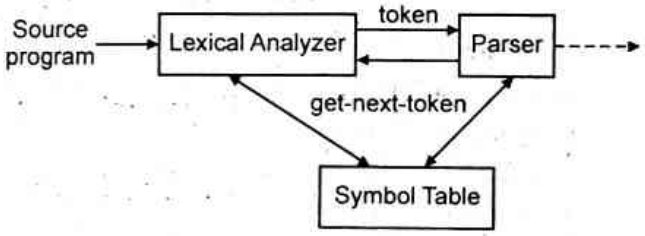


Fig. Lexical analyzer

TOKEN RECONGNITION

Q.9. What are token recognizer? Explain the ways of designing the token recognizer. *S-13(7M)*

OR Explain how the tokens are specified. *S-12(4M)*

OR How tokens are recognized by lexical phase? Explain in details. *W-09(5M)*

Ans. Token recognizer :

- A token is a string of one or more characters that is significant as a group. The process of forming tokens from an input stream of characters is called tokenization.

- Tokens are identified based on the specific rules of the lexer. Some methods used to identify tokens include: regular expressions, specific sequences of characters known as a flag, specific separating characters called delimiters, and explicit definition by a dictionary
- Tokens are often categorized by character content or by context within the data stream.
- Categories are defined by the rules of the lexer. Categories often involve grammar elements of the language used in the data stream.
- Programming languages often categorize tokens as identifiers, operators, grouping symbols, or by data type.
- Written languages commonly categorize tokens as nouns, verbs, adjectives, or punctuation. Categories are used for post-processing of the tokens either by the parser or by other functions in the program.
- A lexical analyzer generally does nothing with combinations of tokens, a task left for a parser.

For example, a typical lexical analyzer recognizes parentheses as tokens, but does nothing to ensure that each "(" is matched with a ")".

Example :

Consider the expression in the C programming language :

sum = 3 + 2

Tokenized and represented by the following table:

Lexeme	Token type
sum	Identifier
=	Assignment operator
3	Integer literal
+	Addition operator
2	Integer literal
;	End of statement

- **Token specification :**

Notational Short-hands

(a) One or more instances :

$(r)^+$ $digit^+$

(b) Zero or one instance :

r^+ is a shorthand for r|ε (E(+|-)?digits)?

(c) Character classes :

[a-z] denotes a|b|c|...|z

[A-Za-z] [A-Za-z0-9]

- **Recognition of tokens :**

Task of recognition of token in a lexical analyzer:

i) Isolate the lexeme for the next token in the input buffer.

ii) Produce as output a pair consisting of the appropriate token and attribute-value, such as <id,pointer to table entry> , using the translation table given in the Fig below.

- Task of recognition of token in a lexical analyzer

Regular expression	Token	Attribute-value
if	if	-
id	id	Pointer to table entry
<	relop	LT

- **Methods to recognition of token :**

Use Transition Diagram : Transition Diagram (Stylized flowchart) depict the actions that take place when a lexical analyzer is called by the parser to get the next token.

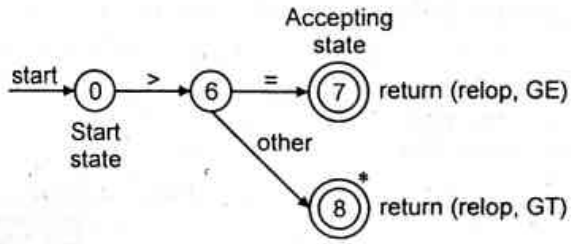


Fig. Transition Diagram

- **Implementing a Transition Diagram**

i) Each state gets a segment of code.

ii) If there are edges leaving a state, then its code reads a character and selects an edge to follow, if possible.

iii) Use nextchar() to read next character from the input buffer.

Q.10. What is regular expression ?State basic rules for constructing regular expression for a given language. **W-10(6M)**

OR Write a short note on Regular expression. *S-13(4M)*

Ans. Regular Expression :

- Regular Expressions are the most popular specification formalisms to describe Lexemes and map them to Tokens.

  Example : An identifier is made by a letter followed by zero or more letters or digits:

  letter (letter | digit)*
- The vertical bar | means "or"; Parentheses are used to group sub-expressions.
- The * means zero or more occurrences.
- Each Regular Expression, say R, denotes a Language, L(R).

  The following are the rules to build them over an alphabet V:

(a) If a $\in$ V $\cup$ {Q} then a is a Regular Expression denoting the language {a};

(b) If R, S are Regular Expressions denoting the Languages L(R) and L(S)

then :

(i) R | S is a Regular Expression denoting L(R) $\cup$ L(S);

(ii) RS is a Regular Expression denoting the concatenation L(R)L(S), i.e. L(R)L(S) = {rs | r $\in$ L(R) and s $\in$ L(S)};

(iii) R* is a Regular Expression denoting L(R)*, zero or more concatenations of L(R), i.e. L(R)*= S∞ i=0 L(R)i;

(iv) (R) is a Regular Expression denoting L(R).

Example :

Let V = {a, b}.

- The Regular Expression a|b denotes the Language {a, b}.
- The Regular Expression (a | b)(a | b) denotes the Language {aa, ab, ba, bb}.
- The Regular Expression a* denotes the Language of all strings of zero or more a's, { ϕ, a, aa, aaa, . . .}.
- The Regular Expression (a|b)* denotes the Language of all strings of a's and b's.

CROSS COMPILER

Q.11. What is cross compiler? How does it use bootstrapping concept? *S-10(6M)*

OR Write short note on bootstrapping. *S-13(4M)*

OR What is cross compiler? Explain how bootstrapping can be achieved. *W-12(5M)*

Ans. Cross compiler :

- A cross compiler is capable of creating executable code for a platform other than the one on which the compiler is running.
- Cross compiler tools are used to generate executables for embedded system or multiple platforms.
- It is used to compile for a platform which is not feasible to do the compiling, like microcontrollers that doesn't support an operating system.
- It becomes more common to use this tool for virtualization where a system may have one or more platforms in use.
- Bootstrapping is the process of writing a compiler (or assembler) in the target programming language which is intended to compile. Applying this technique leads to a self-hosting compiler.
- Bootstrapping a compiler has the following advantages:

(1) It is a non-trivial test of the language being compiled

(2) Compiler developers only need to know the language being compiled.

(3) Compiler development can be done in the higher level language being compiled; improvements to the compiler's back-end improve not only general purpose programs but also the compiler itself.

(4) It is a comprehensive consistency check as it should be able to reproduce its own object code.

- For cross-compiling, we first edit bootstrap.sh and comment out the lines at the end as follows. This is intended to interrupt the bootstrapping process, so we can take some parameters. If this is not done, then a native install will occur.
- **Compiling for multiple machines :**

  For example, a company may wish to support several different versions of an operating system or to support several different operating systems.

By using a cross compiler, a single build environment can be set up to compile for each of these targets.

- **Compiling on a server farm :** Similar to compiling for multiple machines, a complicated build that involves many compile operations can be executed across any machine that is free, regardless of its underlying hardware or the operating system version that it is running.
- **Bootstrapping to a new platform :** When developing software for a new platform, or the emulator of a future platform, one uses a cross compiler to compile necessary tools such as the operating system and a native compiler.

STUDY OF LEX AND YACC

Q.12. Write a short note on YACC. *S-10, W-09,10(5M)*

OR Explain YACC in detail. *S-12(4M)*

Ans. YACC (Yet Another Compiler-Compiler) :

- YACC provides a general tool for describing the input to a computer program.
- The YACC user specifies the structure of his input, together with code to be invoked as each such structure is recognized.
- YACC turns such a specification into a subroutine that handles the input process; frequently, it is convenient and appropriate to have most of the flow of control in the user's application handled by this subroutine.
- Computer program input generally has some structure; in fact, every computer program that does input can be thought of as defining an "input language" which it accepts.
- An input language may be as complex as a programming language, or as simple as a sequence of numbers. Unfortunately, usual input facilities are limited, difficult to use, and often are lax about checking their inputs for validity.
- The input subroutine produced by YACC calls a user-supplied routine to return the next basic input item. Thus, the user can specify his input in terms of individual input characters, or in terms of higher level constructs such as names and numbers.
- The user-supplied routine may also handle idiomatic features such as comment and continuation conventions, which typically defy easy grammatical specification.
- YACC is written in portable C. The class of specifications accepted is a very general one. e.g., LALR(1) grammars with disambiguating rules.
- In addition to compilers for C, APL, Pascal, RATFOR, etc., YACC has also been used for less conventional languages, including a phototype-setter language, several desk calculator languages, a document retrieval system, and a Fortran debugging system.

Q.13. Write a short note on LEX. *W-09(5M)*

OR Explain Lex in details with the help of example. *W-11(8M)*

OR Explain Lex. *S-12(5M)*

Ans. LEX :

- Lex is a program generator designed for lexical processing of character input streams.
- It accepts a high-level, problem oriented specification for character string matching, and produces a program in a general purpose language which recognizes regular expressions.
- Lex is not a complete language, but rather a generator representing a new language feature which can be added to different programming languages, called "host languages".
- Lex can write code in different host languages. Compatible run-time libraries for the different host languages are also provided. This makes Lex adaptable to different environments and different users.
- Lex turns the user's expressions and actions into the host general-purpose language; the generated program is named yylex.
- The yylex program will recognize expressions in a stream and perform the specified actions for each expression as it is detected.

Source -> [Lex] -> yylex

Input -> [yylex] -> Output

Fig. An overview of Lex

- Lex can also be used with a parser generator to perform the lexical analysis phase; it is particularly easy to interface Lex.
- Lex generates a deterministic finite automata from the regular expressions in the source.
- Lex helps write programs whose control flow is directed by instances of regular expressions in the input stream. It is well suited for editor-script type transformations and for segmenting input in preparation for a parsing routine.
- Lex source is a table of regular expressions and corresponding program fragments.
- The table is translated to a program which reads an input stream, copying it to an output stream and partitioning the input into strings which match the given expressions.
- As each such string is recognized the corresponding program fragment is executed.
- The recognition of the expressions is performed by a deterministic finite automata generated by Lex.
- The program fragments written by the user are executed in the order in which the corresponding regular expressions occur in the input stream.

Structure of a Lex :

The structure of a Lex file is intentionally similar to that of a YACC file; files are divided into three sections, separated by lines, as follows :

1) **Definition Section :** The definition section defines macros and imports header files written in C. It is also possible to write any C code here, which will be copied verbatim into the generated source file.
2) **Rules Section :** The rules section associates regular expression patterns with C statements. When the lexer identifies text in the input matching a given pattern, it will execute the associated C code.
3) **C code section :** The C code section contains C statements and functions that are copied verbatim to the generated source file. These statements presumably contain code called by the rules in the rules section. In large programs it is more convenient to place this code in a separate file linked in at compile time.

Q.14. Describe a tool for study of Lex. *W-10(7M)*

Ans.

- Several tools have been built for constructing lexical analyzers from special purpose notations based on regular expressions.
- A tool, called Lex that has been widely used to specify lexical analyzers for a variety of languages, refer to the tool as Lex compiler, and its input specification as the Lex language.
- First, a specification of a lexical analyzer is prepared by creating a program lex.l in the Lex language. Then, lex.l is run through the Lex compiler to produce a C program lex.yy.c.
- The program lex.yy.c consists of a tabular representation of a transition diagram constructed from the regular expression of lex.l, together with a standard routine that uses the table to recognize lexemes.
- The actions associated with regular expression in lex.l are pieces of C code and are carried over directly to lex.yy.c. Finally lex.yy.c is run through the C compiler to produce an object program a.out, which is the lexical analyzer that transforms an input stream into a sequence of tokens.

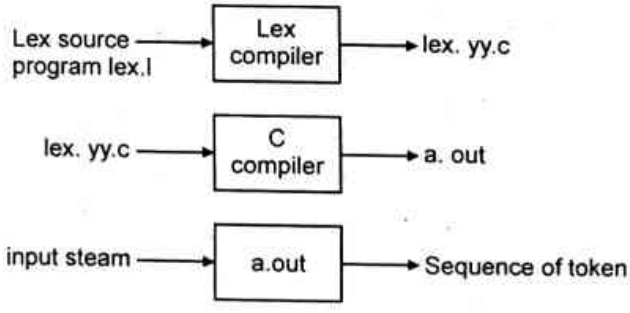


Fig. Creating a lexical analyzer with lex

Q.15. How Lex is used to design lexical analyser? Write lx program to recognize identifier name, integer constant, floating point constant and arithmetic operator. **S-10(9M)**

Ans.

- LPDT (Language Processor Development Tools) generates the analysis phase of a language processor whose source language is L.
- The LPDT requires the following two inputs :
  1) Specification of a grammar of language L.
  2) Specification of semantic actions to be performed in the analysis phase.
- It generates programs that perform lexical, syntax and semantic analysis of the source program and construct the IR.
- These programs collectively form the analysis phase of the language processor.
- These are the lexical analyzer generator LEX and the parse generator YACC.
- The input to these tools is a specification of the lexical and syntactic constructs of L, and the semantic actions to be performed on recognizing the constructs.
- The specification consists of a set of translation rules of the form
  <string specification> {<semantic action>}
  where <semantic action> consists of C code.
- This code is executed when a string matching <string specification> is encountered in the input.
- LEX and YACC generate C programs which contain the code for scanning and parsing, respectively, and the semantic actions contained in the specification.
- Figure shows a schematic for developing the analysis phase of a compiler or language L using LEX and YACC.

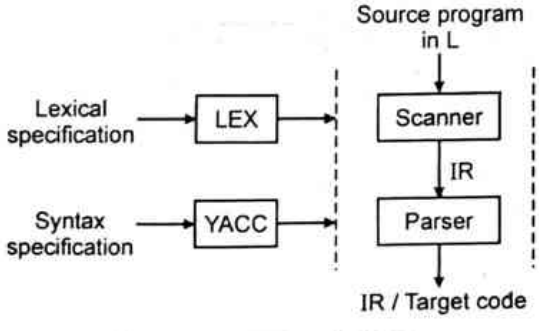


Fig. Using LEX and YACC

LEX :

- LEX accepts an input specification which consists of two components.
- The first component is a specification of strings representing the lexical units in L, e.g. id's and constants.
- This specification is in the form of regular expression.
- The second component is a specification of semantic actions aimed at building an IR.

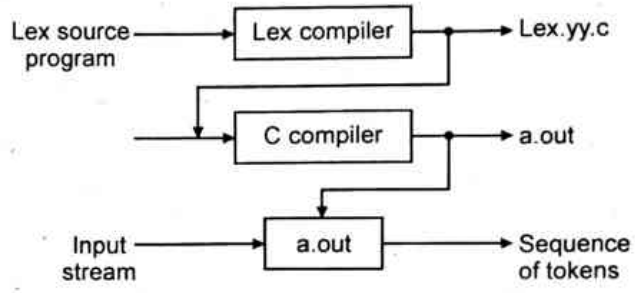


Fig. Creating a lexical analyser with lex

- LEX has been widely used to specify lexical analyzer for variety of languages. Tool has Lex compiler so its input specification has Lex language.
- Lex is a software that takes as input specification of a set of regular expression together with actions to be taken.
- The output of Lex is a program that recognizes the regular expression and acts appropriately on each.

Structure of Lex Programs

The general form of a lex program like lex.1 is
declarations
%%
translation rules
%%
auxiliary functions
lx program to recognize identifier name, integer constant, floating point constant and arithmetic operator is as follows :

```
%{
/*Literal block*/
#include "scan.h" /*Scanner header file*/
int line = 1; /*Current line number*/
```

```
int pos = 0; /*Start position of token*/
int epos = 1; /*End position of token*/
OpType op; /*Last operator scanned*/
void lex_err(char *, char *); /*Reports lexical errors*/
%}
/*Regular definitions*/
letter      [A-Za-z]/*letter can be underscore*/
digit       [0-9]
blank_str   [ \t]+
identifier  ({letter} ({letter} | {digit})*
int_literal {digit}+str_literal \"([^\"\n])*\"
/*Regular definitions to match invalid tokens*/
open_string     \"({^\"\n])*$
%% /*Second section*/
\n {line++; epos = 1;}
{blank_str} {epos +=yyleng;}
[Ii][Ff] {pos = epos; epos += 2; return IF_TK;}
[Tt][Hh][Ee][Nn] {pos = epos; epos +=4; return
THEN_TK;}
[Ee][Ll][Ss][Ee] {pos = epos; epos +=4; return
ELSE_TK;}
[Ee][Nn][Dd]      {pos = epos; epos +=3; return
END_TK;}
[Ww][Hh][Ii][Ll][Ee]     {pos = epos; epos +=5;
return WHILE_TK;}
[Dd][Oo]          {pos = epos; epos +=2; return
DO_TK;}
[Rr][Ee][Aa][Dd] {pos = epos; epos +=4; return
READ_TK;}
[Ww][Rr][Ii][Tt][Ee] {pos = epos; epos +=5; return
WRITE_TK;}
{identifier} {pos = epos; epos += yyleng; return
ID;}
{str_literal} {pos = epos; epos += yylen; return
STRLIT;}
{int_literal} {pos = epos; epos += yyleng; return
INTLIT;}
/*Regular definition to identify arithmetic
operator*/
"+" {op = ADD; pos = epos; epos += 1; return
ADDOP;}
"-" {op = SUB; pos = epos; epos += 1; return
ADDOP;}
"*" {op = MUL; pos = epos; epos += 1; return
MULOP;}
"/" {op = DIV; pos = epos; epos += 1; return
MULOP;}
"=" {op = EQ; pos = epos; epos += 1; return
RELOP;}
"<>" {op = NE; pos = epos; epos += 2; return
RELOP;}
"<" {op = LT; pos = epos; epos += 1; return
RELOP;}
"<=" {op = LE; pos = epos; epos += 2; return
RELOP;}
">" {op = GT; pos = epos; epos += 1; return
RELOP;}
">=" {op = GE; pos = epos; epos += 2; return
RELOP;}
":=" {op = ASGN; pos = epos; epos += 2; return
ASGNOP;}
"," {pos = epos; epos += 1; return ' , ';}
";" {pos = epos; epos += 1; return ' ; ';}
"(" {pos = epos; epos += 1; return ' ( ';}
")" {pos = epos; epos += 1; return ' ) ';}
/*Regular definition to indentify Unsigned
integer or floating point numbers*/
    digit → [0-9]
    digits → digit
    number → digits (. digits)?(E+-]?digits)?
}
```

POINTS TO REMEMBER :

(1) A compiler is a computer program (or set of programs) that transforms source code written in a programming language (the source language) into another computer language (the target language, often having a binary form known as object code).

(2) The most common reason for wanting to transform source code is to create an executable program.

(3) Compilation is a process that translates a program in one language (the source language) into an equivalent program in another language (the object or target language).

(4) A cross compiler is a compiler capable of creating executable code for a platform other than the one on which the compiler is running.

(5) Lex is a program generator designed for lexical processing of character input streams.

(6) YACC provides a general tool for describing the input to a computer program.

SUMMER EXAMINATION - 2012

B.E. (Computer Science & Engg.)

SYSTEM PROGRAMMING

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	26 M
Unit - III	18 M
Unit - IV	19 M
Unit - V	21 M
Unit - VI	26 M
Questions covered in VBD	136 M (100%)

SUMMER EXAMINATION - 2012
QUESTION PAPER

SECTION - A

Q.1.(a) Discuss the formats of data structure needed for two pass assembler design in detail. **9M**

(b) What are the advantages and disadvantages of using the base register in IBM - 360? **4M**

Q.2.(a) Differentiate between the following:

i) Forward Reference and Backward Reference

ii) USING and BALR

iii) Open subroutine and Close subroutine. **9M**

(b) Explain when literals are defined as value of LC. What happens if LTORG is present in the assembly program? Explain with example. **4M**

Q.3. Explain in detail the two pass Macro processor with the help of neat and clean flowchart. What macro facilities are restricted in this design? **13M**

Q.4.(a) Explain how MDI and MDLC help in designing a macro capable of handling macro definition within macro. **4M**

(b) Discuss the advantages and disadvantages of combining a macro processor with pass-1 of assembler. **4M**

(c) What are the two special pseudo-op used in conditional macro? Explain with example. **5M**

Q.5. Consider the following three programs:

```
PGA     START
        ENTRY   PGA 1
        EXTRN   PGB, PGC, PGC2
        DC      A (PGA), A (PGB+3)
PGA1    DC      A(PGA1 – PGA), A(PGC2 – PGC)
        END
PGB     START
        ENTRY   PGB1
        EXTRN   PGA, PGC1
PGB1    DC      A(PGC – 2), A(PGB1)
PGB2    DC      A(PGB+4), A(PGB1 – PGB)
PGB3    DC      A(PGC1+PGB – PGA – 16)
        END
PGC     START
        ENTRY   PGC1, PGC2
        DC      A(PGC2 – PGC)
PGC1    DC      A(PGC1 – 4), (*+4)
PGC2    DC      A(PGC+PGC2) – PGC1)
        END
```

Assume that these programs are loaded at location 400 in the order PGA, PGB, then PGC. Obtain ESD, TXT, RLD, GEST and LESA. **14M**

SECTION - B

Q.6.(a) Write down the structure of a device driver in any operating system. Describe each part in brief. **8M**

(b) Write down the steps for installation of any device driver and specify the setting in system files. **5M**

Q.7.(a) Design a device driver for handling a line printer. **8M**

(b) Explain Symbolic debugger. **5M**

Q.8.(a) Explain in brief the various databases used in lexical phase. **9M**

(b) Explain lexical error in detail. **4M**

Q.9.(a) Explain YACC in detail. Also explain LEX. **9M**

(b) Explain how the tokens are specified. **4M**

Q.10. Write short notes on (any Three). **14M**

(i) Link editor

(ii) Segment header

(iii) Storage classes

(iv) COFF symbol table.

SUMMER EXAMINATION - 2012

SOLUTION

(Unit-wise)

UNIT - I

(Total Marks : 26)

Q.1.(a) Discuss the formats of data structure needed for two pass assembler design in detail. **9M**

Ans. See P.1-15, Q.24

Q.1.(b) What are the advantages and disadvantages of using the base register in IBM - 360? **4M**

Ans. See P.1-10, Q.10

Q.2.(a) Differentiate between the following:

(i) Forward Reference and Backward Reference.

(ii) USING and BALR

(iii) Open subroutine and Close subroutine. **9M**

Ans. See P.1-13, 14, Q.20, Q.21, Q.22.

Q.2.(b) Explain when literals are defined as value of LC. What happens if LTORG is present in the assembly program? Explain with example. **4M**

Ans. See P.1-12, Q.16

UNIT - II

(Total Marks : 26)

Q.3 Explain in detail the two pass Macro processor with the help of neat and clean flowchart. What macro facilities are restricted in this design? **13M**

Ans. See P.2-10, Q.8(a)

Q.4.(a) Explain how MDI and MDLC help in designing a macro capable of handling macro definition within macro. **4M**

Ans. See P.2-12, Q.8(b)

Q.4.(b). Discuss the advantages and disadvantages of combining a macro processor with pass-1 of assembler. **4M**

Ans. See P.2-13, Q.9

Q.4.(c) What are the two special pseudo-op used in conditional macro? Explain with example. **5M**

Ans. See P.2-5, Q.4

UNIT - III

(Total Marks : 18)

Q.5. Consider the following three programs :-

```
PGA     START
        ENTRY   PGA1
        EXTRN   PGB, PGC, PGC2
        DC      A(PGA), A(PGB+3)
PGA1    DC      A(PGA1 – PGA), A(PGC2
                                – PGC)
        END

PGB     START
        ENTRY   PGB1
        EXTRN   PGA, PGC1
PGB1    DC      A(PGC – 2), A(PGB1)
PGB2    DC      A(PGB+4), A(PGB1 – PGB)
PGB3    DC      A(PGC1+PGB – PGA – 16)
        END

PGC     START
        ENTRY   PGC1, PGC2
        DC      A(PGC2 – PGC)
PGC1    DC      A(PGC1 – 4), (* + 4)
PGC2    DC      A(PGC+PGC2 – PGC1)
        END
```

Assume that these programs are loaded at location 400 in the order PGA, PGB, then PGC, Obtain ESD, TXT, RLD, GEST and LESA. **14M**

Ans. See P.3-22, Q.28.

Q.10.(i) Write short note on Link Editor. **4M**

Ans. See P.3-12, Q.16.

UNIT - IV

(Total Marks : 19)

Q.7.(b) Explain symbolic debugger. **5M**

Ans. See P.4-11, Q.13.

Q.10. Write a short note on: **14M**

(i) Segment header

Ans. See P.4-11, Q.12.

Q.10.(ii) Segment header

Ans. See P.4-4, Q.2.

Q.10.(iii) Storage classes

Ans. See P.4-7, Q.5.

Q.10.((iv) COFF symbol table.

Ans. See P.4-5 Q.3.

UNIT - V

(Total Marks : 21)

Q.6.(a). Write down structure of device driver in any operating system. Describe each part in brief. **8M**

Ans. See P.5-3, Q.1.

Q.6.(b) Write down steps for installation of any device driver and specify the setting in system files. **5M**

Ans. See P.5-8, Q.9.

Q.7.(a) Design device driver for handling a line printer. **8M**

Ans. See P.5-13, Q.14.

UNIT - VI

(Total Marks : 26)

Q.8.(a) Explain in brief various databases used in lexical phases. **9M**

Ans. See P.6-5, Q.5.

Q.8.(b) Explain lexical error in details. **4M**

Ans. See P.6-6, Q.7.

Q.9.(a) Explain YACC in details. Also explain Lex. **9M**

Ans. See P.6-10, Q.12 & 13.

Q.9.(b) Explain how the tokens are specified. **4M**

Ans. See P.6-7, Q.9.

Notes :-

WINTER EXAMINATION - 2012

B.E. (Computer Science & Engg.)

SYSTEM PROGRAMMING

Statistical Analysis

Unit Number	Marks
Unit - I	27 M
Unit - II	23 M
Unit - III	24 M
Unit - IV	26 M
Unit - V	21 M
Unit - VI	19 M
Questions covered in VBD	140 M (100%)

WINTER EXAMINATION - 2012
QUESTION PAPER

SECTION - A

Q.1.(a) "Operating system as a user viewpoint." Comment on this. **4M**

(b) What are advantages of using base addressing in IBM 360? **5M**

(c) What are the pseudo instructions? Explain USING, DROP, LTORG, EQU. **5M**

Q.2. Prepare ESD, RLD, and GEST for following program : **13M**

```
PGI.   START
       ENTRY      SYM11
       EXTRN,     PG2, SYM23
       DC         A(PG1), A (PG2 + 4)
Sym11  DC         A(SYM11 – PGI)
                  A (SYM23–PG-2)
       END
PG-2   START
       ENTRY      SYM23
       EXTRN      PG1, SYM11
Sym21  DC         A(Sym11–2), A(Sym21)
Sym22  DC         A(PG2 + 4), A(SYM21–PG2)
Sym23  DC         A(Sym21–PG1)
       END
```

Q.3.(a) Explain various databases required in the design of two pass assembler with their format. **9M**

(b) Draw the neat sketch of IBM 360 machine architecture. **4M**

Q.4. Give the Entries in MNT, ALA and MDT for the following definition and call : **13M**

```
MACRO
OPRTN      & Macnm, &OP
MACRO
&Macnm     &A, &B
  L        1, &A
 & OP      1, &B
  ST       1, &B
MEND
MEND
START
  :
  :
DATA 1     DC     F' 10'
DATA 2     DC     F' 11'
DATA 3     DC     F' 20'
  :
  :
OPRTN      ADD    A
OPRTN      SUB    S
  :
  :
ADD        DATA 1,  DATA 2
  :
  :
SUB        DATA 1,  DATA 3
END
```

Q.5.(a) Explain the following address constants :

(i) Absolute.

(ii) Simple Relocatable

(iii) Complex Relocatable.

What is a role they play in design of loader? **7M**

(b) What is a use of stack frame in macro processor? Give the format of stack frame along with example. **6M**

SECTION - B

Q.6.(a) Explain following entry points and routines for the device drivers of a line printer :

(i) Init.

(ii) Open

(iii) Close

(iv) LP work routine

(v) LP restart routine. **8M**

(b) What is cross compiler? Explain how boot strapping can be achieved. **5M**

Q.7.(a) Design a device driver for handling a RAM Disk. **8M**

(b) Write a short note on STREAM drivers. **5M**

Q.8. Explain in detail common object file format. **13M**

Q.9.(a) Draw a block diagram of the phases of a compiler and indicate the main function of each phase. **9M**

(b) What is a purpose of the uniform symbol? Can uniform symbols be avoided? If so, what are the gains and losses? **5M**

Q.10. Write a short note on (any three) : **13M**

(a) Source code control system.

(b) Positional argument and keyword argument.

(c) Symbolic Debugger.

(d) Binders and Module loaders.

WINTER EXAMINATION - 2012

SOLUTION

(Unit-wise)

UNIT - I

(Total Marks : 27)

Q.1.(a) "Operating system as user viewpoint." Comment on this. **4M**

Ans. See P.1-5, Q.4

Q.1.(b) What are advantages of base addressing in IBM 360? **5M**

Ans. See P.1-10, Q.10

Q.1.(c) What are pseudo instruction ? Explain USING, DROP, LTORG, EQU. **5M**

Ans. See P.1-11, Q.14

Q.3.(a) Explain various database required in design of two pass assembler with their format. **9M**

Ans. See P.1-15, Q.24

Q.3.(b) Draw a neat sketch of IBM 360 machine architecture. **4M**

Ans. See P.1-6, Q.6

UNIT - II

(Total Marks : 23)

Q.4. Give the entries in MNT, ALA, MDT for the following definition and call. **13M**

```
        MACRO
        OPRTN    & Macnm, & OP
        MACRO
                 & Macnm     & A, & B
                 L           1, & A
                 & OP        1, & B
                 ST          1, & B
                 MEND
                 MEND
                 START
                 :
DATA 1   DC       F'10'
DATA 2   DC       F'11'
DATA 3   DC       F'20'
                 :
OPRTN             ADD A
OPRTN             SUB S
                 :
ADD               DATA1, DATA2
                 :
SUB               DATA 1, DATA 3
END
```

Ans. See P.2-15, Q.12

Q.5.(b) What is a use of stack frame in macro processor? Give the format of stack frame along with example. **6M**

Ans. See P. 2-7, Q.7

Q.10.(b) Write short note on : Positional argument and keyword argument. **4M**

Ans. See P. 2-7, Q.6(b)

UNIT - III

(Total Marks : 24)

Q.2. Prepare ESD, RLD and GEST for following program :

```
PGA     START
        ENTRY    SYM11
        EXTRN    PG2, SYM23
        DC       A(PG1), A(PG2 + 4)
Sym11   DC       A(SYM11– PGI),
                 A(SYM23–PG–2)
        END
PG-2    START
```

```
        ENTRY   SYM23
        ENTRN   PG1, SYM11
Sym21   DC      A(Sym11-2), A(Sym21)
Sym22   DC      A(PG2+4),A(SYM21–
                PG2)
Sym23   DC      A(Sym21-PG1)
        END
```

13M

Ans. See P.3-24, Q.29.

Q.5.(a) Explain following address constant. **7M**

(a) Absolute

(b) Simple relocatable

(c) Complex relocatable.

What role they play in design of loader?

Ans. P.3-18, Q.24.

Q.10.(d). Write short note on : **4M**

Binder and Module loader

Ans. P.3-11, Q.13.

UNIT - IV

(Total Marks : 26)

Q.8. Explain in detail common object file format. **13M**

Ans. See P.4-2, Q.1.

Q.10.(a) Write a short note on: **6M**

Source code control system(SCCS)

Ans. See P.4-10, Q.10.

Q.10.(c) Write short note on Symbolic Debugger. **7M**

Ans. See P.4-11, Q.13.

UNIT - V

(Total Marks : 21)

Q.6.(a) Explain following entry points and routines for device driver of a line printer **8M**

(i) init

(ii) open

(iii) close

(iv) LP work routine

(v)LP restart routine

Ans. P.5-14, Q.15.

Q.7.(a) Design a device driver for handling a RAM Disk. **8M**

Ans. See P.5-11, Q.11.

Q.7.(b) Write a short note on STREAM driver. **5M**

Ans. P.5-5, Q.5.

UNIT - VI

(Total Marks : 19)

Q.9.(a) Draw a block diagram of the phases of compiler and indicate the main function of each phase. **9M**

Ans. See P.6-4, Q.4.

Q.9.(b) What is purpose of uniform symbol? Can uniform symbol be avoided? If so, what are gains and losses? **5M**

Ans. See P.6-6, Q.6.

Q.6.(b) What is cross compiler? Explain how boot strapping can be achieved. **5M**

Ans. See P.6-9, Q.11.

SUMMER EXAMINATION - 2013

B.E. (Computer Science & Engg.)

SYSTEM PROGRAMMING

Statistical Analysis

Unit Number	Marks
Unit - I	27 M
Unit - II	13 M
Unit - III	30 M
Unit - IV	19 M
Unit - V	27 M
Unit - VI	21 M
Questions covered in VBD	137 M (100%)

SUMMER EXAMINATION - 2013
QUESTION PAPER

SECTION - A

Q.1.(a) Define system programming. Explain the various system programs in brief. **8M**

(b) With neat sketch explain the general structure of IBM 360 machine. **6M**

Q.2.(a) Explain the design of a single pass assembler with a flowchart. **8M**

(b) Explain the format of MOT and POT in the assembler design. **5M**

Q.3.(a) What databases are used by the two passes of macro processor? **5M**

(b) What are the basic tasks that any macro instruction processor must perform? **5M**

(c) What is the use of conditional macro calls? **3M**

Q.4.(a) What are the functions of loader? **2M**

(b) Explain relocating loader in detail with advantages and disadvantages. **8M**

(c) What is meant by Binder? Also explain the module loader. **3M**

Q.5. Show the entries in ESD, TXT and RLD cards for the following program: **13M**

```
JOHN    START
        EXTRN   SUM, DATA
        ENTRY   LOOP, POINTER
        BALR    15,0
        USING   *, 15
        SR      4, 14
        L       1, FOUR
        A       2, FOUR
        ST      2, FOUR
        BR      14
FOUR    DC      F '4'
LOOP    DC      A(SUM + 4)
POINTER DC      A(LOOP – DATA)
        DC      A(POINTER – LOOP)
        DC      A(POINTER)
        END
```

SECTION - B

Q.6.(a) What is common object file format? Give the typical structure of an object file. **7M**

(b) Explain with example the type field entry of the symbol table. **6M**

Q.7.(a) What are the different kinds of drivers in windows? **3M**

(b) State the different types of UNIX device drivers. **2M**

(c) State the steps involved in installing a device driver in a UNIX system. **4M**

(d) What are the major design issues in the study of device drivers? **4M**

Q.8.(a) Compare the device driver for UNIX and Windows operating system. **6M**

(b) Explain structure of a kernel mode driver. **4M**

(c) Write a short note on driver memory allocation. **4M**

Q.9.(a) State and explain the role of a Lexical Analyzer. **6M**

(b) What are token recognizers? Explain ways of designing token recognizer. **7M**

Q.10. Write notes on any THREE: **13M**

(i) Regular Expression

(ii) Bootstrapping

(iii) Link editor

(iv) Symbolic Debugger.

SUMMER EXAMINATION - 2013

SOLUTION

(Unit-wise)

UNIT - I

(Total Marks : 27)

Q.1.(a) Define system programming. Explain the various system programs in brief. **8M**

Ans. See P.1-3, Q.1

Q.1.(b) With neat sketch explain the general structure of IBM 360 machine. **6M**

Ans. See P.1-6, Q.6

Q.2.(a). Explain the design of a single pass assembler with a flowchart. **8M**

Ans. See P.1-19, Q.27

Q.2.(b) Explain the format of MOT and POT in the assembler design. **5M**

Ans. See P.1-15, Q.24

UNIT - II

(Total Marks : 13)

Q.3.(a) What databases are used by the two passes of macro processor? **5M**

Ans. See P.2-13, Q.10

Q.3.(b) What are the basic tasks that any macro instruction processor must perform? **5M**

Ans. See P.2-4, Q.1(b)

Q.3.(c) What is the use of conditional macro calls? **3M**

Ans. See P.2-5, Q.4

UNIT - III

(Total Marks : 30)

Q.4.(a) What are the functions of loader? **2M**

Ans. See P.3-3, Q.1.

Q.4.(b) Explain relocating loader in detail with advantages and disadvantages. **8M**

Ans. See P.3-7, Q.7.

Q.4.(c) What is meant by Binder? Also explain the module loader. **3M**

Ans. See P.3-11, Q.13.

Q.5. Show the entries in ESD, TXT and RLD cards for the following program :

```
JOHN     START
         ENTRY    SUM, DATA
         EXTRN    LOOP, POINTER
         BALR     15, 0
         USING    *, 15
         SR       4, 14
         L        1, FOUR
         A        2, FOUR
         ST       2, FOUR
         BR       14
FOUR     DC       F '4'
LOOP     DC       A (SUM + 4)
POINTER  DC       A (LOOP – DATA)
         DC       A (POINTER – LOOP)
         DC       A (POINTER)
         END
```

13M

Ans. See P.3-25, Q.30.

Q.10.(iii). Write short note on link editor. **4M**

Ans. See P.3-12, Q.16.

UNIT - IV

(Total Marks : 19)

Q.6.(a) What is common object file format? Give the typical structure of an object file. **7M**

Ans. See P.4-2, Q.1.

Q.6.(b) Explain with example the type field entry of the symbol table. **6M**

Ans. See P.4-6, Q.4.

Q.10.(iii) Write short note on : **3M**
Link editor

Ans. See P.4-11, Q.12.

Q.10.(iv) Write short note on Symbolic Debugger. **3M**

Ans. See P.4-11, Q.13.

UNIT - V

(Total Marks : 27)

Q.7.(a) What are the different kinds of drivers in windows? **3M**

Ans. See P.5-6, Q.6.

Q.7.(b) State different types of UNIX device driver. **2M**

Ans. See P.5-4, Q.3.

Q.7.(c) State steps involved in installing a device driver in a UNIX system. **4M**

Ans. See P.5-8, Q.9.

Q.7.(d) What are major design issue in the study of device drivers? **4M**

Ans. See P.5-3, Q.2.

Q.8.(a) Compare device driver for UNIX and WINDOWS operating system. **6M**

Ans. See P.5-16, Q.16.

Q.8.(b) Explain structure of kernel mode driver. **4M**

Ans. See P.5-7, Q.7.

Q.8.(c) Write a short note on drive memory allocation. **4M**

Ans. See P.5-11, Q.12.

UNIT - VI

(Total Marks : 21)

Q.9.(a) State and explain the role of lexical analyzer. **6M**

Ans. See P.6-7, Q.8.

Q.9.(b) What are token recognizer? Explain the ways of designing the token recognizer. **7M**

Ans. See P.6-7, Q.9.

Q.10. Write a short note on : **8M**

(i) Regular expression

Ans. See P.6-9, Q.10.

(ii) Bootstrapping

Ans. See P.6-9, Q.11.

Notes :-

WINTER EXAMINATION - 2013

B.E. V SEM. (Computer Science & Engg.)

SYSTEM PROGRAMMING

Statistical Analysis

Unit Number	Marks
Unit - I	30 M
Unit - II	20 M
Unit - III	16 M
Unit - IV	22 M
Unit - V	23 M
Unit - VI	21 M
Questions covered in VBD	132 M (100 %)

WINTER EXAMINATION - 2013
QUESTION PAPER

SECTION - A

Q.1.(a) What are Pseudo Instructions? Explain USING DROP and LTORG instruction of IBM 360. **4M**

(b) What is the difference between :
(i) Procedure and program.
(ii) Open subroutine and closed subroutine.
(iii) Assembler and compiler. **6M**

(c) Define the following :
(i) Mnemonic operation codes.
(ii) Symbolic operand specification. **4M**

Q.2.(a) What is the difference between

```
INDEX   EQU  5
INDEX   DC   F'S'
```
3M

(b) What feature of assembly language required us to build a two pass assembler? **3M**

(c) Explain various instruction formats of IBM 360. **7M**

Q.3.(a) In a sense, macro expansion is very similar to subroutine call during program execution. Explain the similarities and differences between them. **4M**

(b) Describe the data structures required for two pass microprocessor design. **5M**

(c) What is conditional macro expansion? Illustrate with an example. **4M**

Q.4.(a) Show the entries in ESD, TXT and RLD cards for the following program :

```
STUDENT   START   0
          ENTRY   A
          EXTRA   SOLN, DELTA
          BARL    15, 0
          USING   STUDENT+2, 15
          SR      4, 4
          L       4, TWO
          L       5, TWO
          ST      5, SAVE
          BR      14
          DC      F '1'
          DC      A (a + 10)
SAVE      DC      F '7'
          DC      A(DELTA)
TWO       DC      F '2'
          DC      A (STUDENT
                  -SOLN)
          END.
```
10M

(b) At what point in time do each of the following loading schemes perform binding :
(i) BSS Loader.
(ii) Absolute Loader.
(iii) Dynamic Linking Loader. **3M**

Q.5.(a) For the following program :
(i) Show the symbol table at the end of pass 1.
(ii) Show the Literal table at the end of pass 1.
(iii) Show the changes in base register table during pass 2.
(iv) Show the Generated machine code from pass 2.

```
TEST    START
BEGIN   BALR    15, 0
        USING   BEGIN +2, 15
        SR      4, 4
        L       3, = F '10'
LOOP    L       2, DATA (4)
        A       2, = F '49'
        ST      2, DATA (4)
        A       4, = F '4'
        BCT     3, * – 16
        BR      14
        LTORG
DATA    DC      F '1,3,3,3,3,4,5,9,0'
        END.
```
10M

(b) What is the purpose of the ID number on the ESD cards? Why is it not needed for locally defined symbols? **3M**

SECTION - B

Q.6.(a) Describe common object file format. **8M**

(b) Explain the concept of storage classes. **5M**

Q.7.(a) What is Prologue? Write a Prologue for Line printer. **7M**

(b) What are Vxd? How UXD communicate with each other? **7M**

Q.8.(a) Explain database and algorithm for lexical phase of compiler. **8M**

(b) What is the purpose of uniform symbol? Can uniform symbols be avoided? If so, what are the gains and losses? **5M**

Q.9.(a) Write short note on STREAM drivers. **5M**

(b) What is LEX? Write a LEX program to count the number of identifiers in the input string. **8M**

Q.10. Write short notes on :

(i) Source Code Control System (SCCS).

(ii) AWK.

(iii) Symbolic debugger. **13M**

WINTER EXAMINATION - 2013 SOLUTION

(Unit-wise)

UNIT - I

(Total Marks : 30)

Q.1.(a) What are Pseudo Instructions? Explain USING DROP and LTORG instruction of IBM 360. **4M**

Ans. See P.1-11, Q.14.

Q.1.(b) What is the difference between : **6M**

(i) Procedure and program.

Ans.

Sr. No.	Procedure	Program
(1)	A procedure is a subroutine that can be called from another part of the program.	Program means a plan, usually step-by-step given instructions.
(2)	A procedure can be defined in as small as in 2 lines (inside procedure may contain hundred of instructions).	Length and range of program can be defined as task or operation that are needed to be performed.

(ii) Open subroutine and closed subroutine.

Ans. See P.1-14, Q.21 (a).

(iii) Assembler and compiler.

Ans. See P.1-14, Q.21 (c).

Q.2.(b) What feature of assembly language required us to build a two pass assembler? 3M

Ans.

(1) Assembler is the program that accepts as input an assembly language program and produce its machine language equivalent along with information for the loader.

(2) Hence this feature of conversion of assembly program to object code of assembly language with the help of assembler is required us to build a two pass assembler.

(3) Since in two pass assembler one pass is used to define symbols and literals while second pass is to generate object program.

(4) In order to build a two pass assembler an assembly programming language should support macro processing, file inclusion. Its definition should be independent of any particular hardware or operating system.

(5) The use of symbolic references is a key feature of assemblers, saving tedious calculations and manual address updates after program modifications.

- We require the following data structure and function to build a two-pass assembler.

Data Structures :

(1) Machine opcode table.

(2) Pseudo opcode table.

(3) Location counter.

(4) End of code segment and start of data segment Flag.

(5) Input source program.

(6) Output program.

(7) Input program line number etc.

Functions :

(1) Read line.

(2) Write line.

(3) SearchMOT.

(4) SearchPOT.

(5) SearchST.

(6) ModifyLC.

(7) InitLC.

(8) Get Attr Forms ST etc.

Q.2.(c) Explain various instruction formats of IBM 360. **7M**

Ans. See P.1-6, Q.7.

Q.5.(a) For the following program :

(i) Show the symbol table at the end of pass 1.

(ii) Show the Literal table at the end of pass 1.

(iii) Show the changes in base register table during pass 2.

(iv) Show the Generated machine code from pass 2.

```
TEST    START
BEGIN   BALR    15, 0
        USING   BEGIN +2, 15
        SR      4, 4
        L       3, = F '10'
LOOP    L       2, DATA (4)
        A       2, = F '49'
        ST      2, DATA (4)
        A       4, = F '4'
        BCT     3, * - 16
        BR      14
        LTORG
DATA    DC      F '1,3,3,3,3,4,5,9,0'
        END.
```

10M

Ans. Symbol Table :

Symbol	Value	Length	Relocation
TEST	0	1	R
BEGIN	0	1	R
LOOP	8	4	R
DATA	44	4	R

Literal Table :

Literal	Value	Length	Relocation
F'10'	32	4	R
F'49'	36	4	R
F'4'	40	4	R

Base Table :

Base	Contents
15	2

Machine Table :

Statement number	Relocative Location	Instruction	
(1)	0	BALR	15,0
(2)	2	SR	4,4
(3)	4	L	3,=F'10'
(4)		L	3,32-2(0,15)
(5)		L	3,30(0,15)
(6)	8	L	2,DATA(4)
(7)		L	2,42(4,15)
(8)	12	A	2,=F'49'
(9)		A	2,34(0,15)
(10)	16	ST	2,DATA(4)
(11)		ST	2,42(4,15)
(12)	20	A	4,=F'4'
(13)		A	4,38(0,15)
(14)	24	BCT	3,6(0, 15)
(15)	28	BCR	15,14

UNIT - II

(Total Marks : 20)

Q.1.(c) Define the following :

(i) Mnemonic operation codes.

(ii) Symbolic operand specification. **4M**

Ans.

(i) Mnemonic operation codes :

- Use of mnemonic operation codes for machine instruction eliminates the need to memorize numeric operation codes.
- It also enables the assembler to provide helpful diagnostic for example indication of misspelt operation codes.

(ii) Symbolic operand specification :

- Symbolic names can be associated with data or instruction .These symbolic name can be used as operand in assembly statements.
- The assembler performs memory binding to these names, the programmer need not know any details of the memory bindings performed by the assembler.

Q.2.(a) What is the difference between

INDEX EQU 5

INDEX DC F'5' **3M**

Ans.

- INDEX EQU 5 will assign 5 to the symbol index. EQU allows the programmer to define variables.
- If one wants to use a different base register only a change in EQU statement will suffice, whereas INDEX DC F'5' instruct the assembler to place a '5' is full word in memory.
- EQU is equal to pseudo op and DC is defined constant pseudo op.

Q.3.(a) In a sense, macro expansion is very similar to subroutine call during program execution. Explain the similarities and differences between them. **4M**

Ans.

- Macros are essentially shorthand representations of arbitrary sections of the source code, which makes the source code succinct, while its (the macro template's) expansion replaces each of its presence prior to compilation. Whatever is there to do with Macros, it is done by the pre-processor, so that the source code is ready for compilation.
- Function is a calling routine or subroutines. Functions are handled in all states of the compiling process.

Sr. No.	Macros	Subroutine
(1)	A MACRO is more or less an abbreviation for some lines of code that are used more than once or twice. A FORM is a local subroutine (which can be called external)	A FUNCTION is (more or less) a subroutine that is called external.
(2)	Macro consumes less time.	Subroutine consumes less memory.
(3)	When a function is called, arguments have to be passed to it, those arguments are accepted by corresponding dummy variables in the function, they are processed, and finally the function returns a value that is assigned to a variable (except for a void function).	While a subroutine replace with macros may look succinct on surface, prior to compilation, all the macro-presences are replaced by their corresponding macro expansions, which consumes considerable memory. On the other hand, even if a function is invoked 100 times, it still occupies the same space.
(4)	Macros can only be used in the program when they are defined in and only after the definition.	Subroutines can be called from both the program when they are defined in and other programs.
(5)	Macros are expanded at compilation or generation.	Subroutines are 'expanded' at runtime.

Q.3.(b) Describe the data structures required for two pass microprocessor design. **5M**

Ans. See P.2-13, Q.10.

Q.3.(c) What is conditional macro expansion? Illustrate with an example. **4M**

Ans. See P.2-5, Q.4.

UNIT - III

(Total Marks : 16)

Q.4.(a) Show the entries in ESD, TXT and RLD cards for the following program :

```
STUDENT   START   0
          ENTRY   A
          EXTRA   SOLN, DELTA
          BARL    15, 0
          USING   STUDENT+2, 15
          SR      4, 4
```

```
        L       4, TWO
        L       5, TWO
        ST      5, SAVE
        BR      14
        DC      F '1'
        DC      A (a + 10)
SAVE    DC      F '7'
A       DC      A(DELTA)
TWO     DC      F '2'
        DC      A (STRUDENT
                -SOLN)
        END.
```

10M

Ans. ESD CARD :

REF NO.	SYMBOL	TYPE	RELATIVE LOCATION	LENGTH
(1)	STUDENT	SD	0	44
(2)	A	LD	32	
(3)	SOLN	ER	-----	
(4)	DELTA	ER	-----	

TXT CARD :

REF NO	RELATIVE LOCATION	OBJECT CODE
(1)	0	BALR 15,0
(2)	2	SR 4,4
(3)	4	L 4,34(0,15)
(4)	8	L 5,34(0,15)
(5)	12	ST 5,26(0,15)
(6)	16	BR 16,14
(7)	20	1
(8)	24	a+10
(9)	28	7
(10)	32	DELTA
(11)	36	2
(12)	40	STUDENT-SOLN

RLD CARD :

REF NO	SYMBOL	FLAG	LENGTH	RELATIVE LOCATION
14	DELTA	+	4	32
16	DELTA	+	4	40

Q.4.(b) At what point in time do each of the following loading schemes perform binding :

(i) BSS Loader.

(ii) Absolute Loader.

(iii) Dynamic Linking Loader. **3M**

Ans.

(i) BSS loader :

- The BSS loaders permit multiple program segments in different languages to be compiled at different times. It required all the translators to produce a common object file format.
- BSS loaders also allowed the development of independent (and separate) data segments [FORTRAN BLOCK DATA]

(ii) Absolute loader :

- In this scheme the object program is placed in secondary devices. The loader only accepts the machine language text and places into core at the location prescribed by the assembler.
- Disadvantage is that the programmer must specify the load address in the program and also if there are multiple subroutines, the programmer must remember the address of each and use that in other subroutines to perform subroutine linkages.

(iii) Dynamic linking loader :

- With dynamic loading, a routine is not loaded until it is called.
- All routines are kept on disk in a reloadable load format.
- The main program is loaded into memory and is executed.
- When a routine needs to call another routine, the calling routine first checks to see whether the other routine has been loaded.
- If not, the reloadable linking loader is called to load the desired routine into memory and to update the program's address tables to reflect this change. Then control is passed to the newly loaded routine.

Q.5.(b) What is the purpose of the ID number on the ESD cards? Why is it not needed for locally defined symbols? **3M**

Ans. See P.3-15, Q.21.

UNIT - IV

(Total Marks : 22)

Q.6.(a) Describe common object file format. **8M**

Ans. See P.4-2, Q.1.

Q.6.(b) Explain the concept of storage classes. **5M**

Ans. See P.4-7, Q.5.

Q.10. Write short notes on : **9M**

(i) Source Code Control System (SCCS).

Ans. See P.4-10, Q.10.

(iii) Symbolic debugger.

Ans. See P.4-11, Q.13.

UNIT - V

(Total Marks : 23)

Q.7.(a) What is Prologue? Write a Prologue for Line printer. **7M**

Ans. Prologue :

- A prologue or prolog is defined in Prolog as an ordered sequence of elements of arbitrary length. The elements contains the following terms, i.e. constants, variables, structures, including other lists.
- Prolog (PROgramming LOGic) rose within the realm of Artificial Intelligence (AI). Prolog represents a fundamentally new approach to computing and became a serious competitor to LISP.
- Prolog lp emulates a line printer, i.e., turn off in program definition.
- We could even place each of the above directive groups in the rules section of the grammar file next to the set of rules that uses the associated semantic type. (In the rules section, we must terminate each of those directives with a semicolon.) And we don't have to worry that some directive (like a %union) in the definitions section is going to adversely affect their functionality in some counter-intuitive manner just because it comes first. Such an organization is not possible using Prologue sections.
- The functionality of Prologue sections can often be subtle and inflexible.
- Syntax for printing a line in prolog i.e syntax of line printer is as follows :

  To print the elements of a list each on a new line :

  print([ ]).

  print([X | Y]): nl, write(X), print(Y).
- Bison provides a %code directive with an explicit qualifier field, which identifies the purpose of the code and thus the location(s) where Bison should generate it.
- For C/C++Code for line printer is as follows :

```
%code requires { #include "type1.h" }
%union { type1 field1; }
%destructor { type1_free ($$); } <field1>
%printer { type1_print (yyoutput, $$); } <field1>
%code requires { #include "type2.h" }
%union { type2 field2; }
%destructor { type2_free ($$); } <field2>
%printer { type2_print (yyoutput, $$); } <field2>
```

- The program below demonstrates the text output function of the Sample programming language by displaying the message "Hello world!".The code can be used to print line using prolog programming code.

  Example : ***// the main program (this is a comment)***

```
Hello :
    nl,
    write('Hello world!' ).
```

Q.7.(b) What are Vxd? How Vxd communicate with each other? **7M**

Ans. Vxd :

- Vxd is a file extension for the virtual device driver program in many Windows operating systems.
- A device driver is a program that maintains the status of hardware devices with changeable settings.
- The device driver controls a specific device attached to the computer by translating the operating system's input/output (I/O) instructions to messages that device can understand.

- A virtual device driver handles software interrupts from the operating system for each of the computer's main hardware devices, including the hard disk drive controller, keyboard, and serial and parallel ports.
- Regular (non-virtual) device drivers handle hardware interrupts.
- Vxd describes the communication procedure control messages between the service callback virtual device driver and is referred to as a Vxd. x is on behalf of the names of various devices, such as virtual keyboard driver (vkd), sale christian louboutin, a virtual mouse driver (vmd) and so on.
- Successful Vxd initialization procedure is the way of hardware. Remember the dos program that they have all the system when they run in virtual machines, Windows machines need to give them a real substitute. Vxd program is the double.
- A Vxd typically control the real hardware and the equipment shared between all virtual machines to manage. However, not saying that every program must have Vxd and a hardware device connected.
- Although the program is used is Vxd virtual hardware devices, but we can also program as a VxD 0 level in the dll.
- For example, If we need to do some level of only 0 to do the work, we can compile a Vxd program to complete the work for you. In this way, because this procedure does not Vxd virtual equipment, we can put it simply as an extension of our program.

Q.9.(a) Write short note on STREAM drivers. **5M**

Ans. See P.5-5, Q.5.

Q.10.(ii) Write short note on AWK. **4M**

Ans. AWK :

- AWK is a data driven programming language designed for processing text-based data, either in files or data streams.
- It is an example of a programming language that extensively uses the string data type , associative arrays (that is, arrays indexed by key strings), and regular expressions.
- AWK is one of the early tools to appear in Version 7 Unix and gained popularity as a way to add computational features to a Unix pipeline.
- A version of the AWK language is a standard feature of nearly every modern Unix-like operating system available today.
- AWK is mentioned in the Single UNIX Specification as one of the mandatory utilities of a Unix operating system. Besides the Bourne shell, AWK is the only other scripting language available in a standard Unix environment.
- It is also present amongst the commands required by the Linux Standard Base specification. Implementations of AWK exist as installed software for almost all other operating systems.
- Structure of AWK is :
  "condition" { "action" }"
- AWK is a language for processing text files. A file is treated as a sequence of records, and by default each line is a record.
- Each line is broken up into a sequence of fields, so we can think of the first word in a line as the first field, the second word as the second field, and so on.
- An AWK program is of a sequence of pattern-action statements. AWK reads the input a line at a time. A line is scanned for each pattern in the program, and for each pattern that matches, the associated action is executed.

UNIT - VI

(Total Marks : 21)

Q.8.(a) Explain database and algorithm for lexical phase of compiler. **8M**

Ans. See P.6-5, Q.5.

Q.8.(b) What is the purpose of uniform symbol? Can uniform symbols be avoided? If so, what are the gains and losses? **5M**

Ans. See P.6-6, Q.6.

Q.9.(b) What is LEX? Write a LEX program to count the number of identifiers in the string. **8M**

Ans. LEX :

- In computer science, Lex is a program that generates lexical analyzers ("scanners" or "lexers").
- Lex is commonly used with the YACC parser generator. Lex, is the standard lexical analyzer generator on many Unix systems, and a tool exhibiting its behaviour is specified as part of the POSIX standard.
- Lex reads an input stream specifying the lexical analyzer and outputs source code implementing the lexer in the C programming language.
- Though traditionally proprietary software, versions of Lex based on the original AT&T code are available as open source, as part of systems such as Open Solaris and Plan 9 from Bell Labs.
- Another popular open source version of Lex is Flex, the "fast lexical analyzer".
- Consider Lex program count number of identifiers in the input string :

```
% {
      unsigned int charcount=0 ;
      int wordcount=0 ;
      int linecount=0 ;
      int blankcount=0 ;
% }
word [ ^ \ t \ n] + eol \ n
% %
[ ] blankcount + + ;
{word} {wordcount + + ; charcount+ = yyleng;}
{eol} {charcount+ +; linecount+ +;}
. {ECHO ; charcount+ + ; }
% %
main (argc, argv)
int argc ;
char **argv ;
{
     if (argc > 1)
     {
        FILE * file ;
        file = fopen (argv[1], "r") ;
        if (!file)
        {
           fprintf (stderr, "cound not open %s\n",
                                        argv[1]) ;
           exit (1) ;
        }
        yyin = file ;
        yylex( ) ;
           printf ("\nThe number of characters =
                                % u\n", charcount) ;
           printf ("\nThe number of wordcount =
                                % u\n", wordcount) ;
           printf ("\nThe number of linecount =
                                % u\n", linecount) ;
           printf ("\nThe number of blankcount =
                                % u\n", blankcount) ;
           return (0) ;
        }
        else
           printf ("Enter the file name along with
                                the program\n") ;
}
```

NOTES

SUMMER EXAMINATION - 2014
B.E. IV SEM. (Computer Science & Engg.) (CBS)
SYSTEM PROGRAMMING

Statistical Analysis

Unit Number	Marks
Unit - I	28 M
Unit - II	26 M
Unit - III	30 M
Unit - IV	22 M
Unit - V	26 M
Unit - VI	28 M
Questions covered in VBD	160 M (100%)

SUMMER EXAMINATION - 2014
QUESTION PAPER

Q.1.(a) What is an operating system? State different services provided by an operating system. **5M**

(b) What is the difference between :
(i) Procedure and program.
(ii) Open subroutine and Closed subroutine. **4M**

(c) Explain IBM 360 machine architecture in detail. **5M**

OR

Q.2.(a) Explain following terms with the help of examples :
(i) USING
(ii) BALR
(iii) DS
(iv) DC
(v) LTORG
(vi) DROP. **6M**

(b) For the following ALP show its symbol table, literal table, base table and generated machine code :

```
TEST    START
BEGIN   BALR    15,0
        USING   BEGIN + 2, 15
        SR      4, 4
        L       3, = F '10'
LOOP    L       2, = F '49'
        ST      2, DATA (4)
        A       4, = F '4'
        BCT     3, * – 16
        BR      14
DATA    DC      F '1, 2, 3, 4'
        END.
```

8M

Q.3.(a) Explain the necessity of stack for implementing macro call within macro definition. **5M**

(b) Explain the different data structure required to implement macro processor. Also write their formats. **8M**

OR

Q.4.(a) What is conditional macro expansion? Illustrate with an example. **5M**

(b) For the following program show MNT, MDT, ALA and expanded assembly language program :

```
        MACRO
        STORE   & ANS
        ST      1, & ANS
        MEND
        MACRO
        TRY     & ADD, & DEST
        SR      1, 1
        MACRO
        & ADD   & A, & B
        L       1, & A
        A       1, & B
        MEND
&DEST   DS      F
        MEND
ARITH   START   0
        USING   *, 15
        TRY     PLUS, RESULT
        PLUS    N1, N2
        STORE   RESULT
N1      DC      F '4'
N2      DC      F '5'
        END.
```

8M

Q.5.(a) Write short notes on :
(i) Direct Linking Loader.
(ii) Absolute Loader. **6M**

(b) Explain in detail Dynamic Linking and Dynamic Loading. **7M**

OR

Q.6.(a) Draw and explain flowchart for pass-2 of Loader. **7M**

(b) Explain GEST and LESA for implementing a two pass Linker. **6M**

Q.7.(a) Describe common object file format. **8M**

(b) Explain the concept of storage classes. **5M**

OR

Q.8.(a) What is symbolic debugger? Explain in brief. **5M**

(b) Write short notes on :
(i) Sources code control system.
(ii) Link Editor. **8M**

Q.9.(a) Explain the various steps involved in installation of UNIX device driver. **7M**

(b) Differentiate between Character Driver and STREAM Driver. **6M**

OR

Q.10.(a) Explain the following entry point and routine for device driver of a line printer :

(i) init

(ii) open

(iii) close. **6M**

(b) What is Device Driver? Explain the necessity of Device Driver. **7M**

Q.11.(a) What is Compiler? Explain in brief the various databases used in lexical phase of compiler. **7M**

(b) Explain YACC in detail. **7M**

OR

Q.12.(a) What is Token? How are tokens specified? Also explain how tokens are recognized. **7M**

(b) Describe a tool for study of Lex. **7M**

SUMMER EXAMINATION - 2014

SOLUTION

(Unit-wise)

UNIT - I

(Total Marks : 28)

Q.1(a). What is an operating system? State different services provided by an operating system. 5M

Ans. See P.1-5, Q.4.

'Operating system provides many services, that generally categorize in the user perspective and system perspective. Services may vary system to system.

(1) Services to user perspective :

(i) Program execution

(ii) Input output operation

(iii) File system manipulation

(iv) Process communication

(v) User interface

(vi) Error detection.

(i) Program execution :

- Operating system provides an environment for the execution of program. It provides certain services to program and to user.
- For execution of any program it is required that the content of program should be loaded into main memory then execution of any program is possible.
- Operating system read file from secondary storage and load it into memory.
- Memory allocation and deallocation is provided by operating system.

(ii) Input/Output operation :

- A running program would need input/output operation for reading input/output data and for displaying of results.
- This I/O may be from/to an I/O device or vice-versa. For each device, some special functions may be necessary. All these operations are managed by an operating system.

(iii) File system manipulation :

- File is a logical structure to store any data on secondary storage.
- It requires some manipulation starting from creation of file to the maintaining of all contents of file to deleting of file, operating system supports the users with all these functions.

(iv) Process communication :

- Generally the nature of processes are co-operative so they need to communicate with each other.
- Operating system provides interprocess communication(IPC), socket, pipe, shared memory, message passing and many more ways to communicate with process.

(v) User interface :

- Operating system provides a user friendly environment to user, so that users can feel a freshness to operate the application.
- Operating system provides graphical user interface(GUI), user interface(UI), command line interface(CLI) and batch interface to user.

(vi) Error Detection :

- Operation is sensitive issue because it may lead to error in either hardware or software.
- Operating system constantly monitors the system to check whether the performance of resources is going properly or not.
- It is also possible to detect the error at hardware and software level.
- Operating system makes sure that the data transfer on media have not changed since they were written.
- Operating system also makes sure that the data is not corrupted.
- In terms of software level, it checks the consistency of data.
- Operating system takes corrective action to overcome and detect error of any case like deadlock, unsafe sequences and show the error message to user.

(2) Resource allocation :

- When multiple users are logged on the system or multiple jobs are running at the same time, resources must be allocated to them.
- Many different types of resources are managed by operating system. These are :

(i) CPU managed by the operating system by proper scheduling and assignment of process to the processor.

(ii) Main memory management without interleaving of address spaces without overlapping of contents by using base and limit register value.

(iii) Secondary storage management HD, magnetic tape, CDROM, with different characteristics and their device drivers.

(iv) Input device like printer, monitor (Video terminal).

(3) Accounting :

- Operating system keeps track of the all users who logged in system, their usage of resource, how many resources holding at which time etc.
- This information is useful in decision making purposes.

(4) Protection :

- As per the system perspective the protection is a huge concern about the system.
- Operating system provides the protection like owner of file, ensures that resources are used logically, authentication.
- Protection ensures that all access to system resources is controlled.
- Password mechanism, locking mechanism, is supported by operating system.

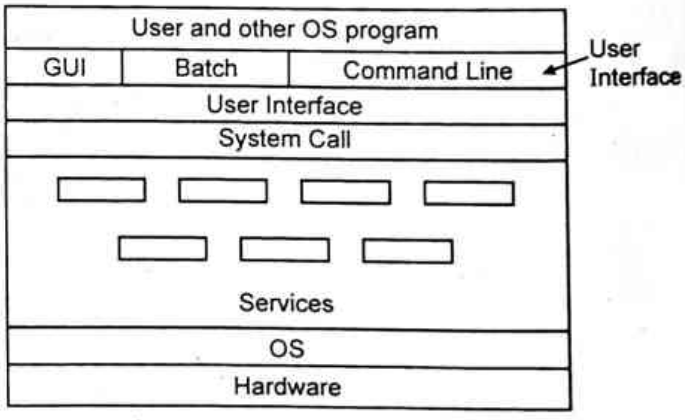


Fig. Services provided by OS

Q.1.(b) What is the difference between :

(i) Procedure and program.

(ii) Open subroutine and Closed subroutine. **4M**

Ans. (i) Procedure and program.

Sr. No.	Procedure	Program
(1)	A procedure is a subroutine that can be called from another part of the program.	Program is given as a set of process. In some cases we may divide a problem into number of parts. At these times we write a separate logic for each part known as process.
(2)	Division of procedure leads to small sub routines or sub procedures.	Division of program can lead number of small sub programs or sub process.
(3)	Procedure is to be written in program.	Program are the collection of procedures or functions.
(4)	Procedure contains set of instructions to perform assigned task.	Program is a sequence of statements/steps stating how to perform a task.
(5)	**Example :** An class of student in c++ is an program containing different types of procedure/methods/ functions.	**Example :** A software is an collection of programs.

(ii) See P.1-14, Q.21(a).

Q.1.(c) Explain IBM 360 machine architecture in detail. **5M**

Ans. See P.1-6, Q.6.

Q.2.(a) Explain following terms with the help of examples :

(i) USING
(ii) BALR
(iii) DS
(iv) DC
(v) LTORG
(vi) DROP **6M**

Ans. See P.1-11, Q.14.

Q.2.(b) For the following ALP show its symbol table, literal table, base table and generated machine code :

```
TEST    START
BEGIN   BALR    15,0
        USING   BEGIN + 2, 15
        SR      4, 4
        L       3, = F '10'
LOOP    L       2, = F '49'
        ST      2, DATA (4)
        A       4, = F '4'
        BCT     3, * – 16
        BR      14
DATA    DC      F '1, 2, 3, 4'
        END.
```

8M

Ans. See P.1-22, Q.29.

UNIT - II

(Total Marks : 26)

Q.3.(a) Explain the necessity of stack for implementing macro call within macro definition. **5M**

Ans. See P.2-7, Q.7.

Q.3.(b) Explain the different data structure required to implement macro processor. Also write their formats. **8M**

Ans. The data structures used to implement the macro processor are as follows :

(1) DEFTAB (Definition Table) :

- It stores the macro definition including macro prototype and macro body.
- The comment lines are omitted.
- It references to the macro instruction parameters are converted to a positional notation for efficiency in substituting arguments.

(2) NAMTAB (Name Table) :

- It stores macro names.
- It serves as an index to DEFTAB.
- It has pointers to the beginning and the end of the macro definition (DEFTAB).

(3) ARGTAB (Argument Table) :

- It stores the arguments according to their positions in the argument list.

- As the macro is expanded the arguments from the Argument table are substituted for the corresponding parameters in the macro body.
- The figure below shows the different data structures described and their relationship.

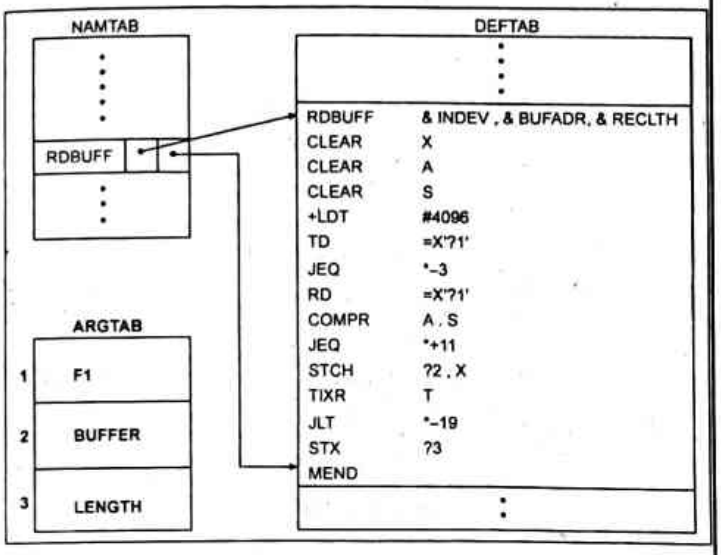


Fig. Data structure of microprocessor

- Formats of data structures are as follows:

(1) DEFTAB (Definition Table) :

Generalized Format is as :

Index or Name of Macro	Label	Card values or Arguments
-----	----	------

Example :

Macro Definition Table MDT

```
&lab    INCR    @ARG1,&ARG2,&ARG#
#0      A       1,#1
        A       2,#2
        A       3,#3
        MEND
```

(2) NAMTAB (Name Table):MACRO NAME TABLE :

Generalized Format is as :

Index	8bytes Names	4bytes MDT index
-----	------	-------

Example :

Note MDT index over here is the index of macro in Macro Definition Table

Macro Name Table

Index	Name	MDT INDEX
.		.
.		.
.		.
3	"INCRbbbb"	15
.		.
.		.
.		.

(3) ARGTAB (Argument Table) or Argument list array :

Generalized Format is as :

Index	Argument Names
------	--------

Example :

Argument list Array

Index	Argument Name
0	"LOOP1bbb"
1	"DATA1bbb
2	"DATA2bbb"

Q.4.(a) What is conditional macro expansion? Illustrate with an example. **5M**

Ans. See P.2-5, Q.4.

Q.4.(b) For the following program show MNT, MDT, ALA and expanded assembly language program :

```
        MACRO
        STORE   & ANS
        ST      1, & ANS
        MEND
        MACRO
        TRY     & ADD, & DEST
        SR      1, 1
        MACRO
        & ADD   & A, & B
        L       1, & A
```

```
        A       1, & B
        MEND
&DEST   DS      F
        MEND
ARITH   START   0
        USING   *, 15
        TRY     PLUS, RESULT
        PLUS    N1, N2
        STORE   RESULT
N1      DC      F '4'
N2      DC      F '5'
        END.
```

8M

Ans. See P.2-14, Q.11.

UNIT - III

(Total Marks : 30)

Q.5.(a) Write short notes on :
(i) Direct Linking Loader
(ii) Absolute Loader. **6M**

Ans. (i) See P.3-8, Q.10.
(ii) See P.3-4, Q.4.

Q.5.(b) Explain in detail Dynamic Linking and Dynamic Loading. **7M**

Ans. See P.3-11, Q.15.

Q.6.(a) Draw and explain flowchart for pass-2 of Loader. **7M**

Ans. See P.3-9, Q.12.

Q.6.(b) Explain GEST and LESA for implementing a two pass Linker. **6M**

Ans. See P.3-14, Q.20.

Q.8.(b)(ii) Write short notes on Link Editor. **4M**

Ans. See P.3-12, Q.16.

UNIT - IV

(Total Marks : 22)

Q.7.(a) Describe common object file format. **8M**

Ans. See P.4-2, Q.1.

Q.7.(b) Explain the concept of storage classes. **5M**

Ans. See P.4-7, Q.5.

Q.8.(a) What is symbolic debugger? Explain in brief. **5M**

Ans. See P.4-11, Q.13.

Q.8.(b)(i) Write short notes on Sources code control system. **4M**

Ans. See P.4-10, Q.10.

UNIT - V

(Total Marks : 26)

Q.9.(a) Explain the various steps involved in installation of UNIX device driver. **7M**

Ans. See P.5-8, Q.9.

Q.9.(b) Differentiate between Character Driver and Stream Driver. **6M**

Ans.

Sr. No.	Parameters	Character Driver	Stream Driver
(1)	Definiton	Character devices handle data serially and transfer data to and from the processor one character at a time, the same as keyboards and low performance printers.	Devices that receive data asynchronously (such as terminal and network devices) are suited to a streams implementation.
(2)	Mechanism	The character I/O mechanism, all ioctl(2) are handled by the single driver associated with the file.	In streams, there can be multiple modules on a stream and each one can have its own set of ioctl(2) calls.
(3)	Operation	The operation of	The operation of the generic

		any specific ioctl(2) is dependent on the device driver. If the driver requires data to be transferred from user space, it will use the kernel ddi_copyin function. It may also use ddi_copyout to transfer any data results to user space.	streams ioctl(2) is generally independent of the presence of any specific module or driver on the stream.
(4)	Entry points	Character I/O drivers have interrupt entry points.	Streams device drivers have similar interrupt entry points at the hardware device interface.
(5)	Validation and reorganisation	All ioctl(2) calls are sent to the driver, which is expected to perform all validation and processing other than file descriptor validity	A set of generic streams ioctl(2) command values recognized and processed by the stream head.
(6)	Entry point applicable to routine	At the system interface, in character driver those same drivers generally have direct entry points (routines) to process open, close, read, and write subroutines, and ioctl operations.	Stream driver have direct entry points only for the open and close subroutines.

Q.10.(a) Explain the following entry point and routine for device driver of a line printer :
(i) init
(ii) open
(iii) close. **6M**

Ans. See P.5-14, Q.15.

Q.10.(b) What is Device Driver? Explain the necessity of Device Driver. **7M**

Ans. See P.5-3, Q.1.

UNIT - VI

(Total Marks : 21)

Q.11.(a) What is Compiler? Explain in brief the various databases used in lexical phase of compiler. **7M**

Ans. See P.6-2, 5, Q.1, 5.

Q.11.(b) Explain YACC in detail. **7M**

Ans. See P.6-10, Q.12.

Q.12.(a) What is Token? How are tokens specified? Also explain how tokens are recognized. **7M**

Ans. See P.6-7, Q.9.

Q.12.(b) Describe a tool for study of Lex. **7M**

Ans. See P.6-11, Q.14.

WINTER EXAMINATION - 2014

B.E. IV SEM. (Computer Science & Engg.) (CBS)

SYSTEM PROGRAMMING

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	28 M
Unit - III	26 M
Unit - IV	26 M
Unit - V	26 M
Unit - VI	28 M
Questions covered in VBD	160 M (100%)

WINTER EXAMINATION - 2014
QUESTION PAPER

Q.1.(a) Explain the difference between : **6M**

(i) Immediate operand and literal operand.

(ii) USING and BALR.

(iii) Machine op and pseudo op.

(b) Why does assembler require more than one pass over input program? Explain your answer with suitable example. **7M**

Q.2.

```
Test    Start   0
        BALR    15, 0
        LTORG
        Using   *, 15
        LR      5, 15
        L       1, Data 1
        Using   *, 10
        BR      14
Data 2  DC      F '10'
Data 1  DC      F '22'
AA      DC      H '2'
BB      DS      F
CC      EQU     1
DD      EQU     2
        BALR    2, 0
        Using   * + CC, AA
        LA      7, = A (BB)
        BR      6
        DC      H '6'
        DROP    DD
        L       9, = A (Data 1)
        A       9, AA
        ST      9, = F '40'
        LTORG
        END
```

For the above code get B.T, L.T, S.T alongwith machine code. **13M**

Q.3. Get MNT, MDT, ALA and expanded code for the following code fragment : **14M**

```
        MACRO
            XUZ &A
        MACRO
            &A &B
        MACRO
            &B &C
            ST 15, A(&C)
        BALR 14, 15
            L 1, = F '3'
            A 1, &C
            ST 1, &C
        MEND
        MEND
        MEND
TEST    START
        BALR
        ⋮
        XYZ     SQRT
        SQRT    TOPOS
        TOPOS   D1
D1      DC      F'10'
        END
```

OR

Q.4.(a) For the given MDT and MNT, write the possible assembly language program : **8M**

MNT :

Index	Macro name	MDTI
1	XYZ	1
2	MIT	4

MDT :

Index	Card
1	XYZ & A
2	ST 1, #1
3	MEND
4	MIT &Z
5	AR 4, #1
6	XYZ #1
7	MEND

(b) Explain the similarities and differences between MACRO expansion and subroutine calls. **6M**

Q.5.(a) Draw detailed flowchart for pass-2 of Direct Linking Loader of IBM 360 type machine. **7M**

(b) Explain which kind of information is generated by the assembler for the 2 pass Link Loader. **6M**

OR

Q.6.(a) Explain the concept of Relocation. Can relocation be avoided? **6M**

(b) How are ESD and RLD cards related? Explain with an example. **7M**

Q.7.(a) Explain with respect to char drivers, data structures and the methods, for the following views :

(i) Code-Flow view.

(ii) Data-Flow view. **6M**

(b) Let the output on the terminal be as follows :

```
ls – l/dev
total 0
crw -------    1 root root  5,  1 console
lrwxrwxrwx     1 root root      3 cdrom → hlc
brw-rw -----   1 root disk  3,  0 hla
crw -------    1 root try   4,  1 tty 1
```

Explain the output with respect to 1st, 5th and 6th column. **7M**

OR

Q.8.(a) Explain the following terms w.r.t. unix : **6M**

(i) Input drivers.

(ii) Input Event Drivers.

(iii) Input sub-system.

(b) Explain Windows Driver Installation in detail. **7M**

Q.9. Explain purpose of following files :

(i) Relocatable object file.

(ii) Executable object file.

(iii) Shared object file.

Your answer should include their structure, creation process and commands used to manipulate these files. **3M**

OR

Q.10.(a) What is Linkage Editor? Which command is used to this effect? Explain how in unix like system, we can link two object files. **8M**

(b) Explain with suitable example, use of gcc to link files. **5M**

Q.11.(a) What are the differences and similarities between an interpreter and a compiler? **8M**

(b) Explain Lexical analysis and Syntax analysis in compilers. **6M**

OR

Q.12.(a) What information must be in the symbol table so that intermediate code generation phase can generate code for the statement :

$x = b + c * d + 70$

Assume suitable data types for the identifier. **7M**

(b) Explain any one of the following with suitable program : **7M**

(i) LEX

(ii) YACC.

WINTER EXAMINATION - 2014

SOLUTION

(Unit-wise)

UNIT - I

(Total Marks : 26)

Q.1.(a) Explain the difference between : **6M**

(i) Immediate operand and literal operand.

Ans.

Sr. no.	Immediate operand	Literal operand
(1)	Immediate operand is a machine recognizable data.	Literal is an assembler directive.
(2)	Immediate : Full word – (opcode, registers)	Full word can be used for literals.
(3)	Immediate data is within the instruction itself.	Values are obtained from data memory-slow.

(ii) USING and BALR.

Ans. **See P.1-13, Q.20.**

(iii) Machine op and pseudo op.

Ans. **See P.1-13, Q.19.**

Q.1.(b) Why does assembler require more than one pass over input program? Explain your answer with suitable example. 7M

Ans.

- Let us pretend that we are the assembler trying to translate the program in the left column of Fig. We read the START instruction and note that it is a pseudo-op instruction (to the assembler) giving JOHN as the name of this program; the assembler must pass the name on to the loader.
- The next instruction is the USING pseudo-op. This tells the assembler (us) that register 15 is the base register and at execution time will contain the address of the first instruction of the program.

Source program			First pass			Second pass		
			Relative address	Mnemonic Instruction		Relative address	Mnemonic instruction	
- JOHN	START	0						
	USING	*, 15						
	L	1, FIVE	0	L	1, – (0, 15)	0	L	1,16 (0, 15)
	A	1, FOUR	4	A	1, – (0, 15)	4	A	1, 12(0, 15)
	ST	1, TEMP	8	ST	1, – (0, 15)	8	ST	1, 20(0, 15)
FOUR	DC	F '4'	12	4		12	4	
FIVE	DC	F '5'	16	5		16	5	
TEMP	DS	1F	20	-		20	-	
	END							

Fig. Intermediate steps in assembling a program

- There is no BALR instruction. This program was presumably called by another program that left the address of the first instruction in register 15.
- Next comes a Load instruction : L 1, FIVE. We can look up the bit configuration for the mnemonic in a table and put the hit configuration for the L in the appropriate place in the machine language instruction.
- Now we need the address of FIVE. At this point, however, we do not know where FIVE is, so we cannot supply its address. Because no index register is being used, we put in 0 for the index register.
- We know that register 15 is being used as the base register, but we cannot calculate the offset. The base register 15 is pointing to the beginning of this program, and the offset is going to be the difference between the location FIVE and the location of the beginning of the program, which is not known at this time.
- We maintain a location counter indicating the relative address of the instruction being processed; this counter is incremented by 4 (length of a load instruction).
- The next instruction is an Add instruction. We look up the op-code, but we do not know the offset for FOUR. The same thing happens with the Store instruction.
- The DC instruction is a pseudo-op directing us to define some data; for FOUR we are to produce a '4'.
- We know that this word will be stored at relative location 12, because the location counter now has the value 12, having been incremented by the length of each preceding instruction.

- The first instruction, four bytes long, is at relative address 0.
- The next two instructions are also four bytes long.
- We say that the symbol "FOUR" has the value 12.
- The next instruction has as a label FIVE and an associated location counter value of 16.
- The label on TEMP has a associated value of 20. We have thus produced the code in the center column of Fig.
- As the assembler, we can now go back through the program and fill in the offsets as is done in the third column of Fig.
- Because symbols can appear before they are defined, it is convenient to make two passes over the input (as this example shows).
- The first pass has only to define the symbols; the second pass can then generate the instructions and addresses.
- Specifically, an assembler must do the following :

(1) Generate instructions :

(a) Evaluate the mnemonic in the operation field to produce its machine code.

(b) Evaluate the subfields-find the value of each symbol, process literals, and assign addresses.

(2) Process pseudo ops

- We can group these tasks into two passes or sequential scans over the input; associated with each task are one or more assembler modules.

Pass 1 : Purpose-define symbols and literals :

(1) Determine length of machine instructions (MOTGET 1)

(2) Keep track of Location Counter (LC)

(3) Remember values of symbols until pass 2 (STSTO)

(4) Process some pseufo ops, e.g. EQU, DS (POTGET 1)

(5) Remember literals (LITSTO)

Pass 2 : Purpose-generate object program :

(1) Look up value of symbols (STGET)

(2) Generate instructions (MOTGET2)

(3) Generate data (for DS, DC, and literals)

(4) Process pseudo ops (POTGET2)

Q.2.

```
Test    Start   0
        BALR    15, 0
        LTORG
        Using   *, 15
        LR      5, 15
        L       1, Data 1
        Using   *, 10
        BR      14
Data 2  DC      F '10'
Data 1  DC      F '22'
AA      DC      H '2'
BB      DS      F
CC      EQU     1
DD      EQU     2
        BALR    2, 0
        Using   * + CC, AA
        LA      7, = A (BB)
        BR      6
        DC      H '6'
        DROP    DD
        L       9, = A (Data 1)
        A       9, AA
        ST      9, = F '40'
        LTORG
        END
```

For the above code get B.T, L.T, S.T alongwith machine code.

13M

Ans.

```
0   BALR   15, 0
2   LR     5, 15
4   L      1, 12 (0, 15)
8   BCR    15, 14
10  10     ....[F'10']
14  22     .....[F' 22']
18  2      ......[H' 2']
20  -
24  BALR   2, 0
26  LA     7, 12 (0, 10)
30  BR     15, 6
```

32 H'6'

34 L 9, 6 (0,10)

38 A 9, 18 (0, 15)

42 ST 9, 29 (0, 2)

48 = A (BB)

52 = A (DATA 1)

56 = F'40'

Base Table (BT) :

Statement	Reg. No.		Contents
4	15	Y	2
7	10	Y	8
16	2	Y	27

Literal Table (LT) :

Literal / Symbol	Value	Length	Relocation
F'10'	10	4	R
F'22'	14	4	R
H'2'	18	2	R
= A (BB)	48	4	R
H'6'	32	2	R
= A (DATA1)	52	4	R
= F'40'	56	4	R

Symbol Table (ST) :

Symbol	Move	Length	Relocation
TEST	0	1	R
DATA 2	10	4	R
DATA 1	14	4	R
AA	18	4	R
BB	20	4	R
CC	1	1	A
DD	2	1	A

UNIT - II

(Total Marks : 28)

Q.3. Get MNT, MDT, ALA and expanded code for the following code fragment : [14M]

```
        MACRO
              XUZ &A
        MACRO
              &A &B
        MACRO
              &B &C
              ST 15, A(&C)
        BALR 14, 15
              L  1, = F '3'
              A  1, &C
              ST 1, &C
        MEND
        MEND
        MEND
TEST    START
              BALR
              :
              XYZ     SQRT
              SQRT    TOPOS
              TOPOS   D1
D1            DC      F'10'
              END
```

Ans. Macro name table :

Index	Macro name	Mdti
1	XYZ	1
2	SQRT	14
3	TOPOS	24

Macro Definition Table (MDT) :

Index	Macro Name
1	XYZ & A
2	MACRO
3	# 1 & B

Index	Macro Name
4	MACRO
5	& B & C
6	ST 15, A (& C)
7	BALR 14, 15
8	L 1, = F'3'
9	A 1, & C
10	ST 1, & C
11	MEND
12	MEND
13	MEND
14	SQRT & B
15	MACRO
16	# 2 & C
17	ST 15, A (& C)
18	BALR 14, 15
19	L 1, = F'3'
20	A 1, & C
21	ST 1, & C
22	MEND
23	MEND
24	TOPOS & C
25	ST 15, A (# 3)
26	BALR 14, 15
27	L 1, = F'3'
28	A 1, # 3
29	ST 1, # 3
30	MEND

Argument List Array (ALA) :

Index	Argument
0	"bbbbbbbb"
1	"XYZbbbbb"
2	"SQRTbbbb"
3	"TOPOSbbb"

Q.4.(a) For the given MDT and MNT, write the possible assembly language program : **8M**

MNT :

Index	Macro name	MDTI
1	XYZ	1
2	MIT	4

MDT :

Index	Card
1	XYZ & A
2	ST 1, #1
3	MEND
4	MIT &Z
5	AR 4, #1
6	XYZ #1
7	MEND

Ans.

```
        MACRO
        XYZ     & A
        ST      1, & A
        MEND
        MACRO
        MLT     & Z
        AR      4, & Z
        XYZ     & Z
        MEND
PROG    START
        USING   *, 15
        MIT     HELLO
        ST      2, 3
        HELLO   YALE
YALE    EQU     5
ALL     DC      F'3'
        END
```

Q.4.(b) Explain the similarities and differences between MACRO expansion and subroutine calls. **6M**

Ans. **See P.PS-17, Q.3(a).**

UNIT - III

(Total Marks : 26)

Q.5.(a) Draw detailed flowchart for pass-2 of Direct Linking Loader of IBM 360 type machine. **7M**

Ans. **See P.3-9, Q.12.**

Q5.(b) Explain which kind of information is generated by the assembler for the 2 pass Link Loader. **6M**

Ans. **See P.3-8, Q.10.**

Q.6.(a) Explain the concept of Relocation. Can relocation be avoided? **6M**

Ans. **See P.3-7, Q.7, 8.**

Q.6.(b) How are ESD and RLD cards related? Explain with an example. **7M**

Ans. **See P.3-15, Q.21.**

See P.3-18, Q.25.

UNIT - IV

(Total Marks : 26)

Q.9. **Explain purpose of following files :**

(i) Relocatable object file.

(ii) Executable object file.

(iii) Shared object file.

Your answer should include their structure, creation process and commands used to manipulate these files. **13M**

Ans.

(i) Relocatable object file :

- A relocatable file holds code and data suitable for linking with other object files to create an executable or a shared object file.
- A program header table, if present, tells the system how to create a process image. Files used to build a process image (execute a program) must have a program header table; relocatable files do not need one.
- A section header table contains information describing the files sections. Every section has an entry in the table; each entry gives information such as the section name, the section size etc.
- Files used during linking must have a section header table; other object files may or may not have one.

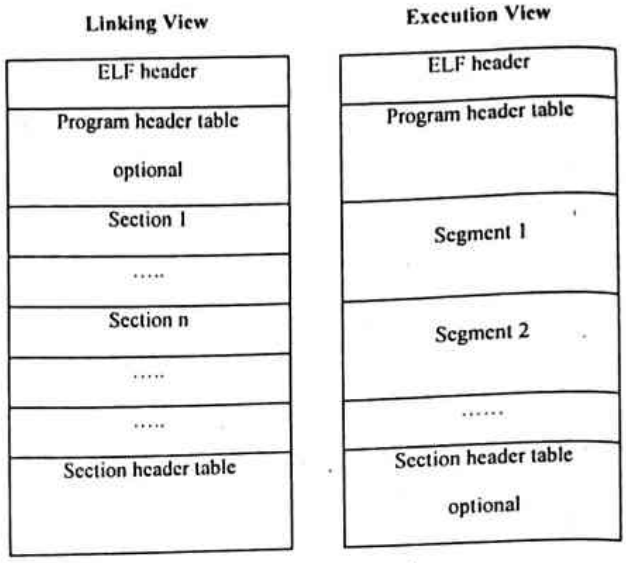


Fig. (a) Object file format

(ii) Executable object file :

- On the other hand, shared object segments typically contain position-independent code. This lets a segments virtual address change from one process to another, without invalidating execution behavior.
- Through the system chooses virtual addresses for individual processes, it maintains the segment's relative positions. Because position-independent code uses relative addressing between segments, the difference between virtual addresses in memory must match the difference between virtual addresses in the file.
- The following table shows possible shared object virtual address assignments for several processes, illustrating constant relative positioning. The table also illustrates the base address computations.

Source	Text	Data	Base Address
File	0x200	0x2a400	0x0
Process 1	0x80000200	0x8002a400	0x80000000
Process 2	0x80081200	0x800ab400	0x80081000
Process 3	0x900c0200	0x900ca400	0x900c0000
Process 4	0x900c6200	0x900f0400	0x900c6000

Fig. (b) Example Shared Object Segment Addresses

```
#define EI_NIDENT        16

typedef struct {
        unsigned char    e_indent (EI_NIDENT);
        Elf32_Half       e_type;
        Elf32_Half       e_machine;
        Elf32_Word       e_version;
        Elf32_Addr       e_entry;
        Elf32_Off        e_phoff;
        Elf32_Off        e_shoff;
        Elf32_Word       e_flags;
        Elf32_Half       e_ehsize;
        Elf32_Half       e_phentsize;
        Elf32_Half       e_phnum;
        Elf32_Half       e_shentsize;
        Elf32_Half       e_shnum;
        Elf32_Half       e_shstrndx;
} Elf32_Ehdr;
```

Fig. (c) ELF Header

- e_ident : The initial bytes mark the file as an object file and provide machine-independent data with which to decode and interpret the file's contents. Complete descriptions appear below, in "ELF Identification".
- e_type : This member identifies the object file type.

Name	Value	Meaning
ET_NONE	0	No file type
ET_REL	1	Relocatable file
ET_EXEC	2	Executable file
ET_DYN	3	Shared object file
ET_CORE	4	Core file
ET_LOPROC	0xff00	Processor-specific
ET_HIPROC	0xffff	Processor-specific

Fig. (d) ELF identification

- Although the core file contents are unspecified, type ET_CORE is reserved to mark the file. Values from ET_LOPROC through ET_HIPROC (inclusive) are reserved for processor-specific semantics. Other values are reserved and will be assigned to new object file types as necessary.

(iii) Shared object file :

- As the system creates or augments a process image, it logically copies a file's segment to a virtual memory segment.
- When and if the system physically reads the file depends on the program's execution behavior, system load, etc. A process does not require a physical page unless it references the logical page during execution and processes commonly leave many pages unreferenced. Therefore delaying physical reads frequently obviates them, improving system performance.
- To obtain this efficiency in practice, executable and shared object files must have segment images whose file offsets and virtual addresses are congruent, modulo the page size.
- Virtual addresses and file offsets for the SYSTEM V architecture segments are congruent modulo 4 KB (0x1000) or larger powers of 2. Because 4 KB is the maximum page size, the files will be suitable for paging regardless of physical page size.

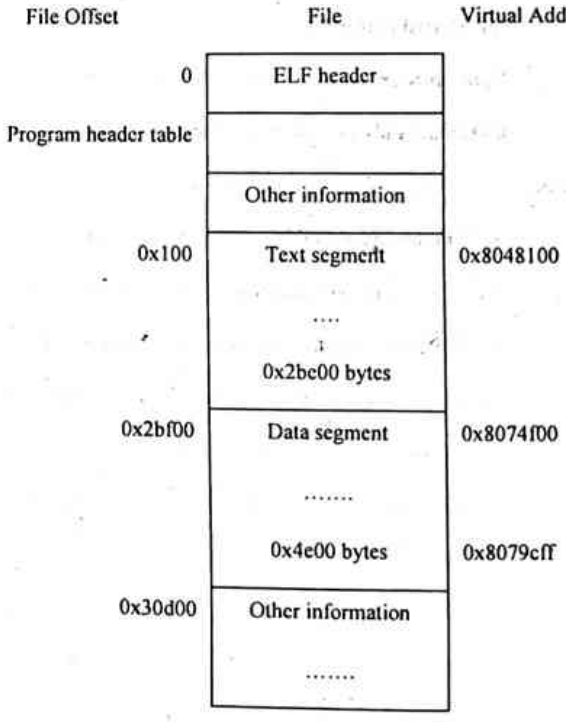


Fig.(e) Executable file

Member	Text	Data
p_type	PT_LOAD	PT_LOAD
p_offset	0x100	0x2bf00
p_vaddr	0x8048100	0x8074f00
p_paddr	Unspecified	Unspecified
p_filesz	0x2bc00	0x4c00
p_memsz	0x2bc00	0x5c24
p_flags	PF_R + PF_X	PF_R + PF_W + PF_X
p_align	0x1000	0x1000

Fig. (f) Program Header Segments

- Although the example's file offsets and virtual addresses are congruent modulo 4 KB for both text and data, up to four file pages hold impure text or data (depending on page size and file system block size).
- The first text page contains the ELF header, the program header table, and other information.
- The last text page holds a copy of the beginning of data.
- The first data page has a copy of the end of text.
- The last data page my contain the information not relevant to the running process.
- Logically, the system enforces the memory permissions as if each segment were complete and separate, segments addresses are adjusted to ensure each logical page in the address space has a single set of permissions. In the example above, the region of the file holding the end of text and the beginning of data will be mapped twice : at one virtual address for text and at a different virtual address for data.
- The end of the data segment requires special handling for unintialized data, which the system defines to begin with zero values. Thus if a files' last data page includes information not in the logical memory page, the extraneous data must be set to zero, not the unknown contents of the executable file.
- "Impurities" in the other three pages are not logically part of the process image; whether the system expunges them is unspecified.
- The memory image for this program follows, assuming 4 KB (0x1000) pages.

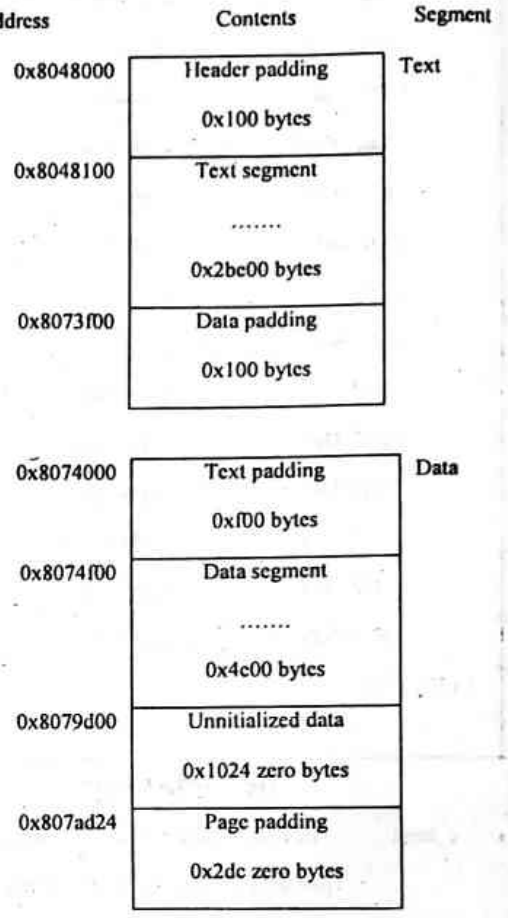


Fig.(g) Process Image Segments

- One aspect of segment loading differs executable files and shared objects.
- Executable file segments typically contain absolute code.
- To let the process execute correctly, the segments must reside at the virtual addresses used to build the executable file. Thus the system uses the p_vaddr values unchanged as virtual addresses.

Q.10.(a) What is Linkage Editor? Which command is used to this effect? Explain how in unix like system, we can link two object files. **8M**

Ans. Linkage Editor :

- The linkage editor link-edits specified object files to create one executable file.

- Invoking the compiler with one of the invocation commands automatically calls the linkage editor unless we specify one of the following compiler options : -E, -P, -c, -S. –q syntax only or -#.
- The linkage editor generates an executable file and places it in our current directory. The default name for an executable file is a.out. To name the executable file explicitly, use the –o file_name option with the compiler invocation command. Where file_name is the name we want to give to the executable file. For example, to compile myfile.c and generate an executable file called myfile, enter :

  If we use the –qmkshrobj option to create a shared library, the shared object created will have a .so filename extension.
- We can invoke the linkage editor explicitly with the ld command. However, the compiler invocation commands set several linkage-editor options, and link some standard files into the executable output by default.
- In most cases, it is better to use one of the compiler invocation commands to link-edit our object files.
- The link editor is called using the command :

  ld [options] filename 1 filename 2 ...
- Link editor command language directives are passed in a file named on the command line. Such a file is often called an ifile or mapfile.
- Any file named on the command line that is not identifiable (from the magic number) as an object module or an archive library is assumed to contain link editor directives. However, to use such an ifile, we must link the object files statically.

Q.10.(b) Explain with suitable example, use of gcc to link files. **SM**

Ans.

- Using the GCC driver to create shared libraries, we may find that they are not always linked with the shared libgcc.
- If GCC finds, at its configuration time, that we have a non-GNU linker or a GNU linker that does not support option –ch-frame-hdr, it links the shared version of libgcc into shared libraries by default. Otherwise, it takes advantage of the linker and optimizes away the linking with the shared version of libgcc, linking with the static version of libgcc by default. This allows exceptions to propagate through such shared libraries, without incurring relocation costs at library load time.

(i) **-static-libasan :**

- When the –fsanitize = address option is used to link a program, the GCC driver automatically links against libasan.
- If libasan is available as a shared library, and the –static option is not used, then this links against the shared version of libasan.
- The –static-libasan option directs the GCC driver to link libasan statically, without necessarily linking other libraries statically.

(ii) **-static-libtsan :**

- When the –fsanitize=thread option is used to link a program, the GCC driver automatically links against libtsan.
- If libtsan is available as a shared library, and the –static option is not used, then this links against the shared version of libtsan statically, without necessarily linking other libraries statically.

(iii) **-static-liblsan :**

- When the –fsanitize=leak option is use to link a program, the GCC driver automatically links against liblsan.
- If liblsan is available as a shared library, and the –static option is not used, then this links against the shared version of liblsan.
- The –static–liblsan option directs the GCC driver to link liblsan statically, without necessarily linking other libraries statically.

(iv) **-static-libubsan :**

- When the-fsanitize=undefined option is used to link a program, the GCC driver automatically links against libubsan.
- If libubsan is available as a shared library, and the –static option is not used, then this links against the shared version of libubsan.
- The –static-libubsan option directs the GCC driver to link libubsan statically, without necessarily linking other libraries statically.

(v) **-static-libstdc++ :**

- When the g++ program is used to link a C++ program, it normally automatically links against libstdc++.
- If libstdc++ is available as a shared library, and the –static option is not used, then this links against the shared version of libstdc++. That is normally fine.

- However, it is sometimes useful to freeze the version of libstdc++ used by the program without going all the way to fully static link.
- The -static-libstdc++ option directs the g++ driver to link libstdc++ statically, without necessarily linking other libraries statically.

(vi) **–symbolic :**

- Bind references to global symbols when building a shared object.
- Warn about any unresolved references (unless overridden by the link editor option –Xlinker -z -Xlinker defs). Only a few systems support this option.

(vii) **–T script :**

- Use script as the linker script. This option is supported by most systems using the GNU linker.
- On some targets, such as bare-board targets without an operating system, the –T option may be required when linking to avoid references to undefined symbols.

(viii) **–Xlinker option :**

- Pass option as an option to the linker. We can use this to supply system specific linker options that GCC does not recognize.
- If we want to pass an option that takes a separate argument, we must use Xlinker twice, once for the argument. For example, to pass –assert definitions, we must write –Xlinker-assert-Xlinker definitions.
- It does not work to write –Xlinker "– assert because this passes the entire string as a single argument, which is not what the linker expects.
- When using the GNU linker, it is usually more convenient to pass arguments to linker options using the option=value syntax than as separate arguments.
- For example, we can specify –Xlinker –Map = output . map rather than Xlinker –Map –Xlinker output.map. Other linkers may not support this syntax for command-line options.

(ix) **–wl, option :**

- Pass option as an option to the linker. If option contains commas, it is split into multiple options at the commas.
- We can use this syntax to pass an argument to the option. For example, –wl, –Map, output.map passes –Map output.map to the linker. When using the GNU linker, we can also get the same effect with –Wl, –Map = output.map.

(x) **–u symbol :**

- Pretend the symbol symbol is undefined, to force linking of library modules to define it.
- We can use –u multiple times with different symbols to force loading of additional library moules.

(xi) **–z keyword :**

- –z is passed directly on to the linker along with the keyword keyword.

UNIT - V
(Total Marks : 26)

Q.7.(a) Explain with respect to char drivers, data structures and the methods, for the following views :
(i) Code-Flow view.
(ii) Data-Flow view. **6M**

Ans. From a code-flow perspective, char drivers have the following :

- An initialization (or init( )) routine that is responsible for initializing the device and seamlessly tying the driver to the rest of the kernel via registration functions.
- A set of entry points (or methods) such as open( ), read( ), ioctl( ), llseek( ) and write( ), which directly correspond to I/O system calls invoked by user applications over the associated/dev node.
- Interrupt routines, bottom halves, timer handlers, helper kernel threads, and other support infrastructure. These are largely transparent to user applications.
- From a data-flow perspective, char drivers own the following key data structures :
- As per-device structure this is the information repository around which the driver revolves struct cdev, a kernel abstraction for character drivers. This structure is usually embedded inside the per device structure referred previously.
- struct file_operations, which contains the addresses of all driver entry points.
- struct file, which contains information about the associated/dev node.

Q.7.(b) Let the output on the terminal be as follows :

```
ls – l/dev
total 0
crw -------    1 root root  5,  1  console
lrwxrwxrwx     1 root root     3  cdrom → hlc
brw-rw ----    1 root disk  3,  0  hla
crw -------    1 root tty   4,  1  tty 1
```

Explain the output with respect to 1st, 5th and 6th column. **7M**

Ans.

```
ls-l /dev
total 0
crw…….. 1     root   root   5,   1   Console
…..
lrwxrwxrwx 1  root   root        3   cdrom-> hdc
……
brw-rw…..1    root   disk   3,   0   hda
brw-rw…..1    root   disk   3,   1   hda1
…..
crw……1        root   tty    4,   1   tty1
crw…..1       root   tty    4,   2   tty2
```

- The first character in each line of the ls output denotes the driver type : c signifies a char driver, b stands for a block driver and / denotes a symbolic link.
- The numbers in the fifth column are called major numbers and those in the sixth column are minor numbers.
- A major number broadly identifies the driver, whereas a minor number pinpoints the exact device serviced by the driver. For example, the IDE block storage driver / dev/ had owns a major number of 3 and is in charge of handling the hard disk on our system, but when we further specify a minor number of 1 (/dev/hda 1), that narrows it down to the first disk partition.
- Char and block drivers occupy different spaces, so we can have same major number assigned to a char as well as a block driver.

Q.8.(a) Explain the following terms w.r.t. unix : **6M**

(i) Input drivers.

(ii) Input event drivers

(iii) Input sub-system

Ans.

(i) Input Drivers :

- Input drivers usually interface with low-level hardware, such as USB, PCI memory or I/O regions, or serial port I/O regions.
- They convert the low-level hardware version of the user input into the standard event format before sending it to the input core.
- The input core uses a standard kernel plugin design, with input_register_device( ) used to add each device and input_unregister_device( ) used to remove it. The argument to these calls is the input_dev structure shown in figure given below.
- Although this structure looks quite large, most of the entries are provided to allow a drive to specify the capabilities of the device.
- Here is a typical example from /proc /bus /input /devices showing a USB mouse :

```
I : Bus = 0003 Vendor = 046d Product = c002 Version = 0120
N : Name= "Logitech USB-PS/2 Mouse M-BA47"
P : Phys = usb-00 : 01. 2-2 . 2/ input 0
H : Handlers = mouse0 event2
B : EV = 7
B : KEY = f 0000 0 0 0 0 0 0 0 0 0
B : REL = 103
```

(ii) Input Event Drivers :

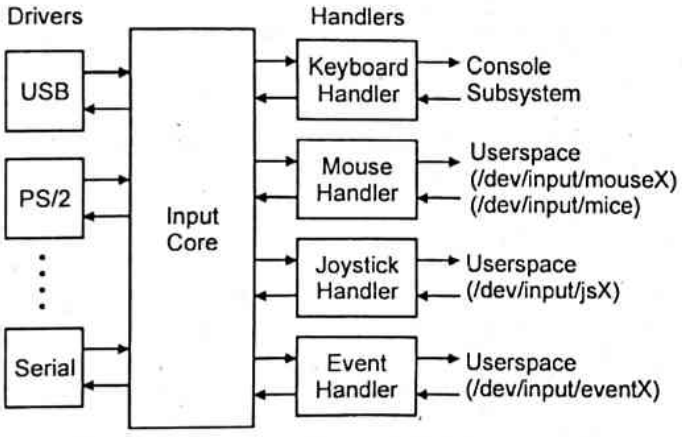
- The interaction between various elements is through events, which are implemented as structures.
- The first field (time) is a simple timestamp, while the other fields are more interesting.
- The type field shows the generic type of event being reported, for example, a key press or button press, relative motion (like moving a mouse) or absolute motion (like moving a joystick).
- The code field tells which of the various buttons or axes are being manipulated, while the value field tells us what the state or motion is. For example, if the type is a key or button, code tells us which key or button it is and value tells us if the button has been pressed or released.
- Similarly, if type is a relative axis, then code tells us which axis, and

value tells us how much motion we have and the direction of that motion on that axis.

- If we move a mouse in a diagonal direction, while simultaneously moving the scroll wheel , we will get three relative events per update : one for motion in the vertical direction (Y-axis), one for motion in the horizontal direction (X-axis) and one for motion of the wheel.

(iii) **Input sub-system :**

- The input subsystem is the part of the linux kernel that manages the various input devices (such as keyboards, mice, joysticks, tablets and a wide range of other devices) that a user uses to interact with the kernel, command line and graphical user interface.
- This subsystem is included in the kernel because these devices usually are accessed through special hardware interfaces (such as serial ports, PS/2 ports, Apple Desktop Bus and the Universal Serial Bus), which are protected and managed by the kernel.
- The kernel then exposes the user input in a consistent, device-independent way to user space through a range of defined APIs.



- The three elements of the input subsystem are the input core, drivers and event handlers.
- The relationship between them is shown in figure given above. Note that while the normal path is from low-level hardware to drivers, drivers to input core, input core to handler and handler to user space, there usually is a return path as well.
- This return path allows for such things as setting the LEDs on a keyboard and providing motion commands to force feedback joysticks. Both directions use the same event definition, with different type identifiers.

Q.8.(b) Explain Windows Driver Installation in detail. **7M**

Ans. See P.5-9, Q.10

UNIT - VI

(Total Marks : 28)

Q.11.(a) What are the differences and similarities between an interpreter and a compiler? **8M**

Ans. Differences :

Sr. no.	Compiler	Interpreter
(1)	Fast, creates executable file that runs directly on the CPU.	Slower, interprets code one line at a time.
(2)	Debugging is more difficult. One error can produce many spurious errors.	Debugging is easier. Each line of code is analyzed and checked before being executed.
(3)	More likely to crash the computer. The machine code is running directly on the CPU.	Less likely to crash as the instructions are being carried out either on the interpreters command line or within a virtual machine environment which is protecting the computer from being directly accessed by the code.
(4)	Easier to protect intellectual property as the machine code is difficult to understand.	Weaker intellectual property as the source code (or bytecode) has to be available at run time. For example if we write a flash actionscript application, we can easily get de-compilers that convert the p-code back into actionscript source code (unless we use encryption, but that is another story).

(5)	Uses more memory -all the execution code needs to be loaded into memory, although tricks like dynamic link libraries lessen this problem.	Uses less memory, source code only has to be present one line at a time in memory.
(6)	Unauthorized modification to the code more difficult. The executable is in the form of machine code. So it is difficult to understand program flow.	Easier to modify as the instructions are at a high level and so the program flow is easier to understand and modify.

Similarity :

Interpreter and compiler both are used to translate HLL program into object code.

Q.11.(b) Explain Lexical analysis and Syntax analysis in compilers. **6M**

Ans. See P.6-4, Q.4.

OR

Q.12.(a) What information must be in the symbol table so that intermediate code generation phase can generate code for the statement :

x = b + c * d + 70

Assume suitable data types for the identifier. **7M**

Ans.

- While translating a source program into a functionally equivalent object code represent –ation, a passer may first generate an intermediate representation.

The given statement is :

X = b + c * d + 70

Three address code :

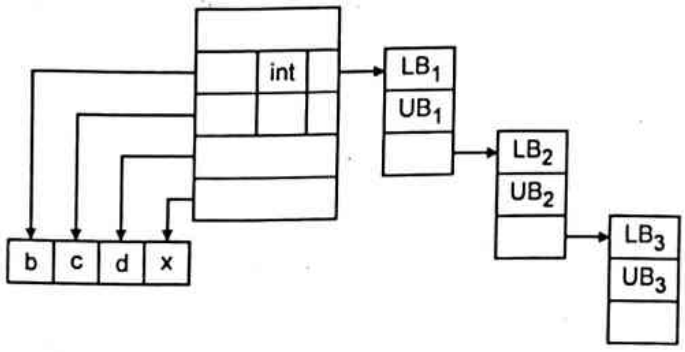
$T_1 = C * d$

$T_2 = b + T_1$

$T_3 = T_2 + 70$

$X = T_3$

Symbol table :



- Symbol table must have following information so that intermediate code generation phase can generate code for the statement :

X = b + c * d + 70

Symbol table :

Symbol	Value	Length	Relocation
b	48	4	R
c	52	4	R
d	56	4	R
x	60	4	R

Assume symbols of full word and stored at given memory locations.

Q.12.(b) Explain any one of the following with suitable program : **7M**

(i) LEX

(ii) YACC.

Ans. See P.6-12, Q.15.

(ii) YACC :

Parser source code :

- The following example shows the contents of the calc.yacc file.
- This file has entries in all three sections of a yacc grammar file : declarations, rules, and programs.

```
% {
#include<stdio.h>

int regs[26];
int base;

%)

%start list
```

```
        {
                $$.a = $1.a & $3.a;
        }
        |
        expr '|' expr
        {
                $$.a = $1.a | $3.a;
        }
        |
        '-' expr %prec UMINUS
        {
                $$.a = -$2.a;
        }
        |
        LETTER
        {
                $$.a = regs [$1.a];
        }

        |
        number
        ;
number: DIGIT
        {
                $$ = $1;
                base = ($1 .a = = 0) ? 8 : 10;
        }       |
        number DIGIT
        {
                $$.a = base * $1.a + $2.a;
        }
        ;
%%
main ( )
{
        return (yyparse ( ));
}
yyerror (s)
char *s;
{
        fprintf (stderr, "%s\n", s);
}

yywrap ( )
{
        return (1);
}
```

- The file contains the following sections :

(1) **Declaration section :** This section contains entries that :

(i) Include standard I /O header file

(ii) Define global variables

(iii) Define the list rule as the place to start processing

(iv) Define the tokens used by the parser

(v) Define the operators and their precedence

(2) **Rules section :** The rules section defines the rules that parse the input stream :

(i) **% start :** Specifies that the whole input should match stat.

(ii) **% union :**

- By default, the values returned by actions and the lexical analyzer are integers.
- yacc can also support values of other types, including structures.
- In addition, yacc keeps track of the types, and inserts appropriate union member names so that the resulting parser will be strictly type checked.
- The yacc value stack is declared to be a union of the various types of values desired.
- The user declares the union, and associates union member names to each token and nonterminal symbol having a value.
- When the value is referenced through a **$$** or **$n** construction, yacc will automatically insert to appropriate union name, so that no unwanted conversions will take place.

(iii) **% type :** Makes use of the members of the %union declaration and gives an individual type for the values associated with each part of the grammar.

(iv) **%token :** Lists the tokens which come from lex tool with their type.

(3) **Programs section :** The programs section contains the following subroutines. Because these subroutines are included in this file, we do not need to use the yacc library when processing this file.

Subroutine Description		
Main	**yyerror(s)**	**Yywrap**
The required main program that calls the yyparse subroutine to start the program.	This error-handling subroutine only prints a syntax error message.	The wrap-up subroutine that returns a value of 1 when the end of input occurs.

Lexical analyzer source code :

This file contains include statements for standard input and output, as well as for the y.tab.h file. If you use the –d flag with the yacccommand, the yacc program generates that file from the yacc grammar file information. The y.tab.h file contains definitions for the tokens that the parser program uses. In addition, the calc.lex file contains the rules to generate these tokens from the input stream. The following are the contents of the calc.lex file.

```
% {
#include <stdio.h>
#include "y.tab.h"
int c;
% }
%%
" ";
[a –z]  {
            c = yytext [0];
            yylval.a = c – 'a';
            return (LETTER);
            }
[0 – 9] {
            c = yytext [0];
            yylval.a = c – '0';
            return (DIGIT);
[^a-z0-9\b]     {
            c = yytext [0];
            return (c);
            }
%%
```

NOTES

SUMMER EXAMINATION - 2015
B.E. IV SEM. (Computer Science & Engg.) (CBS)
SYSTEM PROGRAMMING

Statistical Analysis

Unit Number	Marks
Unit - I	28 M
Unit - II	26 M
Unit - III	26 M
Unit - IV	26 M
Unit - V	26 M
Unit - VI	28 M
Questions covered in VBD	160 M (100%)

SUMMER EXAMINATION - 2015
QUESTION PAPER

Q.1.(a) What are the various data formats used in assembler? Explain them. 7M

(b) Explain in detail the instruction formats used in assembler of IBM 360/370 machine. 7M

OR

Q.2.(a) Explain the databases, flowchart and algorithm for pass-1 assembler. 10M

(b) Write short notes on :

The general machine structure of IBM 360. 4M

Q.3.(a) Explain-MDTC, MNTC and MDTP in detail. 7M

(b) What is conditional macro expansion? Explain in detail. 6M

OR

Q.4.(a) What is a macro call within macro or nested macro call? Explain the utility of stack frame with example. 8M

(b) Explain one pass macro-processor for macro definition with macro definition. 5M

Q.5.(a) What are the four basic functions of loaders? How do relocating loaders perform these functions? 8M

(b) Explain the following types of address format :

(1) Absolute

(2) Simple relocatable. 5M

OR

Q.6.(a) What is the order of RLD, ESD, TXT, END and LDT cards in bunch of object decks? Why they are in this order? 8M

(b) Explain dynamic linking and dynamic loading. 5M

Q.7.(a) Explain in detail the format of common object file. 8M

(b) What is symbolic debugger? Explain. 5M

Q.8.(a) What do you mean by source code control system? 7M

(b) Explain make and link editor. 6M

Q.9.(a) Explain the steps in installation of Unix device drivers in detail. 8M

(b) Write short note on STREAM drivers. 5M

OR

Q.10.(a) Explain all the entry points for line printer device driver. 8M

(b) Design a device driver for handling a RAM disk. 5M

Q.11. Explain compiler and various phases of compiler in detail. 14M

OR

Q.12.(a) What is cross compiler and bootstrapping? 8M

(b) What are token recognizers? Explain ways of designing token recognizers. 6M

SUMMER EXAMINATION - 2015
SOLUTION
(Unit-Wise)

UNIT - I
(Total Marks : 28)

Q.1.(a) What are the various data formats used in assembler? Explain them. 7M

Ans. See P.1-8, Q.8.

Q.1.(b) Explain in detail the instruction formats used in assembler of IBM 360/370 machine. 7M

Ans. See P.1-6, Q.7.

Q.2.(a) Explain the databases, flowchart and algorithm for pass-1 assembler. 10M

Ans. See P.1-17, 19, Q.26 & 27.

Q.2.(b) Write short notes on :

The general machine structure of IBM 360. 4M

Ans. See P.1-6, Q.6.

UNIT - II
(Total Marks : 26)

Q.3.(a) Explain-MDTC, MNTC and MDTP in detail. 7M

Ans. See P.2-10, Q.8(a).

Q.3.(b) What is conditional macro expansion? Explain in detail. 6M

Ans. See P.2-5, Q.4.

Q.4.(a) What is a macro call within macro or nested macro call? Explain the utility of stack frame with example. **8M**

Ans. See P.2-6, 7, Q.5(a) & 7.

Q.4.(b) Explain one pass macro-processor for macro definition with macro definition. **5M**

Ans. See P.2-6, 7, Q.5(b) & 7.

UNIT - III
(Total Marks : 26)

Q.5.(a) What are the four basic functions of loaders? How do relocating loaders perform these functions? **8M**

Ans. See P.3-3, 7, Q.1 & 7.

Q.5.(b) Explain the following types of address format :

(1) Absolute

Ans. See P.3-4, Q.4.

(2) Simple relocatable. **5M**

Ans. See P.3-7, Q.7.

Q.6.(a) What is the order of RLD, ESD, TXT, END and LDT cards in bunch of object decks? Why they are in this order? **8M**

Ans. See P.3-13, Q.19.

See P.3-20, Q.27 (3) for diagram.

Q.6.(b) Explain dynamic linking and dynamic loading. **5M**

Ans. See P.3-11, Q.15.

UNIT - IV
(Total Marks : 26)

Q.7.(a) Explain in detail the format of common object file. **8M**

Ans. See P.4-2, Q.1.

Q.7.(b) What is symbolic debugger? Explain. **5M**

Ans. See P.4-11, Q.13.

Q.8.(a) What do you mean by source code control system? **7M**

Ans. See P.4-10, Q.10.

Q.8.(b) Explain make and link editor.

Ans. See P.4-11, Q.11 & Q.12.

UNIT - V
(Total Marks : 26)

Q.9.(a) Explain the steps in installation of Unix device drivers in detail. **8M**

Ans. See P.5-8, Q.9.

Q.9.(b) Write short note on STREAM drivers. **5M**

Ans. See P.5-5, Q.5.

Q.10.(a) Explain all the entry points for line printer device driver. **8M**

Ans. See P.5-14, Q.15.

Q.10.(b) Design a device driver for handling a RAM disk. **5M**

Ans. See P.5-11, Q.11.

UNIT - VI
(Total Marks : 28)

Q.11. Explain compiler and various phases of compiler in detail. **14M**

Ans. See P.6-2, 4, Q.1 & 4.

Q.12.(a) What is cross compiler and bootstrapping? **8M**

Ans. See P.6-9, Q.11.

Q.12.(b) What are token recognizers? Explain ways of designing token recognizers. **6M**

Ans. See P.6-7, Q.9.

NOTES

WINTER EXAMINATION - 2015
B.E. IV SEM. (Computer Science & Engg.) (CBS)
SYSTEM PROGRAMMING

Statistical Analysis

Unit Number	Marks
Unit - I	28 M
Unit - II	26 M
Unit - III	26 M
Unit - IV	28 M
Unit - V	26 M
Unit - VI	26 M
Questions covered in VBD	160 M (100%)

WINTER EXAM. - 2015
QUESTION PAPER

Q.1.(a) Explain evolution of the components of a programming system. **7M**

(b) Explain operating system user viewpoint : Batch control language. **7M**

OR

Q.2.(a) Give the databases required in pass - 1 and pass-2 of assembler design with database flowchart. **7M**

(b) Explain the format of MOT, POT, LT, ST and BT in the assembler design. **7M**

Q.3. Explain the features of macro facility in detail. **13M**

OR

Q.4. Prepare MDT, MNT and ALA for the following Macro program. Also write expanded source code : **13M**

```
MACRO
DEFINE    & SUB
MACRO
& SUB     & Y
CNOP      0, 4
BAL       1, * + 8
DC        A (&Y)
L         15, = v (& SUB)
BALR      14, 15
MEND
MEND
```

User call the Macro with the statement

```
DEFINE    COS
```

Q.5. Explain compile and go loader and general loader scheme in detail. Also compare them. **13M**

OR

Q.6. Assume that the following three program segments are stored consecutively in the order PRG E1 and E2 with PRG stored at absolute address 1000. Show the assembled program PRG for direct address machine and the complete memory map as loaded by BSS loader. Assume length of E1 is 120 bytes and E2 is 70 bytes :

```
PRG       START
          EXTRN     E1, E2
          L         1, FIVE
          A         1, FOUR
          ST        1, TEMP
          BAL       14, E1
          C         1, TEMP
          BNE       E2
          BR        14
FIVE      DC        F'5'
FOUR      DC        F''4'
TEMP      DS        F
          END
```

13M

Q.7.(a) Explain COFF file structure. **7M**

(b) What is symbolic debugger? Explain. **7M**

OR

Q.8.(a) Explain symbol table structure and its contents. **7M**

(b) What do you mean by source code control system? Explain. **7M**

Q.9.(a) What is device driver? Explain each device driver with example. **7M**

(b) Give and explain device driver installation in UNIX. **6M**

OR

Q.10.(a) Explain the features of character device driver with example. **7M**

(b) Explain the steps involved in designing device driver. **6M**

Q.11.(a) Give and explain different phases of compiler in detail. **8M**

(b) Write a note on cross compiler. **5M**

OR

Q.12.(a) Write a note on following :

(i) LEX

(ii) YACC **8M**

(b) Give and explain different database used in the syntax analysis phase of the compiler. **5M**

WINTER EXAM. - 2015 SOLUTION (Unit-Wise)

UNIT - I

(Total Marks : 28)

Q.1.(a) Explain evolution of the components of a programming system. **7M**

Ans. See P.1-3, Q.1

Q.1.(b) Explain operating system user viewpoint : Batch control language. **7M**

Ans.

- Monitor is a term that refers to the control programs of an operating system. Typically, in a batch system, the jobs are stacked in a card reader and the monitor system sequentially processes each job.
- A job may consist of several separate programs to be executed sequentially, each individual program being called a job step.
- In a batch monitor system, the user communicates with the system by way of a control language.
- In a simple batch monitor system, there are two classes of control cards; execution cards and definition cards.
- For example, an execution card may be in the following format:

  // Step name EXEC name of program to be executed, Argument 1, Argument 2.

  The job control card, a definition card, may take on the following format:

  // job name JOB

  (User name, identification, expected time use, lines to be printed out; expected number of cards to be printed out.)
- Usually, there is an end-of-file card, whose format might consist of /*, signifying the termination of a collection of data. Let us take the following example of a FORTRAN job.

```
// EXAMPLE JOB ABC, T168,1,100,0
//STEP1 EXEC FORTRAN, NOPUNCH
        READ 9100,N
        DO 100 I = 1, n
        I2 = I*I
        I3 = I*I*I
  100   PRINT 9100, I, I2, I3
  9100  FORMAT (3I10)
        END
/*
//STEP2   EXEC LOAD
/*
//STEP3   EXEC OBJECT
    10
/*
```

- The first control card is an example of a definition card. We have defined the user to be ABC.
- The system must set up an accounting file for the user, noting that he expects to use one minute of time, to output a hundred lines of output, and to punch no cards.
- The next control card, EXE FORTRAN, NOPUNCH, is an example of an execution card; that is, the system is to execute the program FORTRAN, given one argument – NOPUNCH. This argument allows the monitor system to perform more efficiently; since no cards are to be punched, it need not utilize the punch routines.
- The data to the compiler is the FORTRAN program shown, terminated by an end-of-file card /*.
- The next control card is another example of an execution card and in this case causes the execution of the loader. The program that has just been compiled will be loaded, together with all the routines necessary for its execution, where upon the loader will "bind" the subroutines to the main program. This job step is terminated by an end-of-file card.
- The EXEC OBJECT card is another execution card, causing the monitor system to execute the object program just compiled. The data card, 10, is input to the program and is followed by the end of file card.
- The simple loop shown in Fig. presents an overview of an implementation of a batch monitor system.
- The monitor system must read in the first card, presumably a job card. In processing a job card, the monitor saves the user's name, account number, allotted time, card punch limit and line print limit.

- If the next control card happens to be an execution card, then the monitor will load the corresponding program from secondary storage and process the job step by transferring control to the executable program.
- If there is an error during the processing, the system notes the error and goes back to process the next job step.

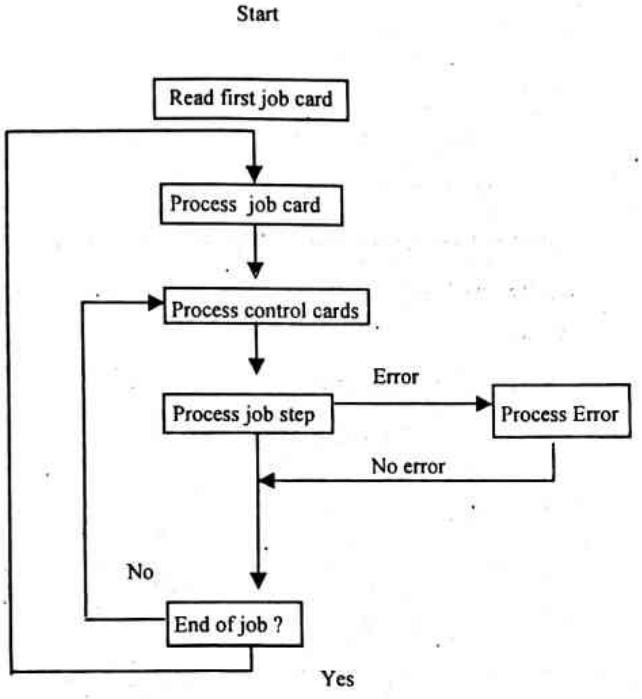


Fig. Main loop of a simple batch monitor system.

Q.2.(a) Give the databases required in pass - 1 and pass-2 of assembler design with database flowchart. **7M**

Ans. **See P.1-17, Q.26**

Q.2.(b) Explain the format of MOT, POT, LT, ST and BT in the assembler design. **7M**

Ans. **See P.1-15, Q.24**

UNIT - II

(Total Marks : 26)

Q.3. Explain the features of macro facility in detail. **13M**

Ans. **See P.2-4, 5, 6, Q.2, 3, 4, 5(a), (b)**

Q.4. **Prepare MDT, MNT and ALA for the following Macro program. Also write expanded source code :** **13M**

```
MACRO
DEFINE      & SUB
MACRO
& SUB       & Y
CNOP        0, 4
BAL         1, * + 8
DC          A (&Y)
L           15, = v (& SUB)
BALR        14, 15
MEND
MEND
```

User call the Macro with the statement

```
DEFINE      COS
```

Ans. **MDT table after Macroprocessing :**

Index		
1.	Define	& SUB
2.	MACRO	
3.	# 1	& y
4.	CNOP	0, 4
5.	BAL	1, * + 8
6.	DC	A (& y)
7.	L	15, = V (# 1)
8.	BALR	14, 15
9.	MEND	
10.	MEND	

MNT :

Index	Name	MDT Index
1	DEFINE	1

After processing of the statement.

DEFINE COS

MDT :

Index		
1.	DEFINE	& SUB
2.	MACRO	
3.	# 1	& y
4.	CNOP	0, 4
5.	BAL	1, * + 8
6.	DL	A (& y)
7.	L	15, = V (# 1)
8.	BALR	14, 15
9.	MEND	
10.	MEND	
11.	COS	& y
12.	CNOP	0, 4
13.	BAL	1, * + 8
14.	DC	A (# 1)
15.	L	15, = V (Cos)
16.	BALR	14, 15
17.	MEND	

MNT :

Index	Name	MDT Index
1	DEFINE	1
2	COS	11

UNIT - III

(Total Marks : 26)

Q.5. Explain compile and go loader and general loader scheme in detail. Also compare them. **13M**

Ans. See P.3-4, 5, Q.3, 5

Sr. No.	Compile and go loader	General loader
(1)	Assembler itself does the process of compiling and load the program in memory.	Assembler outputting the instructions and data as they are assembled circumvents the problem of waisting care for the assembler.
(2)	Assembler is loaded itself in memory.	Loader is loaded into the memory.
(3)	Program translated by assembler is loaded in memory.	Object deck is loaded in the memory.
(4)	Eg. WATFOR FORTRAN compiler.	Eg. Any general loader.

Q.6. Assume that the following three program segments are stored consecutively in the order PRG, E1 and E2 with PRG stored at absolute address 1000. Show the assembled program PRG for direct address machine and the complete memory map as loaded by BSS loader. Assume length of E1 is 120 bytes and E2 is 70 bytes :

```
PRG     START
        EXTRN   E1, E2
        L       1, FIVE
        A       1, FOUR
        ST      1, TEMP
        BAL     14, E1
        C       1, TEMP
        BNE     E2
        BR      14
FIVE    DC      F'5'
FOUR    DC      F''4'
TEMP    DS      F
        END
```

13M

Ans. Assembly of program for "direct-address" 360.

Rel. addr.	Relocation	Object Code	
0	00	E1	
4	00	E2	
8	01	L	1, 34
12	01	A	1, 38
16	01	ST	1, 42
20	01	BAL	14, 0
24	01	C	1, 42

Rel. addr.	Relocation	Object Code	
28	01	BC	7, 4
32	01	BCR	15, 14
34	0	F' 5'	
38	00	1F	
42	00	1F	

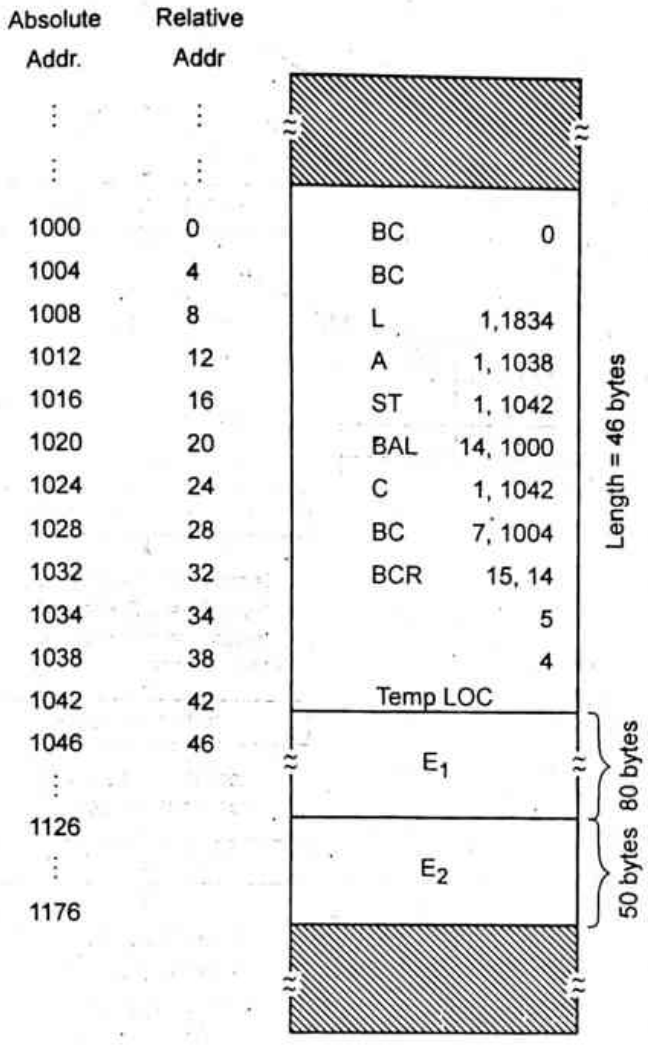


Fig. BSS loading of program for "direct access"

UNIT - IV
(Total Marks : 28)

Q.7.(a) Explain COFF file structure. **7M**

Ans. See P.4-2, Q.1

Q.7.(b) What is symbolic debugger? Explain. **7M**

Ans. See P.4-11, Q.13

Q.8.(a) Explain symbol table structure and its contents. **7M**

Ans. See P.4-5, Q.3

Q.8.(b) What do you mean by source code control system? Explain. **7M**

Ans. See P.4-10, Q.10

UNIT - V
(Total Marks : 26)

Q.9.(a) What is device driver? Explain each device driver with example. **7M**

Ans. See P.5-3, 4, Q.1, 3

Q.9.(b) Give and explain device driver installation in UNIX. **6M**

Ans. See P.5-8, Q.9

Q.10.(a) Explain the features of character device driver with example. **7M**

Ans. See P.5-4, 5, Q.3(2), 4

Q.10.(b) Explain the steps involved in designing device driver. **6M**

Ans. See P.5-3, Q.2

UNIT - VI
(Total Marks : 26)

Q.11.(a) Give and explain different phases of compiler in detail. **8M**

Ans. See P.6-4, Q.4

Q.11.(b) Write a note on cross compiler. **5M**

Ans. See P.6-9, Q.11

Q.12.(a) Write a note on following :

(i) LEX

(ii) YACC **8M**

Ans. See P.6-10, Q.13, 12

Q.12.(b) Give and explain different database used in the syntax analysis phase of the compiler. **5M**

Ans. Databases used by syntax analysis phase are :

(1) Source code.

(2) Output generated by lexical analysis phase i.e. tokens.

(3) Symbol table.

(4) Error management mechanisms table.

(5) Various data structures like id. which holds the important information as explained below :

Syntax analysis :

- The parser has two functions. It checks that the tokens appearing in its input, which is the output of the lexical analyzer, occur in patterns that are permitted by the specification for the source language.
- It also imposes on the tokens a tree-like structure that is used by the subsequent phases of the compiler.
- For example, if a PL/I program contains the expression A + / B then after lexical analysis this expression might appear to the syntax analyzer as the token sequence id + / id on seeing the / the syntax analyzer should detect an error situation, because the presence of these two adjacent binary operations violates the formation rules of a PL/I expression.
- The second aspect of syntax analysis is to make explicit the hierarchical structure of the incoming token stream by identifying which parts of the token stream should be grouped together.

(ii) **The analysis phases :**

- As translation progresses, the compiler's internal representation of the source program changes. We illustrate these representations by considering the translation of the statement position : = initial + rate * 60.
- Fig. shows the representation of this statement after each phase. The lexical analysis phase reads the characters in the source program and groups them into a stream of token in which each token represents a logically cohesive sequence of characters, such as an identifier, a keyword (if, while, etc.) a punctuation character, or a multi-character operator like : = . The character sequences forming a token is called the lexeme for the token.
- Certain tokens will be augmented by a "lexical value." For example, when an identifier like rate is found, the lexical analyzer not only generates a token, say id, but also enters the lexeme rate into the symbol table, if it is not already there.
- The lexical value associated with this occurrence of id points to the symbol-table entry for rate.

Syntax analysis imposes a hierarchical structure on the token stream.

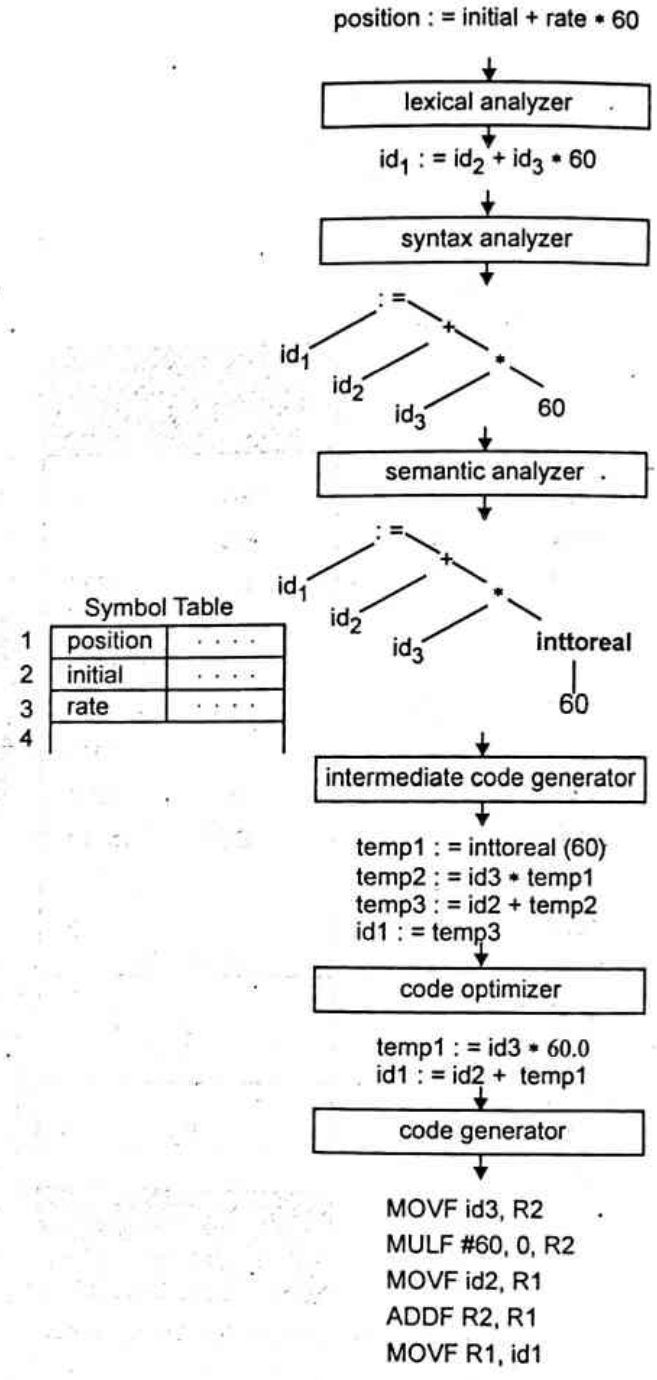


Fig. Translation of a statement.

Notes :-

SUMMER EXAMINATION - 2016
B.E. IV SEM. (Computer Science & Engg.) (CBS)
SYSTEM PROGRAMMING

Statistical Analysis

Unit Number	Marks
Unit - I	28 M
Unit - II	26 M
Unit - III	26 M
Unit - IV	26 M
Unit - V	28 M
Unit - VI	26 M
Questions covered in VBD	160 M (100%)

SUMMER EXAM. - 2016
QUESTION PAPER

Q.1.(a) What is the difference between : **10M**

(i) Procedure and processor

(ii) Procedure and program

(iii) Multiprocessing and multiprogramming

(iv) Open subroutine and closed subroutine

(v) Pure code and impure code

(b) Differentiate between user viewpoint and system viewpoint for design of operating system. **4M**

OR

Q.2.(a) Explain general machine architecture of IBM 360/370. **5M**

(b) Explain the following instructions : **3M**

(i) USING

(ii) EXTERN

(iii) BALR

(c) Explain various instruction formats of IBM 360. **6M**

Q.3.(a) Explain with flowchart simple one pass macro processor. **8M**

(b) What are the features of macro facility? **5M**

OR

Q.4. For the following program show MNT, MDT, ALA and expanded assembly language program. **13M**

```
        MACRO
        STORE   & ANS
        ST      1, &ANS
        MEND
        MACRO
        TRY     & ADD, & DEST
        SR      1, 1
        MACRO
        & ADD   & A, & B
        L       1, & A
        A       1, & B
        MEND
& DEST  DS      F
        MEND
ARITHSTART  O
        USING   *, 15
        TRY     PLUS, RESULT
        PLUS    N1, N2
        STORE   RESULT
N1      DC      F '4'
N2      DC      F '5'
        END
```

Q.5.(a) Draw a detailed flowchart for pass-2 of direct linking loader of IBM 360 m/c. **7M**

(b) Write short notes on any two : **6M**

(i) Compiler and go loader

(ii) Dynamic linking and dynamic loading

(iii) Absolute loader.

OR

Q.6.(a) Explain the term dynamic loading and dynamic linking with suitable example. **6M**

(b) What are the advantages of dividing loader into two separate programs viz. **3M**

(i) Binder

(ii) Module loader

(c) Explain the formats of GEST and LESA of direct linking loader. **4M**

Q.7.(a) Explain coff symbol table and string table. **7M**

(b) Explain the concept of storage classes. **6M**

OR

Q.8.(a) What is symbolic debugger? Explain in brief. **5M**

(b) Write short notes on : **8M**

(i) Source code control system

(ii) Link editor

Q.9.(a) Explain various sections involved in line printer driver. **8M**

(b) What are the major design issues of Unix device driver? Explain. **6M**

OR

Q.10.(a) Explain in detail different types of windows device drivers. **8M**

(b) What is device driver? Explain the necessity of device drivers. **6M**

Q.11.(a) What are the different phases of complier? Explain in brief. **8M**

(b) Explain cross compiler and boot strapping. **5M**

OR

Q.12.(a) Explain YACC in detail. **4M**

(b) List reasons for separating the analysis phase of compiling into lexical analysis and parsing. **3M**

(c) What is complier? Explain in brief the various databases used in lexical phase of compiler. **6M**

SUMMER EXAM. - 2016

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 28)

Q.1.(a) What is the difference between : **10M**

(i) Procedure and processor.

Ans.

Sr. No.	Procedure	Processor
(1)	A procedure is a particular method for performing a task.	A processor is a complex logical circuitry.
(2)	It is a subroutine or function coded to perform a specific task.	It can process the instructions that are written to make computer to perform particular task.
(3)	It is referred as Procedure.	It is also referred as Central Processing Unit (CPU).
(4)	It is also referred as algorithm, method, process, routine.	When it is integrated on single chip or IC it is termed as Microprocessor.
(5)	It makes the use of secondary storage.	It makes the use of register as storage for temporary result storage purpose.

(ii) Procedure and program.

Ans. **See P.PS-15, 26, Q.1(b)(i).**

(iii) Multiprocessing and multiprogramming.

Ans.

Sr. No.	Multiprocessing	Multiprogramming
(1)	Multiprocessing is the ability of an operating system to execute more than one process simultaneously on a multi processor machine.	Multiprogramming is also the ability of an operating system to execute more than one process on a single processor machine.
(2)	More than one process can reside into the main memory at one point of time.	More than one program (i.e. their processes) can reside into the main memory at one point of time.
(3)	In this, a computer uses more than one CPU at a time.	In this, a computer uses only one CPU at a time.
(4)	A computer running processes (relevant to many other executing programs) simultaneously is an example of multiprocessing.	A computer running excel and Firefox browser simultaneously is an example of multiprocessing.

(iv) Open subroutine and closed subroutine.

Ans. **See P.1-14, Q.21(a).**

(v) Pure code and impure code.

Ans. **See P.1-14, Q.21(b).**

Q.1.(b) Differentiate between user viewpoint and system viewpoint for design of operating system. **4M**

Ans. **See P.1-5, 6, Q.4, 5.**

Operating system is designed both by taking user view and system view into consideration. Below is what the users and system thinks about operating system.

User view :

(1) The goal of the operating system is to maximize the work and minimize the effort of the user.

(2) Most of the systems are designed to be operated by single user, however in some systems multiple users can share resources, memory. In these cases operating system is designed to handle available resources among multiple users and CPU efficiently.

(3) Operating system must be designed by taking both usability and efficient resource utilization into view.

(4) In embedded systems (automated systems) user view is not present.

(5) Operating system gives an effect to the user as if the processor is dealing only with the current task, but in background processor is dealing with several processes.

System view :

(1) From the system point of view operating system is a program involved with the hardware.

(2) Operating system is allocator, which allocate memory, resources among various processes. It controls the sharing of resources among programs.

(3) It prevents improper usage, error and handles deadlock conditions.

(4) It is a program that runs all the time in the system in the form of Kernel.

(5) It controls application programs that are not part of Kernel.

Q.2.(a) Explain general machine architecture of IBM 360/370. **5M**

Ans. See P.1-6, Q.6.

Q.2.(b) Explain the following instructions : **3M**

(i) USING

Ans. See P.1-11, Q.14(i).

(ii) EXTERN

Ans. **Extern :**

- The symbols in the operand field are declared as external symbols, symbols that are used in the current program but are declared elsewhere.

Syntax :

Label	Operation	Operand
none	EXTRN	list of symbols

Example :

```
A START  O ..
    JSR  Q    a Jump SubRoutine instruction..
    END
B START  O
P ADD    .. ..
Q L      .. ..
    END
```

- In the example above, the declaration : EXTRN Q or EXTRN P, Q should be added to program A, preferably at the beginning of the program.
- The assembler still does not have a value for Q but, because of the EXTRN declaration, it treats the JSR instruction in a special way and does not consider it an error.
- The programs are separately assembled and, therefore, when the assembler assembles A, it knows nothing about program B (it has erased the symbol table). Thus, in principle, the assembler cannot assemble the JSR instruction.
- Only the loader, while loading all the programs as one executable module, can use values of special symbols from one program in another. Thus our JSR instruction presents a special case, a case that cannot be fully handled by the assembler, and requires the help of the loader.
- It is interesting to note here that a one pass assembler, since it does not use a separate loader, cannot handle separate source files and thus cannot support the EXTRN, ENTRY directives.
- The EXTRN specifies that Q is a special symbol, a symbol defined outside the current program. The assembler stores Q in the symbol table with a special type ext, and without a value.
- The instruction itself is assembled and is written on the object file, without the value of Q in its address field.
- The assembler stores a pointer in the empty address field, to point to the symbol table entry for Q. At the end of pass 2, the symbol table is erased, but the special entries [in the symbol table] are copied by the assembler onto the object file, for later use by the loader.

- The object file thus contains the object code followed by the special symbol table. It will later be shown that there are other items in the object file.
- The following diagram shows the symbol table entry (entry 5) for symbol Q.

	name	value	type
1.	..	..	..
2.	..	.	.
3.	..	.	.
4.	..	.	.
5.	Q	-	ext
6.	..	..	..

- The JSR instruction goes on the object file as the record 'OpCode, 5, id=10'. The id indicates (to the loader) that the 5 in the address field is not the value of a symbol but a pointer to the special-symbol table at the end of the object file.
- That table includes the entry '5 Q ext' among, perhaps, other ones.
- After recognizing the id and finding the entry, the loader knows that the JSR instruction needs the value of symbol Q. This value is defined in program B and should, therefore, be included in the object file generated by that program.

(iii) BALR

Ans. **See P.1-11, Q.14(ii).**

Q.2.(c) Explain various instruction formats of IBM 360. **6M**

Ans. **See P.1-6, Q.7.**

UNIT - II
(Total Marks : 26)

Q.3.(a) Explain with flowchart simple one pass macro processor. **8M**

Ans. **See P.2-7, Q.7.**

Q.3.(b) What are the features of macro facility? **5M**

Ans. **See P.2-4, 5, 6, Q.2, 3, 4, 5(a),(b).**

Q.4. For the following program show MNT, MDT, ALA and expanded assembly language program. **13M**

```
        MACRO
        STORE   & ANS
        ST      1, &ANS
        MEND
        MACRO
        TRY     & ADD, & DEST
        SR      1, 1
        MACRO
        & ADD   & A, & B
        L       1, & A
        A       1, & B
        MEND
& DEST  DS      F
        MEND
ARITH   START   0
        USING   *, 15
        TRY     PLUS, RESULT
        PLUS    N1, N2
        STORE   RESULT
N1      DC      F '4'
N2      DC      F '5'
        END
```

Ans.

(1) Macro Name Table :

Index	Name	MDT Index
.	.	.
.	.	.
.	.	.
3	STORE	15
.	.	.
.	.	.
10	TRY	18
.	.	.
.	.	.
12	#2	12

(2) **Macro Definition Table :**

Index	Card		
.	.		
.	.		
.	.		
.	.		
15		STORE	&ANS
16		ST	1, #1
17		MEND	
18		TRY	&ADD, &DEST
19		SR	1, 1
20		MACRO	
21		#2	&A, &B
22		L	1, #3
23		A	1, #4
24		MEND	
25	#1	DS	F
26		MEND	
.	.		
.	.		

(3) **ALA (Argument List Array) :**

INDEX	ARGUMENT
0	"bbbbbbbb"
1	"RESULTbb"
2	"PLUSbbbb"
3	"N1bbbbbb"
4	"N2bbbbbb"

(4) **Expanded Code :**

```
ARITH    START   0
         USING   *, 15
         SR      1, 1
         L       1, N1
         A       1, N2
RESULT   DS      F
         ST      1, RESULT
N1       DC      F'4'
N2       DC      F'5'
         END
```

UNIT - III

(Total Marks : 26)

Q.5.(a) Draw a detailed flowchart for pass-2 of direct linking loader of IBM 360 m/c. **7M**

Ans. **See P.3-9, Q.12.**

Q.5.(b) Write short notes on any two : **6M**

(i) Compiler and go loader.

Ans. **See P.3-4, Q.3.**

(ii) Dynamic linking and dynamic loading.

Ans. **See P.3-11, Q.15.**

(iii) Absolute loader.

Ans. **See P.3-4, Q.4.**

Q.6.(a) Explain the term dynamic loading and dynamic linking with suitable example. **6M**

Ans. **See P.3-11, Q.15.**

Q.6.(b) What are the advantages of dividing loader into two separate programs viz. **3M**

(i) Binder

(ii) Module loader

Ans. **See P.3-11, Q.13.**

Q.6.(c) Explain the formats of GEST and LESA of direct linking loader. **4M**

Ans. **See P.3-14, Q.20.**

UNIT - IV

(Total Marks : 26)

Q.7.(a) Explain coff symbol table and string table. **7M**

Ans. **See P.4-5, Q.3.**

Q.7.(b) Explain the concept of storage classes. **6M**

Ans. **See P.4-7, Q.5.**

Q.8.(a) What is symbolic debugger? Explain in brief. **5M**

Ans. **See P.4-11, Q.13.**

Q.8.(b) Write short notes on : **8M**

(i) Source code control system.

Ans. **See P.4-10, Q.10.**

(ii) Link editor.

Ans. See P.4-11, Q.12.

UNIT - V
(Total Marks : 28)

Q.9.(a) Explain various sections involved in line printer driver. **8M**

Ans. See P.5-13, 14, Q.14, 15.

Q.9.(b) What are the major design issues of Unix device driver? Explain. **6M**

Ans. See P.5-3, Q.2.

Q.10.(a) Explain in detail different types of windows device drivers. **8M**

Ans. See P.5-6, Q.6.

Q.10.(b) What is device driver? Explain the necessity of device drivers? **6M**

Ans. See P.5-3, Q.1.

UNIT - VI
(Total Marks : 26)

Q.11.(a) What are the different phases of complier? Explain in brief. **8M**

Ans. See P.6-4, Q.4.

Q.11.(b) Explain cross compiler and boot strapping. **5M**

Ans. See P.6-9, Q.11.

Q.12.(a) Explain YACC in detail. **4M**

Ans. See P.6-10, Q.12.

Q.12.(b) List reasons for separating the analysis phase of compiling into lexical analysis and parsing. **3M**

Ans. See P.6-4, 7, Q.4(1, 2), 8, 9.

Q.12.(c) What is complier? Explain in brief the various databases used in lexical phase of compiler. **6M**

Ans. See P.6-2, 5, Q.1, 5.

Notes :-

WINTER EXAMINATION - 2016
B.E. IV SEM. (Computer Science & Engg.) (CBS)
SYSTEM PROGRAMMING

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	26 M
Unit - III	28 M
Unit - IV	26 M
Unit - V	26 M
Unit - VI	28 M
Questions covered in VBD	160 M (100%)

WINTER EXAM. - 2016
QUESTION PAPER

Q.1.(a) Explain the instruction format of IBM 360. **5M**

(b) Explain pass 1 of assembler in detail with the help of flowchart and databases. **8M**

OR

Q.2.(a) For the following ALP show its symbol table, literal table, base table and generated machine code : **8M**

```
TEST    START
BEGIN   BALR    15, 0
        USING   BEGIN +2, 15
        SR      4,4
        L       3, = F '10'
LOOP    L       2, DATA (4)
        A       2, = F '49'
        ST      2, DATA (4)
        A       4, = F '4'
        BCT     3, * - 16
        BR      14
DATA    DC      F '1, 2, 3, 4'
        END
```

(b) Differentiate between : **5M**

(i) Open subroutine and closed subroutine.

(ii) Pure procedure and impure procedure.

Q.3.(a) What is the use of conditional macro calls? **3M**

(b) Explain the necessity of stack for implementing macro call within macro definition. **5M**

(c) What databases are used by the two passes of macro processor? **5M**

OR

Q.4. Explain in detail the two pass macro processor with the help of neat and clean flowchart. What macro facilities are restricted in this design? **13M**

Q.5.(a) Write short notes on : **8M**

(i) Absolute loader.

(ii) Binders and module loader.

(b) Explain in detail dynamic linking and dynamic loading. **6M**

OR

Q.6.(a) What are the basic functions of loader? Explain. **3M**

(b) Show the entries in ESD, TXT and RLD cards for the following program : **11M**

```
JOHN    START
        EXTERN  SUM, DATA
        ENTRY   LOOP, POINTER
        BALR    15, 0
        USING   *, 15
        SR      4, 4
        L       1, FOUR
        A       2, FOUR
        ST      2, FOUR
        BR      14
FOUR    DC      F '4'
LOOP    DC      A (SUM + 4)
POINTER DC      A (LOOP-DATA)
        DC      A (POINTER-LOOP)
        DC      A (POINTER)
        END
```

Q.7.(a) Explain in detail the format of common object file, in detail. **8M**

(b) Write short note on Source Code Control System (SCCS). **5M**

OR

Q.8.(a) Explain make and link editor. **5M**

(b) Explain in detail storage classes. **4M**

(c) What is the structure of COFF symbol table? Explain. **4M**

Q.9.(a) Explain the steps in installation of unix device drivers in detail. **8M**

(b) Design a device driver for handling a RAM disk. **5M**

OR

Q.10.(a) Explain the following entry point and routine for device driver of a line printer : **6M**

(i) init (ii) Lpwork (iii) Open

(b) Differentiate between Character Driver and STREAM Driver. **7M**

Q.11.(a) What is compiler? Explain various phases of compiler. **8M**

(b) Write short note on regular expression. **6M**

OR

Q.12.(a) What is cross compiler? How does it use bootstrapping concept? **6M**

(b) How tokens are recognized by lexical phase? Explain in detail. **8M**

WINTER EXAM. - 2016
SOLUTION
(Unit-Wise)

UNIT - I
(Total Marks : 26)

Q.1.(a) Explain the instruction format of IBM 360. **5M**

Ans. See P.1-6, Q.7.

Q.1.(b) Explain pass 1 of assembler in detail with the help of flowchart and databases. **8M**

Ans. See P.1-19, Q.27.

Q.2.(a) For the following ALP show its symbol table, literal table, base table and generated machine code : **8M**

```
TEST    START
BEGIN   BALR    15, 0
        USING   BEGIN +2, 15
        SR      4,4
        L       3, = F '10'
LOOP    L       2, DATA (4)
        A       2, = F '49'
        ST      2, DATA (4)
        A       4, = F '4'
        BCT     3, * - 16
        BR      14
DATA    DC      F '1, 2, 3, 4'
        END
```

Ans. See P.1-22, Q.29.

Q.2.(b) Differentiate between : **5M**

(i) Open subroutine and closed subroutine.

(ii) Pure procedure and impure procedure.

Ans. See P.1-14, Q.21(a),(b).

UNIT - II
(Total Marks : 26)

Q.3.(a) What is the use of conditional macro calls? **3M**

Ans. See P.2-5, Q.4.

Q.3.(b) Explain the necessity of stack for implementing macro call within macro definition. **5M**

Ans. See P.2-7, Q.7.

Q.3.(c) What databases are used by the two passes of macro processor? **5M**

Ans. See P.2-13, Q.10.

Q.4. Explain in detail the two pass macro processor with the help of neat and clean flowchart. What macro facilities are restricted in this design? **13M**

Ans. See P.2-10, Q.8(a).

UNIT - III
(Total Marks : 28)

Q.5.(a) Write short notes on : **8M**

(i) Absolute loader. (ii) Binders and module loader.

Ans. See P.3-4, 11, Q.4, 13.

Q.5.(b) Explain in detail dynamic linking and dynamic loading **6M**

Ans. See P.3-11, Q.15.

Q.6.(a) What are the basic functions of loader? Explain. **3M**

Ans. See P.3-3, Q.1.

Q.6.(b) Show the entries in ESD, TXT and RLD cards for the following program : **11M**

```
JOHN    START
        EXTERN  SUM, DATA
        ENTRY   LOOP, POINTER
        BALR    15, 0
        USING   *, 15
        SR      4, 4
        L       1, FOUR
        A       2, FOUR
        ST      2, FOUR
        BR      14
FOUR    DC      F '4'
LOOP    DC      A (SUM + 4)
POINTER DC      A (LOOP-DATA)
        DC      A (POINTER-LOOP)
        DC      A (POINTER)
        END
```

Ans. See P.3-25, Q.30.

UNIT - IV

(Total Marks : 26)

Q.7.(a) Explain in detail the format of common object file, in detail. **8M**

Ans. See P.4-2, Q.1.

Q.7.(b) Write short note on Source Code Control System (SCCS). **5M**

Ans. See P.4-10, Q.10.

Q.8.(a) Explain make and link editor. **5M**

Ans. See P.4-11, Q.11, 12.

Q.8.(b) Explain in detail storage classes. **4M**

Ans. See P.4-7, Q.5.

Q.8.(c) What is the structure of COFF symbol table? Explain. **4M**

Ans. See P.4-5, Q.3.

UNIT - V

(Total Marks : 26)

Q.9.(a) Explain the steps in installation of unix device drivers in detail. **8M**

Ans. See P.5-8, Q.9.

Q.9.(b) Design a device driver for handling a RAM disk. **5M**

Ans. See P.5-11, Q.11.

Q.10.(a) Explain the following entry point and routine for device driver of a line printer : **6M**

(i) init (ii) Lpwork (iii) Open

Ans. See P.5-14, Q.15(i),(iv),(ii).

Q.10.(b) Differentiate between Character Driver and STREAM Driver. **7M**

Ans. See P.PS-29, Q.9(b).

UNIT - VI

(Total Marks : 28)

Q.11.(a) What is compiler? Explain various phases of compiler. **8M**

Ans. See P.6-2, 4, Q.1, 4.

Q.11.(b) Write short note on regular expression. **6M**

Ans. See P.6-8, Q.10.

Q.12.(a) What is cross compiler? How does it use bootstrapping concept? **6M**

Ans. See P.6-9, Q.11.

Q.12.(b) How tokens are recognized by lexical phase? Explain in detail. **8M**

Ans. See P.6-7, Q.9.

SUMMER EXAMINATION - 2017

B.E. IV SEM. (Computer Science & Engg.) (CBS)

SYSTEM PROGRAMMING

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	26 M
Unit - III	28 M
Unit - IV	26 M
Unit - V	26 M
Unit - VI	28 M
Questions covered in VBD	160 M (100%)

SUMMER EXAM. - 2017
QUESTION PAPER

Q.1.(a) Explain pass I of assembler in detail with the help of flowchart and databases. **8M**

(b) Differentiate between : **5M**

(i) Open subroutine and closed subroutine.

(ii) Pure procedure and impure procedure.

OR

Q.2. For the following program draw the symbol table, literal table, base table and object code. **13M**

```
PROG    START   0
        BALR    15, 0
        USING   *, 15
        LR      5, 15
        LH      1, Data 1
        USING   *, 10
        BR      14
DATA2   DC      F '11'
DATA1   DC      H '22'
TRS     DC      H '23'
BCK     DS      F
AA      EQU     1
DP      EQU     2
        BALR    2, 0
        USING   * + AA, TRS
        LA      7, = A (BCK)
        BR      6,
        DC      H '64'
        DROP    DP
        L       9, = A (DATA I)
        A       9, TRS
        LTORG
        ST      Q, = F '100'
        END
```

Q.3.(a) What is conditional macro expansion? Explain in detail with example. **6M**

(b) What databases are used by the two passes of macro processor? Explain advantages and disadvantages of combining macro processor with assembler. **7M**

OR

Q.4. For the following assembly program prepare MDT, MNT and ALA. Also write expanded source code. **13M**

```
MACRO
XYZ     & A
ST      1, & A
MEND
MACRO
MIT     & Z
MACRO
& Z     & W
AR      Y, & W
XYZ     ALL
MEND
ST      & Z, ALL
MEND
PROG START
        USING   *, 15
        MIT     HELLO
        ST      2, 3
        HELLO   YALE
YALE    EQU     5
ALL     DC      F '3'
        END
```

Q.5.(a) Explain relocating loader in detail with advantages and disadvantages. **8M**

(b) Explain in detail about dynamic linking and dynamic loading with example. **6M**

OR

Q.6.(a) Show the entries in ESD, TXT and RLD cards for the following program : **10M**

```
JOHN      START
          ENTRY     SUM, DATA
          EXTRN     LOOP, POINTER
          BALR      15, 0
          USING     *, 15
          SR        4, 14
          L         1, FOUR
          A         2, FOUR
          ST        2, FOUR
          BR        14
FOUR      DC        F '4'
LOOP      DC        A (SUM + 4)
POINTER   DC        A (LOOP-DATA)
          DC        A (POINTER-LOOP)
          DC        A (POINTER)
          END
```

(b) Write short note on link editor. **4M**

Q.7.(a) Explain in detail the format of common object file. **8M**

(b) Write a note on link editor. **5M**

OR

Q.8.(a) Describe source code control system with example. **7M**

(b) What is symbolic debugger? Explain in brief. **6M**

Q.9. Describe in detail the anatomy and types of device driver in UNIX SYSTEM. **13M**

OR

Q.10.(a) What are the major design issues in the study of device driver? Elaborate. **6M**

(b) Write a note on driver memory allocation. **7M**

Q.11. What is compiler? Draw a block diagram of the phases of compiler and indicate the main function of each phase. **14M**

OR

Q.12.(a) Describe LEX as a tool, for system programming. **7M**

(b) What is a cross compiler? Explain how boot strapping can be achieved. **7M**

SUMMER EXAM. - 2017

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 26)

Q.1.(a) Explain pass 1 of assembler in detail with the help of flowchart and databases. **8M**

Ans. **See P.1-19, Q.27.**

Q.1.(b) Differentiate between : **5M**

(i) Open subroutine and closed subroutine.

(ii) Pure procedure and impure procedure.

Ans. **See P.1-14, Q.21(a), (b).**

Q.2. For the following program draw the symbol table, literal table, base table and object code. **13M**

```
PROG      START     0
          BALR      15, 0
          USING     *, 15
          LR        5, 15
          LH        1, Data 1
          USING     *, 10
          BR        14
DATA2     DC        F '11'
DATA1     DC        H '22'
TRS       DC        H '23'
BCK       DS        F
AA        EQU       1
DP        EQU       2
          BALR      2, 0
          USING     * + AA, TRS
          LA        7, = A (BCK)
          BR        6,
          DC        H '64'
          DROP      DP
          L         9, = A (DATA 1)
          A         9, TRS
          LTORG
          ST        Q, = F '100'
          END
```

Ans. **Symbol table :**

Symbol	Value	Length	Relocation
PROG	0	1	R
DATA2	8	4	R
DATA1	12	2	R
TRS	14	2	R
BCK	16	4	R
AA	1	1	A
DP	2	1	A

Literal table :

Literal	Value	Length	Relocation
= A(BCK)	40	4	R
= A (DATA1)	44	2	R
= F '100'	46	4	R

Base table :

After statement no.	Register	Availability	Contents
3	15	Y	2
6	10	Y	8
15	14	Y	23
19	2	N	-

Object code :

Statement no.	Rel. Loc.	Code
2	0	BALR 15, 0
4	2	LR 5, 15
5	4	LH 1, 12(0, 15)
7	8	BCR 15, 14
8	10	11
9	14	22
10	16	23
11	18	--
14	22	BALR 2, 0
16	24	LA 7, 34(0, 10)
17	28	BCR 15, 2
18	30	64
20	32	L 9, 21(0, 14)
21	36	A 9, 12(0, 10)
22	40	18
	44	14
	46	100
23	50	ST 9, 40(0, 10)

UNIT - II

(Total Marks : 26)

Q.3.(a) What is conditional macro expansion? Explain in detail with example. **6M**

Ans. See P.2-5, Q.4.

Q.3.(b) What databases are used by the two passes of macro processor? Explain advantages and disadvantages of combining macro processor with assembler. **7M**

Ans. See P.2-13, Q.10, 9.

Q.4. For the following assembly program prepare MDT, MNT and ALA. Also write expanded source code. **13M**

```
MACRO
XYZ     & A
ST      1, & A
MEND
MACRO
MIT     & Z
MACRO
& Z     & W
AR      Y, & W
XYZ     ALL
MEND
ST      & Z, ALL
MEND
PROG START
        USING   *, 15
        MIT     HELLO
        ST      2, 3
        HELLO   YALE
YALE    EQU     5
ALL     DC      F '3'
        END
```

Ans. MDT :

Index	Card	
1	XYZ	&A
2	ST	1, #3
3	MEND	
4	MIT	&Z
5	MACRO	
6	#1	&W
7	AR	Y, &W
8	XYZ	ALL
9	MEND	
10	ST	#1, ALL
11	MEND	
12	HELLO	&W
13	AR	Y, #2
14	XYZ	ALL
15	MEND	

MNT :

Index	Name	MDT Index
1	XYZ	1
2	MIT	4
3	HELLO	12

ALA :

Index	Argument
0	"bbbbbbbb"
1	"HELLObbb"
2	"YALEbbbb"
3	"ALLbbbbb"

Expanded code :

```
PROG    START
        USING   *, 15
        AR      Y, YALE
        ST      1, ALL
        ST      HELLO, ALL
        ST      2, 3
        AR      Y, YALE
        ST      1, ALL
YALE    EQU     5
ALL     DC      F'3'
        END
```

UNIT - III

(Total Marks : 28)

Q.5.(a) Explain relocating loader in detail with advantages and disadvantages. **8M**

Ans. **See P.3-7, Q.7.**

Q.5.(b) Explain in detail about dynamic linking and dynamic loading with example. **6M**

Ans. **See P.3-11, Q.15.**

Q.6.(a) Show the entries in ESD, TXT and RLD cards for the following program : **10M**

```
JOHN    START
        ENTRY   SUM, DATA
        EXTRN   LOOP, POINTER
        BALR    15, 0
        USING   *, 15
        SR      4, 14
        L       1, FOUR
        A       2, FOUR
        ST      2, FOUR
        BR      14
FOUR    DC      F '4'
LOOP    DC      A (SUM + 4)
POINTER DC      A (LOOP- DATA)
        DC      A (POINTER-LOOP)
        DC      A (POINTER)
        END
```

Ans. **See P.3-25, Q.30.**

Q.6.(b) Write short note on link editor. **4M**

Ans. **See P.3-12, Q.16.**

UNIT - IV

(Total Marks : 26)

Q.7.(a) Explain in detail the format of common object file. **8M**

Ans. See P.4-2, Q.1.

Q.7.(b) Write a note on link editor. **5M**

Ans. See P.4-11, Q.12.

Q.8.(a) Describe source code control system with example. **7M**

Ans. See P.4-10, Q.10.

Q.8.(b) What is symbolic debugger? Explain in brief. **6M**

Ans. See P.4-11, Q.13.

UNIT - V

(Total Marks : 26)

Q.9. Describe in detail the anatomy and types of device driver in UNIX SYSTEM. **13M**

Ans. See P.5-4, Q.3.

Q.10.(a) What are the major design issues in the study of device driver? Elaborate. **6M**

Ans. See P.5-3, Q.2.

Q10.(b) Write a note on driver memory allocation. **7M**

Ans. See P.5-11, Q.12.

UNIT - VI

(Total Marks : 28)

Q.11. What is compiler? Draw a block diagram of the phases of compiler and indicate the main function of each phase. **14M**

Ans. See P.6-2, 4, Q.1, 4.

Q.12.(a) Describe LEX as a tool, for system programming. **7M**

Ans. See P.6-10, 11, Q.13, 14.

Q.12.(b) What is a cross compiler? Explain how boot strapping can be achieved. **7M**

Ans. See P.6-9, Q.11.

WINTER EXAMINATION - 2017
B.E. IV SEM. (Computer Science & Engg.) (CBS)
SYSTEM PROGRAMMING

Statistical Analysis

Unit Number	Marks
Unit - I	28 M
Unit - II	26 M
Unit - III	26 M
Unit - IV	26 M
Unit - V	28 M
Unit - VI	26 M
Questions covered in VBD	160 M (100%)

WINTER EXAM. - 2017
QUESTION PAPER

Q.1.(a) Define system programming. Explain the components involved in the foundation of system programming. **7M**

(b) Explain different instruction formats of IBM 360 / 370 machine. **7M**

OR

Q.2.(a) Explain the formats of following tables used in assembler design : **6M**

(i) MOT - Machine - OP table

(ii) POT - Pseudo - OP table

(iii) ST - Symbol table

(b) With neat schematic explain PASS - I of a two pass assembler. **8M**

Q.3.(a) Explain the formats of MNT, MDT and ALA with example. What is the difference between ALA prepared in pass - 1 and pass - 2 of microprocessor? **7M**

(b) Explain the stack organization for macro call within macros. What is the significance of storing old stack pointer? **6M**

OR

Q.4. Draw and explain the flowchart for macro defining macros. State the significance of READ flowchart. **13M**

Q.5.(a) Assuming that the following program segment is stored at absolute address 1000. Show the assembled program PRG for a direct address machine and the complete memory map as loaded by BSS loader. Assume length of E1 = 120 bytes and E2 = 70 bytes **8M**

```
PRG   START
      EXTRN    E1, E2
      L        1, FIVE
      A        1, FOUR
      ST       1, TEMP
      BAL      14, E1
      C        1, TEMP
      BNE      E2
      BR       14
      FIVE     DC      F'5'
      FOUR DC  F'4'
      TEMP DS  1 F
               END
```

(b) Describe in detail with neat diagram a compile and go loader. **5M**

OR

Q.6.(a) Explain the direct linking loader scheme. Also discuss about all the cards it uses. **9M**

(b) Write a note on dynamic loading and dynamic linking. **4M**

Q.7.(a) Explain in detail common object file format. **8M**

(b) What is symbolic debugger? Explain. **5M**

OR

Q.8.(a) Explain the structure of executable file. **5M**

(b) Explain different storage classes with example of each. **8M**

Q.9.(a) What is device driver? Explain the significance of device driver. **4M**

(b) Explain different UNIX device drivers in detail. **10M**

OR

Q.10.(a) Explain all the entry points for line printer device driver. **8M**

(b) Give and explain the steps of installation of UNIX device driver. **6M**

Q.11. Explain various phases of compiler in detail with database involved in each phase. **13M**

OR

Q.12.(a) Explain how LEX tool helps in the design of lexical analyzer. **6M**

(b) What is cross compiler? Explain how bootstrapping can be achieved. **7M**

WINTER EXAM. - 2017
SOLUTION
(Unit-Wise)

UNIT - I
(Total Marks : 28)

Q.1.(a) Define system programming. Explain the components involved in the foundation of system programming. **7M**

Ans. See P.1-3, 4, Q.1, 2.

Q.1.(b) Explain different instruction formats of IBM 360 / 370 machine. **7M**

Ans. See P.,1-6 Q.7.

Q.2.(a) Explain the formats of following tables used in assembler design. **6M**

(i) MOT - Machine - OP table

(ii) POT - Pseudo - OP table

(iii) ST - Symbol table

Ans. See P.1-15, Q.24.

Q.2.(b) With neat schematic explain PASS - 1 of a two pass assembler. **8M**

Ans. See P.1-19, Q.27.

UNIT - II
(Total Marks : 26)

Q.3.(a) Explain the formats of MNT, MDT and ALA with example. What is the difference between ALA prepared in Pass - 1 and Pass - 2 of microprocessor? **7M**

Ans. See P.PS-27, Q.3(b).

Q.3.(b) Explain the stack organization for macro call within macros. What is the significance of storing old stack pointer? **6M**

Ans. See P.2-7, Q.7.

Q.4. Draw and explain the flowchart for macro defining macros. State the significance of READ flowchart. **13M**

Ans. See P.2-6, 7, Q.5(b), 7.

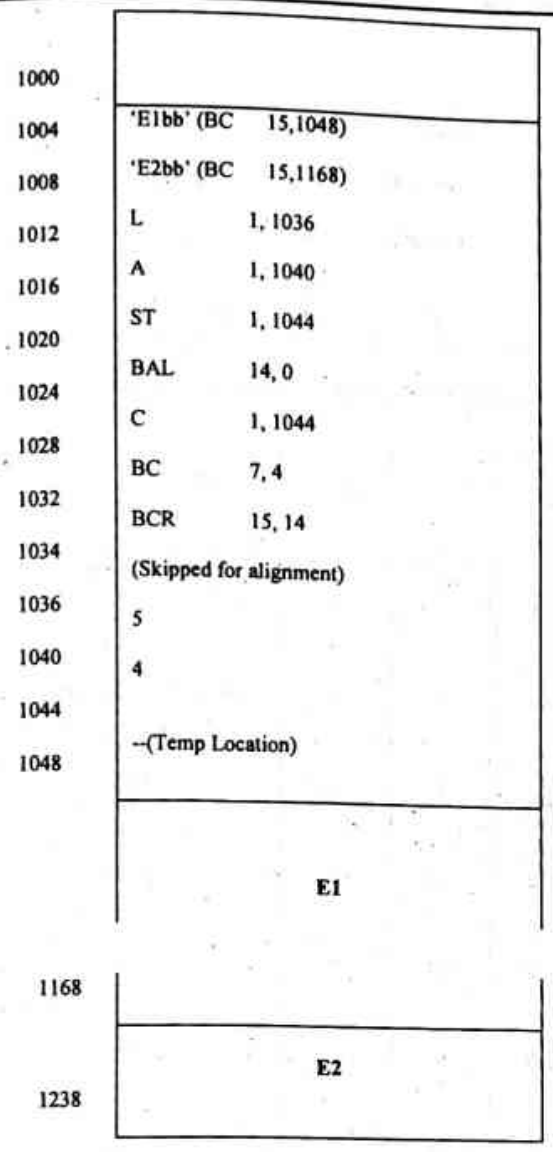
UNIT - III
(Total Marks : 26)

Q.5.(a) Assuming that the following program segment is stored at absolute address 1000. Show the assembled program PRG for a direct address machine and the complete memory map as loaded by BSS loader. Assume length of E1 = 120 bytes and E2 = 70 bytes. **8M**

```
PRG  START
     EXTRN   E1, E2
     L       1, FIVE
     A       1, FOUR
     ST      1, TEMP
     BAL     14, E1
     C       1, TEMP
     BNE     E2
     BR      14
FIVE DC      F'5'
FOUR DC      F'4'
TEMP DS      1 F
     END
```

Ans. Assembled program PRG for a direct address machine and the complete memory map as loaded by BSS loader.

Abs. Add.	Rel. Add.	Relocation	Object code
1000	0	00	'E1bb' (BC 15,1048)
1004	4	00	'E2bb' (BC 15,1168)
1008	8	01	L 1, 1036
1012	12	01	A 1, 1040
1016	16	01	ST 1, 1044
1020	20	01	BAL 14, 0
1024	24	01	C 1, 1044
1028	28	01	BC 7, 4
1032	32	01	BCR 15, 14
1034	34	0	(Skipped for alignment)
1036	36	00	5
1040	40	00	4
1044	44	00	--(Temp Location)



Q.5.(b) Describe in detail with neat diagram a compile and go loader. **5M**

Ans. See P.3-4 Q.3.

Q.6.(a) Explain the direct linking loader scheme. Also discuss about all the cards it uses. **9M**

Ans. See P.3-8, 9, Q.10, 12.

Q.6.(b) Write a note on dynamic loading and dynamic linking. **4M**

Ans. See P.3-11, Q.15.

UNIT - IV
(Total Marks : 26)

Q.7.(a) Explain in detail common object file format. **8M**

Ans. See P.4-2, Q.1.

Q.7.(b) What is symbolic debugger? Explain. **5M**

Ans. See P.4-11, Q.13.

Q.8.(a) Explain the structure of executable file. **5M**

Ans. See P.PS-38, Q.9(ii).

Q.8.(b) Explain different storage classes with example of each. **8M**

Ans. See P.4-7, Q.5.

UNIT - V
(Total Marks : 28)

Q.9.(a) What is device driver? Explain the significance of device driver. **4M**

Ans. See P.5-3, Q.1.

Q.9.(b) Explain different UNIX device drivers in detail. **10M**

Ans. See P.5-4, Q.3.

Q.10.(a) Explain all the entry points for line printer device driver. **8M**

Ans. See P.5-14, Q.15.

Q.10.(b) Give and explain the steps of installation of UNIX device driver. **6M**

Ans. See P.5-8, Q.9.

UNIT - VI
(Total Marks : 26)

Q.11. Explain various phases of compiler in detail with database involved in each phase. **13M**

Ans. See P.6-4, 5, Q.4, 5.
See P.PS-58 Q.12(b).

Q.12.(a) Explain how LEX tool helps in the design of lexical analyzer. **6M**

Ans. See P.6-12, Q.15.

Q.12.(b) What is cross compiler? Explain how bootstrapping can be achieved. **7M**

Ans. See P.6-9, Q.11.

SUMMER EXAMINATION - 2018
B.E. IV SEM. (Computer Science & Engg.) (CBS)
SYSTEM PROGRAMMING

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	28 M
Unit - III	28 M
Unit - IV	26 M
Unit - V	26 M
Unit - VI	26 M
Questions covered in VBD	160 M (100%)

SUMMER EXAM. - 2018
QUESTION PAPER

Q.1.(a) Illustrate the various instruction formats of IBM 360 with an example of each. **8M**

(b) What is the advantage of using a base register for formation of address? What is the limitation of using base register and how can it be resolved? **5M**

OR

Q.2.(a)(i) Which of the following is not true about BALR instruction? **5M**

(i) BALR used to load base register.

(ii) If second operated of BALR is O, the execution proceeds with the next instruction.

(iii) BALR is a RX types of instruction.

(a) Only i (b) Only ii

(c) Only iii (d) ii and iii

Justify your answer.

(ii) Definition of symbols and literals is done in pass 2 of assembler state true or false. Also justify your answers.

(iii) Which of the following is not a static table :

(a) Symbol table (b) Machine op table.

(c) Pseudo op table. (d) None of the above.

(b) For the following AL code, generate the symbol table, literal table, base table and machine code using the 2 pass assembler algorithm. **8M**

```
            START    0
Begin       BALR     15, 0
            USING    *, 15
            L        3, OLDOH
            A        3, RECPT
            S        3, ISSUE
            ST       3,NEWOH
OLDOH       DC       F '9'
RECP        DC       F'4'
ISSUE       DC       F'6'
NEWOH       DS       F
            END      BEGIN
```

Q.3.(a) What is the difference between a macro and a subroutine call? **3M**

(b) Explain the features of a macro facility. **8M**

(c) List the names of databases used by a microprocessor, alongwith their contents. **3M**

OR

Q.4. For the given AL, generate the expanded code (i.e. no macros) using an appropriate microprocessor algorithm. Also show the MDT, MNT and ALA after processing. **14M**

```
            MACRO
            RTMNU    &Z
            MACRO
            &Z       &N
            AR       4, & N
            ST       1, ALL
            MEND
            ST       &Z, ALL
            MEND
BEGIN       START    O
            USING    *, 15
            RTMNU    HELLO
            ST       2,3
            HELLOW   FOURTH SEM
FOURTH SEM  EQU      20
ALL         DC       F'5'
            END
```

When does the inner macro get defined in MDT? Does the macro processor algorithm work correctly if macro definition is done after macro call?

Q.5.(a) Draw and explain the flowchart for pass-1 of direct linking loader algorithm. **8M**

(b) Explain the concept of dynamic binding. What are the advantages of dynamic binding over other loader schemes? **6M**

OR

Q.6.(a) Consider the following program and identify each of the address constants below as absolute, simple relocatable or complex

relocatable. **7M**

```
PROGRAM   START   0
          ENTRY   MATH
          EXTRN   ABS
ONE       EQU     1
          ⋮
LOOP      ...
MATH      ...
          ⋮
BASE      ...
          ⋮
          END
```

Address constants :

(i) A(ABS)

(ii) A(BASE-LOOP+ONE)

(iii) A(MATH)

(iv) A(ABS-LOOP)

Also explain how the value of each address constant would be calculated and corrected?

(You need not show object deck).

(b) How is a Local External Symbol Array (LESA) created? Is it necessary for a loader to create a separate LESA for every segment (Program)? Justify your answer. What is the role of LESA in pass 2 of direct linking loader algorithm? **7M**

Q.7.(a) Explain the format of symbol table in an object file. **6M**

(b) What is the use of symbolic debugger system utility? Illustrate the use of GNU Debugger with a suitable example. (Use various commands of GNU debugger). **7M**

OR

Q.8.(a) Explain the 'Make' system utility. Illustrate with example various steps of creating and using make file and make command. **7M**

(b) Explain the source code control system. How is it used for version control? List the commands that are used in source code control system. **6M**

Q.9.(a) Explain in detail the process of driver installation on Unix system. **7M**

(b) Explain the anatomy of device drivers. **6M**

OR

Q.10.(a) Explain the different types of Unix Device drivers. **8M**

(b) Compare the device drivers for Unix and windows operating systems. **5M**

Q.11.(a) What is the significance of a 'Symbol Table' in compiler? Which phases of compiler refer/manipulate the symbol table? **5M**

(b) Explain the machine independent code optimization techniques of compiler. **8M**

OR

Q.12.(a) Write a LEX program for counting the number of words in a given input string. Also explain the process of executing a LEX program. **8M**

(b) Explain each of the following term with an example of each : **5M**

(i) Pattern.

(ii) Token.

(iii) Lexeme.

SUMMER EXAM. - 2018

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 26)

Q.1.(a) Illustrate the various instruction formats of IBM 360 with an example of each. **8M**

Ans. **See P.1-6, Q.7.**

Q.1.(b) What is the advantage of using a base register for formation of address? What is the limitation of using base register and how can it be resolved? **5M**

Ans. **See P.1-10, Q.10, 11, 12.**

Q.2.(a)(i) Which of the following is not true about BALR instruction? **5M**

(i) BALR used to load base register.

(ii) If second operated of BALR is O, the execution proceeds with the next instruction.

(iii) BALR is a RX types of instruction :

(a) Only i (b) Only ii

(c) Only iii (d) ii and iii

Justify your answer.

Ans. **(c) See P.1-11, Q.14(ii).**

Q.2.(a)(ii) Definition of symbols and literals is done in pass 2 of assembler state true or false. Also justify your answers.

Ans. **False, See P.1-16, Q.25.**

Q.2.(a)(iii) Which of the following is not a static table :

(a) Symbol table (b) Machine op table.

(c) Pseudo op table. (d) None of the above.

Ans. **(a) Symbol table, See P.1-15, Q.24.**

Q.2.(b) For the following AL code, generate the symbol table, literal table, base table and machine code using the 2 pass assembler algorithm. **8M**

```
            START   0
Begin       BALR    15, 0
            USING   *, 15
            L       3, OLDOH
            A       3, RECPT
            S       3, ISSUE
            ST      3,NEWOH
OLDOH       DC      F '9'
RECPT       DC      F'4'
ISSUE       DC      F'6'
NEWOH       DS      F
            END     BEGIN
```

Ans.

```
0           START   0
0   Begin   BALR    15, 0
2           USING   *, 15
2           L       3, OLDOH
6           A       3, RECPT
10          S       3, ISSUE
14          ST      3,NEWOH
18  OLDOH   DC      F '9'
22  RECPT   DC      F'4'
26  ISSUE   DC      F'6'
30  NEWOH   DS      F
34          END     BEGIN
```

Symbol table :

Symbol	Value	Length	Relocation
BEGIN	0	4	R
OLDOH	18	4	R
RECPT	22	4	R
ISSUE	26	4	R
NEWOH	30	4	R

Literal table : No Literal Table Created.

Bas Table : Only showing the entry of register used as base register.

Register Number	Availability Indicator	Contents
15	"Y"	00 00 00

Machine Code :

```
0     START   0
0     BALR    15, 0
2     USING   *, 15
2     L       3, 18(0, 15)
6     A       3, 22 (0, 15)
10    L       3, 25(0, 15)
14    L       3, 30(0, 15)
18    F'9'
22    F'4'
26    F'6'
30    ...
34    END
```

UNIT - II

(Total Marks : 28)

Q.3.(a) What is the difference between a macro and a subroutine call? **3M**

Ans. **Subroutines vs Macros :**

- Although subroutines and macros work similarly, they have subtle differences. Any logic may use both macros and subroutines. The main difference is in the way they are used. Macros may be used in

any expression field, but the macro may only contain an expression (i.e., Entries(LOC1), U(5, 1)). When a macros is used in a logic field, the macro may include any logic element valid in that logic field.

- Subroutines may also be used in an expression field provided that the RETURN statement is used to return a value to the expression field. When a subroutine is used to represent one or more logic statements, the subroutine may only include statements valid for the particular context.

Sr. No.	Macros	Subroutines
(1)	This is not case with macros.	Only subroutines can be used when you need to pass arguments, get a return value, or activate the independent execution to logic.
(2)	Only macros can be used when defining run-time interface parameters.	Subroutine cannot be used when defining runtime interface parameters.
(3)	This is not the case with macros.	When a subroutine is used to represent one or more logic statements, the subroutine may only include statements valid for the particular context.
(4)	No such RETURN statement is used to return a value to the expression field.	Subroutines may also be used in an expression field provided that the RETURN statement is used to return a value to the expression field.

Q.3.(b) Explain the features of a macro facility. **8M**

Ans. **See P.2-4, 5, 6, Q.2, 3, 4, 5(a), 5(b).**

Q.3.(c) List the names of databases used by a microprocessor, alongwith their contents. **3M**

Ans. **See P.2-13, Q.10.**

Q.4. For the given AL, generate the expanded code (i.e. no macros) using an appropriate microprocessor algorithm. Also show the MDT, MNT and ALA after processing. **14M**

```
        MACRO
        RTMNU   &Z
        MACRO
        &Z      &N
        AR      4, & N
        ST      1, ALL
        MEND
        ST      &Z, ALL
        MEND
BEGIN   START   O
        USING   *, 15
        RTMNU   HELLO
        ST      2.3
        HELLOW  FOURTH SEM
FOURTH  EQU     20
SEM
ALL     DC      F'5'
        END
```

When does the inner macro get defined in MDT? Does the macro processor algorithm work correctly if macro definition is done after macro call?

Ans. **MDT :**

INDEX	CARD
1	RTMNU &Z
2	MACRO
3	#1 &N
4	AR 4,&N
5	ST 1, ALL
6	MEND
7	ST #1, ALL
8	MEND
9	HELLO &N
10	AR 4, #1
11	ST 1, ALL
12	MEND

MNT :

INDEX	NAME	MDT INDEX
1	RTMNU	1
2	HELLO	9

ALA FOR MACRO RTMNU :

INDEX	ARGUMENT
0	"bbbbbbbb"
1	"HELLObbb"

ALA FOR MACRO RTMNU :

INDEX	ARGUMENT
0	"bbbbbbbb"
1	"FORTHSE"

EXPANDED CODE :

```
BEGIN       START
            USING     *, 15
            AR        4, FORTH SEM
            ST        1, ALL
            ST        HELLO, ALL
            AR        4, FORTH SEM
            ST        1, ALL
FORTH SEM   EQU       20
ALL         DC        F'5'
            END
```

The inner macro is defined only after the outer macro has been called. This is the problem of the single pass macro which provide macro definition within macros.

No, the microprocessor algorithm does not work correctly if macro definition is done after macro call. Macro definitions must be processed before calls because macros must be defined to the processor before it can expand calls to them.

UNIT - III

(Total Marks : 28)

Q.5.(a) Draw and explain the flowchart for pass-1 of direct linking loader algorithm. **8M**

Ans. See 3-8, Q.11.

Q.5.(b) Explain the concept of dynamic binding. What are the advantages of dynamic binding over other loader schemes? **6M**

Ans. See 3-13, 11, Q.18, 15.

Q.6.(a) Consider the following program and identify each of the address constants below as absolute, simple relocatable or complex relocatable. **7M**

```
PROGRAM     START     0
            ENTRY     MATH
            EXTRN     ABS
ONE         EQU       1
            ⋮
LOOP        ...
MATH        ...
            ⋮
BASE        ...
            ⋮
            END
```

Address constants :

(i) A(ABS)

(ii) A(BASE-LOOP+ONE)

(iii) A(MATH)

(iv) A(ABS-LOOP)

Also explain how the value of each address constant would be calculated and corrected?

(You need not show object deck).

Ans.

(i) **A(ABS) :** It is a Complex Relocatable.

If a symbol is a entry to some other program then it is Complex Relocatable. Symbol ABS is a external symbol i.e. ABS is define in some other program.

The assembler places a minus relative value of ABS for the address constant. The loader then subtract from this constant the address of the beginning of the program, thereby setting the address constant to – (absolute value of ABS).

(ii) **A(BASE-LOOP+ONE) :** It is a Simple Relocatable.

If one symbol is a relocatable symbol within the procedure and other symbol is an absolute number (e.g. ONE EQU 1) is called Simple Relocatable.

The assembler can calculate the difference between relative location of the symbols, but the loader must perform the relocation by adding the program load address.

The relative address of BASE, LOOP and ONE (found in symbol table) is determined by the assembler, to this the loader adds the address of the beginning of the program.

(iii) **A(MATH) :** It is a absolute type of address constant.

If the relocatable symbols define internals to program - the assembler calculate the actual value. Symbol MATH is a relocatable symbol define internal to the program and hence the assembler calculate the actual value.

(iv) **A(ABS-LOOP) :** It is a Complex Relocatable.

If a symbol is a entry to some other program then it is Complex Relocatable. Symbol ABS is a external symbol i.e. ABS is define in some other program. Whereas the symbol LOOP is a relocatable symbol define internal to the procedure. Though the value of symbol LOOP known to the assembler but the value of ABS is only calculated by the loader.

The assembler places a minus relative value of ABS for the address constant. The loader then subtracts from this constant the value of LOOP supplied by the assembler.

Q.6.(b) How is a Local External Symbol Array (LESA) created? Is it necessary for a loader to create a separate LESA for every segment (Program)? Justify your answer. What is the role of LESA in pass 2 of direct linking loader algorithm? **7M**

Ans. **See P.3-14, Q.20.**

UNIT - IV

(Total Marks : 26)

Q.7.(a) Explain the format of symbol table in an object file. **6M**

Ans. **See P.4-5, 6, Q.3, 4.**

Q7.(b) What is the use of symbolic debugger system utility? Illustrate the use of GNU debugger with a suitable example. (Use various commands of GNU debugger). **7M**

Ans. **See P.4-11, 12, Q.13, 14.**

Q.8.(a) Explain the 'Make' system utility. Illustrate with example various steps of creating and using make file and make command. **7M**

Ans. **See P.4-11, Q.11.**

Q.8.(b) Explain the source code control system. How is it used for version control? List the commands that are used in source code control system. **6M**

Ans. **See P.4-10, Q.10.**

UNIT - V

(Total Marks : 26)

Q.9.(a) Explain in detail the process of driver installation on Unix system. **7M**

Ans. **See P.5-8, Q.9.**

Q.9.(b) Explain the anatomy of device drivers. **6M**

Ans. **See P.5-3, Q.1.**

Q.10.(a) Explain the different types of Unix Device drivers. **8M**

Ans. **See P.5-4, Q.3.**

Q.10.(b) Compare the device drivers for Unix and windows operating systems. **5M**

Ans. **See P.5-16, Q.16.**

UNIT - VI

(Total Marks : 26)

Q.11.(a) What is the significance of a 'Symbol Table' in compiler? Which phases of compiler refer/manipulate the symbol table? **5M**

Ans. **See P.6-6, Q.6.**

Q.11.(b) Explain the machine independent code optimization techniques of compiler. **8M**

Ans. **Machine independent code optimization techniques :**

In this optimization, the compiler takes in the intermediate code and transforms a part of the code that does not involve any CPU register and / or absolute memory locations.

- The code optimization in the synthesis phase is a program transformation technique, which tries to improve the intermediate code by making it consume fewer resources (i.e. CPU, Memory) so that faster-running machine code will result.
- Compiler optimizing process should meet the following objectives :

(i) The optimization must be correct, it must not, in any way, change the meaning of the program.

(ii) Optimization should increase the speed and performance of the program.

(iii) The compilation time must be kept reasonable.

(iv) The optimization process should not delay the overall compiling process.

Optimization of the code is often performed at the end of the development stage since it reduces readability and adds code that is used to increase the performance.

Types of code optimization : The optimization process can be broadly classified into two types :

(1) **Machine independent optimization :**

This code optimization phase attempts to improve the intermediate code to get a better target code as the output. The part of the intermediate code which is transformed here does not involve any CPU registers or absolute memory locations.

Machine independent optimization techniques includes :

(i) Peephole optimization

(ii) Local optimizations

(iii) Global optimizations

(a) Inter-procedural

(b) Intra-procedural

(iv) Loop optimization.

(2) **Machine dependent optimization :**

Machine dependent optimization is done after the target code has been generated and when the code is transformed according to the target machine architecture. It involves CPU registers and may have absolute memory references rather than relative references. Machine - dependent optimizers put efforts to take maximum advantage of the memory hierarchy.

Q.12.(a) Write a LEX program for counting the number of words in a given input string. Also explain the process of executing a LEX program

Ans. **See P.PS-20, Q.9(b).**

Q.12.(b) Explain each of the following term with an example of each : **5M**

(i) Pattern.

(ii) Token.

(iii) Lexeme.

Ans. **See P.6-3, Q.3.**

WINTER EXAMINATION - 2018

B.E. IV SEM. (Computer Science & Engg.) (CBS)

SYSTEM PROGRAMMING

Statistical Analysis

Unit Number	Marks
Unit - I	28 M
Unit - II	26 M
Unit - III	28 M
Unit - IV	26 M
Unit - V	26 M
Unit - VI	26 M
Questions covered in VBD	160 M (100%)

WINTER EXAM. - 2018
QUESTION PAPER

Q.1.(a) What is system programming? Explain the components of system programming. **6M**

(b) For the following ALP show the entries in ST, LT, BT and MDT. Also write the machine language program at the end of pass 2. **8M**

```
PROGRAM          START 0
                 USING * 15
                 LA 15, SETUP
                 SR TOTAL, TOTAL
AC               EQU 2
INDEX            EQU 3
TOTAL            EQU 4
DATABASE         EQU 13
SETUP            EQU *
                 USING SETUP, 15
                 L DATABASE, = A (DATA1)
USING DATAAREA, DATABASE
                 SR INDEX, INDEX
LOOP             L AC, DATA 1 (INDEX)
                 AR TOTAL, AC
                 A AC, = F '5'
                 ST AC, SAVE (INDEX)
                 A INDEX, = F '4'
                 C INDEX, = F '8000'
                 BNE LOOP
                 LR 1, TOTAL
                 BR 14
                 LTORG
SAVE             DS 2000F
DATA AREA        EQU*
DATA 1           DC F '25, 26, 97, 101 ----,
                 [2000 numbers]
                 END
```

OR

Q.2.(a) Draw and explain the flowchart and algorithm of pass-2 of assembler. **8M**

(b) Explain IBM 360/370 machine architecture in detail. **6M**

Q.3.(a) For the following CULP, Prepare MNT, MDT, ALA databases and also write expanded source code. **7M**

```
MACRO
& lab1 INCR & arg1, & arg2, & arg3
& lab1 A 1, & arg 1
       A 2, & arg 2
       A 3, & arg 3
       MEND
       :
       :
LOOP 1 INCR data1, data2, data3
       :
       :
LOOP 2 INCR data3, data2, data1
       :
       :
data 1 DC F '5'
data 2 DC F '6'
data 3 DC F '7'
```

(b) What are the basic task that any macroprocessor must perform? How do you define a macro in a program? Give a general format. **6M**

OR

Q.4.(a) Explain with the flowchart of simple one pass macro-processor. **7M**

(b) Explain the concept of conditional macro expansion with example. **6M**

Q.5.(a) Draw a detail flowchart for pass 1 of direct linking loader of IBN 360 machine. **6M**

(b) Show the entries in ESD, TXT and RLD cards for the following prog. **8M**

```
JOHN     START
         ENTRY SUM, DATA
         EXTRN LOOP, POINTER
         BALR 15, 0
         USING *, 15
         SR 4, 14
         L 1, FOUR
```

```
            A 2, FOUR
            ST 2, FOUR
            BR 14,
FOUR        DC F '4'
LOOP        DC A (SUM + 4)
POINTER     DC A (LOOP – DATA)
            DC A (POINTER – LOOP)
            DC A (POINTER)
            END
```

Q.6.(a) Write the difference between loader and linker. **7M**

(b) Explain in detail the general loading scheme with advantages and disadvantages. **7M**

Q.7.(a) Explain in detail the format of common object file. **7M**

(b) Explain with suitable diagram the concept of link editor. **6M**

OR

Q.8.(a) Write short note on **any three.** **13M**

(1) SCCS.

(2) Symbolic debugger.

(3) GNU debugger.

(4) String table.

Q.9.(a) What is device drivers? Explain the necessity of the device driver. **7M**

(b) Explain in details the steps for the installation of device driver in UNIX. **6M**

OR

Q.10.(a) Explain in detail with neat diagram the types of device driver in UNIX. **7M**

(b) Differentiate between device drivers for UNIX and WINDOWS. **6M**

Q.11.(a) What is lexical analyzer? How the tokens are recognized by lexical phases? Explain. **7M**

(b) Explain any one of the following with suitable program : **6M**

(i) LEX

(ii) YAAC

OR

Q.12.(a) Explain various phases of compiler in detail. Draw the block diagram and explain the functionality of each phase with suitable example. **13M**

WINTER EXAM. - 2018

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 28)

Q.1.(a) What is system programming? Explain the components of system programming. **6M**

Ans. **See P.1-3, Q.1.**

Q.1.(b) For the following ALP show the entries in ST, LT, BT and MDT. Also write the machine language program at the end of pass 2. **8M**

```
PROGRAM     START 0
            USING * 15
            LA 15, SETUP
            SR TOTAL, TOTAL
AC          EQU 2
INDEX       EQU 3
TOTAL       EQU 4
DATABASE    EQU 13
SETUP       EQU *
            USING SETUP, 15
            L DATABASE, = A (DATA1)
            USING DATAAREA, DATABASE
            SR INDEX, INDEX
LOOP        L AC, DATA 1 (INDEX)
            AR TOTAL, AC
            A AC, = F '5'
            ST AC, SAVE (INDEX)
            A INDEX, = F '4'
            C INDEX, = F '8000'
            BNE LOOP
            LR 1, TOTAL
            BR 14
            LTORG
SAVE        DS 2000F
DATA AREA   EQU*
DATA 1      DC F '25, 26, 97, 101 —,
```

[2000 numbers]

END

Ans. **Symbol table :**

Symbol	Value	Length	Relocation
PROGRAM	0	1	R
AC	2	1	A
INDEX	3	1	A
TOTAL	4	1	A
DATABASE	13	1	A
SETUP	6	1	R
LOOP	12	4	R
SAVE	64	4	R
DATAAREA	8064	1	R
DATA1	8064	4	R

Literal table :

Literal	Value	Length	Relocation
A(DATA1)	48	4	R
F'5'	52	4	R
F'4'	56	4	R
F'8000'	60	4	R

Base table : (Showing only base register in use)

REGISTER NO.	BASE ADDRESS
After statement 2 :	
15	0
After statement 10 :	
15	6
After statement 12 :	
13	8064
15	6

MDT : No MACRO is defined in this program. So no MDT will be generated.

Generated machine code :

0	LA	15, 6 (0, 15)
4	SR	4, 4
6	L	13, 42 (0, 15)
10	SR	4, 4
12	L	2, 0 (3, 13)
16	AR	4, 2
18	A	1, 46 (0, 15)
22	ST	2, 58 (3, 15)
26	A	3, 50 (0, 15)
30	C	3, 54 (0, 15)
34	BC	7, 6 (0, 15)
38	LR	1, 4
40	BCR	15, 14
48	8064	
52	X'0000 0005	
56	X'0000 0004	
60	8000	

Q.2.(a) Draw and explain the flowchart and algorithm of pass-2 of assembler. **8M**

Ans. **See P.1-19, 21, Q.27, 28.**

Q.2.(b) Explain IBM 360/370 machine architecture in detail. **6M**

Ans. **See P.1-6, Q.6.**

UNIT - II

(Total Marks : 26)

Q.3.(a) **For the following CULP, Prepare MNT, MDT, ALA databases and also write expanded source code.** **7M**

```
MACRO
& lab1 INCR & arg1, & arg2, & arg3
& lab1   A 1, & arg 1
         A 2, & arg 2
         A 3, & arg 3
         MEND
         :
         :
LOOP 1 INCR data1, data2, data3
         :
         :
LOOP 2 INCR data3, data2, data1
         :
```

:

data 1 DC F '5'

data 2 DC F '6'

data 3 DC F '7'

Ans. MNT :

Index	Name	MDT index
1	"INCRbbbb"	1

MDT :

Index	Card
1	$LAB INCR &arg1, &arg2, &arg3
2	A 1, &arg1
3	A 2, &arg2
4	A 3, &arg3
5	MEND

ALA :

For : LOOP1 INCR data1, data2, data3

Index	Argument
0	"LOOP1bbb"
1	"data1bbb"
2	"data2bbb"
3	"data3bbb"

For – LOO2 INCR data3, data2, data1

Index	Argument
0	"LOOP2bbb"
1	"data3bbb"
2	"data2bbb"
3	"data1bbb"

Expanded source code :

Source code :

Source Cdoe			Expanded Source Code		
	MACRO				
&lab1	INCR	&arg1, &arg2, &arg3			
&lab1	A	1, &arg1			
	A	2, &arg2			
	A	3, &arg3			
	MEND				
	⋮			⋮	
LOOP1	INCR	data1, data2, data3	LOOP1	A	1,data1,
				A	2,data2
				A	3,data3
	⋮			⋮	
LOOP2	INCR	data3, data2, data1	LOOP2	A	1,data3,
				A	2,data2
				A	3,data3
	⋮			⋮	
data1	DC	F'5'	data1	DC	F'5'
data2	DC	F'6'	data2	DC	F'5'
data2	DC	F'7'	data2	DC	F'7'

Q.3.(b) What are the basic task that any macroprocessor must perform? How do you define a macro in a program? Give a general format. **6M**

Ans. See P.2-3, 4, Q.1, 1(b).

Q.4.(a) Explain with the flowchart of simple one pass macro-processor. **7M**

Ans. See P.2-7, Q.7.

Q.4.(b) Explain the concept of conditional macro expansion with example. **6M**

Ans. See P.2-5, Q.4.

UNIT - III
(Total Marks : 28)

Q.5.(a) Draw a detail flowchart for pass 1 of direct linking loader of IBN 360 machine. **6M**

Ans. See P.3-8, Q.11.

Q.5.(b) Show the entries in ESD, TXT and RLD cards for the following prog. **8M**

```
JOHN      START
          ENTRY SUM, DATA
          EXTRN LOOP, POINTER
          BALR 15, 0
          USING *, 15
          SR 4, 14
          L 1, FOUR
          A 2, FOUR
          ST 2, FOUR
          BR 14,

FOUR      DC F '4'
LOOP      DC A (SUM + 4)
POINTER   DC A (LOOP – DATA)
          DC A (POINTER – LOOP)
          DC A (POINTER)
          END
```

Ans. See P.3-25, Q.30.

Q.6.(a) Write the difference between loader and linker. **7M**

Ans.

Sr. No.	Loader	Linker
(1)	The loader is the system software that loads the executable file generated by the linker to the main memory.	Linker is the software that links the object code with additional files such as header files and creates an executable file with exe extension.
(2)	The loader loads the executable file.	The linker links the object code and the header files.
(3)	It loads the executable module to the main memory.	It generates the executable module of a source program by linking the object code and the header files.
(4)	It takes input as an executable module generated by a linker.	It takes as input, the object code generated by an assembler or by language translator.
(5)	It allocates the addresses to an executable module in main memory for execution.	It combines all the object modules of a source code to generate an executable module.
(6)	e.g. Absolute loading, Relocatable loading and Dynamic Run-time loading.	e.g. Linkage Editor, Dynamic linker.

Q.6.(b) Explain in detail the general loading scheme with advantages and disadvantages. **7M**

Ans. See P.3-5, Q.5.

UNIT - IV
(Total Marks : 26)

Q.7.(a) Explain in detail the format of common object file. **7M**

Ans. See P.4-2, Q.1.

Q.7.(b) Explain with suitable diagram the concept of link editor. **6M**

Ans. **See P.4-11, Q.12.**

Q.8.(a) Write short note on **any three.** **13M**

(1) SCCS.

(2) Symbolic debugger.

(3) GNU debugger.

(4) String table.

Ans. **See P.4-10, 11, 12, 5, Q.10, 13, 14, 3.**

UNIT - V
(Total Marks : 26)

Q.9.(a) What is device drivers? Explain the necessity of the device driver. **7M**

Ans. **See P.5-3, Q.1.**

Q.9.(b) Explain in details the steps for the installation of device driver in UNIX. **6M**

Ans. **See P.5-8, Q.9.**

Q.10.(a) Explain in detail with neat diagram the types of device driver in UNIX. **7M**

Ans. **See P.5-4, Q.3.**

Q.10.(b) Differentiate between device drivers for UNIX and WINDOWS. **6M**

Ans. **See P.5-16, Q.16.**

UNIT - VI
(Total Marks : 26)

Q.11.(a) What is lexical analyzer? How the tokens are recognized by lexical phases? Explain. **7M**

Ans. **See P.6-7, Q.8, 9.**

Q.11.(b) Explain any one of the following with suitable program : **6M**

(i) LEX

(ii) YAAC

Ans. **(i) See P.6-12, Q.15.**

(ii) See P.PS-45, Q.12(b)(ii).

Q.12.(a) Explain various phases of compiler in detail. Draw the block diagram and explain the functionality of each phase with suitable example. **13M**

Ans. **See P.6-4, Q.4.**

Notes :-

SUMMER EXAMINATION - 2019
B.E. IV SEM. (Computer Science & Engg.) (CBS)
SYSTEM PROGRAMMING

Statistical Analysis

Unit Number	Marks
Unit - I	28 M
Unit - II	26 M
Unit - III	26 M
Unit - IV	26 M
Unit - V	26 M
Unit - VI	28 M
Questions covered in VBD	160 M (100%)

SUMMER EXAM. - 2019
QUESTION PAPER

Q.1.(a) Explain pass 1 of assembler in detail with the help of flowchart and databases. **8M**

(b) Explain following terms with help of examples.

(i) USING (ii) BALR

(iii) DS (iv) DC

(v) LTORG (vi) DROP **6M**

OR

Q.2. For the given AL code get base table literal table, symbol table, machine opcode **14M**

Table along with machine code

```
PROG    START   0
        BALR    15, 0
        USING   *, 15
        LR      5, 15
        LH      1, Data 1
        USING   *, 10
        BR      ,14
Data 2  DC      F '11'
Data 1  DC      H '22'
PQR     DC      H '23'
BCK     DS      F
MN      EQU     1
XY      EQU     2
        BALR    2, 0
        USING   * + MN, PQR
        LA      7, = A (BCK)
        BR      6
        DC      H '64'
        DROP    XY
        L       9, = A (Data 1)
        A       9, PQR
        LTORG
        ST      9, = F'100'
        END
```

Q.3.(a) List the names of databases used in pass 1 and pass 2 of microprocessor. **5M**

(b) Explain conditional macro expansion and macro instruction defining macro facilities brief your answer when, how and which MACRO facilities are implemented for a particular AL code. **8M**

OR

Q.4. Get MDT, MNT, ALA and expanded code for the following code fragment - **12M**

```
        MACRO
        XYZ & A
        MACRO
        & A     & B
        MACRO
        & B     & C
        ST      15, A (& C)
        BALR    14, 15
        L       1, = F '3'
        A       1, & C
        ST      1, & C
        MEND
        MEND
        MEND
PROG    START
        BALR
        ⋮
        XYZ     MIT
        MIT     HELLO
        HELLO   DI
DI      DC      F'10'
        END
```

Q.5.(a) How relocating loader performs the function of loader? Explain in detail. **5M**

(b) Why 2 passes are required by direct linking loader? Draw and explain flowchart of pass 2 of loader. **8M**

OR

Q.6. Show the entries in ESD, TXT and RLD card for following program. **13M**

```
JOHN    START
        ENTRN   PGA, PGB
        EXTRY   PGC, PGD
```

```
        BALR    15, 0
        USING   *, 15
        SR      4, 14
        L       1, FIVE
        A       2, FIVE
        ST      2, FIVE
        BR      14
FIVE    DC      F '4'
PGC     DC      A (PGA + 4)
PGD     DC      A (PGC-PGB)
        DC      A (PGD-PGC)
        DC      A (PGD)
        END
```

Q.7.(a) Explain the COFF symbol table and string table. **8M**

(b) Write a short note on link editor. **5M**

OR

Q.8.(a) Explain Source Code Control System (SCCS). **7M**

(b) Explain the GNU debugger. Enlist the findings concred by GNU debugger. **6M**

Q.9.(a) Explain the different types of unix device drivers. **8M**

(b) Explain gross anatomy of device drivers. **5M**

OR

Q.10.(a) Explain the process of driver installation on unix system in detail. **8M**

(b) Compare the device drivers for unix and windows operating system. **5M**

Q.11.(a) Explain database and algorithm for lexical phase of compiler. **7M**

(b) Explain YACC in detail. **7M**

OR

Q.12.(a) What is cross compiler? How does it use bootstrapping concept? **7M**

(b) Describe a tool for study of Lex. **7M**

SUMMER EXAM. - 2019
SOLUTION
(Unit-Wise)

UNIT - I
(Total Marks : 28)

Q.1.(a) Explain pass 1 of assembler in detail with the help of flowchart and databases. **8M**

Ans. **See P.1-19, Q.27.**

Q.1.(b) Explain following terms with help of examples.

(i) USING (ii) BALR

(iii) DS (iv) DC

(v) LTORG (vi) DROP **6M**

Ans. **See P.1-11, Q.14.**

Q.2. **For the given AL code get base table literal table, symbol table, machine opcode** **14M**

Table along with machine code

```
PROG    START   0
        BALR    15, 0
        USING   *, 15
        LR      5, 15
        LH      1, Data 1
        USING   *, 10
        BR      14
Data 2  DC      F '11'
Data 1  DC      H '22'
PQR     DC      H '23'
BCK     DS      F
MN      EQU     1
XY      EQU     2
        BALR    2, 0
        USING   * + MN, PQR
        LA      7, = A (BCK)
        BR      6
        DC      H '64'
        DROP    XY
        L       9, = A (Data 1)
        A       9, PQR
        LTORG
        ST      9, = F'100'
        END
```

Ans. **Symbol table :**

Symbol	Value	Length	Relocation
PROG	0	1	R
Data 2	10	4	R

Data 1	14	2	R
PQR	16	2	R
BCK	18	4	R
MN	1	1	A
XY	2	1	A

Literal table :

Literal	Value	Length	Relocation
=A(BCK)	40	4	R
=A(DATA1)	44	2	R
=F'100'	46	4	R

Base table :

After Statement No.	Register	Availability	Contents
3	15	Y	2
6	10	Y	8
15	15	Y	23
19	2	N	--

Object code :

Statement No.	Rel. Loc.	Code
2	0	BALR 15,0
4	2	LR 5, 15
5	4	LH 1, 12(0,15)
7	8	BCR 15,14
8	10	11
9	14	22
10	16	23
11	18	--
14	22	BALR 2, 0
16	24	LA 7, 34(0,10)
17	28	BCR 15,2
18	30	64
20	32	L 9, 21(0,15)
21	36	A 9, 8(0,10)
22	40	18
	44	14
	46	100
23	50	ST 9, 38(0,10)

UNIT - II

(Total Marks : 26)

Q.3.(a) List the names of databases used in pass 1 and pass 2 of microprocessor. **5M**

Ans. See P.2-13, Q.10.

Q.3.(b) Explain conditional macro expansion and macro instruction defining macro facilities brief your answer when, how and which MACRO facilities are implemented for a particular AL code. **8M**

Ans. See P.2-5, 6, Q.4, 5(b).

Q.4. Get MDT, MNT, ALA and expanded code for the following code fragment - **13M**

```
        MACRO
        XYZ & A
        MACRO
        & A         & B
        MACRO
        & B         & C
        ST          15, A (& C)
        BALR        14, 15
        L           1, = F '3'
        A           1, & C
        ST          1, & C
        MEND
        MEND
        MEND
PROG    START
        BALR
        :
        XYZ         MIT
        MIT         HELLO
        HELLO       DI
DI      DC          F'10'
        END
```

Ans. **MDT :**

Index	Card
1	XYZ &A
2	MACRO
3	#1 &B
4	MACRO
5	#1 &C
6	ST 15, A(#2)
7	BALR 14, 15
8	L 1, =F'3'
9	A 1, #2
10	ST 1, #2
11	MEND
12	MEND
13	MEND

MNT :

Index	Name	MDT Index
1	XYZ	1
2	MIT	4
3	HELLO	12

ALA :

Index	Argument
0	"bbbbbbbb"
1	"MITbbbbb"
2	"HELLObbb"
3	"DIbbbbbb"

Expanded code :

```
PROG  START
          BALR
          :
          :
          ST        15, A(DI)
          BALR      14, 15
          L         1,=F'3'
          A         1, DI
          ST        1, DI
DI        DC        F'10'
          END
```

UNIT - III

(Total Marks : 26)

Q.5.(a) How relocating loader performs the function of loader? Explain in detail **5M**

Ans. **See P.3-7, Q.7.**

Q.5.(b) Why 2 passes are required by direct linking loader? Draw and explain flowchart of pass 2 of loader. **8M**

Ans. **See P.3-9, Q.12.**

Q.6. **Show the entries in ESD, TXT and RLD card for following program.** **13M**

```
JOHN    START
        ENTRN     PGA, PGB
        EXTRY     PGC, PGD
        BALR      15, 0
        USING     *, 15
        SR        4, 14
        L         1, FIVE
        A         2, FIVE
        ST        2, FIVE
        BR        14
FIVE    DC        F '4'
PGC     DC        A (PGA + 4)
PGD     DC        A (PGC-PGB)
        DC        A (PGD-PGC)
        DC        A (PGD)
        END
```

Ans. **ESD card :**

Name	Type	ID	Relative Location	Total Length
JOHN	SD	01	0	38
PGA	ER	02	-	

PGB	ER	03	-	-
PGC	LD	-	22	
PGD	LD	-	26	

TXT card :

Source Card	Reference	Relative Address	Contents	Comments
4	BALR 15, 0	0	BALR 15, 0	
6	SR 4, 14	2	SR 4, 14	
7	L 1, FIVE	4	L 1, 16(0,15)	
8	A 2, FIVE	8	A 2, 16(0,15)	
9	ST 2, FIVE	12	ST 2, 16(0.15)	
10	BR 14	16	BCR 15, 14	
11	FIVE DC F'4'	18	F'4'	
12	PGC DC A(PGA + 4)	22	A(PGA + 4)	PGA - External Symbol
13	PGD DC A(PGC-PGB)	26	A(22 + PGB)	PGB - External Symbol
14	DC A(PGD-PGC)	30	4	
15	DC A(PGD)	34	26	
16	END	38	END	

RLD card :

Source Card	ESD ID	Length	Flag	Relative Address
12	02	4	+	22
13	03	4	+	26
14	-	4	+	30
15	-	4	+	34

UNIT - IV

(Total Marks : 26)

Q.7.(a) Explain the COFF symbol table and string table. **8M**

Ans. **See P.4-5, Q.3.**

Q.7.(b) Write a short note on link editor. **5M**

Ans. **See P.4-11, Q.12.**

Q.8.(a) Explain source code control system (SCCS). **7M**

Ans. **See P.4-10, Q.10.**

Q.8.(b) Explain the GNU debugger. Enlist the findings concred by GNU debugger. **6M**

Ans. **See P.4-12, Q.14.**

UNIT - V

(Total Marks : 26)

Q.9.(a) Explain the different types of unix device drivers. **8M**

Ans. **See P.5-4, Q.3.**

Q.9.(b) Explain gross anatomy of device drivers. **5M**

Ans. **See P.5-4, 7, Q.3, 7.**

Q.10.(a) Explain the process of driver installation on unix system in detail. **8M**

Ans. **See P.5-8, Q.9.**

Q.10.(b) Compare the device drivers for unix and windows operating system. **5M**

Ans. **See P.5-16, Q.16.**

UNIT - VI

(Total Marks : 28)

Q.11.(a) Explain database and algorithm for lexical phase of compiler. **7M**

Ans. **See P.6-5, Q.5.**

Q.11.(b) Explain YACC in detail. **7M**

Ans. **See P.6-10, Q.12.**

Q.12.(a) What is cross compiler? How does it use bootstrapping concept? **7M**

Ans. **See P.6-9, Q.11.**

Q.12.(b) Describe a tool for study of Lex. **7M**

Ans. **See P.6-11, Q.14.**

WINTER EXAMINATION - 2019

B.E. IV SEM. (Computer Science & Engg.) (CBS)

SYSTEM PROGRAMMING

Statistical Analysis

Unit Number	Marks
Unit - I	28 M
Unit - II	26 M
Unit - III	26 M
Unit - IV	26 M
Unit - V	26 M
Unit - VI	28 M
Questions covered in VBD	160 M (100%)

WINTER EXAM. - 2019
QUESTION PAPER

Q.1.(a) Explain Pass 1 of Assembler in detail with the help of flowchart and databases. **7M**

(b) Explain different instruction formats of IBM 360/370 machine. **7M**

OR

Q.2.(a) Differentiate between: **6M**

(i) Open subroutine and Closed subroutine.

(ii) Pure procedure and Impure procedure

(b) For the following AL Code, generate the symbol table, literal table, base table and machine code using the 2 pass assembler algorithm. **8M**

```
        START   0
BEGIN   BALR    15,0
        USING   *,15
        L       3,OLDOH
        A       3,RECPT
        S       3, ISSUE
        ST      3,NEWOH
OLDOH   DC      F'9'
RECPT   DC      F'4'
ISSUE   DC      F'6'
NEWOH   DS      F
        END     BEGIN
```

Q.3. Explain in detail the two pass macro processor with the help of neat and clean flowchart. What macro facilities are restricted in this design? **13M**

OR

Q.4.(a) Explain the stack organization for macro call within macros. What is the significance of storing old stack pointer? **6M**

(b) Explain the formats of MNT, MDT and ALA with example. What is the difference between ALA prepared in Pass-1 and Pass-2 of macroprocessor? **7M**

Q.5.(a) Explain in detail about Dynamic linking and dynamic loading with example. **6M**

(b) Explain the direct linking loader scheme. Also discuss about all the cards it uses. **7M**

OR

Q.6.(a) Assuming that the following program segment is stored at absolute address 1000. Show the assembled program PRG for a direct address machine and the complete memory map as loaded by BSS loader. Assume length of E1=120 bytes and E2 = 70 bytes. **7M**

```
PRG     START
        EXTRN   E1,E2
        L       1,FIVE
        A       1,FOUR
        ST      1,TEMP
        BAL     14,E1
        C 1,    TEMP
        BNE     E2
        BR      14
FIVE    DC      F'5'
FOUR    DC      F'4'
TEMP    DS      1F
        END
```

(b) How is a Local External Symbol Array (LESA) created? Is it necessary for a loader to create a separate LESA for every segment (program)? Justify your answer. What is the role of LESA in pass 2 of Direct Linking loader algorithm? **6M**

Q.7.(a) Explain in detail the format of common object file in detail. **8M**

(b) What is symbolic debugger? Explain. **5M**

OR

Q.8.(a) Describe source code control system with example. **6M**

(b) Explain different storage classes with example of each. **7M**

Q.9. Describe in detail the anatomy and types of device driver in UNIX system. **13M**

OR

Q.10.(a) What are the major design issues in the study of device driver? Elaborate. **7M**

(b) Compare the device drivers for UNIX and WINDOWS operating systems. **6M**

Q.11. What is compiler? Draw a block diagram of the phases of compiler and indicate the main function of each phase. **14M**

OR

Q.12.(a) How tokens are recognized by lexical phase? Explain in detail. **7M**

(b) Describe LEX as a tool, for system programming. **7M**

WINTER EXAM. - 2019

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 28)

Q.1.(a) Explain Pass 1 of Assembler in detail with the help of flowchart and databases. **7M**

Ans. See P.1-19, Q.27.

Q.1.(b) Explain different instruction formats of IBM 360/370 machine. **7M**

Ans. See P.1-6, Q.7.

Q.2.(a) Differentiate between : **6M**

(i) Open subroutine and closed subroutine.

(ii) Pure procedure and Impure procedure.

Ans. See P.1-14, Q.21.

Q.2.(b) For the following AL Code, generate the symbol table, Literal table, base table and machine code using the 2 pass assembler algorithm. **8M**

```
          START   0
BEGIN     BALR    15.0
          USING   *,15
          L       3,OLDOH
          A       3,RECPT
          S       3, ISSUE
          ST      3,NEWOH
OLDOH     DC      F'9'
RECPT     DC      F'4'
ISSUE     DC      F'6'
NEWOH     DS      F
          END     BEGIN
```

Ans. See P.PS-86, Q.2(b).

UNIT - II

(Total Marks : 26)

Q.3. Explain in detail the two pass macro processor with the help of neat and clean flowchart. What macro facilities are restricted in this design? **13M**

Ans. See P.2-10, Q.8(a).

Q.4.(a) Explain the stack organization for macro call within macros. What is the significance of storing old stack pointer? **6M**

Ans. See P.2-6, 7, Q.5(a), 7.

Q.4.(b) Explain the formats of MNT, MDT and ALA with example. What is the difference between ALA prepared in Pass-1 and Pass-2 of microprocessor? **7M**

Ans. See P.PS-27, Q.3(b).

UNIT - III

(Total Marks : 26)

Q.5.(a) Explain in detail about dynamic linking and dynamic loading with example. **6M**

Ans. See P.3-11, Q.15.

Q.5.(b) Explain the direct linking loader scheme. Also discuss about all the cards it uses. **7M**

Ans. See P.3-8, 9, Q.10, 12.

Q.6.(a) Assuming that the following program segment is stored at absolute address 1000. Show the assembled program PRG for a direct address machine and the complete memory map as loaded by BSS loader. Assume length of E1=120 bytes and E2 = 70 bytes. **7M**

```
PRG       START
          EXTRN   E1,E2
          L       1,FIVE
          A       1,FOUR
          ST      1,TEMP
          BAL     14,E1
          C 1,    TEMP
          BNE     E2
```

```
        BR      14
FIVE    DC      F'5'
FOUR    DC      F'4'
TEMP    DS      1F
        END
```

Ans. See P.PS-57, Q.6.

Q.6.(b) How is a Local External Symbol Array (LESA) created? Is it necessary for a loader to create a separate LESA for every segment (program)? Justify your answer. What is the role of LESA in pass 2 of Direct Linking loader algorithm? **6M**

Ans. See P.3-14, Q.20.

UNIT - IV
(Total Marks : 26)

Q.7.(a) Explain in detail the format of common object file, in detail. **8M**

Ans. See P.4-2, Q.1.

Q.7.(b) What is symbolic debugger? Explain. **5M**

Ans. See P.4-11, Q.13.

Q.8.(a) Describe source code control system with example. **6M**

Ans. See P.4-10, Q.10.

Q.8.(b) Explain different storage classes with example of each. **7M**

Ans. See P.4-7, Q.5.

UNIT - V
(Total Marks : 26)

Q.9. Describe in detail the anatomy and types of device driver in UNIX system. **13M**

Ans. See P.5-4, Q.3.

Q.10.(a) What are the major design issues in the study of device driver? Elaborate. **7M**

Ans. See P.5-3, Q.2.

Q.10.(b) Compare the device drivers for UNIX and WINDOWS operating systems. **6M**

Ans. See P.5-16, Q.16.

UNIT - VI
(Total Marks : 28)

Q.11. What is compiler? Draw a block diagram of the phases of compiler and indicate the main function of each phase. **14M**

Ans. See P.6-2, 4, Q.1, 4.

Q.12.(a) How tokens are recognized by lexical phase? Explain in detail. **7M**

Ans. See P.6-7, Q.9.

Q.12.(b) Describe LEX as a tool, for system programming. **7M**

Ans. See P.6-10, 11, Q.13, 14.

APPENDIX

OVERVIEW OF DATABASES AND ALGORITHM FOR PHASES OF COMPILERS

Q.1. Describe the way of Implementing the Lexical phase.

Ans. Way of Implementing Lexical Phase:

- These are many ways of implementing the Lexical phase. One way is described below:

  Figure depicts the results of the lexical phase for PL/I program. The input string is separated into tokens by break characters.
- Break characters are denoted by the contents of a special field in the terminal table. Source characters are read in, checked for legality and tested to see if they are break characters. Consecutive non-break characters are accumulated into tokens. Strings between break characters are tokens, as are nonblank break characters. Blanks may serve as break characters but are otherwise ignored.
- Lexical analysis recognizes three types of tokens: Terminal symbols, possible identifiers and literals. It checks all tokens by first comparing them with the entries in the terminal table. Once a match is found, the token is classified as a terminal symbol and lexical analysis creates a uniform symbol of type 'TRM' and inserts it in the uniform symbol table.
- If a token is not a terminal symbol, the lexical analysis proceeds to classify it as a possible identifier or literal. Those tokens that satisfy the lexical rules for forming identifiers are classified as "possible identifiers". In PL/I these are strings that begin with an alphabetic character and contain up to 30 more alphameric characters or underscores. If a token does not fit into one of these categories, it is an error and is flagged as such.
- After a token is classified as a "possible identifier", the identifier table is examined. If this particular token is not in the table, a new entry is made. Since the only attribute that we know about an identifier is its name, that is all that goes into the table. The remaining information is discovered and inserted by later phases.
- Regardless of whether or not an entry had to be created, lexical analysis creates a uniform symbol of type 'IDN' and inserts it into the uniform symbol table.
- Numbers, quoted character strings and other self-defining data are classified as "literals". After a token has been classified as such the literal table is examined. If the literal is not yet there, a new entry is made. In contrast to the case of identifiers, lexical analysis can determine the attributes and the internal representation of a literal by looking at the characters that represent it. Thus each new entry made in the literal table consists of the literal and all its attributes.
- Regardless of whether or not a new entry is made, a uniform symbol of type 'LIT' is created and put into the uniform symbol table.
- The lexical analysis may make one complete pass over the source code and produce the entire uniform symbol table. Another scheme is to have the lexical analyzer called only when the syntax phase needs the next token.

Terminal Table:

	Symbol	Break	Other
1	:	Yes	
2	;	Yes	
3	(	Yes	
4	)	Yes	
5	.	Yes	
6	,	Yes	
7	PROCEDURE	No	
8	DECLARE	No	
9	RETURN	No	
10	END	No	
	+		
	−		
	⋮		
	etc.		

Uniform Symbol Table:

Class	Index	Tokens
IND	1	WCM
TRM	1	:
TRM	7	PROCEDURE
TRM	3	(
IDN	2	RATE
TRM	5	,
IDN	3	START
TRM	5	,
IDN	4	FINISH
TRM	4	)
TRM	2	;
TRM	8	DECLARE
TRM	3	(
IDN	5	COST
TRM	5	,
IDN	2	RATE
⋮		⋮
etc.		

Identifier Table:

	Name	Attributes
1	WCM	
2	RATE	
3	START	Filled in by later phases
4	FINISH	
5	COST	

Literal Table:

Literal	Base	Scale	Precision	Other	Address
31	DECIMAL	FIXED	2		
2	DECIMAL	FIXED	1		
100	DECIMAL	FIXED	3		

Fig. Lexical phase - terminal, identifier, uniform symbol tables for example.

Q.2. Explain in detail the database used in the Syntax Phase.

OR What do you mean by the syntax analysis phase in the compiler? Elaborate the various database used in syntax analysis.

OR Explain the database and algorithm required for the syntax analysis phase of the compiler with a neat sketch.

OR Write a short note on database requirements for the syntax analysis phase in the compiler.

OR Explain in brief algorithms and databases used in the syntax analysis phase of the compiler.

OR Consider the following general form of rule. Elaborate the conventions used in the rule: Label: Old Top of Stack/Action Routines/New Top of Stack/Next Reduction.

Ans. Syntax Phase:

- The function of the syntax phase is to recognize the major constructs of the language and to call the appropriate action routines that will generate the intermediate form or matrix for these constructs.
- In some compilers, this phase is implemented by one large program that recognizes each construct. We choose to present a more general implementation.
- Our syntax phase will be an interpreter for general rules or reductions that define the major constructs of the language and the appropriate action to be taken for each construct.
- The reductions are dependent upon the syntax of the source language and thus must be modified should the syntax of the source language change. The format of the reductions is quite arbitrary, but the information contained in them is essential. For reasons of efficiency, many compilers do not have interpreters as their syntactic phase. Rather, the phase consists of fixed code.
- If one wished to make the compiler more efficient, one could "compile" the reductions. We find that writing reductions are a systematic approach to implementing the syntax phase; if the designer wishes, he way then hand-translate reductions into a single program.

DataBases:

Uniform Symbol Table:

This table is created by the lexical analysis phase and contains the source program in the form of uniform symbols. It is used by the syntax and interpretation phases as the source of input to the stack. Each symbol from the UST enters the stack only once.

Table	Index

Uniform symbol table entry.

Stack:

- The stack is the collection of uniform symbols that are currently being worked on by the syntax analysis and interpretation phases. Additions

to or deletions from the stack are made by the phases that use it. The stack is organized on the Last In-First Out (LIFO) basis. This book always shows the top of the stack toward the bottom of the page, representing the way a stack might be organized in a computer. Actually the term 'Top of Stack' refers to the most recent entry and 'Bottom of Stack' to the oldest entry. The stack may be incorporated into the uniform symbol table or vice versa.

Reductions:

- The syntax rules of the source language are contained in the reduction table. The syntax analysis phase is an interpreter driven by the reductions.
- The general form of the rule or reduction is:

  Label: Old Top of Stack/Action Routines/New Top of Stack/Next Reduction
- The following conventions are used:

(A) Label - optional,

(B) Old Top of Stack - to be compared to a top of the stack.

(a) Blank or null - always a match, regardless of what is on the stack.

(b) Nonblank - One or more items from the following categories.

(i) <syntactic type> such as an identifier or literal – matches any uniform symbol of this type.

(ii) <any> matches a uniform of any type.

(iii) Symbolic representation of a keyword, such as "Procedure" or "If" or " ; " – matches only the uniform symbol for this keyword.

(C) Action Routines - to be called if Old Top of stack matches Top of Stack

(a) Blank or null - no action routines called,

(b) Name of action routine(s) - call the routine(s).

(D) New Top of Stack - changes to be made to Top of Stack after action routines are executed.

(a) Blank or null - no change,

(b) " – delete Top of Stack (i.e. pattern that has been matched)[2]

(c) Syntactic type, keyword or stack item (Sm) – delete Old Top of Stack (i.e. pattern that has been matched in 2)[2] and replace with this item (s)

(d) * – get the next uniform symbol from the uniform symbol table and put it on top of the stack.

(E) Next Reduction:

(a) Blank or null - interpret the next sequential reduction.

(b) n - interpret reduction n

- Within the reductions we have written both stack fields with the oldest item on the left and the newest on the right. $S_m\ S_{m-1} \ldots S_2 S_1$ denotes a field m long, where S_1 refers to the top of the stack, S_2 to the second item on the stack, etc.
- The S_n notation will be used in the New Top of Stack field to represent the nth item on the stack. We have also placed the symbolic representation of the terminal symbols into the Old Top of Stack field, instead of the uniform symbols that may be the actual stack elements in efficient implementation of reductions.

Algorithm:

The algorithm for the syntax analysis phase is as follows:

(i) Reductions are tested consecutively for a match between the Old Top of Stack field and the actual Top of Stack until a match is found.

(ii) When a match is found, the action routines specified in the action field are executed in order from left to right.

(iii) When control returns to the syntax analyzer, it modifies the Top of Stack to agree with the New Top of Stack field.

(iv) Step 1 is then repeated starting with the reduction specified in the next reduction field.

Q.3. Explain in detail the database used in Interpretation Phase.

OR What do you mean by the interpretation phase in the compiler? Elaborate the various database used in interpretation.

OR Explain the database and algorithm required for the interpretation phase of the compiler with a neat sketch.

OR Write a short note on database requirements for the interpretation phase in the compiler.

OR Explain in brief algorithms and databases used in the interpretation phase of the compiler.

Ans. Interpretation Phase:

- The interpretation phase is typically a collection of routines that are called when a construct is recognized in the syntactic phase. The purpose of these routines (called action routines) is to create an intermediate form of the source program and add information to the identifier table.
- The separation of the syntactic phase from the interpretation phase is a logical division. The former phase recognizes syntactic constructs (phrases that have associated meaning, e.g., statements or clauses) while the latter interprets the precise meaning into the matrix or identifier table.
- In actual situations it may not be obvious where the recognizing of a construct ends and the interpretation of its meaning begins. In other words, it may be arbitrary to what size phrase of its meaning begins.

This may be illustrated by the problem of handling arithmetic expressions. An entire arithmetic expression may be considered a syntactic entity so that its recognition should be a problem for syntax analysis and its parsing a problem for an action routine. However; each subexpression (two operands and an operator) can also be considered a syntactic entity and therefore should be recognized by the syntax analysis phase. For the first case, the action routine may construct matrix entries for the entire expression and for the second case the action routines would only construct one entry for a subexpression.

- The tradeoff is between how much computation is performed by the syntax phase versus how much by the action routines.

DataBases:

Uniform Symbol Table:

- Uniform Symbol Table consists of a full or partial list of the tokens as they appear in the program. Table is created by Lexical analysis and used for syntax analysis and interpretation phase. There is one uniform symbols table for every token in the program.
- Each uniform symbol contains the identification of tables of which the token is a member and its index within that table

Table	Index

Uniform symbol table entry

Stack:

Contains the tokens currently being parsed by the syntax and interpretation phases.

Identifier Table:

- Initialized by lexical analysis to completely describe all identifiers used in the source program. The lexical phase entered the identifier (or pointer to its character string) in the name field. The interpretation phase enters all the attributes. The storage assignment phase will enter the address.
- This information is used in later phases for assignment of storage and for generating the proper code for accessing storage associated with these entries. Most attributes may be determined during the parsing of the declaration statement.

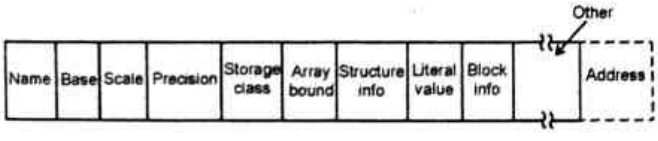


Fig. Identifier table entry.

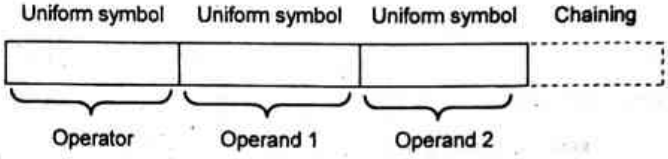
Matrix:

- The primary intermediate form of the program. There are two reasons for having a matrix:

(i) It allows the first four phases of the compiler to be machine-independent;

(ii) It allows machine-independent optimization of the program to occur before the code generation phase.

- A simple form of a matrix entry consists of a triplet where the first element is a uniform symbol denoting the terminal symbol or operator and the other two elements are uniform symbols denoting the arguments. The code generation phase will be driven by this matrix and will produce the appropriate object code.
- There may also be fields reserved for chaining of entries that can be utilized by the optimization phase to add or delete entries.



Matrix entry.

- Although the entries in the matrix are uniform symbols, for clarity in our examples we will show the symbol itself. Some of the matrix operators do not have a direct parallel in either the source language or the machine language. For example, a compound statement such as IF THEN...ELSE, may be translated into multiple matrix entries, each of which is translated into several machine instructions.
- Matrix operands are uniform symbols of the type IDN, LIT, or TRM and a fourth form, MTX. A uniform symbol | MTX | n | denotes the result of the n^{th} matrix entry and points to the corresponding entry in the temporary storage area of the identifier table line.

Temporary Storage Table:

(May be implemented as part of the identifier table) The interpretation phase enters into the temporary storage table all information about the associated values of MTX symbols, i.e., the attributes of the temporary computations resulting from matrix lines, such as data type, precision, source statement in which it is used, etc. This information will be used in storage assignments and in generating code for access.

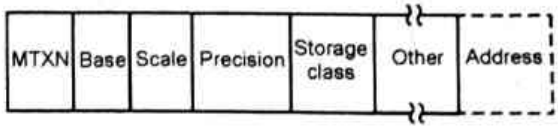


Fig. Temporary Storage Table.

Algorithm:

There is no one algorithm for this phase since it is a collection of individual action routines that accomplish specific tasks when invoked by the syntax analysis phase. In particular, these routines may:

(1) Do any necessary additional parsing - this permits action routines to add symbols to or delete them from the stack as they deem necessary.

(2) Create new entries in the matrix or add data attributes to the identifier table. In the former case, the routines must be able to determine the proper operator and operands and insert them into the matrix. In the latter case, they must decide exactly what attributes have been declared and put them into the identifier table. In both these cases, the complexity of the action routines will depend on how much has been done by the reductions and vice versa.

Q.4. Explain in detail the database used in the optimization phase.

OR What do you mean by the optimization phase in the compiler? Elaborate the various database used in optimization.

OR Explain the database and algorithm required for the optimization phase of the compiler with a neat sketch.

OR Write a short note on database requirements for the optimization phase in the compiler.

OR Explain in brief algorithms and databases used in the optimization phase of the compiler.

OR Discuss the elimination of a common subexpression optimization technique.

Ans. Optimization Phase:

- The two types of optimization performed by a compiler, machine-dependent and machine-independent.
- Machine-dependent optimization is so intimately related to the instructions that get generated that it was incorporated into the code generation phase, whereas machine-independent optimization was done in a separate optimization phase.
- In deciding whether or not to incorporate any of these optimization schemes into a compiler, it is necessary to weigh the gains it would bring in increased efficiency for the object program against the increased cost in compilation time and complexity.
- The outcome of the decision should depend heavily on the circumstances under which the compiler is being produced and on the use to which it will be put.

DataBases:

Matrix:

- This is the major database used in the optimization phase. To facilitate the elimination or insertion of entries into the matrix, we add to each entry chaining informing, forward and backward pointers. This avoids the necessity of reordering and relocating matrix entries when an entry is added or deleted.
- The forward pointers allow the code generation phase to go through the matrix in the proper order. The backward pointer allows backward sequencing through the matrix as may be needed by the optimization technique that moves invariant computations outside DO_LOOPS.
- A matrix entry now looks like:

Operator	Operand 1	Operand 2	Forward pointer	Backward pointer

where a forward pointer is the index of the next matrix entry in the program and the backward pointer is the index of the previous one.

Identifier Table - accessed to delete un-needed temporary storage and obtain information about identifiers.

Literal Table - new literals that may be created by certain types of optimization.

Algorithm:

We discuss possible algorithms for four optimization techniques:

(i) Elimination of Common Subexpressions:

- The common subexpressions must be identical and must be in the same statement. For example, in the following program, we could not eliminate the common matrix entries associated with the second START-FINISH since the value of START is changed in between the evaluation of the two expressions.

```
COST = RATE * (START – FINISH);
START = START / 2;
COST2 = RATE * (START - FINISH - 100);
```

- In some languages such as PL/I where interrupts may change the value of variables at any time, elimination of common subexpressions can be done only if the programmer specifies that non of his interrupt routines change the value of any variables. For example, in PL/I a programmer may declare ON OVERFLOW (FINISH=0). This could possibly make our two subexpressions unequivalent even if they occurred in the same statement.

- The elimination algorithm must:

(a) Place the matrix in a form so that common subexpressions can be recognized.

(b) Recognize two subexpressions as being equivalent.

(c) Eliminate one of them.

(d) Alter the rest of the matrix to reflect the elimination of this entry.

- Once it has modified entries within a given statement, this algorithm might also want to check to see if its modifications have created any additional common subexpressions.

For example, Figure represents the matrix created from several statements of a program. A possible algorithm for the elimination of common subexpression would:

(a) Order the operands of the + and * operators alphabetically – check each entry and change those that are backwards (only M3 and M4 need to be switched).

(b) Realize statement boundaries - in this case merely separate them using the = operators.

(c) Look for common subexpression – compare M2 and M3, M4, M5. M3 is the only match so delete M3 by changing the pointers of M2 and M4.

(d) Alter rest of matrix to reflect deleted entry, e.g. change all references of M3 to M2.

(e) Rescan matrix for possible created common subexpressions – repeat 1 through 5 until no changes occur.

(f) Eliminate from the temporary storage table any MTX entries that are no longer needed.

Source Code:

B = A

A = C * D * (D * C + B)

Matrix before optimization

M1	=	B	A	0	2
M2	*	C	D	1	3
M3	*	D	C	2	4
M4	+	M3	B	3	5
M5	*	M2	M4	4	6
M6	=	A	M5	5	?

Matrix after steps 1 and 2

M1	=	B	A	0	2	Statement
M2	*	C	D	1	3	Statement
M3	*	C	D	2	4	
M4	+	B	M3	3	5	
M5	*	M2	M4	4	6	
M6	=	A	M5	5	?	

Matrix after step 4

M1	=	B	A	0	2
M2	*	C	D	1	(4)
M3	*	C	D	2	4
M4	+	B	(M2)	(2)	5
M5	*	M2	M4	4	6
M6	=	A	M5	5	?

Fig. Example of elimination of common subexpressions.

(ii) **Compile Time Compute:**

- Doing computations involving constants at compile time saves both space and execution time for the object program. This the especially helpful if there is such an arithmetic computation within a loop. For example, if we had the statement.

A = 2 * 276 / 92 * B

the compiler could perform the multiplication and the division so indicated and substitute 6 * B for the original expression. This would enable it to delete two matrix entries, thus reducing the amount of object code that is generated.

Matrix	Before optimization		
M1	*	2	276
M2	/	M1	92
M3	*	M2	B
M4	=	A	M3

Matrix	Before optimization		
M1			
M2			
M3	*	6	B
M4	=	A	M3

Fig. Compile-time-compute

- A simple algorithm for this optimization technique would first scan the matrix, looking for operators, both of whose operands were literals. Whenever it found such an operation, it would evaluate it, create a new literal, delete the old lines and replace all references to it with the uniform symbol for the new literal and then continue scanning the matrix for more possible computations.

(iii) **Boolean Expression Optimization:**

We may use the properties of Boolean expressions to shorten their computation; e.g. in a statement IF a OR b OR c, THEN ..... when a, b and c are expressions. Rather than generate code that will always test each of the expressions a, b, c, we generate code so that if a is computed as true, then b OR c is not computed and similarly for b.

(iv) **Move Invariant Computations outside of Loops:**

If a computation within a loop depends on a variable that does not change within that loop, the computation may be moved outside the loop.

This involves a reordering of a part of the matrix. There are three general problems (a, b, c) that need to be solved in an algorithm of this type.

(a) **Recognition of Invariant Computations:**

- For example, in the following program what should we try to move outside a loop?

```
                DO   I = 1 To 10;
OUTLOOP:   ≡
                DO   J = 1 To 10;
        LA:A = 10;
        LB:B = Y(I + 2);
        LC:C = C + 10;
                END;
                END;
```

- The statement LA will always produce the same result and thus could be moved outside the loop. In LB the subscript I + 2 is also invariant and could be moved outside the inner loop. We might also want to check. If the value of Y is changed inside the inner loop. In this case, it is not, so this too is an invariant computation that can be moved.
- LC is a computation that involves C, which is a variable that is changed in the inner loop so it cannot be moved to the outer one. Note that if there are any statements (within the loop) before LA that reference A or B, we could not move their assignment statements out of the inner loop, all we could move would be the computation of the subscript.

(b) **Discovering where to move the Invariant Computation:**

- In general we want to move the computation to a position directly preceding the LOOP from which it comes. This requires us to find the starting points of DO_LOOPS by examining the matrix. The interpretation phase would have to place a unique matrix entry at the beginning of every DO_LOOP.
- It would also have to supply backward pointers in the matrix so that the optimization phase could work its way back to outer loops.

(c) **Moving the Invariant Computation:**

Using the same methods of chaining discussed in the elimination of common subexpressions, we would delete the invariant computation from its original position in the matrix and insert it into the appropriate place.

Q.5. Explain in detail the database used in Storage Assignment Phase.

OR What do you mean by the storage assignment phase in the compiler? Elaborate the various database used in storage assignments.

OR Explain the database and algorithm required for the storage assignment phase of the compiler with a neat sketch.

OR Write a short note on database requirements for the storage assignment phase in the compiler.

OR Explain in brief algorithms and databases used in the storage assignment phase of the compiler.

Ans. Storage Assignment Phase:

- The purpose of this phase is to:

(i) Assign storage to all variables referenced in the source program.

(ii) Assign storage to all temporary locations that are necessary for intermediate results, e.g. the results of matrix lines. These storage references were reserved by the interpretation phase and did not appear in the source code.

(iii) Assign storage to literals.

(iv) Ensure that the storage is allocated and appropriate locations are initialized (literals and any variables with the initial attribute).

- To do this, it must insert information for the code generation phase into the identifier table, the literal table and the matrix.

Data Bases:

Identifier Table:

Created by lexical analysis and furnished with all attribute entries by interpretation phase. Storage assignment designates locations to all identifiers that denote data. The next two phases use this information to generate access to the identifiers.

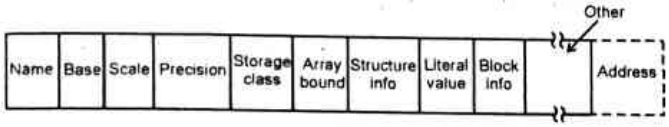


Fig. Identifier table entry.

Temporary Storage Table:

- (Part of identifier table) Created by the interpretation phase to describe the temporary results of computations in the matrix. This table may be implemented as part of the identifier table since much of the information is of the same format.
- Further, note that all storage of this type is implemented as automatic, e.g. it is not physically allocated until the procedure is activated. This is satisfactory since no other procedure ever references it, nor is the space needed unless the procedure that contains the storage is activated.

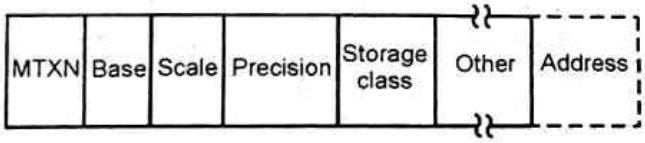


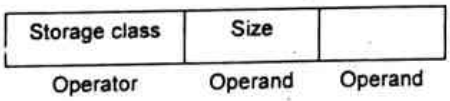
Fig. Temporary storage entry

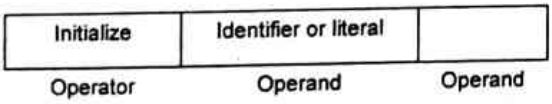
Literal Table:

The storage assignment phase assigns all literal addresses and places an entry in the matrix to denote that code generation should allocate this storage.

Matrix:

Storage assignment place entries into the matrix to ensure that code generation creates enough storage for identifiers, temporary storage and literal. It also ensures that the appropriate ones are initialized.

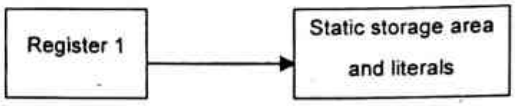




Matrix entries by storage allocation

Algorithm:

- Before creating an algorithm for the storage assignment phase, consideration must be given to the format we wish storage to appear in at execution time to allow efficient accessing.
- On a 360-type machine, with base and index registers, it is particularly desirable to layout the user's storage in continuous blocks, with a register pointing to the beginning of each storage block.



- This requires storage assignment to calculate only the offsets for all static storage. The code generation phase will:

(i) Use these offsets to generate the address used in instructions and

(ii) Generate appropriate code that will load a register with a pointer to the appropriate storage block at execution time and before that storage is accessed.

- The storage allocation phase first scans through the identifier table, assigning locations to each entry with a storage class of static. It uses a location counter, initialized at zero, to keep track of how much storage it has assigned. Whenever it finds a static variable in the scan, the storage allocation phase does the following four steps:

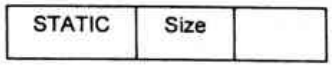
(i) Updates the location counter with any necessary boundary alignment.

(ii) Assigns the current value of the location counter to the address field of the variable.

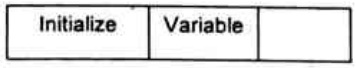
(iii) Calculates the length of the storage needed by the variable (by examining its attributes).

(iv) Updates the location counter by adding this length to it.

- Once it has assigned relative address to all identifiers requiring STATIC storage location, this phase creates a matrix entry:



- This allows code generation to generate the proper amount of storage. For each variable that requires initialization, the storage allocation phase generates a matrix entry:



- This tells code generation to put into the proper storage location the initial value that the action routines saved in the identifier table.
- A similar scan of the identifier table is made for automatic storage and controlled storage. The scan enters relative locations for each entry. An "automatic" entry and a "controlled" entry are also made

in the matrix. Code generation uses the relative location entry to generate the address part of instructions. No storage is generated at compile time for automatic or controlled. However, the matrix entry automatic does cause code to be generated that allocates this storage at execution time, i.e. when the generated code is executed, it allocates automatic storage.

- The literal table is similarly scanned and locations are assigned to each literal and a matrix entry | LIT | Size | | is made. Code generation generates storage for all literals in the static area and initializes the storage with the values of the literals.
- Temporary storage is handled differently since each source statement may reuse the temporary storage (intermediate matrix result area) of the previous source statement. A computation is made of the amount of temporary storage that is required for each source statement.
- The statement requiring the greatest amount of temporary storage determines the amount that will be required for the entire program. A matrix entry is made of the form | AUTOMATIC | size | | . This enables the code generation phase to generate code to create the proper amount of storage.
- Temporary storage is automatic since it is only referenced by the source program and only needed while the source program is active.

Q.6. Explain in detail the database used in the code generation phase.

OR What do you mean by the code generation phase in the compiler? Elaborate the various database used in code generation.

OR Explain the database and algorithm required for the code generation phase of the compiler with a neat sketch.

OR Write a short note on database requirements for the code generation phase in the compiler.

OR Explain in brief algorithms and databases used in the code generation phase of the compiler.

OR Write in brief the simple machine-dependent optimization algorithm.

Ans. Code Generation phase:

- The purpose of the code generation phase is to produce the appropriate code (assembly or machine language). We have again chosen a general way of depicting this phase, but one which exhibits the issues of code generation and certain machine-dependent optimization techniques.
- It uses code productions (macros definitions) which define the operators that may appear in the matrix to produce code. It also references the identifier tables and literal tables in order to generate proper address and code conversions.

DataBases:

Matrix:

Each entry has its operator defined in the code pattern database. Code generation examines each entry and looks up the code to be generated in the code patterns. In other words, the operator of each entry serves as a macro with the operands as arguments.

Identifier Table, Literal Table:

Identifier table, Literal table are used to determine the data type and locations of the variables (operands of the matrix) so that proper accessing code with the correct address is generated. Also if there are mixed data type expressions, code generation must generate proper code for conversions.

Code Production (Macro Definition):

A permanent database defining all possible matrix operators.

Algorithm:

The code generation process in a way analogous to that used for the assembler macro processor. The operation field of each matrix line is treated as a "macro-call" and the matrix operands on the line are used as "macro arguments."

Simple Machine-dependent Optimization:

Figure depicts the matrix for the source statement A = B + C + D. The centre column is the code generated using the simple definitions of + and = of Figure.

Matrix	"Original Code"		"Better Code"	
+ B C	(1) L	1, B	(1) L	1, B
	(2) A	1, C	(2) A	1, C
	(3) ST	1, M1		----
+ M1 D	(4) L	1, M1		----
	(5) A	1, D	(3) A	1, D
	(6) ST	1, M2		----
= A M2	(7) L	1, M2		----
	(8) ST	1, A	(4) ST	1, A

Fig. Code for statement A = B + C + D.

- There are two types of optimization involved. First, if a value is already contained in a register, there is no need to reload, this allows us to eliminate the L 1, M1 and L 1, M2 references. Second, careful

consideration of the matrix generation process will show that a matrix temporary, Mi is used once at most.

- Therefore, if we are going to immediately use the value of Mi that was left in the register, there is no need to store its value. This allows us to eliminate the ST 1, M1 and ST 1, M2 instructions.
- There are three possible ways to perform this code generation optimization:

(i) During Matrix Optimization

(ii) During Code Generation or

(iii) During a post pass after assembly code is generated.

- The matrix, even when optimized, is still intended to be a machine-independent representation of the source program. If we optimize register usage in the matrix, it becomes machine-dependent, for example, what does it mean to a machine without "registers" such as the IBM 1620 and 1401.
- Using a post-generation optimization has many shortcomings. For example, a 1000-line matrix would generate about 4000 assembly cards. Whereas, we might fit the entire matrix in core memory, it would be necessary to write out and then reread the assembly cards to eliminate the unnecessary instructions - a time-consuming process.
- The best solution is to eliminate unnecessary instructions before they are generated (i.e. during the code selection pass).

Simple Machine-dependent Optimization Algorithm:

- Figure depicts an algorithm we could use to eliminate unnecessary Load (L) and Store (ST) instructions for the "+" operation.
- There are two points that should be noted about the algorithm. First, if an operand, Mi, is (at execution time) already in the register, it is not reloaded. Second, the responsibility for storing the matrix temporary is left to the next matrix line code generation since it is the only line that knows whether the previous temporary will be needed immediately.
- Rather than write a separate assembler subroutine for the code generation of each operator, a code pattern language is used to represent the appropriate algorithm.

For the "+" operator the code pattern of Figure may be employed.

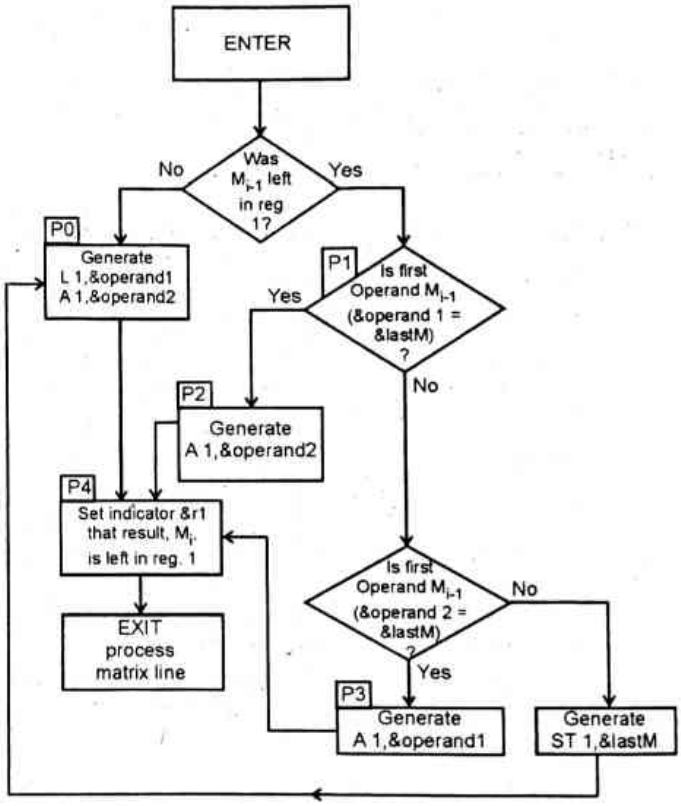
- It is assumed that the variables &i, &r1, &M and &lastM were initialized at the beginning of code generation as follows (using 360 macro language notation):

```
        GBLA    &i
        GBLC    &r1, &M, &lastM
&i      SETA    1
&r1     SETC    'null'
&M      SETC    'M1'
&lastM  SETC    'null'
```

- The macro NEXTM used in the code pattern for "+" can be defined as follows:

```
        MACRO
        NEXTM
&lastM  SETC    'M&i'     set new value for Mi-1
&i      SETA    &i+1      set new value for i
&M      SETC    'M&i'     set new value for Mi
```

Fig. Code generation language for "+" operator with optimized register usage



The labels P0, P1 etc. used in the code pattern for "+' correspond to the labelled boxes in the flowchart of figure.

Q.7. Explain in detail databases used in Assembly Phase.

OR What do you mean by the assembly phase in the compiler? Elaborate the various database used in the assembly phase.

OR Explain the database and algorithm required for the assembly phase of the compiler with a neat sketch.

OR Write a short note on database requirements for the assembly phase in the compiler.

OR Explain in brief algorithms and databases used in the assembly phase of the compiler.

Ans. Assembly Phase:

- The task of the assembly phase depends on how much has been done in code generation. If a lot of work has been done in code generation, then the assembly phase (simple) must resolve label references in the object program, format the object deck and format the appropriate information for the loader (e.g. for direct linking loading schemes prepare TXT, RLD, ESD cards).
- At the other extreme, if code generation has simply generation has simply generated symbolic machine instructions and labels, the assembly phase (complicated) must:

(i) Resolve label references.

(ii) Calculate addresses.

(iii) Generate binary machine instructions and

(iv) Generate storage, convert literals.

Data Bases:

Identifier Table:

The simple assembly phase uses this database to enter the value of all labels into the identifier table.

Literal Table:

The complicated assembly phase would initialize all literal locations, that is, place the literals on the appropriate TXT cards.

Object Code:

The output of the code generation.

Algorithm:

- The simple assembly phase scans the object code, resolving all label references and producing the TXT cards. It then scans the identifier table to create the ESD cards. The RLD cards are created using the object code, ESD cards and the identifier table.
- In actual implementation, certain functions and databases may be eliminated as they are duplicated in the other phases of the compiler, for example, the symbol table is similar to the identifier table.

INTEL 64 AND IA-32

Q.8. Give the case study and basic architecture of Intel®64 and IA-32 Processors.

Ans. Intel 64 and IA-32 Processors:

- **IA-32 Architecture** refers to systems based on 32-bit processors generally compatible with the Intel Pentium® II processor, (for example, Intel® Pentium 4 processor or Intel Xeon processor), or processors from other manufacturers supporting the same instruction set, running a 32-bit operating system.
- **Intel 64 Architecture** refers to systems based on IA-32 architecture processors which have 64-bit architectural extensions, for example, Intel Core 2 processor family), running a 64-bit operating system such as Microsoft Windows XP Professional x64 Edition or Microsoft Windows Vista x64.
- If the system is running a 32-bit version of the Windows operating system, then IA-32 architecture applies instead.

Systems based on AMD processors running a 64-bit version of Windows are also supported by Intel compilers for Intel® 64 architecture applications.

Table: Key Features of Most Recent Intel 64 Processors.

Intel Processor	Date Introduced	Micro Architecture	Highest Processor Base Clock Frequency at Introduction	Transistors	Register Sizes	System Bus Bandwidth	Max. External Address Space	On-Die Cache
Intel Xeon Processor E7-8870	2012	Intel Turbo Boost Technology, Intel microarchitecture code name Westmere; Ten core; Hyperthreading Technology; Intel 64 Architecture; Intel Virtualization Technology.	2.40 GHz	2.2B	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	QPI: 6.4 GT/s; Memory: 102 GB/s	16 TB	L1: 64 KB L2: 256KB L3: 30MB

Intel Core i7-2600K Processor	2011	Intel Turbo Boost Technology, Intel microarchitecture code name Sandy Bridge; Four core; Hyperthreading Technology; Intel 64 Architecture; Intel Virtualization Technology.	3.50 GHz	2.2B	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128 YMM: 256	DMI: 5 GT/s; Memory: 21 GB/s	1 TB	L1: 64 KB L2: 256KB L3: 8MB
Intel Core i7-620M Processor	2010	Intel Turbo Boost Technology, Intel microarchitecture code name Westmere; Dual-core; Hyperthreading Technology; Intel 64 Architecture; Intel Virtualization Technology., Integrated graphics	2.66 GHz	383 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	QPI: 6.4 GT/s; 32 GB/s	64 GB	L1: 64 KB L2: 256KB L3: 4MB
Intel Core i7-965 Processor Extreme Edition	2008	Intel microarchitecture code name Nehalem; Quadcore; Hyper-Threading Technology; Intel QPI; Intel 64 Architecture; Intel Virtualization Technology.	3.20 GHz	731 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	QPI: 6.4 GT/s; Memory: 25 GB/s	64 GB	L1: 64 KB L2: 256KB L3: 8MB
Intel Core 2 Duo Processor E6850	2007	Intel Core Microarchitecture; Dual Core; Intel 64 Architecture; Intel Virtualization Technology; Intel Trusted Execution Technology	3.00 GHz	291 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	10.6 GB/s	64 GB	L1: 64 KB L2: 4MB (4MB Total)
Intel Xeon Processor 5472	2007	Enhanced Intel Core Microarchitecture; Quad Core; Intel 64 Architecture; Intel Virtualization Technology	3.00 GHz	820 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	12.8 GB/s	256 GB	L1: 64 KB L2: 6MB (12MB Total)
Intel Core 2 Extreme Processor QX6700	2006	Intel Core Microarchitecture; Quad Core; Intel 64 Architecture; Intel Virtualization Technology.	2.66 GHz	582M	GP: 32,64 FPU: 80 MMX: 64 XMM: 128	8.5 GB/s	64 GB	L1: 64 KB L2: 4MB (4MB Total)

Intel Pentium 4 Processor 672	2005	Intel NetBurst Microarchitecture; Intel Hyperthreading Technology; Intel 64 Architecture; Intel Virtualization Technology.	3.80 GHz	164 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	6.4 GB/s	64 GB	12K µop Execution Trace Cache; 16 KB L1; 2MB L2
64-bit Intel Xeon Processor with 800 MHz System Bus	2004	Intel NetBurst Microarchitecture; Intel Hyperthreading Technology; Intel 64 Architecture	3.60 GHz	125 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	6.4 GB/s	64 GB	12K µop Execution Trace Cache; 16 KB L1; 1 MB L2

Table: Key Features of Most Recent IA-32 Processors.

Intel Processor	Date Introduced	Micro Architecture	Highest Processor Base Clock Frequency at Introduction	Transistors	Register Sizes	System Bus Bandwidth	Max. External Address Space	On-Die Cache
Intel Pentium M Processor 755	2004	Intel Pentium M Processor	2.00 GHz	140 M	GP: 32 FPU: 80 MMX: 64 XMM: 128	3.2 GB/s	4 GB	L1: 64 KB L2: 2 MB
Intel Core Duo Processor T2600	2006	Improved Intel Pentium M Processor Microarchitecture; Dual-Core; Intel Smart Cache, Advanced Thermal Manager	2.16 GHz	152M	GP: 32 FPU: 80 MMX: 64 XMM: 128	5.3 GB/s	4 GB	L1: 64 KB L2: 2 MB (2MB Total)
Intel Atom Processor Z5xx series	2008	Intel Atom Microarchitecture; Intel Virtualization Technology	1.86 GHz – 800 MHz	47M	GP: 32 FPU: 80 MMX: 64 XMM: 128	Up to 4.2 GB/s	4 GB	L1: 56 KB L2: 512KB

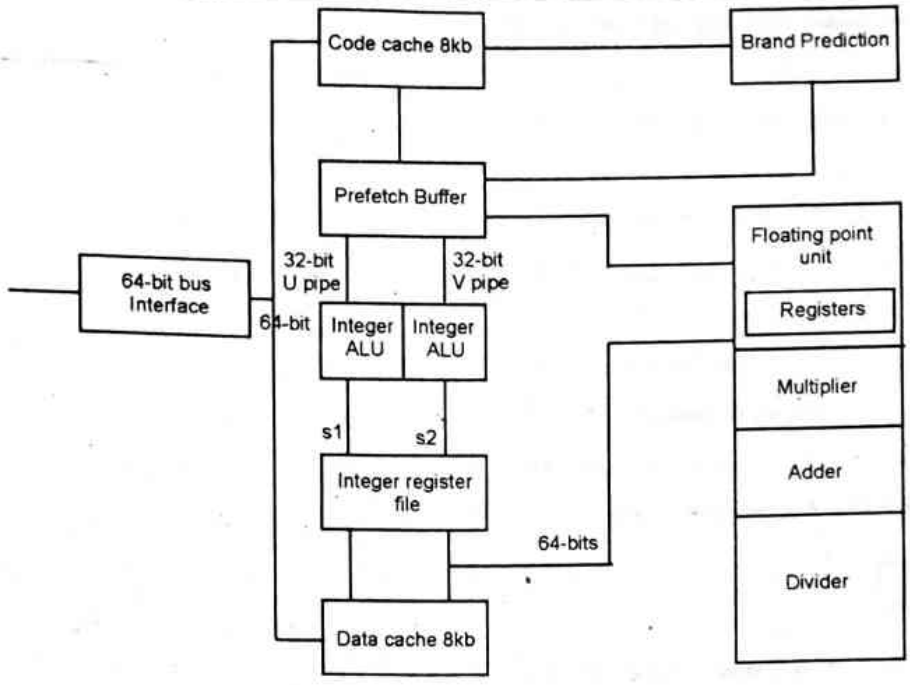
Table: Key Features of Previous Generations of IA-32 Processors.

Intel Processor	Date Introduced	Highest Processor Base Clock Frequency at Introduction	Transistors	Register Sizes	Ext. Data Bus Size	Max. External Address Space	Caches
Pentium M Processor	2003	1.60 GHz, Intel NetBurst Microarchitecture	77 M	32 GP 80 FPU 64 MMX 128 XMM	64	4 GB	L1: 64KB L2: 1 MB
Intel Xeon Processor	2002	2.20 GHz, Intel NetBurst Microarchitecture, Hyperthreading Technology	55 M	32 GP 80 FPU 64 MMX 128 XMM	64	64 GB	12K µop Execution Trace Cache; L1: 8KB L2: 512KB
Pentium 4 Processor	2000	1.50 GHz, Intel NetBurst Microarchitecture	42 M	32 GP 80 FPU 64 MMX 128 XMM	64	64 GB	12K µop Execution Trace Cache; L1: 8KB L2: 256 KB
Pentium III and Pentium III Xeon Processors	1999	700 MHz	28 M	32 GP 80 FPU 64 MMX 128 XMM	64	64 GB	L1: 32 KB L2: 256 KB
Pentium II Processor	1997	266 MHz	7 M	32 GP 80 FPU 64 MMX	64	64 GB	L1: 32 KB L2: 256 KB - 512 KB
Intel486 DX Processor	1989	25 MHz	1.2 M	32 GP 80 FPU	32	4 GB	L1: 8 KB
8086	1978	8 MHz	29 K	16 GP	16	1 MB	None

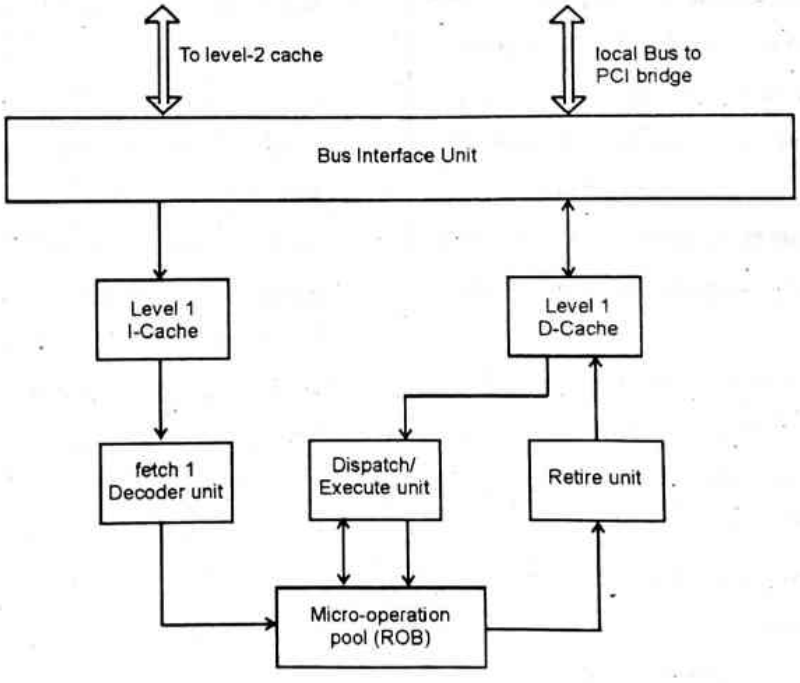
IA-32 Processor Basic Architecture:

- IA-32 processor has 32-bit memory addresses and 32-bit data operands.

Two successive instructions are dispatched in parallel to the two integer units U and V, which are capable of executing instructions in parallel.



- Fig. above shows simplified Pentium Architecture. It includes two on-chip caches, one for data and one for instructions, each of 8k bytes.
- The data cache feeds both integer and floating-point operations. The data Cache is dual-ported. The code cache is read-only, feeds into a prefetch buffer.
- The main execution units are two integer arithmetic logic units (ALUs) which can execute in parallel. A floating-point unit with its own registers and its own multiplier adds and divide components.
- The two 32-bit ports can be used separately to interface to two integer ALU units and can be combined for a 64-bit interface to a floating-point unit.



- It consists of a fetch/decode unit, dispatch/execute unit, a high-level pipeline. They communicate through an instruction pool. Information in the instruction pool is stored in a table called the Re-order Buffer pool (ROB).

- The fetch/decode unit fetches instructions and break them into micro-operation for storage in ROB.
- The dispatch/Execute unit takes micro-operations from the ROB and execute them.
- The retire units' complete execution of each micro-operation and updates the registers. Instruction enters the ROB in order, can be executed out of order, but are retired in order again.
- The bus interface unit is responsible for communicating with the memory system, both the L2 cache and the main memory.
- The L2 cache is not connected to the local bus, so the bus interface unit is responsible for fetching data loading all the caches.

COMPUTER LANGUAGE

Q.9. Write about Computer Programming Language or System Programming Language.

Ans. System Programming Language:

A system programming language is a programming language used for system programming; such languages are designed for writing system software, which usually requires different development approaches when compared with application software.

- A computer programming language consists of a vocabulary containing a set of grammatical rules intended to convey instructions to a computer or computing device to perform specific tasks.
- Each programming language has a unique set of keywords along with a special syntax to organize the software's instructions.
- There are low-level and high-level programming languages which, although simple compared to human languages, are more complex than machine languages.
- Low-level languages include assembly and machine languages. An assembly language contains a list of basic instructions and is much harder to read than a high-level language.
- High-level languages, on the other hand, are designed to be easy to read and understand, allowing programmers to write source code naturally, using logical words and symbols.
- Throughout the evolution of computers, hundreds of different programming languages have been created for various types of development.
- The field of programming is very wide, so the use of a particular language will depend on the objectives to be achieved.
- General-purpose programming languages tend to focus on generic features to allow programs written in the language to use the same code on different platforms.
- Examples of such languages include ALGOL and Pascal. This generic quality typically comes at the cost of denying direct access to the machine's internal workings, and this often has negative effects on performance.
- System languages, in contrast, are designed not for compatibility, but for performance and ease of access to the underlying hardware while still providing high-level programming concepts like structured programming. Examples include SPL and ESPOL, both of which are similar to ALGOL in syntax but tuned to their respective platforms. Others are cross-platform but designed to work close to the hardware, like BLISS, JOVIAL and BCPL.
- Some languages straddle the system and application domains, bridging the gap between these uses. The canonical example is C, which is used widely for both system and application programming. Some modern languages also do this such as Rust and Swift.

Main features of Computer Programming Languages:

The popularity of a programming language depends on the features and utilities it provides to programmers. The features that a programming language must have to stand out are the following:

(i) Simplicity:

The language must offer clear and simple concepts that facilitate learning and application, in a way that is simple to understand and maintain. Simplicity is a difficult balance to strike without compromising the overall capability.

(ii) Naturalness:

This means that its application in the area for which it was designed must be done naturally, providing operators, structures and syntax for operators to work efficiently.

(iii) Abstraction:

It is the ability to define and use complicated structures or operations while ignoring certain low-level details.

(iv) **Efficiency:**

Programming languages must be translated and executed efficiently so as not to consume too much memory or require too much time.

(v) **Structuring:**

The language allows programmers to write their code according to structured programming concepts, to avoid creating errors.

(vi) **Compactness:**

With this characteristic, it is possible to express operations concisely, without having to write too many details.

(vii) **Locality:**

Refers to the code concentrating on the part of the program with which you are working at a given time.

SYSTEM SOFTWARE DEVELOPMENT

Q.10. Write a note on System Software Development or System Design.

Ans. System Software Development:

- System programming involves designing and writing computer programs that allow the computer hardware to interface with the programmer and the user, leading to the effective execution of application software on the computer system.
- Typical system programs include the operating system and firmware, programming tools such as compilers, assemblers, I/O routines, interpreters, schedulers, loaders and linkers as well as the runtime libraries of the computer programming languages.
- System programming is an essential and important foundation in any computer's application development, and always evolving to accommodate changes in the computer hardware.
- This kind of programming requires some level of hardware knowledge and is machine dependent; the system programmer must therefore know the intended hardware on which the software is required to operate.
- Additionally, the programmer may make some assumptions on the hardware and other system components.
- The software is usually written in a low-level programming language which can operate efficiently in a resource-constrained environment, and with little runtime overhead using a small library or none at all.
- The low-level language enables direct control over memory access and allows the program to be written directly in an assembly language. The majority of programs are written using assembly languages such as C, C++ and C#.
- System programming leads to the development of computer system software that manages and controls the computer operations.
- The low-level codes are very close to the hardware level and deal with things such as registers and memory allocations.
- The system programs or system software coordinates data transfer across the various components and deals with the compiling, linking, starting and stopping of programs, reading from files as well as writing to files.
- The system programming enhances or extends the functions of an operating system and may comprise components such as drivers, utilities and updates.
- They enable efficient management of hardware resources such as memory, file access, I/O operations, device management and process management such as process administration and multi-tasking.
- An example is an operating system, which usually acts as the interface between the user, the application software, and computer hardware.
- The OS provides an environment that enables users to execute other programs efficiently. Comprising of a set of system programs, the operating system functions include storage management, file handling, memory management, CPU and device scheduling and management, error handling, process control and more.
- Systems design has three main areas: architectural design, logical design and physical design:
- The architectural design deals with the structure of the system and how each component behaves within it.
- Logical design deals with abstract representations of data flow (inputs and outputs within the system).
- Physical design deals with the actual inputs and outputs. The physical design establishes specific requirements on the components of the system, such as input requirements, output requirements, storage requirements, processing requirements, and system backup and recovery protocols.

- Another way of expressing this is to say that physical design deals with user interface design, data design, and process design.
- Systems designers operate within key design principles when they are creating the system architecture.
- Some of the key tenets of good system design are:

(i) **Be Explicit:** All assumptions should be expressed.

(ii) **Design for Iteration:** Good design takes time, and nothing is ever perfect the first time.

(iii) **Keep Digging:** Complex systems fail for complex reasons.

(iv) **Be Open:** Comments and feedback will improve the system.

- The system programmers are the ones responsible for executing the vision of the system designers.

Q.11. What are the different types of Software Development?

Ans. Types of Software Development:

- There are several different types. They can be grouped into four basic categories:

(i) Application development that provides functionality for users to perform tasks. Examples include office productivity suites, media players, social media tools, and booking systems. Applications can run on the user's computing equipment or servers hosted in the cloud or by an internal IT department. Media streaming development is one example of application development for the cloud.

(ii) System software development to provide the core functions such as operating systems, storage systems, databases, networks, and hardware management.

(iii) Development tools that provide software developers with the tools to do their job, including code editors, compilers, linkers, debuggers, and test harnesses.

(iv) Embedded software development that creates the software used to control machines and devices, including automobiles, phones and robots.

- Each of the categories of the different types of development typically attracts different types of software developers, with different personalities, characteristics and specialisms.
- For example, medical device software development requires software developers that are fully conversant with developing safety-critical systems.
- This is a specialism that a typical application software developer is unlikely to possess, but which is critical for all aspects of healthcare software development.
- A music software developer will need to have musical ability, as well as a grounding in formal music theory, to be successful in developing software for the notation and performance of music.
- A security software developer needs to have more than just development skills, they must also be experienced in all aspects of IT security.
- Mobile development specializes in developing application software for mobile devices such as phones and PDAs.
- Enterprise software development is used to develop a suite of applications that are used across an organization.
- The software that is developed can remain proprietary within the organization, or be sold to others.
- It can also be made freely and widely available to anybody that wants to use it; this is called open-source development.

RECENT TRENDS IN SOFTWARE DEVELOPMENT

Q.12. What are the Recent Trends in System Software Development?

Ans. Recent Trends in System Software Development:

- There are many different software development methodologies, with many forms of both traditional waterfall development and agile development.
- The waterfall model of development, sometimes known as V model development, was formerly the most common development strategy.
- Rapid Application Development develops prototypes rapidly for testing features and functions without worrying about how the end product will be affected in the process.
- It is a perfect solution for people who have worked in the waterfall model and find it hard to change the core features and functions of the software.
- A less common model is the spiral model of development, using a risk-based approach to guide the team to select the most appropriate software development models for a particular project.

- In most software development frameworks, six stages are followed in order:

(i) Capture Requirements

(ii) Perform Analysis

(iii) Design the Solution

(iv) Code the Software

(v) Test for Bugs

(vi) Deployment and System Maintenance

- Today, the top development methodologies are all based on agile principles and methods. Any agile software development methodology follows lean development principles that are designed to maximize efficiency and throughput and eliminate waste.
- Agile development with Scrum used to be the most popular agile model, but today it is being overtaken by agile software development models that provide more flexibility where code can be deployed at any time, not just in regular increments.
- Software development is moving toward simplification, with a new approach to creating new apps known as low-code development.
- Low-code platforms enable practically anyone to develop software without traditional coding but via graphical user interfaces and configuration.
- Such programming environments have great potential because they allow people to implement their ideas into digital solutions with limited or no specific development skills.

LEVELS AND TYPES OF COMPUTER SOFTWARE

Q.13. Write a note on levels and types of Computer Software.

Ans. Following are the levels and types of computer software:

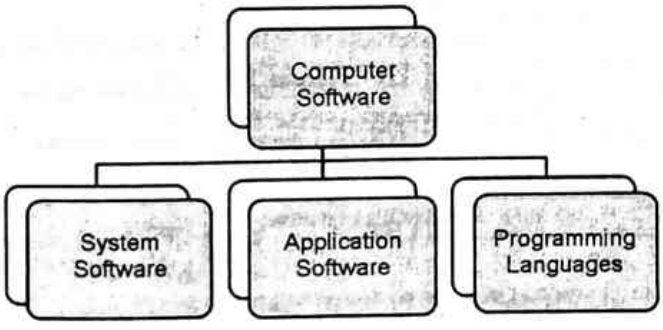


Fig. Types of Computer Software.

(i) System Software:

- The system software is a type of computer software that is designed for running the computer hardware parts and the application programs. It is the platform provided to the computer system where other computer programs can execute.
- The system software act as a middle layer between the user applications and hardware. The operating system is the type of system software. The operating system is used to manage all other programs installed on the computer.
- The other purpose of system software is to translate inputs received from other sources and convert them into language so that the machine can understand.
- The BIOS (Basic Input/Output System) is another type of system software that works when the computer system starts and is used to manage the data between the hardware devices (video adapter, mouse, keyboard and printer) and the opening system.
- The system software provides the functionality for the user to use the hardware directly using the device drivers program.
- The boot is the system software program that loads the operating system in the main memory of the computer or can load in Random Access Memory (RAM).
- The example of system software is an assembler which has the functionality to take computer instructions as input and then convert it into bits so that the processor can read that bit and perform computer operations.
- The other example of system software is a device driver which is used to control some specific device that is connected to computer systems like mouse or keyboard. The device driver software is used to convert input/output instructions of OS to messages so that the device can read and understand.
- The system software can be run in the background or can be executed directly by the user.

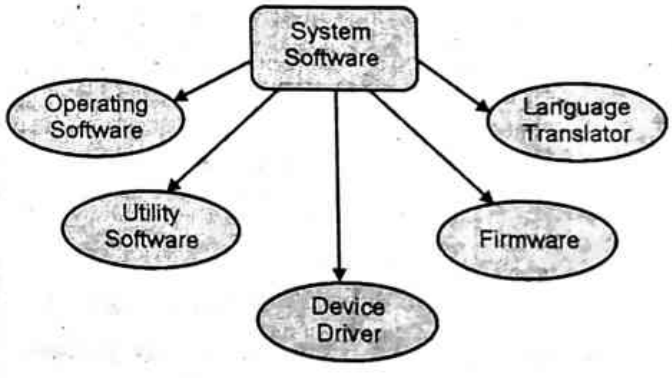


Fig. Types of System Software.

(ii) **Application Software:**

- The other category of software is application software that is designed for the users to perform some specific tasks like writing a letter, listening to music or seeing any video. For all these requirements there required a specific software for each type and that specific software that is designed for some specific purpose is known as application software. The operating software runs the application software in the computer system.
- The difference between system software and application software is the difference in the user interface. In system software, there is no user interface present whereas in application software the user interface is present for each software so that users can easily use the software.
- The user cannot see the system software like an operating system and cannot work in system software application, software users can see the application software using a graphical user interface and can also work in the application software.
- The user also has an option to create its user-written software and use the software for personal use.
- The templates are present which can be used by the user to create user-written programs. The application software can be bundled together and that bundle is known as an application suite. An example of an application suite is Microsoft Office. The word processor software is designed by combining various small programs to make one single program that can be used for writing text, creating a spreadsheet or creating presentations. The other type of application software is Mozilla Firefox, an internet explorer.
- These kinds of application software can be used for searching any article, text on the web and interacting with the outside world.

(iii) **Programming Languages:**

- The programming language is the third category of computer software which is used by the programmers to write their programs, scripts and instructions which can be executed by a computer. The other name of the programming language is a computer language that can be used to create some common standards.
- The programming language can be considered as a brick that can be used to construct computer programs and operating systems. Examples of programming languages are JAVA, C, C++ and other languages.
- There is always some similarities between the programming languages the only difference is the syntax of programming language which makes them different. The programmer uses the syntax and rules of programming languages to write their programs. Once the source code is written by a programmer in the IDE (Integrated Development Environment) the programmer then compiles that code as machine language which can be understood by the computer.
- The use of programming language is in developing a website, applications and many other programs.
- The programming language can be broadly divided into two major elements syntax and semantics. The programming language follows some sequence of operations so that the desired output can be achieved.
- The programming language is also known as high-level language as the programs written by a programmer are easy to read and easy to understand. The JAVA, C, C++ programming languages are considered high-level languages. The other category of a programming language is a low-level language.
- The low level of language includes machine language and assembly language. The assembly language contains a list of instructions that are not easy to read and understand.
- The machine language contains binary codes that can be read by the CPU directly and are not present in a human-readable form. The low level of language can be directly understood by computer hardware.

FEATURES OF SYSTEM SOFTWARE

Q.14. Enlist the features of the System Software.

Ans. Feature of system software are as follows:

(i) **High Speed:**

System software must be as efficient as possible to provide an effective platform for higher-level software in the computer system.

(ii) Hard to Manipulate:

It often requires the use of a programming language, which is more difficult to use than a more intuitive user interface (UI).

(iii) Written in a low-level Computer Language:

System software must be written in a computer language the Central Processing Unit (CPU) and other computer hardware can read.

(iv) Close to the System:

It connects directly to the hardware that enables the computer to run.

(v) Versatile:

System software must communicate with both the specialized hardware it runs on and the higher-level application software that is usually hardware-agnostic and often has no direct connection to the hardware it runs on. The system software also must support other programs that depend on it as they evolve and change.

MACHINE LANGUAGE

Q.15. Write a note on Machine Language.

Ans. Machine Language:

- Sometimes referred to as machine code or object code, machine language is a collection of binary digits or bits that the computer reads and interprets. Machine language is the only language a computer is capable of understanding.
- The exact machine language for a program or action can differ by the operating system. The specific operating system dictates how a compiler writes a program or action into machine language.
- Machine language is a language of 0 and 1, which is also called binary language. It consists of a sequence of 0 and 1. These are treated as 2 states in which the electronic components work. A core is magnetized in anticlockwise (0 state) or clockwise (1 state), a switch is either open (0) or closed (1), electrical pulses are either present (1) or absent (0).
- With the use of only 2 states, the circuit is simplified, the cost is reduced and reliability is improved. Programs are written in any other language (assembly or high-level language) are translated to machine language.
- Computer programs are written, in one or more programming languages, like C++, Java, or Visual Basic. A computer cannot directly understand the programming languages used to create computer programs, so the program code must be compiled.
- Once a program's code is compiled, the computer can understand it because the program's code is turned into machine language.
- Machine language is created by a software called "assemblers," "compilers" and "interpreters." These conversion programs turn the programmer's source code into machine language.
- A machine-level language is not portable as each computer has its machine instructions, so if we write a program in one computer will no longer be valid in another computer.
- The different processor architectures use different machine codes, for example, a PowerPC processor contains RISC architecture, which requires different code than intel x86 processor, which has a CISC architecture.

ELEMENTS OF ASSEMBLY LANGUAGE

Q.16. Write a note on Elements of Assembly Language.

Ans. Elements of Assembly Language:

- Assembly language is basically like any other language, which means that it has its words, rules and syntax.
- An assembly language is a machine-dependent language, a low-level programming language that is specific to computers.
- Each statement in assembly language is corresponding to an instruction or declaration statement or directive to the assembler.
- The basic elements of assembly language are:

Constants, Expressions and Numeric Literal:

- A numeric literal is a combination of digits and other optional parts: a sign, a decimal point and an exponent.

Here are some examples: 5, 5.5, – 5.5, 26.E + 05

Integer constants can end with a radix symbol that identifies the numeric base.

- The bases are: h = hexadecimal, q (or o) = octal, d = decimal, b = binary. If no radix is specified, decimal is the default.

  26 - decimal

  0F6h - hexadecimal

  1101b - binary

  36q - octal

- When a hexadecimal constant begins with a letter, it must contain a leading zero.

  Although the radix can be uppercase, we recommend that lowercase be used consistently for a more uniform appearance.

- A constant expression consists of combinations of numeric literals, operators and defined symbolic constants.
- The expression value must be able to be determined at assembly time, and its value cannot change at runtime. Here are some examples of expressions involving only numeric literals:

  5

  26.5

  4 * 20

  – 3 * 4 / 6

  – 2.301E + 04

- A symbolic constant is created by assigning a constant expression to a name.

  For example, rows = 5 columns = 10

  tablePosition = rows * columns

Character or String Constant:

A constant may also represent a string of characters enclosed in either single or double quotation marks.

Embedded quotes are permitted, as the examples show: 'ABC' 'X' "This is a test" '1024'

Variable:

A variable is a location in a program's data area that has been assigned a name.

For example: count1 db 50; a variable (memory allocation)

Labels:

- A label is a special type of symbol used to represent a textual version of an address in ROM or RAM memory.
- They are always placed at the beginning of a program line. It is very complicated to call a subroutine or execute some of the jump or branch instructions without them. They are easily used.

  A symbol (label) with some easily recognizable name should be written at the beginning of a program line from which a subroutine starts or where the jump should be executed.

- It is sufficient to enter the name of the label instead of the address in the form of a 16-bit number in instructions calling a subroutine or jump.
- During the process of compiling, the assembler automatically replaces such symbols with appropriate addresses.

  LabelX: MOV AX,0

  MOV BX,0

Names:

A name identifies a label, variable, symbol, or keyword. It may contain any of the following characters:

Character	Description
A . . . z, a . . . z	Letters
0 . . . 9	Digits
?	Question mark
_	Underscore
@	@ Sign
$	Dollar sign
.	Period

- A maximum of 247 characters in MASM.
- There is no distinction between uppercase and lowercase letters.
- The first character can be a letter, '@', '_', or '$'. Subsequent characters can be the same, or they can also be decimal digits.
- Avoid using '@' as the first character, because many predefined symbol names start with it.
- A programmer-chosen name cannot be the same as an assembler reserved word.

Statements:

- An assembly language statement consists of a name, an instruction mnemonic, operands and a comment. Statements generally fall into two classes, instructions and directives.
- Instructions are executable statements, and directives are statements that provide information to tell the assembler how to generate executable code.
- The general format of a statement is:

  [name] [mnemonic] [operands] [;comment]
- Statements are free-form, meaning that they can be written in any column with any number of spaces between each operand.
- Blank lines are permitted between statements.
- A statement must be written on a single line and cannot pass column 128.
- We can continue a line onto the next line if the last character in the first line is \ (backslash):

Directives:

- Unlike instructions being compiled and written to chip program memory, directives are commands of assembly language itself and do not influence the operation of the microcontroller.
- Some of them are an obligatory part of every program while some are used only to facilitate or speed up the operation. Directives are written in the column reserved for instructions. There is a rule allowing only one directive per program line.

(a) **ORIGIN:** The syntax of this directive is

ORIGIN < address spec >

This directive indicates that **Location Counter (LC)** should be set to the address given by < address spec >.

LC: It will be the same as that of the program counter which points address of the next instruction to execute.

The ORIGIN statement is useful when the target program does not consist of consecutive memory words.

e.g. ORIGIN 200: - Location counter will shift to ROM location 200.

(b) **EQU:** The syntax of this directive is

< symbol > EQU < address spec >

e.g. A EQU 100: - A is assigned to address spec 100.

(c) **LTORG:**

- This directive is used to assign memory locations to immediate value in between program.

  If we are not using LTORG then for immediate value are getting locations after the END statement.
- It permits the programmer to specify where literals should be placed by default assembler places the literals after END statement.
- At every LTORG and END statement, assembler allocates memory to literals as a literal pool.

  This pool consists all the literals used in a program since the start of the program or since the last LTORG statement.

(d) **IF, ENDIF and ELSE**

- These directives are used to create so-called conditional blocks in the program. Each of these blocks starts with directive IF and ends with directive ENDIF or ELSE.
- The statement or symbol (in parentheses) following the IF directive represents a condition which specifies the part of the program to be compiled:

(i) If the statement is correct or if the symbol is equal to one, the program will include all instructions up to directive ELSE or ENDIF.

(ii) If the statement is not correct or if the symbol value is equal to zero, all instructions are ignored, i.e. not compiled, and the program continues with instructions following directives ELSE or ENDIF.

(e) **Control Directives:**

Control directives start with a dollar symbol $. They are used to determine which files are to be used by the assembler during compilation, where the executable file is to be stored as well as the final layout of the compiled program called Listing. There are many control directives, but only a few of them is of importance.

(f) **SINCLUDE Directive:**

This directive enables the assembler to use data stored in other files during compilation. For example:

#INCLUDE (TABLE.H)

Program header with basic data

```
;Example of a program which generates a sequence of pulses with
;the frequency of 1KHz. The output pin is P0.1.

        SMOD8253                                ;Program is written for 8253 MCU
        DSEG                                    ;Next segment refers to internal RAM
                    ORG     20h
        Var1        DS      1                   ;Byte at location 20h is reserved
        STATE       BIT     Var1.0              ;Bit "STATE" is assigned an address
        OUTPUT      BIT     P1.0                ;Bit "OUTPUT" is assigned an address

        CSEG                                    ;Next segment refers to program memory
                    ORG     0h                  ;Program starts at address 0000h
                    AJMP    START               ;Jump to the lable "START"

                    ORG     0Bh                 ;Timer T0 interrupt-vector address
                    AJMP    INTERRUPT           ;Jump to the lable "INTERRUPT"

        START       MOV     IE,#82h             ;Interrupt enabled on Timer T0 overflow
                    MOV     TMOD,#01            ;16-bit Timer
                    MOV     TH0,#FEh            ;Starting value of Timer is FE0Ch
                    MOV     TL0,#0Ch
                    SETB    STATE               ;Bit "STATE" is set
                    SETB    TR0                 ;Timer T0 starts operating

        LOOP        NOP
                    SJMP    LOOP                ;Program remains in endless loop

        INTERRUPT   CLR     TR0                 ;Timer must be stopped before overflow
                    MOV     TH0,#FEh            ;Timer T0 starting value is rewritten
                    MOV     TL0,#0Ch
                    SETB    TR0                 ;Timer starts recounting
                    CPL     STATE               ;Current state is complemented
                    MOV     C,STATE             ;Bit"STATE" is coppied to C bit
                    MOV     OUTPUT,C            ;C bit is coppied to bit "OUTPUT"
                    RETI                        ;Return from interrupt routine
                    END
```

Directives

Labels

Instructions (mnemonics) (operands)

Comments

Q.17. Differentiate between Machine Language and Assembly Language.

Ans.

Sr. No.	Parameters	Assembly Language	Machine Language
(1)	Code representation	Symbolic uses English-like language.	Combination of 0's and 1's.
(2)	Readability	The readability of assembly language is very high as it is written in English which is understandable by human beings.	The readability of machine language is less as a compared to assembly language because it is written in the form of binary code which normal human beings cannot understand. But this is the actual language of the computer.

(3)	**Language generation**	Second generation language is known as assembly language.	First-generation language is known as machine language.
(4)	**Platform dependency**	The assembly language is platform-dependent and so at present most programs are written in the third-generation language.	Machine language varies from the platform. Different platforms have different machine language codes.
(5)	**Modifiable**	It is necessary for the survival of software to evolve. So in order to avail this advantage the language should be easily modifiable. And assembly language is easily modifiable and its support changes.	Machine language cannot be changed easily and it does not support modification.
(6)	**Quality**	The risk of occurrence of error is reduced in assembly language.	The risk of the existence of error is high in machine language.
(7)	**Memorability**	Memorability is high in assembly language because it is easy to remember variable names instead of binary code.	The binary code cannot be memorized.
(8)	**Need of Translator**	In the case of assembly language, an interpreter is needed which is known as an assembler in the case of assembly language to convert its code into machine code in the form of bits and bytes.	In the case of machine language, there is no need for a compiler or interpreter because it is the mother language of a computer. So the computer does not need an interpreter to understand its language.
	Example:	L 2, FOUR (i.e. L 2, 924 (0,1))	5820139C (010110000010000000001001110011100)

Q.18. Give the differences between Compiler and Assembler.

Ans.

Sr. No.	Compiler	Assembler
(1)	Compiler converts the high-level language source code into machine level language code.	An assembler converts the assembly level language code into the machine level language code.
(2)	The input of a compiler is high-level language source code.	Its input is low-level assembly code.
(3)	Compiler converts the whole source code to machine code at once.	The assembler does not convert the code in one go.
(4)	It has the following phases: lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization, code generation and error handling.	An assembler completes the task in two passes.
(5)	It produces a machine code in form of mnemonics.	It produces binary code in form of 0s and 1s.
(6)	Examples are Java, C, C++ etc. compilers.	Examples of assemblers are GAS, GNU etc.

FINITE-STATE AUTOMATA IN LEXICAL ANALYSIS

Q.19. Write about Finite State Automata in Lexical Analysis.

Ans. Finite State Automata in Lexical Analysis:

- A finite automaton (FA) is a simple idealized machine used to recognize patterns within input taken from some character set (or alphabet) C.
- The job of an FA is to accept or reject an input depending on whether the pattern defined by the FA occurs in the input.
- Finite automata are a state machine that takes a string of symbols as input and changes its state accordingly.
- Finite automata come in two flavours:

(i) Nondeterministic finite automata (NFA) have no restrictions on the labels of their edges. A symbol can label several edges out of the same state, and e, the empty string, is a possible label.

(ii) Deterministic finite automata (DFA) have, for each state, and each symbol of its input alphabet exactly one edge with that symbol leaving that state.

- Both deterministic and nondeterministic finite automata are capable of recognizing the same languages.
- When a regular expression string is fed into finite automata, it changes its state for each literal.
- If the input string is successfully processed and the automata reach its final state, it is accepted, i.e. the string just fed was said to be a valid token of the language in hand.
- As the first phase of a compiler, the main task of the lexical analyzer is to read the input characters of the source program, group them into lexemes, and produce as output a sequence of tokens for each lexeme in the source program. The stream of tokens is sent to the parser for syntax analysis.
- The mathematical model of finite automata consists of:

(i) Finite set of states (Q)

(ii) Finite set of input symbols (Σ)

(iii) One Start state (q0)

(iv) Set of final states (qf)

(v) Transition function (δ)

- The transition function (δ) maps the finite set of state (Q) to a finite set of input symbols (Σ), $Q \times \Sigma \rightarrow Q$ Finite Automata Construction
- Let L(r) be a regular language recognized by some finite automata (FA).

States: the States of FA are represented by circles. State names are written inside circles.

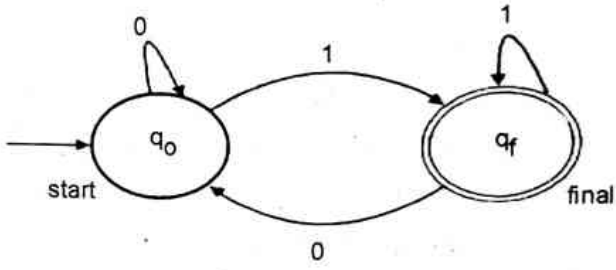
Start state: The state from where the automata start, is known as the start state. Start state has an arrow pointing towards it.

Intermediate states: All intermediate states have at least two arrows; one pointing to and another pointing out from them.

Final state: If the input string is successfully parsed, the automata is expected to be in this state. The final state is represented by double circles. It may have any odd number of arrows pointing to it and even many arrows pointing out from it. The number of odd arrows is one greater than even, i.e. odd = even+1.

Transition: The transition from one state to another state happens when a desired symbol in the input is found. Upon transition, automata can either move to the next state or stay in the same state. Movement from one state to another is shown as a directed arrow, where the arrows point to the destination state. If automata stay on the same state, an arrow pointing from a state to itself is drawn.

Example: We assume FA accepts any three-digit binary value ending in digit 1. FA = {Q (q_0, q_f), Σ (0,1), q_0, q_f, δ}



Disadvantages of Finite State Machine:

(i) The expected character of deterministic finite state machines can be not needed in some areas like computer games.

(ii) The implementation of huge systems using FSM is hard for managing without any idea of design.

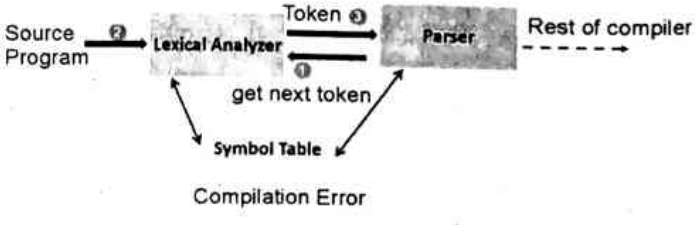
(iii) Not applicable for all domains.

DESIGN OF LEXICAL ANALYZER

Q.20. Explain the design of Lexical Analyzer.

Ans. Design of Lexical Analyzer:

- The main task of lexical analysis is to read input characters in the code and produce tokens.
- Lexical analyzer scans the entire source code of the program. It identifies each token one by one.
- Scanners are usually implemented to produce tokens only when requested by a parser.
- Here is how recognition of tokens in compiler design works



(i) "Get next token" is a command which is sent from the parser to the lexical analyzer.

(ii) On receiving this command, the lexical analyzer scans the input until it finds the next token.

(iii) It returns the token to Parser.

- Lexical Analyzer skips white spaces and comments while creating these tokens. If any error is present, then the Lexical analyzer will correlate that error with the source file and line number.

Roles of the Lexical Analyzer:

The lexical analyzer performs below given tasks:

(i) Helps to identify tokens into the symbol table.

(ii) Removes white spaces and comments from the source program.

(iii) Correlates error messages with the source program.

(iv) Helps you to expand the macros if it is found in the source program.

(v) Read input characters from the source program.

Advantages of Lexical Analysis:

(i) Lexical analyzer method is used by programs like compilers which can use the parsed data from a programmer's code to create a compiled binary executable code.

(ii) It is used by web browsers to format and display a web page with the help of parsed data from JavsScript, HTML, CSS.

(iii) A separate lexical analyzer helps you to construct a specialized and potentially more efficient processor for the task.

Disadvantages of Lexical Analysis:

(i) Need to spend significant time reading the source program and partitioning it in the form of tokens.

(ii) Some regular expressions are quite difficult to understand compared to PEG or EBNF rules.

(iii) More effort is needed to develop and debug the lexer and its token descriptions.

(iv) Additional runtime overhead is required to generate the lexer tables and construct the tokens.

Issues in Lexical Analysis:

- Simpler design is the most important consideration. The separation of lexical analysis from syntax analysis often allows us to simplify one or the other of these phases.
- Compiler efficiency is improved.
- Compiler portability is enhanced.

ROLE OF CONTEXT-FREE GRAMMER IN SYNTAX ANALYSIS

Q.21. Give the role of Context-Free Grammer in Syntax Analysis.

Ans. Context-Free Grammars:

- A regular expression is a concept in formal language theory which is a sequence of characters that define a search pattern.
- Context-Free Grammar is a type of formal grammar in formal language theory, which is a set of production rules that describe all possible strings in a given formal language.
- Context-free grammar can generate context-free languages. They do this by taking a set of variables that are defined recursively, in terms of one another, by a set of production rules.
- Context-free grammar is named as such because any of the production rules in the grammar can be applied regardless of the context it does not depend on any other symbols that may or may not be around a given symbol that is having a rule applied to it.
- A context-free grammar can be generated by pushdown automata just as regular languages can be generated by finite state machines. Since all regular languages can be generated by CFGs, all regular languages can too be generated by pushdown automata.
- Any language that can be generated using regular expressions can be generated by context-free grammar.
- The way to do this is to take the regular language, determine its finite state machine and write production rules that follow the transition functions.
- In some cases, like C++, the language definition will note that the grammar given is actually a superset of the language because it specifies programs that will still be rejected by the compiler. So real languages tend to be a little bit fuzzy around the edges and not precisely defined by context-free grammar.
- A compiler will typically use a parser generator that can take a context-free language description as input. However, parser generators may be more limited in what they can do than full context-free languages;
- The generator may have additional restrictions to eliminate ambiguity or limit the amount of lookahead that must be done.
- Syntactic analysis, or parsing, is needed to determine if the series of tokens given are appropriate in a language - that is, whether or not the sentence has the right shape/form.

- However, not all syntactically valid sentences are meaningful, further semantic analysis has to be applied for this. For syntactic analysis, context-free grammars and the associated parsing techniques are powerful enough to be used - this overall process is called parsing.

Components:

Context-Free Grammars have the following components:

- A set of terminal symbols which are the characters that appear in the language/strings generated by the grammar. Terminal symbols never appear on the left-hand side of the production rule and are always on the right-hand side.
- A set of nonterminal symbols (or variables) which are placeholders for patterns of terminal symbols that can be generated by the nonterminal symbols. These are the symbols that will always appear on the left-hand side of the production rules, though they can be included on the right-hand side. The strings that a CFG produces will contain only symbols from the set of nonterminal symbols.
- A set of production rules which are the rules for replacing nonterminal symbols. Production rules have the following form: a variable → string of variables and terminals.
- A start symbol which is a special nonterminal symbol that appears in the initial string generated by the grammar.
- Context-Free Grammar can be described by a four-element tuple (V, Σ, R, S), where

V is a finite set of variables (which are non-terminal);

Σ is a finite set (disjoint from V) of terminal symbols;

R is a set of production rules where each production rule maps a variable to a string $s \in (V \cup \Sigma)^*$;

S (which is in V) which is a start symbol.

- To create a string from a Context-Free Grammar, follow these steps:

(i) Begin the string with a start symbol.

(ii) Apply one of the production rules to the start symbol on the left-hand side by replacing the start symbol with the right-hand side of the production.

(iii) Repeat the process of selecting nonterminal symbols in the string, and replacing them with the right-hand side of some corresponding production, until all nonterminal has been replaced by terminal symbols.

INTERPRETER: OVERVIEW AND BENEFITS

Q.22. What is an Interpreter? Give types, advantages and disadvantages.

Ans. Interpreters/Interpretation:

- All high-level languages need to be converted to machine code so that the computer can understand the program after taking the required inputs.
- The software by which the conversion of the high-level instructions is performed line-by-line to machine level language, other than compiler and assembler, is known as Interpreter.
- If an error is found on any line, the execution stops till it is corrected.
- This process of correcting errors is easier as it gives a line-by-line error but the program takes more time to execute successfully.
- An Interpreter directly executes instructions written in a programming or scripting language without previously converting them to an object code or machine code.
- Interpreters make working with the source code much easier. Hence, they are highly suitable, especially for beginners.
- On the negative side, interpreted programs can only run on the computers that have the respective interpreters.

Types of an Interpreter in Programming:

(i) Bytecode Interpreters:

- The source code is firstly converted to bytecode.
- Bytecode is a compressed and optimized representation of source code. But it is not the machine code.
- The bytecode interpreters then execute this compiled code
- Compiler and interpreter both are used hence, the name 'compreters'.
- Each instruction starts with a byte. Therefore, they have up to 256 instructions.

(ii) Threaded Code Interpreters:

- Similar to bytecode interpreters but, they use pointers.
- Each instruction is a word acting as a pointer. This pointer points to a function or instruction sequence.

- There is no restriction on the number of instructions. Considering that, memory and address space is available.

(iii) **Abstract Syntax Tree Interpreters:**

- It converts the source code to an abstract syntax tree (AST). Then it executes the program according to this tree.
- Each sentence is parsed just once.
- The program structure and the relation between statements remain the same.
- Provides better analysis during run time

(iv) **Self-Interpreters:**

- These are special types of interpreters.
- They are programming language interpreters written in a programming language that can interpret itself.
- An example can be a BASIC interpreter written in BASIC.
- Self-interpreters are created in case if no compiler exists for a language.
- Their creation requires the implementation of that language in a host language. This host language can be another programming language.

Advantages:

(i) **Cross-Platform:** In interpreted language, we directly share the source code which can run on any system without any system incompatibility issue.

(ii) **Easier To Debug:** Code debugging is easier in interpreters since it reads the code line by line, and returns the error message on the spot. Also, the client with the source code can debug or modify the code easily if they need to.

(iii) **Less Memory and Step:** Unlike the compiler, interpreters don't generate new separate files. So, it doesn't take extra Memory and we don't need to perform one extra step to execute the source code, it is executed on the fly.

(iv) **Execution Control:** Interpreter reads code line by line so you can stop the execution and edit the code at any point, which is not possible in a compiled language. However, after being stopped it will start from the beginning if you execute the code again.

(v) An interpreter takes less time to analyze source code.

Disadvantages:

(i) **Slower in operation:** Interpreter is often slower than compiler as it reads, analyzes and converts the code line by line.

(ii) A client or anyone with the shared source code needs to have an interpreter installed in their system, to execute the code.

(iii) **Less Secure:** Unlike compiled languages, an interpreter doesn't generate any executable file so to share the program with others we need to share our source code which is not secure and private. So, it is not good for any company or corporations who are concerned about their privacy.

(iv) Interpreted programs can run on only those computers which have the same interpreter.

(v) The interpreter doesn't generate a separate machine code as an output program. So, it checks the source code every time during the execution.

Q.23. Give the difference between Compiler and Interpreter in detail.

Ans. Difference between Compiler and Interpreter:

Parameter	Compiler	Interpreter
Programming Steps	Write a program in source code. The compiler will analyze your program statements and check their correctness. If an error is found in a program, it throws an error message. If the program contains no error, then the compiler will convert the source code program into machine code. The compiler links all the code files into a single runnable program, which is known as the exe file. Finally, it runs the program and generates output.	Write a program in source code. No linking of files happens, or no machine code will generate separately. The source code programming statements are executed line-by-line during their execution. If an error is found at any specific statement interpreter, it stops further execution until the error gets removed.
Translation type	A compiler translates complete high-level programming code into machine code at once.	An interpreter translates one statement of programming code at a time into machine code.

Advantage	As the source code is already converted into machine code, the code execution time becomes short.	As the source code is interpreted line-by-line, error detection and correction become easy.
Disadvantage	If you want to change your program for any reason, either by error or logical changes, you can do it only by going back to your source code.	Interpreted programs can run on only those computers which have the same interpreter.
Machine code	It stores the converted machine code from your source code program on the disk.	It never stores the machine code at all on the disk.
Running time	A compiler takes an enormous time to analyze source code. However, overall compiled programming code runs faster as compression to an interpreter.	An interpreter takes less time to analyze source code as compared to a compiler. However, overall interpreted programming code runs slower as compression to the compiler.
Program generation	The compiler generates an output of a program (in the form of an exe file) that can run separately from the source code program.	The interpreter doesn't generate a separate machine code as an output program. So, it checks the source code every time during the execution.
Execution	The process of program execution takes place separately from its compilation process. Program execution only takes place after the complete program is compiled.	The process of program execution is a part of interpretation steps, so it is done line-by-line simultaneously.
Memory requirement	A compiled program is generated into an intermediate object code, and it further required linking. So, there is a requirement for more memory.	An interpreted program does not generate an intermediate code. So, there is no requirement for extra memory.
Best suited for	The compiled program is bounded to the specific target machine. It requires the same compiler on the machine to execute; C and C++ are the most popular programming language based on the compilation model.	In web environments, compiling takes place relatively more time to run even small code, which may not run multiple times. As load time is essential in the web environment, interpreters are better. JavaScript, Python, Ruby is based on the interpreter model.
Error execution	The compiler shows the complete errors and warning messages at program compilation time. So, it is not possible to run the program without fixing program errors. Doing debugging of the program is comparatively complex while working with a compiler.	An interpreter reads the program line-by-line; it shows the error if present at that specific line. You must have to correct the error first to interpret the next line of the program. Debugging is comparatively easy while working with an Interpreter

JAVA LANGUAGE ENVIRONMENT

Q.24. What is the Java Language?

Ans. Java Language:

- Java programs are written as source code in the Java language. This is a human-readable programming language, which is class-based and object-oriented. It is considered to be relatively easy to read and write (if occasionally a bit verbose).

- Java is intended to be easy to learn and to teach. It builds on industry experience with languages like C++ and tries to remove complex features as well as preserve "what works" from previous programming languages.
- Overall, Java is intended to provide a stable, solid base for companies to develop business-critical applications.
- As a programming language, it has a relatively conservative design and a slow rate of change. These properties are a conscious attempt to serve the goal of protecting the investment that businesses have made in Java technology.
- The language has undergone gradual revision since its inception in 1996.
- Java 8 has added the most radical changes seen in the language for almost a decade. Features like lambda expressions and the overhaul of the core collections code will change forever the way that most Java developers write code.
- The Java language is governed by the Java Language Specification (JLS), which defines how a conforming implementation must behave.

Q.25. What is the Java Ecosystem?

Ans. Java Ecosystem:

- The Java language is easy to learn and contains relatively few abstractions, compared to other programming languages. The JVM provides a solid, portable, high-performance base for Java (or other languages) to execute on.
- Taken together, these two connected technologies provide a foundation that businesses can feel confident about when choosing where to base their development efforts.
- Since Java's inception, an extremely large ecosystem of third-party libraries and components has grown up. This means that a development team can benefit hugely from the existence of connectors and drivers for practically every technology imaginable-both proprietary and open source.
- In the modern technology ecosystem, it is now rare indeed to find a technology component that does *not* offer a Java connector.
- From traditional relational databases to NoSQL, to every type of enterprise monitoring system, to messaging systems - everything integrates with Java.
- It is this fact that has been a major driver of the adoption of Java technologies by enterprises and larger companies. Development teams have been able to unlock their potential by making use of preexisting libraries and components.
- This has promoted developer choice and encouraged open, best-of-breed architectures with Java technology cores.

Q.26. What is the Java Programming Environment?

Ans. Java Programming Environment:

- Java is a general-purpose, class-based, object-oriented programming language designed for having lesser implementation dependencies. It is a computing platform for application development.
- Java is fast, secure, and reliable, therefore. It is widely used for developing Java applications in laptops, data centres, game consoles, scientific supercomputers, cell phones, etc.
- The Java language environment results in software applications that are portable, highly secure and high performance.
- Java runtime environment, consisting of:

(i) A programming language

(ii) An API specification

(iii) A virtual machine specification

- Java has the following characteristics:

(i) **Object-Oriented:**

Java provides the basic object technology of C++ with some enhancements and some deletions.

(ii) **Architecture Neutral:**

Java source code is compiled into architecture-independent object code. The object code is interpreted by a Java Virtual Machine (JVM) on the target architecture.

(iii) **Portable:**

Java implements additional portability standards. For example, ints are always 32-bit, 2's-complemented integers. User interfaces are built through an abstract window system that is readily implemented in Solaris and other operating environments.

(iv) **Distributed:**

Java contains extensive TCP/IP networking facilities. Library routines support protocols such as HyperText Transfer Protocol (HTTP) and file transfer protocol (FTP).

(v) **Robust:**

Both the Java compiler and the Java interpreter provide extensive error checking. Java manages all dynamic memory, checks array bounds, and other exceptions.

(vi) **Secure:**

- Features of C and C++ that often result in illegal memory accesses are not in the Java language. The interpreter also applies several tests to the compiled code to check for illegal code.
- After these tests, the compiled code causes no operand stack over-or underflows, performs no illegal data conversions, performs only legal object field accesses, and all opcode parameter types are verified as legal.

(vii) **High Performance:**

Compilation of programs to architecture-independent machine-like language results in a small efficient interpreter of Java programs. The Java environment also compiles the Java bytecode into native machine code at runtime.

(viii) **Multithreaded:**

Multithreading is built into the Java language. It can improve interactive performance by allowing operations, such as loading an image, to be performed while continuing to process user actions.

(ix) **Dynamic:**

Java does not link invoked modules until runtime.

(x) **Simple:**

Java is similar to C++, but with most of the more complex features of C and C++ removed.

- Java does not provide:

(i) Programme-controlled dynamic memory,

(ii) Pointer arithmetic,

(iii) struct,

(iv) typedefs,

(v) #define.

JAVA VIRTUAL MACHINE (JVM)

Q.27. Write a note on Java Virtual Machine (JVM).

Ans. Java Virtual Machine (JVM):

- Java Virtual Machine (JVM) is an engine that provides a runtime environment to drive the Java Code or applications.
- It converts Java bytecode into machine language. JVM is a part of the Java Run Environment (JRE).
- In other programming languages, the compiler produces machine code for a particular system. However, the Java compiler produces code for a Virtual Machine known as Java Virtual Machine.
- Java applications are primarily called Write once Run Anywhere (WORA) because of their ability to run on any platform.
- JVM offers a runtime environment for executing Java Code or applications, further converting Java bytecode into machine language. Usually, in other programming languages, the compiler produces machine code for a specific system, but the Java compiler initiates for JVM.
- Besides being a part of the Java Run Environment (JRE), JVM boasts an array of tools, frameworks, and libraries. As one of the key components of the Java platform, JVM executes source code with platform independence, which is why you can run a Java program on any platform without any restrictions.
- JVM also has a Just-in-Time (JIT) compiler to convert Java source code into low-level machine language, which explains the fast speed of Java applications.
- The JVM is an abstract computing machine, having an instruction set that uses memory. Virtual machines are often used to implement a programming language.
- The JVM is the cornerstone of the Java programming language. It is responsible for Java's cross-platform portability and the small size of its compiled code.
- The JVM is used to execute Java applications. The Java compiler, javac, outputs bytecodes and puts them into a .class file. The JVM then interprets these bytecodes, which can then be executed by any JVM implementation, thus providing Java's cross-platform portability.
- The next two figures illustrate the traditional compile-time environment and the new portable Java compile-time environment.

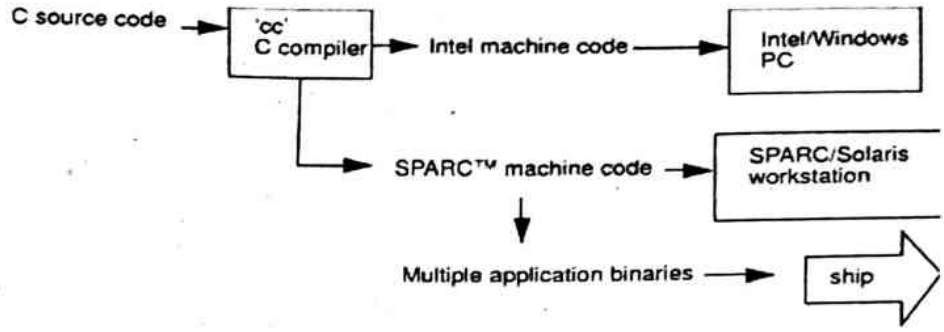


Fig. Traditional Compile-Time Environment.

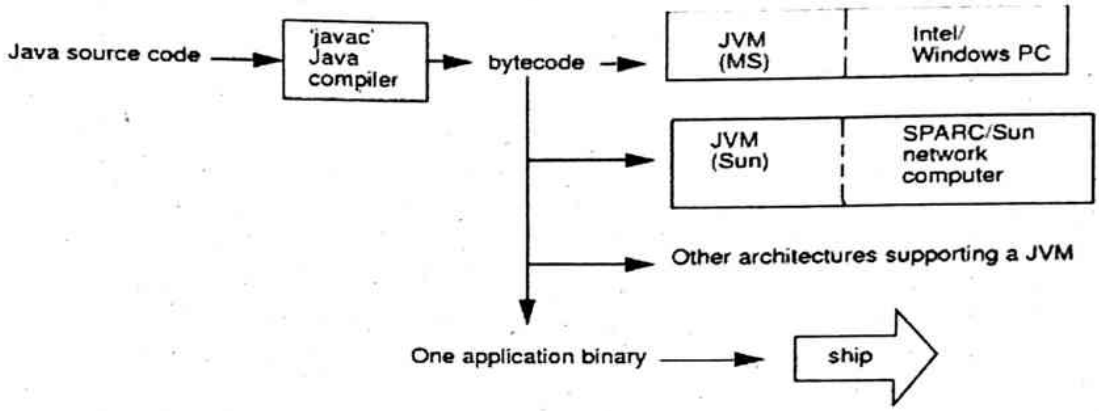


Fig. Portable Java Compile-Time Environment.

Components of JVM

- Class Loader Subsystem,
- Loading,
- Initialization,
- Linking,
- Stack,
- Runtime Data Area, which includes heap area, method area, etc.

Features of JVM:

(i) JVM provides a platform-independent way of executing Java source code.

(ii) It has numerous libraries, tools, and frameworks.

(iii) Once you run a Java program, you can run it on any platform and save lots of time.

(iv) JVM comes with JIT (Just-in-Time) compiler that converts Java source code into low-level machine language. Hence, it runs faster than a regular application.

Multithreading JVM:

- The Java programming language requires that multithreading (MT) programs be supported. All Java interpreters provide an MT programming environment.
- However, many of these interpreters support only uniprocessor multithreading, so Java program threads are executed one at a time.
- The JVM interpreter takes full advantage of multiprocessor systems by using the intrinsic Solaris multithread facilities. These allow multiple threads of a single process to be scheduled simultaneously onto multiple CPUs.

- An MT Java program run under the Solaris JVM will have a substantial increase in concurrency over the same program run on other platforms.

Java Runtime Environment (JRE):

- JRE is responsible to run a Java program. All-in-all, JRE is installed on the computer system that takes hold of Java code, blends it with the desired libraries, and allows JVM to execute it.
- A programmer needs JDK, and if you are one, then you will need JDK. However, JRE's role is limited to running Java programs as it doesn't flaunt tools related to development like compiler or debugger.
- But if you wish to run Java applets, it is imperative to have JRE installed in your system. As it is platform-dependent, you have to download and install JRE that is compatible with your operating system.
- Apart from this, JRE has class libraries, JVM, and utilizes various package classes like swing, math, util, lang, and runtime libraries.

Java components list in JRE:

- Code libraries, resource files as well as property settings
- DLL files
- When it comes to extension, Java component example includes files related to locale specification like localedata.jar
- Security management files like java. security
- Applet support classes
- True Type font files

TYPES OF ERRORS IN JAVA

Q.28. What are the types of Errors in Java?

Ans. Types of Errors in Java:

- Error is an illegal operation performed by the user which results in the abnormal working of the program.
- Programming errors often remain undetected until the program is compiled or executed.
- Some of the errors inhibit the program from getting compiled or executed.
- Thus, errors should be removed before compiling and executing.
- The most common errors can be broadly classified as follows:

(a) Run Time Error:

- Runtime errors are errors that cause a program to terminate abnormally.
- They occur while a program is running if the environment detects an operation that is impossible to carry out.
- Input mistakes typically cause runtime errors.
- Run Time errors occur or we can say, are detected during the execution of the program. Sometimes these are discovered when the user enters invalid data or data which is not relevant.
- Runtime errors occur when a program does not contain any syntax errors but asks the computer to do something that the computer is unable to reliably do.
- During compilation, the compiler has no technique to detect these kinds of errors. It is the JVM (Java Virtual Machine) that detects it while the program is running.
- Exception is a runtime error that can be handled or prevented and occurs during the execution of the program.
- Some exceptions can be detected by the java compiler while compiling and are called checked exceptions while those detected during runtime by JVM are called unchecked exceptions.
- To handle the error during the run time we can put our error code inside the try block and catch the error inside the catch block.

Example 1: If the user inputs a data of string format when the computer is expecting an integer, there will be a runtime error.

Example 2: Runtime Error caused by dividing by zero.

Example 3: Runtime Error caused by Assigning/ Retrieving Value from an array using an index that is greater than the size of the array.

Example 4: An input error occurs when the program is waiting for the user to enter a value, but the user enters a value that the program cannot handle.

(b) Compile Time Error:

- Compile Time Errors are sometimes also referred to as Syntax errors.
- Compile Time Errors are those errors that prevent the code from running because of an incorrect syntax such as a missing semicolon at the end of a statement or a missing bracket, class not found, etc.
- These errors are detected by the java compiler and an error message is displayed onto the screen while compiling.

- These kinds of errors are easy to spot and rectify because the java compiler finds them for you.
- The compiler will tell the user which piece of code in the program got in trouble and its best guess as to what you did wrong.
- Usually, the compiler indicates the exact line where the error is, or sometimes the line just before it, however, if the problem is with incorrectly nested braces, the actual error may be at the beginning of the block.
- In effect, syntax errors represent grammatical errors in the use of the programming language.
- Since a single error will often display many lines of compile errors, it is a good practice to fix errors from the top line and work downward. Fixing errors that occur earlier in the program may also fix additional errors that occur later.

Example 1: Misspelled variable name or method names

Example 2: Missing semicolons, parenthesis, square brackets, or curly braces

Example 3: Incorrect format of selection statements or loops

(c) **Logical Error:**

- Logical errors are also called Semantic Errors.
- Logic errors occur when a program does not perform the way it was intended to. When a program compiles and executes, but does the wrong thing or returns an incorrect result or no output when it should be returning an output.
- These errors are detected neither by compiler nor by JVM. The Java system has no idea what the program is supposed to do, so it provides no additional information to help you find the error.
- These errors are caused due to an incorrect idea or concept used by a programmer while coding.
- Syntax errors are grammatical errors whereas, logical errors are errors arising out of an incorrect meaning.

Example 1: if a programmer accidentally adds two variables when he or she meant to divide them, the program will give no error and will execute successfully but with an incorrect result.

Example 2: Accidentally using an incorrect operator on the variables to perform an operation (Using '/' operator to get the modulus instead of using '%')

Example 3: Displaying the wrong message

DEBUGGING PROCEDURES

Q.29. What is Debugging? Explain in detail.

Ans. Debugging:

- Debugging is the process of finding and resolving defects or problems within a computer program that prevent the correct operation of computer software or a system.
- Once errors are known during a program code, it's necessary to initial establish the precise program statements liable for the errors and so to repair them.

Challenges in Debugging:

- Debugging is finished through the individual that evolved the software program and it's miles difficult for that person to acknowledge that an error was made.
- Debugging is typically performed under a tremendous amount of pressure to fix the supported error as quickly as possible.
- It can be difficult to accurately reproduce input conditions.
- Compared to the alternative software program improvement activities, relatively little research, literature and formal preparation exist at the procedure of debugging.

Benefits of Debugging:

(i) Debugging can immediately report an error condition whenever it occurs.

(ii) It prevents hampering the result by detecting the bugs in the earlier stage, making software development stress-free and smooth.

(iii) It offers relevant information related to the data structures that further helps in easier interpretation.

(iv) Debugging assist the developer in reducing impractical and disrupting information.

(v) With debugging, the developer can easily avoid complex one-use testing code to save time and energy in software development.

Steps involved in Debugging:

Following are the different steps that are involved in debugging:

(i) **Identify the Error:**

Identifying an error in a wrong may result in the wastage of time. It is very obvious that the production errors reported by users are hard to interpret, and sometimes the information we receive is misleading. Thus, it is mandatory to identify the actual error.

(ii) **Find the Error Location:**

Once the error is correctly discovered, you will be required to thoroughly review the code repeatedly to locate the position of the error. In general, this step focuses on finding the error rather than perceiving it.

(iii) **Analyze the Error:**

The third step comprises error analysis, a bottom-up approach that starts from the location of the error followed by analyzing the code. This step makes it easier to comprehend the errors. Mainly error analysis has two significant goals, i.e. evaluation of errors all over again to find existing bugs and postulating the uncertainty of incoming collateral damage in a fix.

(iv) **Prove the Analysis:**

After analyzing the primary bugs, it is necessary to look for some extra errors that may show up on the application. By incorporating the test framework, the fourth step is used to write automated tests for such areas.

(v) **Cover Lateral Damage:**

The fifth phase is about accumulating all of the unit tests for the code that requires modification. As when you run these unit tests, they must pass.

(vi) **Fix and Validate:**

The last stage is the fix and validation that emphasizes fixing the bugs followed by running all the test scripts to check whether they pass.

Debugging Approaches:

- When it comes to debugging, the tools and techniques we use are essential and can determine how easy it is to fix problems within our code.
- Following are the few popular methods:

(i) **Brute Force Method:**

This is the foremost common technique of debugging however is the least economical method. During this approach, the program is loaded with print statements to print the intermediate values with the hope that a number of the written values can facilitate to spot the statement in error.

- This approach becomes a lot of systematic with the utilization of a symbolic program (also known as a source code debugger). as a result of values of various variables will be simply checked and breakpoints and watch-points can be easily set to check the values of variables effortlessly.

(ii) **Backtracking:**

- This is additionally a reasonably common approach. during this approach, starting from the statement at which an error symptom has been discovered, the source code is derived backwards till the error is discovered.
- Due to the variety of supply lines to be derived back will increase, the quantity of potential backward methods will increase and should become unimaginably large so limiting the utilization of this approach.

(iii) **Cause Elimination Method:**

- In this approach, a listing of causes that may presumably have contributed to the error symptom is developed and tests are conducted to eliminate every error.
- A connected technique of identification of the error from the error symptom is that the package fault tree analysis.

(iv) **Program Slicing:**

- This technique is analogous to backtracking. Here the search house is reduced by process slices.
- A slice of a program for a specific variable at a particular statement is that the set of supply lines preceding this statement which will influence the worth of that variable

(v) **Using Breakpoints:**

- A breakpoint is an intentional way to stop or pause at a specific line in a program that has been set up for debugging purposes. The debugger puts a piece of code on that line, and during the execution process, it can call the debugger or create an interruption at the CPU level, which passes to the exception handler of the debugger.
- In Java, we can use either conditional breakpoints or exception breakpoints. Conditional breakpoints are best for Eclipse IDE as they can be used to remove bugs in your program through specified conditions where the thread will stop at the targeted line when it's true.

- Exception breakpoints can be used in Eclipse and NetBeans; they are also not associated with single code lines, thus applicable throughout.
- Different debuggers have different implementations of their debugging control algorithms that can add a condition to the breakpoint, which helps to stop or pause the code at a particular stage conditionally.
- Some have the option to manipulate a piece of data before resuming the execution again.

(vi) **Print Debugging:**

- Print debugging is the most used technique that every developer uses at a certain point in their development career, dumping values during an application's runtime to the application console or user interface. This can be used for Java and other programming languages as well.
- Developers do this to understand the flow inside a function or a service, sometimes to understand the shape of fetched or manipulated data. Print statements are added to the code based on where developers assume the bug resides - there is no set convention.
- Though this is not a very reliable method of debugging, it is handy in cases where either a silly mistake is involved, the developer couldn't understand the flow of the application, or the code block responsible for the bug is identifiable.

(vii) **Remote Debugging:**

- This is a widespread technique that involves debugging a process remotely when the debugger and the application are not running on the same platform/server. Remote debugging is at times done in tandem with production debugging.
- When we develop software for small devices with minimal resources that cannot contain both the codes for running the application and the application doing the debugging, remote debugging is the go-to method.
- Also, it is useful when we try to debug large applications with multiple components connected and communicate with each other to serve a client's requests, e.g. microservices.
- As we know, a debugger needs to be connected to both the OS and the application via basic hooks so that it can trigger the debugging process.
- When we install a remote debugging code in our application server, which itself is not a debugger, it just connects to the application running the server. It also connects to the remote debugger via network and sends or receives commands to the remote debugger.
- One example is Lightrun; they offer Lightrun Cloud, a free-to-use production debugger with logging capabilities and staging environments. It operates on the IntelliJ IDE for Java applications and is great for Monolith, microservices, Kubernetes, Docker Swarm, ECS, Big Data workers, serverless, and more such applications.
- Another good example is Eclipse. While Eclipse is also an IDE, it has a lot of advanced features for debugging which makes it a holistic tool. It is for Java, Python, Ruby, C# and PHP. A common use for Eclipse is managing remote debugging configurations and it allows for the same very easily, making it quite popular in the Java development community.

(viii) **Memory-dump Debugging:**

- When a program crashes, the system creates a snapshot of the memory and the data it holds at that specific time. This is called the memory dump. So, this is useful because developers can easily find exactly where the process crashed and the memory dump was taken to see a "stack-trace" of the functions that were invoked at that point and values of the local or global variables that are accessible.
- But the critical fact here is that we can get a memory dump without having a crash because there are different kinds of bugs where the process never crashes. We are talking about the memory dump of a process, but we can also get a memory dump of the whole system where multiple processes run from different applications.
- This is helpful when you have more than one piece of a Java application communicating with each other, and you can't identify exactly who is responsible for creating the bug.

- But if a kernel crash is different from a process or system crash, sometimes it's challenging to get the memory dump because it frees the memory before taking the dump.

(ix) Replay Debugging

- Replay debugging is another helpful technique. The idea comes from reverse engineering, where we have a well-defined flow of the actual process, and from there, we can track back to the issue.
- The first step of replay debugging is to record the process during its execution where all events invocation, memory management, data changes in hard disks, user inputs, and other connected device outputs are added to the snap.
- This piece of recording is generated by executing the process and then monitoring different events that trigger.
- Finally, when the process ends, it contains all the relevant information in the execution cycle.
- Developers or Quality Assurance Engineers can use this recording to walk through the execution path of the process or connect to a debugger and try to identify what went wrong.
- This might come in handy when we try to regenerate a bug because it gives us a holistic view of the whole process.

DESIGN OF MACRO PROCESSOR

Q.30. Explain the design of a single pass macro processor with the facility of MACRO definition within MACRO definitions and MACRO calls within MACROS.

Ans.

- A single pass macro processor with the facility of MACRO definition within MACRO definition is given in the figure.
- If we wanted to provide macro definitions within macros. The basic problem is that the inner macro is defined only after the outer one has been called to provide for any use of the inner macro, we would have to repeat both the macro definition and macrocell passes. However, there is a simpler solution that has the added advantages of reducing all macros processing to a single pass.
- In addition to MDTC and MNTC we have a Macro-definitions indent indicator and a Macro Definition Level Counter MDLC. The MDI and MDLC are switches (counters) used to keep track of macrocells and macro definitions.
- The MDI indicator has the value "ON" during the expansion of a macro cell and the value "OFF" at all other times. The actual expansion of macro calls is performed in the READ box, which is detailed in the second flowchart of the figure. Read Tests the switch MDI. If it is "ON" lines are read from the Macro Definition Table (MDT).
- The reading of a MEND line indicates the end of a macro and terminates the expansion of a call. MDI is reset to "OFF" and the next line is obtained from the Regular input stream. After setting all the counts as Maro Name Table MNT is searched for a match with operation code if the macro name is found, the lines are read from MDT. If it is a macro Pseudo-op, the entire macro definition that follows is saved in the next available locations in the MDT.
- The first line of the definition is the macro name line. The name is entered into the Macro Name Table, along with a pointer to the first location of the MDT entry of the definition. When the END Pseudo-op is encountered, all of the macro definitions have been processed for assembler processing.
- Again the algorithm tests the operation mnemonic of each input live to see if it is a name in the MNT. When a call is found, the call processes set a pointer, the Macro Definition Table P. to the corresponding macro definition stored in the MDT. The initial value of the MDTP is obtained from the MDT index field of MNT entry. The macro expander prepares the Argument list array (ALA) consisting of a table of dummy arguments indices and corresponding arguments to the call.
- The Macro definition Level counter is incremented by 1 when MACRO pseudo-op is encountered and decremented by 1 when a MENO pseudo-op occurs.
- The MDLC is used to ensure that the entire macro definition including MACROs and MENDs gets stored in the MDT.

To see the rest part of the question i.e. the design of a processor with the facility of macro calls within the macro. **[See the solution of Que No.7, P.2-7]**

Q.31. What is the use of Argument List Array in the design of a Macro processor? Explain with example.

Ans. Argument List Array:

- The Argument List Array (ALA) is used during both pass 1 and pass 2 but for somewhat reverse functions. During pass 1, to simplify later argument replacement during macro expansion, dummy arguments in the macro definition are replaced with positional indicators when the definition is stored.
- The i^{th} dummy argument on the macro name card is represented in the body of the macro by the index maker symbol # where # is a symbol reserved for the use of the macro processor (i.e. not available to the programmers). These symbols are used in conjunction with the argument list prepared before the expansion of a macro call. The symbols are used in conjunction with the argument list prepared before the expansion of a macro call.
- The symbolic dummy arguments are retained on the macro name card to enable the macro processor to handle argument replacement by name rather than by position.

As an example, consider the macro INCR used in example 3. The stored macro definition would be:

Macro Definition Table MDT

```
&LAB INCR  &ARG1, &ARG2, &ARG3
#0         A      1, #1
           A      2, #2
           A      3, #3
           MEND
           :
```

- During pass 2 it is necessary to substitute macro call arguments for the index markers stored in the macro definition. Thus upon encountering the call

LOOP INCR DATA1, DATA2, DATA3

The macro call expander would prepare an argument list array:

Argument List Array

8 bytes per entry

Index	Argument
0	"LOOP1bbb"
1	"DATA1bbb"
2	"DATA2bbb"
3	"DATA3bbb"

(b denotes the blank character)

Q.32. Explain the pass1 and pass2 with flowcharts of the macro definition and expansion algorithms.

Ans.

- Figure 1 and figure 2 show the flowcharts of the macro definition and expansion algorithms. Each of the algorithms makes a line-by-line scan over its input.
- The 'READ' boxes refer to the fetching of successive input lines from secondary storage into a workspace.

PASS 1 - MACRO DEFINITION:

- The algorithm for pass 1 (Fig.1) tests each input line. If it is a MACRO pseudo-op, the entire macro definition that follows is saved in the next available locations in the Macro Definition Table (MDT).
- The first line of the definition is the macro name line. The name is entered into the Macro Name Table (MNT), along with a pointer to the first location of the MDT entry of the definition. When the END pseudo-up is encountered, all of the macro definitions have been proposed so control transfers to pass 2 to process macro calls.

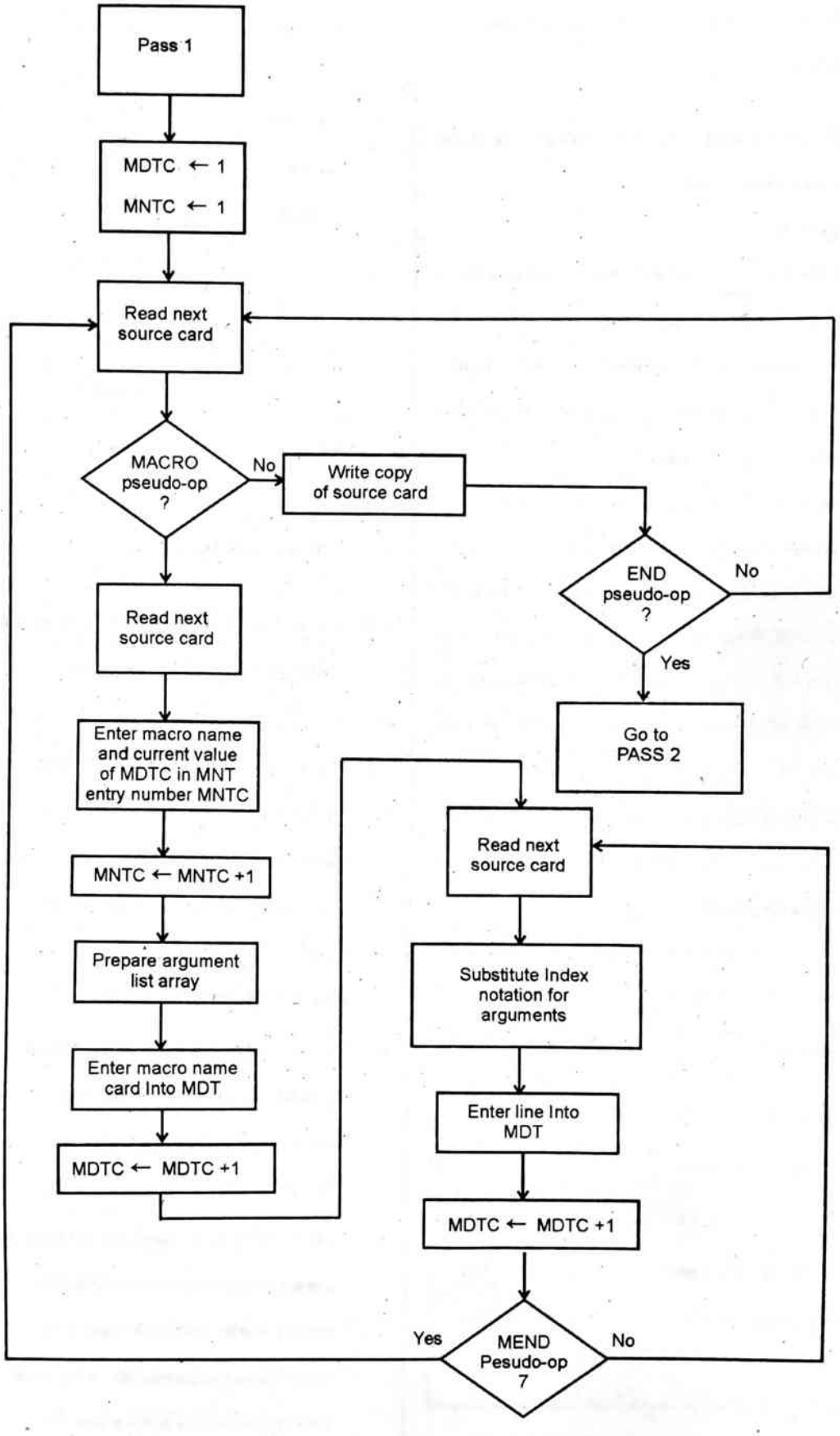


Fig.1. Pass 1 - Processing macro definitions.

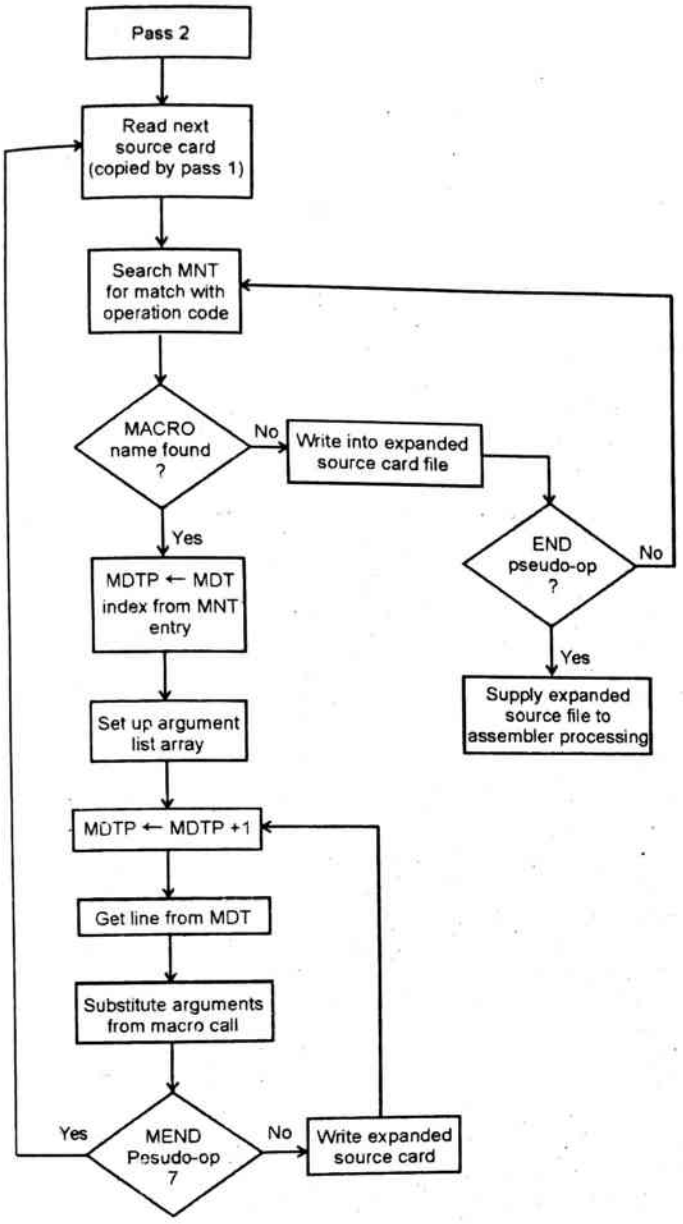


Fig.2. Pass 2 - Processing macro calls and expansion.

Pass 2 - MACRO CALLS and EXPANSION:

- The algorithm for pass 2 (Fig 2) tests the operation mnemonic of each input line to see if it is a name in the MNT. When a call is found, the call processor sets a pointer, the Macro Definition Table Pointer (MDTP), to the corresponding macro definition stored in the MDT. The initial value of the MDTP is obtained from the "MDT index" field of the MNT entry.
- The macro expander prepares the Argument List Array (ALA) consisting of a table of dummy argument indices and corresponding arguments to the call. This list is simply a succession of symbols ordered to match the dummy arguments on the name card. Arguments not represented in a call are considered blank and superfluous arguments are ignored.
- In the case of argument reference by position, this scheme is completely straightforward. For references by name, the macro processor locates the dummy argument on the macro name line to determine the proper index.
- Reading proceeds from the MDT; as each successive line is read, the values from the argument list are substituted for dummy argument indices in the macro definition. Reading of the MEND line the MDT terminates expansion of the macro and scanning continues from the input file. When the END pseudo-op to encountered, the expanded source deck is transferred to the assembler for further processing.

Q.33. Explain a simple one-pass macro processor with a flowchart.

Ans.

- The flowcharts in figure 1 and figure 2 shows the simple one-pass macro processor and detail of READ function used for macro expansion respectively.
- There are two additional variables introduced in the one-pass design; a Macro Definition Input (MDI) indicator and a Macro Definition Level Counter (MDLC). The MDI and MDLC are switches (counters) used to keep track of macro calls and macro definitions.
- The MDI indicator has the value "ON" during the expansion of a macro call and the value "OFF" at all other times. The actual expansion of macro calls is performed in the READ box, which is detailed in the second flowchart (figure 2). Read tests the switch MDI. If it is "ON", lines are read from the Macro Definition table (MDT).

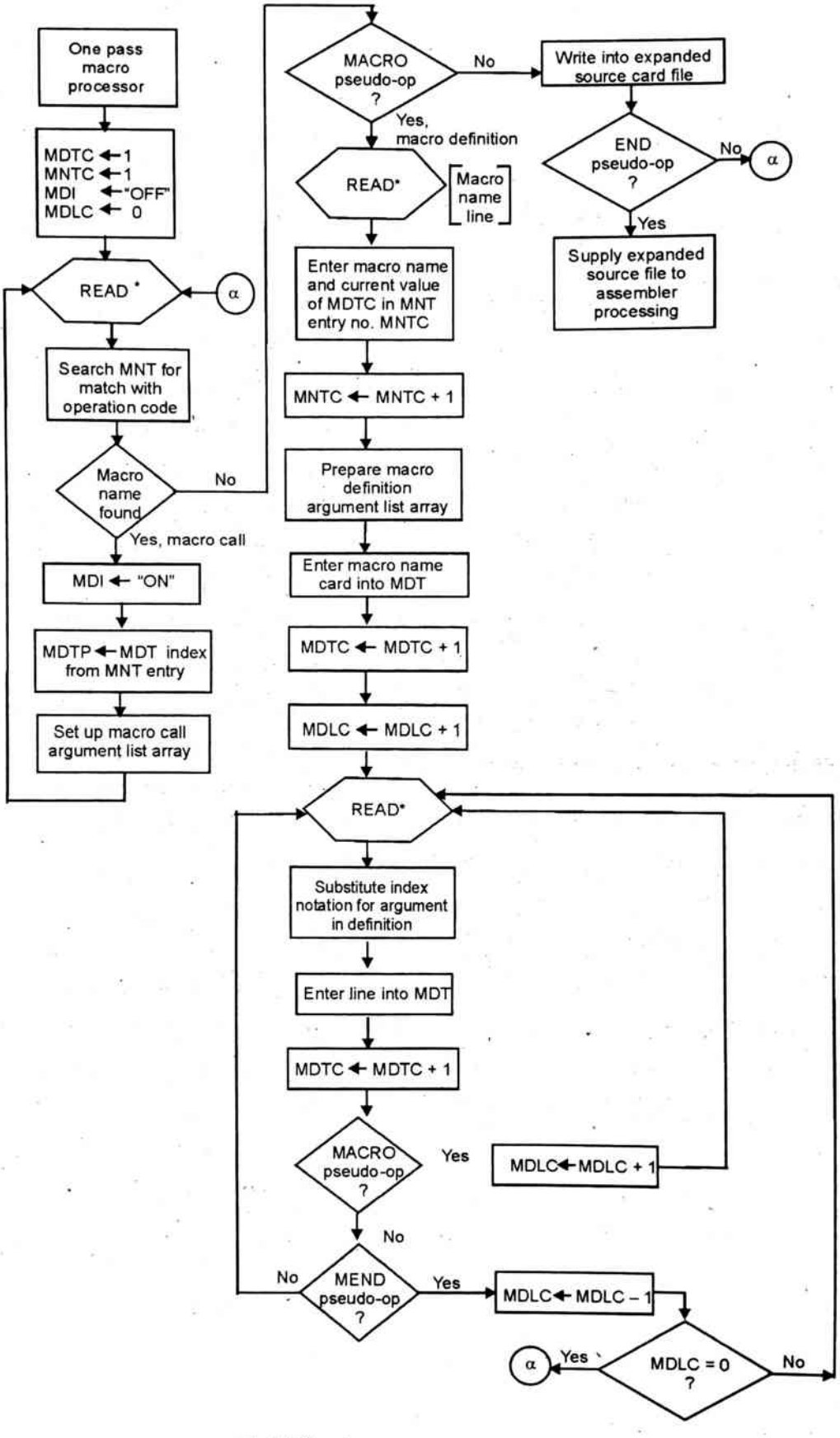


Fig.(1) Simple one-pass macro processor.

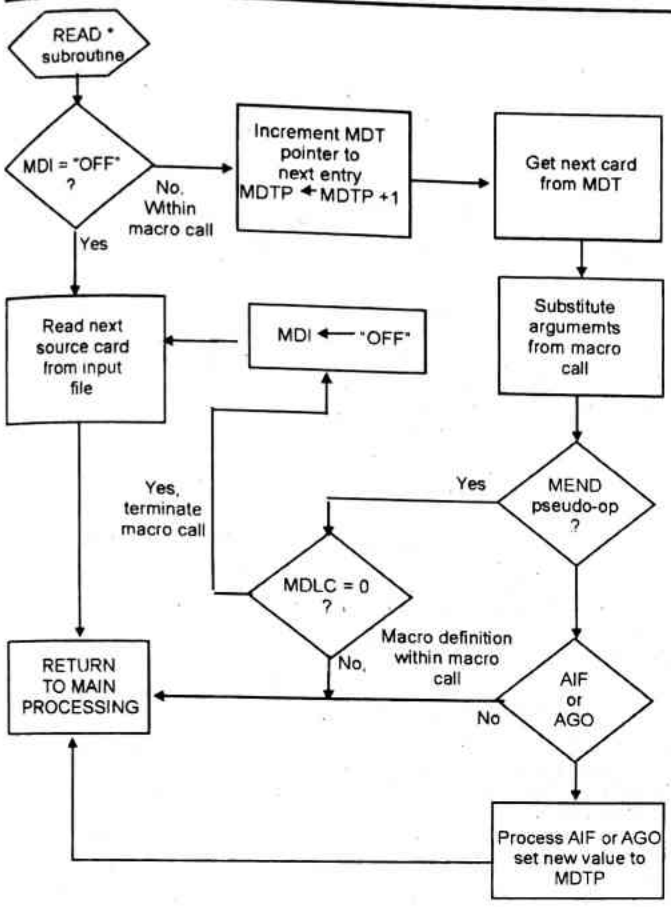


Fig.(2) Detail of READ function used for macro expansion.

- The reading of a MEND line indicates the end of a macro and terminates the expansion of a call; MDI is reset to "OFF" and the next line is obtained from the regular input stream. Note that lines returned by READ may include macro definitions; expanded macro code comes out of READ looking just like any other code and may therefore include macro definitions.
- The Macro Definition Level Counter is incremented by 1 when a MACRO pseudo-op is encountered and decremented by 1 when a MEND pseudo-op occurs.
- The MDLC is used to ensure that the entire macro definition, including MACROs and MENDs, gets stored in the MDT. This is analogous to treating statements with nested parenthesis –

  e.g. "(2 × (A + B) + C)". If a program is to read the entire expression, it must not stop at the first right parenthesis, but rather it must keep a count of left and right parentheses and recognize that the second parenthesis is the true end of the expression.
- The MDLC acts as a counter for the difference between the number of MACROs and MENDs encountered; it tells how many more MACROs than MENDs have been read.

DESIGN ISSUES OF MACRO PROCESSOR

Q.34. What are the design issues of the Macro processor?

Ans. Design Issues of Macro Processor:

- A macro processor is system software. A macro is a name given to a block of statements as a preprocessor directive.
- Being a pre-processor, the block of code is communicated to the compiler before entering into the actual coding function. Macro is the Section of code that the programmer defines once and then can use or invokes many times.
- A more general problem involves the tokens of the programming language-for example, identifiers, constants, operators and keywords. Languages differ substantially in their restrictions on the length of identifiers and the rules for the formation of constants.
- Sometimes the rules for such tokens are different in FORTRAN or DATA DIVISION in COBOL). In some languages, there are multiple-character operators such as ** in FORTRAN and := in Pascal. Problems may arise if these are treated by a macro processor as two separate characters rather than as a single operator.
- Even the arrangement of the source statements in the input file may create difficulties. The macro processor must be concerned with whether or not blanks are significant, with the way statements are continued from one line to another and with special statement formatting conventions such as those found in FORTRAN and COBOL.
- Another potential problem with general-purpose macro processors involves the syntax used for macro definitions and macro invocation statements. With most special-purpose macro processors, macro invocations are very similar in form to statements in the source programming language.

Data Structure:

- Data structure has a strong influence on the nature of the macro design. Often, a change in the data structure is a far better solution than writing a macro to preserve it.

- The macro needs modification for the slightest changes in the scope of the problem. The macro lacks the qualities of reusability and maintenance.

  For example, if the data variable names contain important information, it is quite likely that the macro code will need to extract and manipulate the information in code.

Parameter Name and Position:

- Parameter names should describe the value that is being passed to the macro.
- They should not be too long or too short and remember to be consistent with the naming scheme within, as well as between macros.
- The position of the parameters should be considered, in addition to using proper names.
- In a complex macro, it helps to organize the parameters into groups.

  For example, it is common to have all input parameters before all output parameters. Adding such a structure to the parameters helps both the reader of the macro and the reader of the code.
- If such structure is attempted with positional parameters, then adding new parameters requires you to break the implied structure or to break existing code.
- Hence, positional parameters not only make it hard to see what the code means, but they also make it hard to extend the functionality of an existing macro.

The Size of the Program in Micro:

- The pre-processor will replace all the macros in the program by its real definition before the compilation process of the program.
- If the micro is called multiple times and the length of micro definition is itself a bit lengthy then, runtime size may be an issue for the compiler.

Local vs Global Macro Variables:

- When a macro is executed, it creates its environment for storing local variables.
- Any variables outside of that execution are independent of those variables and do not change or clash with variables inside the macro.

  However, any call to a macro can include the variables created outside the macro environment.

  This means a macro can protect itself from outside influence by declaring inner variables local with a local statement, but it cannot protect itself from the macros that it invokes.
- When a macro uses a global macro variable that must be assigned by the user, it increases the amount of information required to use the macro.

  Even if the user need not assign the value, they must be aware of the name of the variable to prevent inadvertently using that name.
- Local variables hide the information from the caller and protect them from the need to know about the macro's internal code.
- Another problem with using global macro variables is it makes the code difficult to reuse.

OPERATING SYSTEM

Q.35. Define Operating System. State its types.

Ans. Operating System:

The operating system can be defined as a collection of system programs that control, the overall operation of computer systems. It is also defined as a program that acts as an intermediary between a user of computer & computer hardware.

Types of O.S.:

(i) Batch System:

- Early computers were physically enormous machines run from a console. The common input devices were card readers & tape drives. The common output devices were line printers, tape drives, and card punches. The user did not interact directly with the computer systems. Rather, the user prepared a job that consisted of the program, the data, and some control information about the nature of the job (control cards) and submitted it to the computer operator. The job was usually in the form of punch cards.
- At some later time, the output appeared. The o/p consisted of the result of the program, as well as a dump of the final memory and register contents for debugging.

- The operating system in these early computers was fairly simple. Its major task was to transfer control automatically from one job to the next.
- To speed up processing, operators batched together jobs with similar needs and run them through the computer as a group. Thus. the programmers would leave their programs with the operator. The operator would sort the program into batches with similar requirements and, as the computer became available, would run each batch. The o/p from each job would be sent back to the appropriate programmer.

(ii) Time-Sharing System:

- The operating system which executes several jobs or programs simultaneously is known as a multiprogramming operating system.
- Time sharing is a logical extension of multiprogramming. The CPU executes multiple jobs by switching among them, but the switching occurs so frequently that the users can interact with each program while it is running.
- A time-shared operating system allows many users to share the computer simultaneously. Since each action or command in a time-shared system tends to be short, only a little CPU time is needed for each user. As the system switches rapidly from one user to the next, each user is given the impression that the entire computer system is dedicated to his use, even though it is being shared among many users.
- A time-shared operating system uses CPU scheduling and multiprogramming to provide each user with a small portion of a time-shared computer.
- Time-sharing operating systems are even more complex than multprogrammed operating systems. In both, several jobs must be kept simultaneously in memory, so the system must have memory management and protection. The Time-sharing system also provides a file system.

(iii) Real-Time System:

- Another form of a special-purpose operating system is the real-time system. A real-time system is used when rigid time requirements have been placed on the operation of a processor or the flow up data; thus it is often used as a control device in a dedicated application.
- A real-time system has well defined, fixed time constraints. Processing must be done within the defined constraints or the system will fail.
- Real-time system comes in two flavours: A hard real-time system and a soft real-time system.
- A hard real-time system guarantees that critical tasks be completed on time. This goal requires that all delays in the system be bounded, from the retrieval of stored data to the time that it takes the operating system to finish any request made of it. Such time constraints dedicate the facilities that are available in hard real-time systems.

  A less respective type of real-time system is a soft real-time system, where a critical real-time task gets priority over the other task, retains that priority over the other task, retains that priority until it completes. A Soft-real-time system is an achievable goal that can be mixed with other types of systems.

(iv) Distributed System:

- A network, in the simplest terms, is a communication path between two or more systems. Distributed systems depend on networking for their functionality. By being able to communicate, distributed systems can share computational tasks and provide a rich set of features to users.
- Networks vary by the protocol used, the distances between nodes, and the transport media. TCP/IP is the most common network protocol, although ATM and other protocols are in widespread use.
- Most operating systems support TCP/IP, including the WINDOWS and UNIX operating system. To an operating system, a network protocol simply needs an interface device a network adapter. Networks are typecast, based on the distances between their nodes.
- A (LAN) Local Area Network, exists within a room, a floor, or a building. A wide-area network (WAN), usually exists between buildings, cities, or countries. A metropolitan area network (MAN) could link buildings within a city.

NOTES

Due to Covid - 19

Summer - 2020 to Winter - 2021 exams were conducted online by the respective Colleges.

Notes:-

VBD

Based on New Syllabus

COMPUTER SCIENCE ENGINEERING

Our Most Useful Titles for B.Tech. IV Sem. (RTMNU)

VBD
DISCRETE MATHEMATICS AND GRAPH THEORY
Solved Question Bank + University Paper Solutions
VBD

VBD
SYSTEM PROGRAMMING
Solved Question Bank + University Paper Solutions
VBD

VBD
DATABASE MANAGEMENT SYSTEM
WITH VIVA VOCE
Solved Question Bank + University Paper Solutions
VBD

VBD
DATA STRUCTURE AND PROGRAM DESIGN
WITH VIVA VOCE
Solved Question Bank + University Paper Solutions
VBD

VBD
COMPUTER NETWORKS
Solved Question Bank + University Paper Solutions
VBD

VBD
THEORY OF COMPUTATION
Solved Question Bank + University Paper Solutions
VBD

VBD PUBLICATIONS

- Sales Office -

Plot No. 236, Near Ram Coolers, Besides Ganesh Temple, Singada Market,
Navi Shukrawari, Mahal, Nagpur-18
Ph. : 0712-2722235, 2722236, 7620266004

For online purchases, visit WWW.VBDPUBLICATIONS.COM